



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Programmazione di sistema

Docenti:

Gianpiero Cabodi
Giovanni Malnati

Studente:

Cristiano Berardo

Indice

I	API Programming	1
1	Architettura e memoria	2
1.1	Obiettivi	2
1.2	Perché studiare come è organizzata la memoria?	2
1.3	Programmi e loro esecuzione	2
1.4	Spazio di indirizzamento	3
1.5	Effetti pratici	3
1.6	Gerarchia di memoria	4
1.7	Esecuzione e programmi di alto livello	4
1.8	Modello di esecuzione di un programma	4
1.9	Livelli di astrazione	4
1.10	Librerie di esecuzione	4
1.11	Modello di esecuzione nei linguaggi C e C++	5
1.12	Esecuzione sequenziale sincrona, senza limitazioni	5
1.13	Flusso di esecuzione	5
1.14	Stack	5
1.15	Heap	6
1.16	Organizzazione dello spazio di indirizzamento	7
1.17	Ciclo di vita delle variabili	7
1.18	Variabili globali e locali	7
1.19	Variabili dinamiche	7
1.20	Allocazione della memoria	8
1.21	Rilascio della memoria	8
1.22	Puntatori	8
1.23	Il costo del rilascio	8
1.24	Puntatori in C/C++	9
1.25	Le ambiguità dei puntatori in C/C++	9
1.26	Problemi legati ai puntatori	9
1.27	Responsabilità del programmatore	9
1.28	Rischi	10
1.29	Problemi dei puntatori in C/C++	10
1.30	Le risposte di Rust	10
1.31	Gestire i puntatori	10
1.32	Possesso della memoria	11
1.33	Dipendenze	11
1.34	Ma non basta scrivere un programma "giusto"?	11
1.35	Perché questo problema non si verifica in altri linguaggi?	11
1.36	Confronto	12
2	Possesso	13
2.1	Possesso	13
2.2	Movimento	13
2.3	Copia	14
2.3.1	Copia e Movimento	15
2.4	Clonazione	15
2.5	Box<T> - Puntatore con possesso	15
2.6	Riferimenti	16

2.6.1	Riferimenti e prestiti	17
2.6.2	Riferimenti mut - mutual exclusion	17
2.6.3	Riferimenti: disposizione in memoria	17
2.7	Tempo di vita dei riferimenti	18
2.7.1	Esistenza in vita	18
2.8	Riferimenti e funzioni	19
2.9	Riferimenti e strutture dati	20
2.10	Elisione dei tempi di vita	20
2.11	Possesso - riassunto delle regole	21
2.12	Puntatori in Rust	21
2.13	Slice	22
2.14	Vantaggi introdotti dal concetto di possesso	22
3	Tipi composti	23
3.1	Struct in Rust	23
3.1.1	Come utilizzare una struct	23
3.1.2	Accesso alla struct	24
3.1.3	Struct fatte a tupla	24
3.1.4	Probabile rappresentazione in memoria	24
3.1.5	Rappresentazione in memoria	24
3.2	Visibilità	25
3.3	Metodi	25
3.3.1	Definizione di un metodo	25
3.4	Costruttori	27
3.5	Gestione delle risorse	27
3.6	RAII - Resource Acquisition Is Initialization	27
3.7	Distruttori	27
3.8	Metodi statici	28
3.9	Enum	28
3.9.1	Rappresentazione in memoria	28
3.9.2	Enumerazioni e clausole match	28
3.10	Destrutturazione	29
4	Gestione degli errori	30
4.1	Errori ed eccezioni	30
4.1.1	Spazio di scelta	30
4.1.2	Supporto sintattico alla gestione degli errori	31
4.1.3	I limiti della gestione delle eccezioni	31
4.1.4	Gestioni delle eccezioni in Rust	31
4.1.5	Elaborare i risultati	32
4.1.6	Gestire gli errori	32
4.1.7	Panic	32
4.1.8	Ignorare gli errori	32
4.1.9	Propagare gli errori	33
4.1.10	Altri modi di esprimere il fallimento	33
4.1.11	Propagare errori eterogenei	33
5	Moduli	36
5.1	Modularità	36
5.1.1	Perché rendere modulare un progetto?	36
5.2	Struttura di un progetto Rust: package, crate, moduli	36
5.3	Crates	37
5.4	Organizzazione del codice	37
5.4.1	Create binario	37
5.4.2	Le librerie	37
5.4.3	Package con più file binari	38
5.5	Moduli e visibilità	38
5.5.1	Visibilità di default - privata	38
5.5.2	Visibilità pubblica - pub	38

5.5.3	Cammino assoluto	39
5.5.4	Cammino relativo	39
5.5.5	Cammino e *	39
5.6	Moduli e file sorgente	39
5.6.1	Struttura di un progetto eseguibile Rust	39
5.6.2	Struttura di un progetto con due o più eseguibili	39
5.7	Librerie	40
5.8	Librerie interne ed esterne	40
5.8.1	Librerie ed interoperabilità	40
5.9	Collegamento delle librerie	41
5.10	Preludio	41
5.11	Clap - libreria per la gestione dei parametri da linea di comando	42
6	Polimorfismo	43
6.1	Polimorfismo: cos'è e perché serve	43
6.2	Implementare il polimorfismo	43
6.3	Il polimorfismo in Rust si basa sui tratti	43
6.3.1	Definire un tratto	44
6.3.2	Implementare un tratto per un dato tipo	44
6.3.3	Un tratto può essere implementato da molti tipi	44
6.3.4	Un tratto può essere implementato dai tipi esistenti	44
6.3.5	Invocare i metodi di un tratto	45
6.4	I tratti come vincoli	45
6.4.1	Due modi di usare un tratto	45
6.5	Invocazione dinamica	45
6.6	Oggetti-tratto	45
6.7	Vincoli degli oggetti-tratto	45
6.8	Tratti con implementazioni di default	46
6.9	Tipi e tratti	46
6.10	Programmazione generica	46
6.11	Funzioni generiche	46
6.12	Dall'astrazione al codice concreto	47
6.13	Tipi generici	47
6.14	Sintassi dei tipi generici	47
6.15	Tratti con tipi associati	47
6.16	Tipi associati e generics	48
6.17	Super-tratti	48
6.18	&dyn Trait e T: Trait a confronto	48
6.19	Approfondimento: tratti nella libreria standard	48
6.20	Confronto tra valori	49
6.21	Visualizzare i contenuti	49
6.22	Copia e duplicazione	49
6.22.1	Clone	49
6.22.2	Copy	49
6.23	Rilasciare le risorse	50
6.24	Conversione tra tipi	50
7	Test	51
7.1	Test	51
7.2	Test di unità	51
7.2.1	Quanti test fare	52
7.3	Test dei casi limite	52
7.4	Refactoring: miglioriamo il codice	52
7.5	Test di integrazione	53
7.5.1	Test di integrazione: modulo Repository	53
7.5.2	Test di integrazione: modulo Service	54
7.5.3	Test di integrazione: esempio di test	54
7.5.4	Test di integrazione: razionale	54
7.6	Test complessivi - end-to-end test	54

7.7	Cosa fanno i test	54
7.8	Il flusso di lavoro con i test	55
7.9	Test in Rust	55
7.9.1	Sintassi dei test	55
7.9.2	Asserzioni	56
7.10	Eseguire i test	56
7.11	Automatizzare la creazione di test	56
7.11.1	Test parametrici con rstest	56
7.12	Riassunto	57
8	Collezione di dati	58
8.1	Complessità	58
8.2	Metodi comuni a tutte le collezioni	60
8.3	Vec<T>	60
8.3.1	Metodi di Vec<T>	60
8.4	VecDeque<T>	61
8.5	LinkedList<T>	62
8.6	Mappe	62
8.6.1	HashMap<K,V>	62
8.6.2	BTreeMap<K,V>	63
8.6.3	Mappe e possesso	63
8.6.4	Entry<'a,K,V>	64
8.6.5	Operare con le mappe	64
8.7	Insiemi	64
8.7.1	HashSet<T>	64
8.7.2	BTreeSet<T>	65
8.7.3	BinaryHeap<T>	65
9	File I/O	67
9.1	File e file system	67
9.2	Percorsi	67
9.3	Navigare il file system	67
9.4	Manipolare i file nel file system	68
9.5	Operazioni con i file	68
9.6	Leggere e scrivere file	68
9.6.1	Aprire un file	68
9.7	I tratti relativi a I/O	69
9.7.1	std::io::Read	69
9.7.2	Metodi del tratto Read	69
9.8	std::io::BufRead	70
9.9	std::io::Write	70
9.10	std::io::Seek	71
9.11	Gestione degli errori di I/O	71
9.11.1	ErrorKind: casi comuni e strategie	72
9.11.2	Buone pratiche	72
9.12	Lettura e scrittura di contenuti strutturati	72
9.12.1	Uso del framework Serde	73
10	Chiusure	74
10.1	Funzioni e chiusure	74
10.1.1	Puntatori a funzione - fn(T1, ..., Tn) -> U	74
10.2	Chiusure: codice + ambiente catturato	75
10.2.1	Differenza essenziale: una chiusura cattura l'ambiente	75
10.2.2	Chiusure che non catturano	75
10.2.3	Modalità di cattura in Rust	75
10.2.4	Modello mentale: le chiusure come struct anonime	76
10.2.5	Cattura delle variabili in Rust	76
10.2.6	Gerarchia dei tratti funzionali	76
10.3	FnOnce vs FnMut	76

10.4	Fn	77
10.5	Chiusure come argomenti	77
10.5.1	Restituire una chiusura in sicurezza	77
10.5.2	Esempio di generatore	77
11	Iteratori	78
11.1	Iteratori	78
11.1.1	Uso degli iteratori	78
11.1.2	Caratteristiche degli iteratori	78
11.2	Iteratori in Rust	79
11.2.1	Implementare un iteratore in Rust	79
11.3	Iteratori e possesso	80
11.3.1	Derivare un iteratore	81
11.4	Adattatori	81
11.4.1	Adattatori: trasformare gli elementi	81
11.4.2	Adattatori: filtrare	82
11.4.3	Adattatori: combinare	82
11.4.4	Adattatori: funzionalità ulteriori	82
11.4.5	fold(...): riduzione esplicita di un iteratore	83
11.4.6	Consumatori: aggregare	83
11.4.7	Consumatori: cercare e verificare	84
11.4.8	Consumatori: confrontare	84
12	Smart Pointers	85
12.1	Smart Pointer	85
12.1.1	Uso dei puntatori	85
12.2	Smart pointer in Rust	85
12.2.1	<code>std::Box<T></code>	86
12.3	Conteggio dei Riferimenti	87
12.3.1	<code>std::rc::Rc<T></code>	87
12.3.2	<code>std::rc::Weak<T></code>	88
12.4	Interior Mutability	88
12.4.1	<code>std::cell::Cell<T></code>	88
12.4.2	<code>std::cell::RefCell<T></code>	89
12.4.3	<code>std::borrow::Cow<'a, B></code>	90
12.4.4	Smart pointer e metodi (<code>self</code>)	90
12.5	Esempio: Albero doppiamente collegato	90
12.6	Cheat Sheet dei Container in Rust	92
13	Concorrenza	93
13.1	Programmazione concorrente	93
13.2	Thread nativi	93
13.2.1	Processi, thread e modelli di esecuzione	94
13.2.2	Cosa implica la concorrenza	94
13.2.3	Concorrente e parallelismo	95
13.2.4	Concorrenza in pratica	95
13.3	Modello di memoria	95
13.3.1	Problemi aperti	96
13.4	La risposta dei processori	96
13.4.1	Errori comuni	96
13.5	Thread e memoria	97
13.5.1	Esecuzione e non determinismo	97
13.6	Esecuzione e non determinismo	97
13.6.1	Interferenza	98
13.7	Sincronizzazione	98
13.7.1	Strutture native di sincronizzazione	99
13.8	Correttezza	99
13.8.1	Safe Rust	99
13.9	Correttezza: non solo sicurezza, ma anche progressione	100

13.9.1	Accesso condiviso: i possibili problemi	100
13.9.2	Accesso condiviso: le possibili soluzioni	100
13.9.3	Mutex	100
13.9.4	Quale strumento utilizzare?	101
13.10	Uso dei thread	101
13.11	Thread in Rust	101
13.11.1	Configurare un thread	102
13.12	I tratti della concorrenza	102
13.12.1	I tratti della concorrenza: riassunto	103
13.13	Modello di concorrenza	103
13.14	Condivisione dello stato	104
13.15	Rilasciare i mutex	104
13.16	Mutex in Rust	104
13.16.1	<code>std::sync::Mutex<T></code>	105
13.16.2	<code>std::sync::Arc<T></code>	106
13.16.3	<code>std::sync::RwLock<T></code>	107
13.17	Tipi atomici	107
13.18	<code>std::sync::Weak<T></code>	108
13.19	Attese condizionate	108
13.19.1	Condvar in C++	109
13.19.2	Condvar in Rust	109
13.19.3	Condvar - metodi principali	109
13.19.4	Pattern Condvar	109
13.20	Meccanismo di funzionamento	110
13.21	Notifiche spurie	110
13.22	Notifiche perse	110
13.23	Attesa temporizzata	110
13.24	Stato condiviso o scambio di messaggi?	111
13.25	Condivisione di messaggi	111
13.26	Canali sincroni	112
13.27	Condvar esempi	112
13.28	Gestione dell'attesa	115
13.29	Canali: regole pratiche	115
13.30	La libreria Crossbeam	115
13.31	Uso della libreria Crossbeam	115
13.31.1	Fan-out / Fan-in	115
13.31.2	Pipeline	116
13.32	Producer/consumer - Thread pool	116
13.33	Checklist finale	117
14	Programmazione asincrona	118
14.1	Programmazione Asincrona	118
14.1.1	Concorrenza non è parallelismo	118
14.2	Operazioni bloccanti attese	118
14.2.1	Il costo dei thread	119
14.3	Tre strategie di esecuzione	119
14.4	Separare richiesta e continuazione (callback)	120
14.5	I limiti delle callback	120
14.5.1	Async/await: stessa idea, forma diversa	120
14.6	Funzioni asincrone	121
14.6.1	La parola chiave <i>async</i>	121
14.6.2	L'operatore <i>.await</i>	121
14.6.3	Un future è inerte: serve un esecutore	121
14.7	Il tratto Future	122
14.7.1	Chi invoca <code>poll()</code> : executor, reactor, Waker	122
14.8	Esecutori	122
14.9	Possesso nel tempo	123
14.10	Tokio: i costrutti di base	123

14.10.1	Impostare un progetto con Tokio	123
14.10.2	Attendere un Future: esecuzione sequenziale	123
14.10.3	tokio::spawn - avviare task concorrenti	123
14.10.4	Non perdere il controllo dei task	124
14.10.5	join! - attendere più Future insieme	124
14.10.6	select! - il primo che si completa	125
14.10.7	Gestione del tempo	125
14.10.8	Cancellare un task	125
14.10.9	Esempio	125
14.11	Guida alla progettazione	126
14.11.1	Esempio: sequenza o concorrenza?	126
14.11.2	Il caso speciale: attività che richiedono molta CPU	126
14.11.3	Quattro modi di coordinare i Future	127
14.12	Condividere e comunicare	127
14.12.1	Architettura di elaborazione	127
14.13	Condividere dati tra task	127
14.13.1	Stato condiviso: Arc<tokio::sync::Mutex<T>»	128
14.13.2	Stato condiviso vs canali	128
14.13.3	Comunicare invece di condividere: i canali	128
14.13.4	Backpressure	130
14.13.5	Quando usare cosa	130
14.13.6	Errori tipici in async Rust	130
14.14	Come funziona internamente	131
14.14.1	Esecuzione parziale	131
14.14.2	Generare la macchina a stati	131
14.14.3	Implementare la funzione	132
14.15	Prestazioni a confronto	133
14.15.1	Quando l'esecuzione asincrona non conviene?	133

II OS Programming

134

15	Main memory	135
15.1	Memoria principale	135
15.1.1	Località spaziale e località temporale	135
15.2	Problema della riallocazione	136
15.3	Protezione	136
15.3.1	Protezione degli indirizzi hardware	136
15.4	Binding degli indirizzi	136
15.5	Memory Management Unit - MMU	137
15.5.1	Spazio di indirizzamento logico vs. fisico	137
15.6	MMU	137
15.6.1	Registro di riallocazione	137
15.6.2	Caricamento dinamico	137
15.6.3	Linking Statico vs Dinamico	138
15.7	Allocazione dei programmi	138
15.7.1	Allocazione contigua	138
15.7.2	Partizione variabile	139
15.7.3	Problema dell'allocazione dinamica della memoria	139
15.7.4	Frammentazione	139
15.7.5	Compattazione	139
15.8	Paginazione	140
15.8.1	Schema di traduzione degli indirizzi	140
15.9	Hardware di paginazione	140
15.9.1	Quante pagine?	141
15.9.2	Implementazione della page table	142
15.9.3	Soluzione hardware - Translation Look-Aside Buffer	142
15.9.4	Tempo di accesso effettivo	142
15.10	Codice C e page fault	143

15.11	Page fault	143
15.11.1	Come rilevare i page fault?	145
15.11.2	Passi nella gestione dei page fault	145
15.11.3	Costo di un page fault	145
15.12	Aspetti nel richiedere paging	146
15.13	Riavvio dell'Istruzione	146
15.14	Strategia di sostituzione delle pagine	146
15.14.1	Passi nel richiedere Paging - Caso Peggior	146
15.14.2	Ottimizzazioni della Paginazione su Richiesta	147
15.14.3	Copia su Scrittura (Copy-on-Write)	147
15.14.4	Cosa succede se non ci sono frame liberi?	148
15.14.5	Sostituzione di pagina di base	148
15.15	Algoritmi di sostituzione di pagine e frame	148
15.15.1	Frequenza di Page Fault - probability empirica	149
15.15.2	Selezione delle vittime usando il modify bit	149
15.15.3	Algoritmo FIFO	149
15.15.4	Algoritmo ottimale	150
15.15.5	Algoritmo LRU - Least Recently Used	150
15.15.6	Algoritmi LRU approssimati	151
15.15.7	Algoritmi basati sul conteggio	152
15.15.8	Page-Buffering Algorithms	152
15.15.9	Applicazioni e Sostituzione delle Pagine	152
15.16	Allocazione dei frame ai processi	153
15.16.1	Allocazione fissa	153
15.16.2	Allocazione globale vs. locale	153
15.16.3	Recupero delle pagine	153
15.16.4	Non-Uniform Memory Access	154
15.16.5	Thrashing	154
15.16.6	Paging su richiesta e thrashing	154
15.16.7	Working-Set Model	154
15.16.8	Tenere traccia del Working Set	155
15.16.9	Frequenza dei page fault	155
15.17	Page Fault Frequency Algorithm	156
15.18	Allocazione in memoria del kernel	157
15.19	Buddy System	157
15.20	Slab Allocator	158
15.21	Prepaging	158
15.22	Dimensione della Pagina	158
15.23	TLB Reach	159
15.24	I/O interlock	159
15.24.1	Esempio Windows	160
15.24.2	Esempio Solaris	160
15.25	Performance del Demand Paging	160
15.25.1	Priority Allocation	161
15.25.2	Compressione della Memoria	161
16	Memoria Virtuale	163
16.1	Swapping delle Pagine	163
16.1.1	Tempo di Context Switch con lo Swapping	163
16.1.2	Swapping sui Sistemi Mobili	164
16.2	Struttura della Tabella delle Pagine	164
16.2.1	Shared Pages	165
16.2.2	Paginazione gerarchica	165
16.2.3	Tabelle delle Pagine con Hashing	165
16.2.4	Tabella delle Pagine Invertita	166
16.3	Oracle SPARC Solaris	166
16.4	Segmentazione	167
16.4.1	Paginazione vs Segmentazione	168

16.5 Esempio: Architettura ARM	169
17 Sistemi di Archiviazione di Massa	170
17.1 Dispositivi di Memoria Non Volatile	170
17.2 Algoritmi del Controller NAND Flash	171
17.3 Memoria Volatile	171
17.4 Struttura del Disco	171
17.4.1 Collegamento del Disco	171
17.5 Scheduling del Disco	171
17.5.1 FCFS	172
17.5.2 SCAN - algoritmo dell'ascensore	172
17.5.3 C-SCAN	172
17.5.4 Scelta dell'Algoritmo di Scheduling del Disco	173
17.6 Pianificazione per la NVM	173
17.7 Rilevamento e Correzione degli Errori	173
17.7.1 Gestione dei dispositivi di storage	173
17.7.2 Gestione dello Spazio di Swap	174
17.7.3 Collegamento dello Storage all'Host	174
17.7.4 Archiviazione Cloud	174
17.7.5 Storage Array	175
17.7.6 Storage Area Network - SAN	175
17.7.7 Array Ridondante di Dischi Indipendenti - RAID	175
17.8 Altre Funzionalità	176
17.9 Estensioni	176
17.10 Object Storage	176
18 Interfaccia del File System	177
18.1 Cos'è un File?	177
18.1.1 Attributi del File	177
18.1.2 Operazioni sui File	177
18.1.3 File Aperti	178
18.1.4 Open file locking	178
18.1.5 Struttura del File	178
18.2 Accesso e Operazioni sui File	178
18.2.1 Blocco dei File	178
18.2.2 Metodi di Accesso	179
18.3 File Mappati in Memoria	180
18.4 Dischi e File System	180
18.4.1 Tipi di File System	180
18.5 Struttura delle Directory	181
18.5.1 Organizzazione delle Directory	181
18.5.2 Directory a Singolo Livello	181
18.5.3 Directory a Due Livelli	181
18.5.4 Directory ad Albero	182
18.5.5 Directory a Grafo Aciclico	182
18.6 File Sharing	182
18.6.1 Condivisione di File - Sistemi Remoti	183
18.6.2 Condivisione di File - Modalità di Guasto	183
18.7 Protezione	183
18.7.1 Liste di Accesso e Gruppi	183
19 I/O System	184
19.1 Panoramica	184
19.2 I/O Hardware	184
19.3 Polling	185
19.4 Interrupt	185
19.5 Latenza (Latency)	186
19.6 Direct Memory Access (DMA)	186
19.7 Interfaccia di I/O per le applicazioni	186

19.8	Caratteristiche dei dispositivi di I/O	186
19.9	Dispositivi a blocchi e a caratteri	187
19.10	Dispositivi di rete	188
19.11	Clock e Timer	188
19.12	I/O Non bloccante e Asincrono	188
19.13	Due metodi di I/O	188
19.14	Vectored I/O	188
19.15	Sottosistema di I/O del Kernel	189
19.16	Device-status Table	189
19.17	Velocità disp. I/O	189
20	File System Implementation	190
20.1	Struttura del File System	190
20.1.1	Driver dei Dispositivi - I/O Controller	190
20.1.2	Basic File Systeme	190
20.1.3	File organization module	190
20.1.4	Logical File System	190
20.2	Tipi di File System	191
20.3	Dall'API all'Implementazione	191
20.3.1	Operazioni del File System	191
20.3.2	File Control Block - FCB	191
20.3.3	Il Linux iNode	191
20.3.4	Strutture del File System in Memoria	192
20.4	Implementazione delle Directory	193
20.4.1	Lista Lineare	193
20.4.2	Tabella Hash	193
20.5	Metodo di Allocazione	193
20.5.1	Metodo di Allocazione Contigua	193
20.5.2	Allocazione Collegata - Linked	194
20.5.3	Metodo di Allocazione File-Allocation Table FAT	194
20.5.4	Metodo di Allocazione Indicizzata	194
20.5.5	Prestazioni	194
20.6	Gestione dello Spazio Libero	195
20.6.1	Lista Collegata dello Spazio Libero su Disco	195
20.6.2	Lista Collegata dello Spazio Libero	195
20.6.3	TRIMing Unused Blocks	196
20.6.4	Efficiency and Performance of File Systems	196
20.6.5	Page Cache	196
20.6.6	Recovery	196
20.6.7	Log Structured File Systems	196
20.7	File System in altri OS	196
20.7.1	File System di Windows	196
20.8	Thread in OS161	197
20.9	Memoria in OS161	197
20.10	Lab 4	197

Parte I

API Programming

Capitolo 1

Architettura e memoria

1.1 Obiettivi

- Comprendere lo spazio di indirizzamento virtuale
- Conoscere la differenza tra stack e heap
- Saper spiegare perché i puntatori sono pericolosi
- Conoscere quali bug tipici nascono in C/C++
- Spiegare perché Rust introduce i concetti di possesso, prestito, smart pointer

1.2 Perché studiare come è organizzata la memoria?

90% dei bug di sistema → Accessi illeciti alla memoria o mancata sincronizzazione in programmi concorrenti.
Rust offre una risposta a questi problemi

```
int* p = (int*)malloc(sizeof(int));  
free(p);  
*p = 5; // cosa succede?
```

1.3 Programmi e loro esecuzione

Un programma eseguibile è costituito da un insieme di istruzioni macchina, dati, valori di configurazione e controllo rappresentati come sequenze di numeri binari. Il significato delle istruzioni macchina è cablato all'interno del processore specifico per cui il programma è stato creato. Il significato delle informazioni restanti è contenuto, in parte, nelle istruzioni macchina e, per la parte restante, dipende dal sistema operativo (se c'è) o da particolari caratteristiche del processore stesso.

Per poter essere eseguito, un programma deve essere caricato all'interno della memoria accessibile al processore.

- Nei sistemi più piccoli (es., Arduino) questo avviene grazie ad hardware specifico che memorizza in modo semi-permanente quanto necessario nella memoria flash.
- Nei sistemi più grandi, un modulo del sistema operativo (**loader**) si occupa di trasferire dal disco alla memoria RAM il contenuto del file eseguibile

Una volta che il programma è presente in memoria, può essere eseguito. Il processore preleva dalla memoria, una alla volta, le istruzioni macchina del programma e provvede ad eseguirle.

L'esecuzione di un'istruzione determina quali effetti collaterali debbano avvenire e quale debba essere la prossima istruzione da eseguire. Modifiche nelle informazioni contenute nel processore stesso e/o nella memoria. Eventuali interazioni con altre periferiche collegate al processore stesso (I/O).

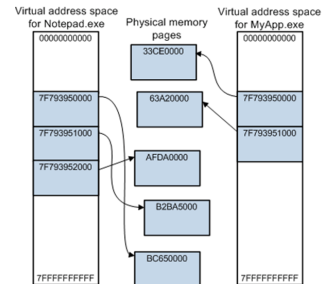
Il modello base di esecuzione prevede l'alternarsi continuo di tre fasi:

- Fetch - Un'istruzione viene trasferita dalla memoria all'interno del processore

- Decode - L'istruzione prelevata viene trasformata in comandi da eseguire
- Execute - I comandi vengono eseguiti

1.4 Spazio di indirizzamento

L'esecuzione di un programma avviene nel suo **spazio di indirizzamento**. Sottinsieme delle celle indirizzabili, gestito dal sistema operativo. Dati, istruzioni, informazioni di controllo, ... sono memorizzati come valori al suo interno. Lo spazio di indirizzamento può essere immaginato come un enorme array di byte, che possono essere letti individualmente, a coppie, a gruppi di 4, 8 alla volta, in base al parallelismo del processore (8, 16, 32 o 64 bit). Il numero di celle **potenzialmente** presenti nello spazio di indirizzamento dipende dal processore. Il numero di celle **effettivamente** presenti, è limitato dalla quantità di RAM disponibile ed è controllato dal sistema operativo. Nel caso di CPU Intel x64, il processore opera a 64 bit. Ma lo spazio di indirizzamento si estende tra 0 e $2^{48} - 1$ (scelta di implementazione). All'interno di questo, però, solo una piccola parte di indirizzi è effettivamente presente.



Gli indirizzi che un programma utilizza non corrispondono - in generale - all'effettiva posizione che un dato assume in memoria

- Il codice fa riferimento ad **indirizzi virtuali**
- L'hardware utilizza **indirizzi fisici**

Un apposito blocco hardware presente nel processore (**MMU - Memory Management Unit**) si occupa di tradurre l'indirizzo virtuale in indirizzo fisico.

Tale traduzione garantisce che:

- Programmi diversi contemporaneamente in esecuzione non possano interferire tra loro
- Consente di controllare quali operazioni siano possibili (lettura, scrittura, esecuzione)
- Permette inoltre di scrivere programmi che manipolano dati più grossi della memoria fisica effettivamente presente

1.5 Effetti pratici

- Segmentation fault
 - Tentativo di accedere ad indirizzo non mappato o senza gli adeguati permessi
 - Il processore blocca l'esecuzione e rimanda al sistema operativo (che termina brutalmente il programma)
- Page fault
 - Tentativo di accedere ad un indirizzo appartenente ad un blocco il cui contenuto è stato salvato su disco
 - Il processore richiede l'intervento del sistema operativo che ricerca una pagina fisica libera e vi trasferisce il contenuto del blocco presente nel file di paginazione
 - Al termine il processore ritenta l'accesso e procede normalmente
- Trashing
 - Se il programma cerca di accedere rapidamente ad una grande quantità di dati paginati, il sistema passa più tempo a spostare pagine tra disco e memoria fisica di quanto ne possa dedicare all'esecuzione vera e propria

1.6 Gerarchia di memoria

Nei sistemi non elementari, il processore **non legge/scrive** direttamente dalla memoria RAM. Tra i due elementi viene interposta la memoria **cache**. Questa ha il compito di velocizzare l'accesso alle informazioni cercando di anticipare il bisogno di informazioni da parte della CPU.

Si basa sul **principio di località** dei programmi: se è stato appena fatto accesso all'indirizzo I , è molto probabile che venga fatto subito dopo accesso all'indirizzo $I + \Delta$ (con Δ molto piccolo).

La memoria cache si basa su dispositivi il cui tempo di accesso è molto più rapido di quello offerto dalla memoria RAM tradizionale, ma dotati di una capacità molto inferiore (pochi KB/MB).

Per questo $\text{Vec}<T>$ è quasi sempre meglio di $\text{LinkedList}<T>$

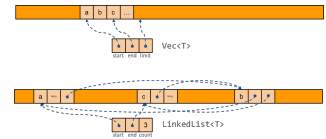


Figura 1.1: Località

1.7 Esecuzione e programmi di alto livello

Utilizzando linguaggi di programmazione ad alto livello, i dettagli legati al meccanismo di esecuzione sono normalmente invisibili.

Tuttavia, per poter garantire il livello di astrazione necessario, essi devono introdurre delle sovrastrutture che impattano sul codice che viene scritto dal programmatore... e introducono un insieme di **vincoli e restrizioni** di cui è necessario essere pienamente coscienti.

1.8 Modello di esecuzione di un programma

Ogni linguaggio di programmazione propone uno specifico modello di esecuzione, un insieme di comportamenti che attuati dall'elaboratore a fronte dei costrutti di alto livello del linguaggio.

Tale modello per lo più non corrisponde a quello di un dispositivo reale, come fa il processore ad eseguire solo Fetch, Decode ed Execute? Utilizzando strutture dati intermedie e da un insieme di run-time library. È compito del compilatore introdurre uno strato di adattamento che implementi il modello nei termini offerti dal dispositivo sottostante.

Questo strato è composto, da un lato, da una specifica modalità di traduzione dei costrutti di alto livello in istruzioni macchina e, dall'altro, da un insieme di funzionalità di supporto offerte sotto forma di libreria di supporto all'esecuzione (run-time library).

1.9 Livelli di astrazione

La compilazione trasforma il programma sorgente in un nuovo programma dotato di un modello di esecuzione ed un insieme di istruzioni più semplice. Che può essere eseguito da una macchina fisica (CPU) o subire un'ulteriore trasformazione (esecuzione virtuale)



1.10 Librerie di esecuzione

Offrono agli applicativi meccanismi di base per il loro funzionamento

- Supportano le astrazioni del linguaggio di programmazione
- Forniscono un'interfaccia uniforme tra i diversi S.O. per le funzioni ad essi demandate

Costituite da due tipi di funzioni

- Alcune, invisibili al programmatore, sono inserite in fase di compilazione per supportare l'esecuzione (controllo dello stack, copia di aree di memoria ...)
- Altre offrono al programmatore funzionalità standard, gestendo opportune strutture dati ausiliarie e/o richiedendo al S.O. quelle non altrimenti realizzabili (malloc o fopen)

1.11 Modello di esecuzione nei linguaggi C e C++

In C/C++ sono pensati come se fossero gli unici utilizzatori di un elaboratore, completamente dedicato loro. Le uniche interazioni si riducono al più all'uso di risorse persistenti come il file system o le connessioni di rete .

L'isolamento è reso possibile dal meccanismo di indirizzamento virtuale.

Un programma C/C++ assume di poter accedere a qualsiasi indirizzo di memoria presente nello spazio di indirizzamento, all'interno del quale può leggere o scrivere dati o dal quale può eseguire codice macchina. Ma sappiamo che invece lo spazio di indirizzamento è:

- Limitato
- Non contiguo
- Non tutto lo spazio consente ogni tipo di operazione

1.12 Esecuzione sequenziale sincrona, senza limitazioni

Un programma C/C++ è formato da un insieme di istruzioni eseguite, una per volta, nell'ordine indicato dal programmatore. Non ci sono limiti sul numero di istruzioni da eseguire, sul tempo richiesto alla loro esecuzione, né sulla memoria necessaria .

In realtà, tali limiti esistono e condizionano la scrittura dei programmi. Da qui nasce la necessità di un modello di gestione della memoria sicuro .

All'interno del flusso principale di esecuzione è possibile attivare flussi di elaborazione secondari, detti **thread**. Questi abilitano l'esecuzione concorrente (effettivamente parallela se la CPU è multicore) e introducono un ordine di grandezza nel livello di complessità della scrittura dei programmi. È il S.O. che si occupa di gestire tutti questi flussi di esecuzione indirizzandoli nei vari core fisici.

1.13 Flusso di esecuzione

Il programma è costituito, come minimo, da un flusso di esecuzione il cui punto di partenza è predefinito:

- La funzione `main()` nel linguaggio C
- I costruttori delle variabili globali in C++, seguiti dalla chiamata alla funzione `main()`

Vi sono inoltre due strutture dati:

- Lo stack che permette di gestire chiamate annidate tra funzioni. Essa **supporta la ricorsione** e l'uso di **variabili locali**. In C++, supporta anche la gestione strutturata delle eccezioni
- Lo heap offre supporto per gestire dinamicamente l'utilizzo della memoria. Secondo logiche non strettamente correlate al procedere dell'esecuzione

1.14 Stack

Blocco di memoria allocato automaticamente all'avvio di un programma, circa 1 MB, al cui interno sarà possibile salvare:

- Variabili locali e temporanee
- Gli argomenti passati ad ogni invocazione di funzione
- Valori di ritorno
- Traccia la storia delle chiamate a funzione

Viene utilizzato a partire da un estremo, espandendosi verso l'estremo opposto ogni volta che avviene una chiamata e contraendosi verso l'inizio quando una funzione ritorna.

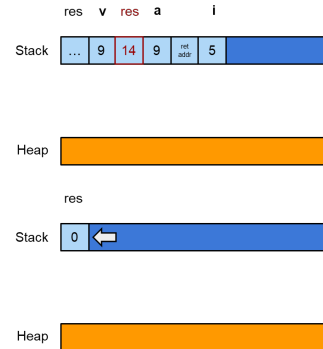
Poiché ha dimensione finita, limita la profondità massima di ricorsione. Così come la dimensione totale delle variabili locali complessivamente utilizzabili.

Se, a seguito di un numero troppo elevato di chiamate o all'allocazione di variabili troppo grandi, si espande oltre il proprio limite, il sistema operativo (o il processore) interviene terminando l'esecuzione del programma generando un Stack overflow.

Nota: Stack = automatico, veloce, tempo di vita deterministico

```
int f(int a) {
    int i = 5;
    if (a>0) {
        return a+i;
    } else {
        return a-1;
    }
}

int main() {
    int v = 9;
    v = f(v);
    return 0;
}
```



L'allocazione ed il rilascio sullo stack sono estremamente efficienti, grazie alla particolare politica di espansione / contrazione adottata.

Per contro, la durata dei valori memorizzati al suo interno è strettamente correlata alla durata dell'invocazione di una funzione. Non è possibile memorizzare nello stack un dato che duri più a lungo della funzione in cui è stato allocato. Per questo motivo, il chiamante di una funzione ha la responsabilità di pre-allocare lo spazio in cui dovrà essere memorizzato il risultato prodotto dalla funzione chiamata.

Inoltre, poiché lo spazio complessivo dello stack è definito a priori, non è possibile memorizzare un dato la cui dimensione superi tale spazio. In generale, occorre limitare la dimensione massima del dato allocato ad una grandezza compatibile con il livello di profondità di chiamate e la dimensione degli altri dati che devono essere memorizzati complessivamente nello stack.

1.15 Heap

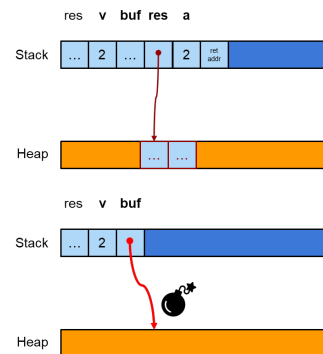
Tutte le volte in cui un dato ha un ciclo di vita **non collegato al tempo di esecuzione** della funzione in cui è stato creato o la cui dimensione è **cospicua** oppure **non nota** in fase di compilazione, occorre richiederne la memorizzazione in una struttura a parte. I linguaggi C e C++ offrono un'apposita area, chiamata Heap o Free Store, all'interno della quale è possibile allocare e rilasciare blocchi di dimensione arbitraria (entro il limite della memoria disponibile).

A differenza di quanto avviene per lo stack, le aree allocate sullo heap non sono associate, a livello di linguaggio sorgente, ad un nome di variabile bensì si accede al loro contenuto esclusivamente tramite **puntatori**. Utilizzando l'operatore di dereferenziazione, *, posso accedere al contenuto memorizzato in memoria.

Questo spazio concesso dal S.O. è un prestito, dunque è responsabilità del programmatore rilasciare, prima o poi, la memoria allocata. Altrimenti si verificherà una perdita di memoria (memory leak) che, se reiterata, porta il sistema operativo a distruggere il processo.

```
int* f(int a) {
    if (a>0) {
        return new int[a];
    } else {
        return nullptr;
    }
}

int main() {
    int v = 2;
    int* buf = f(v);    //process buf
    delete[] buf;
    return 0;
}
```



Il puntatore è probabile che sia buono, ma non lo è, non gli è stato assegnato null, il linguaggio non te lo impone.

1.16 Organizzazione dello spazio di indirizzamento

- Codice eseguibile
 - Contiene le istruzioni in codice macchina
 - Accesso in lettura ed esecuzione
- Costanti
 - Accesso in sola lettura
- Variabili globali
 - Accesso lettura/scrittura
- Stack
 - Contiene indirizzi e valori di ritorno, parametri e variabili locali
 - Accesso lettura/scrittura
- Heap
 - Insieme di blocchi di memoria disponibili per l'allocazione dinamica
 - Gestiti tramite funzioni di libreria (`malloc`, `new`, `free`, ...) che li frammentano e ricompattano in base alle richieste del programma

Nel sistema operativo Linux, è facile verificare come sia organizzato lo spazio di indirizzamento di un programma in esecuzione (processo). Ogni processo è identificato univocamente da un numero intero (PID) e con il comando `ps -ef`, il sistema operativo crea, per ciascun processo attivo, un file virtuale denominato `/proc/<pid>/maps` che descrive lo spazio di indirizzamento con informazioni sui diversi segmenti al suo interno.

1.17 Ciclo di vita delle variabili

Il modello di esecuzione dei linguaggi C/C++ distingue diverse tipologie di variabili

Globali locali dinamiche

Ciascuna classe ha un proprio ciclo di vita: Intervallo di tempo in cui è garantito l'accesso alle informazioni contenute al loro interno.

1.18 Variabili globali e locali

- Le variabili globali hanno un indirizzo fisso, determinato dal compilatore e dal linker. Sono accessibili in ogni momento e all'avvio del programma, contengono l'eventuale **valore di inizializzazione**.
- Le variabili locali hanno un indirizzo relativo alla cima dello stack. Il ciclo di vita coincidente con quello della funzione/blocco in cui sono dichiarate. Il valore **iniziale è casuale**.

1.19 Variabili dinamiche

Le variabili dinamiche hanno un indirizzo assoluto, determinato in fase di esecuzione. Sono accessibili solo tramite puntatori, è il programmatore a controllarne il ciclo di vita e il valore iniziale può essere inizializzato o meno.

L'uso di questo tipo di variabili presuppone un'infrastruttura di supporto che offra meccanismi di allocazione e di rilascio fornita dalla libreria di esecuzione e dal S.O.

1.20 Allocazione della memoria

La libreria standard C offre vari meccanismi per ottenere l'indirizzo di un blocco allocato dinamicamente

```
void *malloc(size_t s)
void *calloc(int n, size_t s) //per gli array
void *realloc(void* p, size_t s) //rialloca aggiungendo memoria
```

In caso di fallimento, viene restituito NULL, tranne nella *realloc(...)* dove il blocco originale non viene toccato e il puntatore *p* resta valido.

In C++ viene definito il costrutto: `new NomeTipo{argomenti...}`

1.21 Rilascio della memoria

Opportune funzioni di libreria permettono di restituire i blocchi precedentemente allocati. Poiché ogni funzione di allocazione mantiene le proprie strutture dati, occorre che un blocco sia rilasciato dalla funzione duale di quella con cui è stato allocato:

- `malloc(...)` / `free(...)`
- `new` / `delete`
- `new ...[num\_elements]` / `delete[ ]`

Se il blocco non viene rilasciato si crea una perdita di memoria, che, a lungo andare, provoca l'esaurimento dello spazio di indirizzamento.

Se il blocco viene rilasciato più volte o viene rilasciato con la funzione sbagliata si corrompono le strutture dati degli allocatori, con conseguenze imprevedibili.

1.22 Puntatori

Le variabili sullo heap possono essere gestite solo tramite puntatori perché la loro posizione non è nota a priori.

Vi è un problema, potentissimi per implementare di una vasta gamma di algoritmi come liste, grafo, etc., ma sono estremamente pericolosi. I puntatori sono intrinsecamente ambigui implica che il programmatore gestisca esplicitamente l'allocazione ed il rilascio dei blocchi al loro interno e ne manipoli il contenuto attraverso l'uso dei puntatori.

Il loro utilizzo espone infatti il programmatore ad una vasta gamma di errori, le cui conseguenze portano - nella maggior parte dei casi - a **comportamenti non definiti** (undefined behavior), le cui conseguenze sono disastrose.

1.23 Il costo del rilascio

I linguaggi gestiti, come Java, JavaScript, Python, C#, etc. gestiscono automaticamente il rilascio dei blocchi di memoria utilizzando il Garbage Collector. Per farlo, tengono traccia di chi punta a chi altro. Se, a partire dalle variabili presenti nello stack, un blocco non risulta più raggiungibile, questo può essere liberato.

Per poter applicare questo algoritmo occorre "fermare il mondo". Ovvero **sospendere temporaneamente l'esecuzione del programma intero** per poter osservare una situazione statica e procedere con la compattazione dello heap. Questo può comportare lo spostamento di blocchi e il cambiamento del valore contenuto nei rispettivi puntatori.

L'operazione di rilascio può comportare pause non deterministiche. Talvolta incompatibili con contesti di esecuzione in tempo reale.

1.24 Puntatori in C/C++

Permettono l'accesso diretto ad un blocco di memoria, tale blocco può appartenere ad altri oggetti:

```
int A=10;
int* pA = &A;
```

o essere allocato allo scopo `int* pB = new int{24}`

Possono essere esplicitamente resi invalidi:

- Il valore `0`
- La macro `NULL` $\Rightarrow ((void *)0)$
- La parola-chiave `nullptr` (C++11 e superiori)

Quando si usano i puntatori, occorre stabilire quali responsabilità/permessi sono associati al loro uso.

1.25 Le ambiguità dei puntatori in C/C++

- Contiene un indirizzo valido?
- Quanto è grosso il blocco puntato?
- Fino a quando è garantito l'accesso?
- Se ne può modificare il contenuto?
- Occorre rilasciarlo?
- Lo si può rilasciare o altri conoscono lo stesso indirizzo?
- Viene usato come modo per esprimere l'opzionalità del dato?

Nota: In C/C++ il programmatore è responsabile; in Rust la responsabilità è codificata nel sistema dei tipi

1.26 Problemi legati ai puntatori

In C, la gestione della memoria è totalmente affidata al programmatore

- Non ci sono supporti sintattici per automatizzare il ciclo di vita del blocco puntato
- Né meccanismi per associare una specifica semantica al puntatore

In C++ sono state introdotte nuove astrazioni (riferimenti, smart pointer) che sollevano il programmatore da alcune responsabilità, facilitando la creazione di programmi più robusti. Ma richiedono in ogni caso una comprensione approfondita del problema e l'uso attento e coerente dei puntatori.

1.27 Responsabilità del programmatore

- Limitare gli accessi ad un blocco
 - Nello spazio: non accedere alle locazioni che lo precedono o che lo seguono
 - Nel tempo: non accedere al blocco al di fuori del suo ciclo di vita
- Non assegnare a puntatori valori che corrispondono ad indirizzi non mappati
 - Possibile nel caso di cast o di assegnazione improprie
- Rilasciare tutta la memoria dinamica allocata
 - Una e una sola volta, usando la funzione duale di quella usata per la sua allocazione

1.28 Rischi

Accedere ad un indirizzo quando il corrispondente ciclo di vita è terminato, ha **effetti imprevedibili**:

- Dangling pointer
- La memoria indirizzata può essere inutilizzata, in uso ad altre parti del programma o non mappata

Non rilasciare la memoria non più in uso, **spreca risorse** del sistema

- Memory leakage
- Se si continua ad allocare senza mai rilasciare, si può saturare lo spazio di indirizzamento

Rilasciare la memoria più volte **corrompe le strutture** usate dallo heap, double free.

Se si assegna ad un puntatore un indirizzo non mappato nello spazio di indirizzamento e si usa quel puntatore, viene generata una interruzione del processore. Il S.O. intercetta tale interruzione e termina il processo.

Se non si inizializza un puntatore e lo si usa, *Wild pointer*, il suo contenuto potrebbe puntare ovunque. Può causare una violazione di accesso oppure corrompere un'area di memoria in uso ad altre parti del programma.

1.29 Problemi dei puntatori in C/C++

Problema	Conseguenza
dangling / wild pointer	crash / UB
double free	corruzione heap
leak	esaurimento della memoria
data race	risultati casuali

1.30 Le risposte di Rust

Problema C / C++	Strumento Rust
dangling / wild pointer	borrow checker + tempo di vita
double free	gestione del possesso
leak	pattern RAI
nullptr	tipo Option<T>
gestione del rilascio	tipi Box<T> / RC<T> / Arc<T>

1.31 Gestire i puntatori

Chi alloca un blocco di memoria è **responsabile** di mettere in atto un meccanismo che ne garantisca il successivo **rilascio**. Viene detto "possessore" del blocco.

Cosa capita se si effettua una copia di un puntatore?

Il secondo puntatore si trova coinvolto, suo malgrado, nel ciclo di vita del dato. Occorre introdurre un meccanismo per gestire efficacemente la semantica di un puntatore.

1.32 Possesso della memoria

Il vincolo di rilascio risulta problematico per via dell'**ambiguità** del concetto di puntatore all'interno del linguaggio. Dato un indirizzo non nullo, non è possibile distinguere se sia **valido o meno**, né **a quale area appartenga**, né se occorra o meno **rilasciarlo**.

Ogni volta in cui si alloca un blocco dinamico e se ne salva l'indirizzo in una variabile puntatore, quella variabile diventa proprietaria del blocco: ha la responsabilità di liberarlo.

Non tutti i puntatori posseggono il blocco a cui puntano. Se ad un puntatore viene assegnato l'indirizzo di un'altra variabile, la proprietà è della libreria di esecuzione.

Il problema si complica se un puntatore che possiede il proprio blocco viene copiato. Quale delle due copie è responsabile del rilascio?

1.33 Dipendenze

Quando si introducono strutture dati complesse, è comune che tali strutture debbano appoggiarsi a blocchi di memoria ausiliari in cui memorizzare le informazioni che esse gestiscono.

Un oggetto di tipo `std::vector<T>`, in C++, incapsula un array dinamico di dati di tipo generico T: tali dati possono essere aggiunti, rimossi, sostituiti, ... tramite opportuni metodi sull'oggetto stesso. Tali blocchi ausiliari sono di proprietà dell'oggetto: vengono gestiti dai suoi metodi e devono essere rilasciati quando l'oggetto viene distrutto.

Analogamente, altre strutture dati possono mantenere riferimenti a oggetti del sistema operativo (handle a file, socket, timer, ...). Anche in questo caso le risorse corrispondenti risultano di proprietà dell'oggetto che le gestisce.

Collettivamente, questi tipi di dato prendono il nome di **dipendenze**.

Il linguaggio C non offre nessun supporto sintattico per la gestione delle dipendenze e lascia al programmatore la completa responsabilità della loro gestione.

Al contrario, il linguaggio C++ offre, per gli oggetti di tipo *class*, *struct* e *union* (!), la possibilità di specificare particolari funzioni membro dette rispettivamente costruttori e distruttori.

- Un costruttore è una funzione membro che ha il compito di inizializzare un oggetto, provvedendo ad acquisire - se necessario - le sue dipendenze
- Il distruttore è una funzione membro che viene invocata automaticamente dal compilatore quando l'oggetto raggiunge la fine della propria esistenza ed ha il compito di rilasciare correttamente le dipendenze possedute dall'oggetto stesso

1.34 Ma non basta scrivere un programma "giusto"?

Sì, basterebbe se fosse possibile essere certi che lo è! In realtà le cose sono molto più complesse.

Raramente i programmi sono scritti da una singola persona. Per lo più si appoggiano a librerie scritte da altri che hanno fatto opportune assunzioni su come debbano essere utilizzate, ma non è detto che tali assunzioni siano rispettate né, talvolta, conosciute. Spesso si lavora in gruppo e la comunicazione è imperfetta.

Al crescere della dimensione del programma, la quantità di particolari a cui occorre badare cresce molto più in fretta. Ed è estremamente probabile dimenticarsi delle proprie assunzioni.

1.35 Perché questo problema non si verifica in altri linguaggi?

Nella maggior parte dei linguaggi di alto livello il problema della gestione della memoria non si pone, pur offrendo, a vario titolo, il concetto di puntatore.

Java, C#, Python, JavaScript, ..., basano il proprio meccanismo di rilascio su un algoritmo di Garbage Collection. Libera il programmatore dalla responsabilità di rilasciare esplicitamente la memoria dinamica allocata, in cambio della perdita di controllo su **quando** e **come** tale rilascio avvenga.

La maggior parte di tali linguaggi basa la propria strategia di gestione della memoria su una combinazione dei seguenti algoritmi:

- Reference counting

- Mark and Sweep

1.36 Confronto

Gestione manuale in C/C++	Gestione automatica (Java/C#/Python/Js)
Il rilascio avviene invocando la funzione free() o l'operatore delete	Il rilascio è eseguito automaticamente dal garbage collector
Il momento in cui avviene il rilascio è controllato dal programmatore	Il momento in cui avviene il rilascio è controllato dal garbage collector, che viene eseguito periodicamente
In C++, gli oggetti dispongono di un distruttore che gestisce il rilascio esplicito delle dipendenze	Il concetto di distruttore non è parte del linguaggio (ecc. Java, con il metodo <i>finalize()</i> ora deprecato)
Il rilascio non comporta tempi supplementari di attesa, che sarebbero incompatibili con i sistemi in tempo reale	Quando si attiva la garbage collection, il programma si arresta fino al termine dell'algoritmo, mettendo a rischio il funzionamento di un sistema in tempo reale
Si possono verificare memory leak, doppi rilasci, dangling pointer, wild pointer	Non possono verificarsi errori sulla gestione della memoria

Capitolo 2

Possesso

2.1 Possesso

In Rust ogni valore introdotto nel programma è **posseduto** da una ed una sola variabile e il suo valore salvato in memoria.

- Il **Borrow Checker** verifica formalmente questo fatto per ogni punto dell'esecuzione del programma
- Ogni violazione porta ad un **fallimento** della compilazione del programma con conseguente errore e spiegazione di quest'ultimo.

Possedere un valore significa essere responsabile del suo **rilascio**.

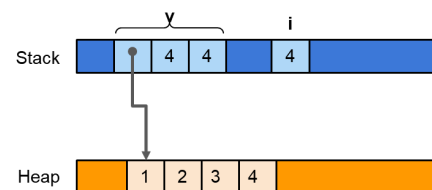
In particolare se il valore contiene una risorsa quale un puntatore a memoria dinamica, handle di file o socket, etc, questa deve essere liberata. Una volta liberata deve essere restituita al proprietario cui molto spesso è il Sistema Operativo.

Il rilascio avviene quando la variabile che possiede esce dal proprio scope sintattico o quando viene **assegnata** ad un nuovo valore.

Rust offre un meccanismo di drop mediante il quale è possibile associare azioni arbitrarie da eseguire prima che sia liberata la memoria. Il rilascio può essere rimandato ad un secondo momento se il contenuto della variabile viene trasferito (mosso) in un'altra variabile, la quale diventa responsabile del suo rilascio.

Nota: Questo meccanismo rende l'utilizzo del Garbage collector, presente in linguaggi di programmazione come Java o Python, non necessario.

```
fn main() {  
    let mut v = Vec::with_capacity(4);  
    // v possiede il vec  
  
    for i in 1..=5 {  
        v.push(i);  
    }  
  
    println!("{:?}", v);  
}
```



Inserendo valori all'interno del vettore, lo spazio precedentemente allocato viene via via riempito. Se necessario il blocco viene riallocato per fare spazio ad un maggior numero di elementi.

Finché le variabili sono in scope, *i* e *v*, le risorse che posseggono sono accessibili. Una volta usciti dal proprio scope la variabile si occupa dello smaltimento della porzione di stack, ed eventualmente heap, precedentemente allocato seguendo le informazioni contenute in Drop.

2.2 Movimento

Quando una variabile viene inizializzata, essa prende il possesso del relativo valore. Se ad una variabile mutabile viene assegnato un nuovo valore, quello precedentemente posseduto viene rilasciato e la variabile diventa proprietaria del nuovo valore.

```
let mut b = a      f(a)      return a
```

In questi tre casi, assegnazione, passaggio ad una funzione e ritorno di una variabile, in Rust, succede qualcosa che in altri linguaggi di programmazione non succede: il valore viene **mosso**, il possesso viene passato dalla variabile originale alla destinazione.

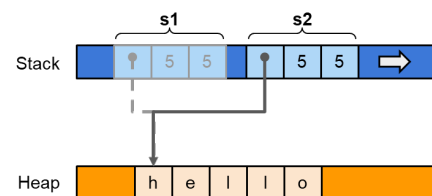
Questo porta ad alcune particolarità:

- La variabile originale cessa di possedere il valore, ovvero non è più responsabile della sua distruzione ed il possesso passa alla variabile di destinazione, o parametro della funzione invocata
- La variabile originale rimane allocata fino a quando non termina la sua visibilità, ovvero al chiusura del blocco in cui è stata definita
- Accessi in **lettura** porteranno ad **errori di compilazione**
- Accessi in **scrittura** alla variabile originale avranno successo e ne riabiliteranno la lettura (si crea un nuovo oggetto)
- La variabile destinazione conterrà una copia bit a bit del valore originale, ammesso che il compilatore non riesca a riusare i dati originali al loro posto

```
let mut s1 = "hello".to_string();
println!("s1: {}", s1);

let s2 = s1;
println!("s2: {}", s2);

//s1 non è più accessibile
...
s1 = "world".to_string();  \s1 ritorna
↪ accessibile
```



Il movimento in Rust è la soluzione al problema del doppio possesso di parti di aree di memoria. Fisicamente nessuno ha cancellato l'area di memoria ma il compilatore marca *s1* come inaccessibile.

Il movimento previene l'ambiguità che introduce C/C++: un puntatore è valido/non valido, dobbiamo rilasciare noi la memoria o ci pensa qualcun altro? In Rust è solo il possessore, univoco, della variabile ad occuparsene, i precedenti possessori altro non fanno che contrarre lo stack per liberare lo spazio occupato.

```
let mut s1 = "hello".to_string();
let mut s2 = s1 \s2 possiede "hello"
...
let mut s1 ... \Questo s1 nasconde il precedente s1
```

2.3 Copia

Alcuni tipo, tra cui quelli numerici, sono definiti **copiabili**.

Questa proprietà è definita attraverso il tratto **Copy**. Quando una valore assegnato ad un'altra variabile o usato come argomento in una chiamata a funzione, il valore originale rimane accessibile in **sola lettura**.

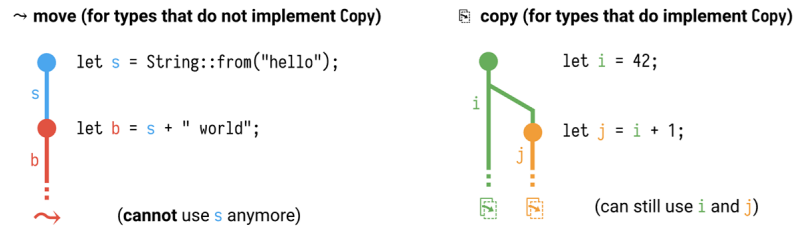
Questo è possibile quando il valore contenuto non costituisce una risorsa che richiede altre azioni di rilascio come ad esempio possono essere File o Socket.

Se *Vec* e *String* posseggono il tratto *Drop*, etichetta per lo smaltimento specifico dato che sono puntatori a zone di heap, *Int*, *Bool*, etc. assumono il tratto *Copy*.

Nota: Drop e Copy sono mutualmente esclusivi. Posso esistere anche tipi né Copy né Drop come per esempio gli oggetti Thread.

I tipo semplici e le loro combinazioni, tuple e array di numeri, sono copiabili. Sono copiabili anche i riferimenti a valori non mutabili.

Viceversa i riferimenti a valori mutabili NON sono copiabili.

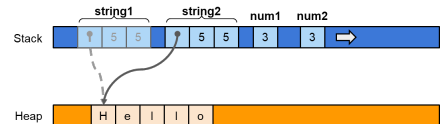


Nota: Dal punto di vista del codice generato, non cambia nulla tra copia e movimento. L'istruzione di assegnazione o passaggio come argomento comporta la duplicazione, bit a bit, del valore originale. In caso di copia il Borrow Checker non impedisce l'ulteriore accesso in sola lettura al dato originale.

2.3.1 Copia e Movimento

Copia e Movimento diventano esattamente la medesima istruzione assembly *mov*, quello che cambia sono le tabelle e le decisioni prese dal compilatore.

```
let string1 = "Hello".to_string();
let string2 = string1;
//da qui in poi, string1 è inaccessibile in lettura
//a meno che non venga riassegnato
let num1: i32 = 3;
let num2 = num1; //nessun vincolo su num1!
```



2.4 Clonazione

Alcune volte non vogliamo cedere il possesso ma solamente avere una copia e se un tipo implementa il tratto *Clone* può essere duplicato invocando il metodo *clone()*.

A differenza di copia e movimento, la clonazione può implementare una **copia profonda** dei valori. Questo implica un elevato costo di clonazione se il tipo risulta occupare molta memoria.

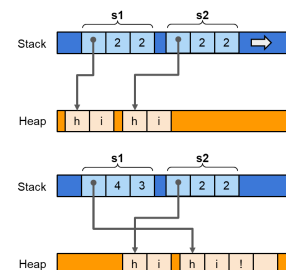
L'implementazione dell'operazione di clonazione, a differenza di copia e movimento che sono sotto il controllo esclusivo del compilatore ed è basata sull'invocazione della funzione *memcpy(...)*, è modificabile dal programmatore.

Nota: Affinché un tipo possa implementare il tratto Copy, occorre che implementi anche il tratto Clone. Tuttavia, non occorre che le due implementazioni coincidano.

```
let mut s1 = "hi".to_string();
let s2 = s1.clone();

s1.push('!');

println!("s1: {}", s1); //hi!
println!("s2: {}", s2); //hi
```



2.5 Box<T> - Puntatore con possesso

Una variabile locale è SEMPRE allocata sullo stack. Questo si estende di una quantità pari alla dimensione del valore che deve essere memorizzato nella variabile. Quando la variabile esce dal proprio scope sintattico, lo stack si contrae ed il valore viene rilasciato.

In alcune situazioni:

- Non è nota la dimensione a priori
- Occorre prolungare il tempo di vita del valore oltre al blocco sintattico in cui è definito
- La dimensione è maggiore dello stack stesso

l'oggetto deve essere allocato sullo heap, utilizzando il tipo generico **Box<T>**. Una variabile di questo tipo contiene un puntatore al valore.

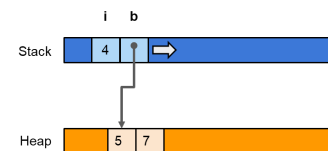
Per allocare un valore con il tipo Box si scrive:

```
let b = Box::new(v);
```

dove v è un qualsiasi valore. L'istruzione precedente definisce una variabile b la quale è un puntatore ad un area di memoria sullo heap contenente il valore v .

Si accede al valore contenuto con l'espressione $*b^1$, e se la variabile è definita come mutabile è possibile modificare il contenuto a cui si punta.

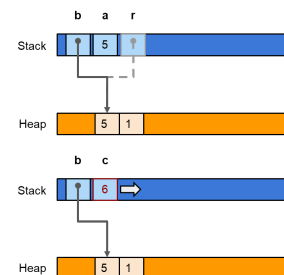
```
fn useBox() {
    let i = 4;
    let mut b = Box::new( (5, 2) );
    (*b).1 = 7;
    println!("{:?}", *b); // (5,7)
}
```



Quando l'esecuzione del programma raggiungerà la fine del blocco di codice in cui la variabile b è stata definita, il blocco verrà rilasciato a patto che il contenuto di b , il puntatore al blocco sia stato **mosso** in un'altra variabile.

```
fn makeBox(a: i32) -> Box<i32,i32> {
    let r = Box::new( (a, 1) );
    return r;
}

fn main() {
    let b = makeBox(5);
    let c = b.0 + b.1;
}
```



2.6 Riferimenti

Per risolvere i problemi di accesso, Rust introduce diverse forme di puntatori, volte a rendere espliciti responsabilità e diritti di chi le maneggia. Il borrow checker si occupa di garantire che il codice sia scritto in conformità con tali regole e impedisce la compilazione in caso contrario.

Un **riferimento** è un puntatore in sola lettura ad un blocco di memoria **posseduto da un'altra variabile**.

- Permette di accedere ad un valore senza trasferirne la proprietà
- Il riferimento può esistere solo mentre esiste la variabile che possiede il dato
- non serve utilizzare $*$, quando si accede al valore tramite l'operatore $.$

```
let point = (1.0, 0.0); //point possiede il valore
let reference = &point; //reference PUÒ accedere al valore in lettura
//finché esiste point
println!("{}, {}", reference.0, reference.1);

point = (2.0, 3.0); //tentativi di accesso a reference generano errore
```

¹Si può scrivere pure $b.1$, il compilatore lo de-referenzia in autonomia $(*b).1$

2.6.1 Riferimenti e prestiti

Un riferimento prende in prestito, borrows, in sola lettura, l'indirizzo di memoria in cui esiste il valore.

Fino a che il riferimento è accessibile, non è possibile modificare il valore, né tramite il riferimento, il quale è in sola lettura, né tramite la variabile che possiede il valore.

È possibile creare ulteriori riferimenti a partire dal dato originale o da altri riferimenti ad esso.

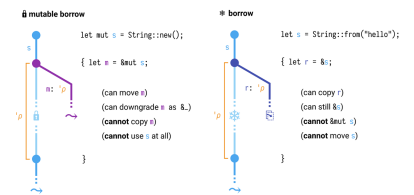
- I riferimenti sono **copiabili**: viene duplicato il puntatore
- Il compilatore, attraverso il BC, garantisce che un riferimento punti sempre ad un dato valido
- Finché esiste almeno un riferimento, il dato originale **non potrà essere né modificato né distrutto**

2.6.2 Riferimenti mut - mutual exclusion

A partire da una variabile che possiede un valore è possibile estrarre **un solo riferimento mutabile** per volta. `let r = &mut v;`

Mentre esiste un riferimento mutabile

- Non possono esistere riferimenti semplici, in sola lettura
- Non è possibile modificare né muovere la variabile che possiede il valore



È possibile creare un riferimento mutabile solo se la variabile che possiede il dato è a sua volta dichiarata come mutabile.

2.6.3 Riferimenti: disposizione in memoria

I riferimenti sono implementati diversamente in base al tipo puntato

- Puntatori semplici, se il compilatore conosce la dimensione del dato puntato
- Puntatore + dimensione (fat pointer), se la dimensione del dato è solo nota in fase di esecuzione. Nelle stringhe tolgo il problema di inserire il terminatore.
- Puntatore doppio, se il tipo di dato puntato è noto solo per l'insieme di tratti che implementa. Esempi File o Socket.

```
let i: i32 = 10;
let r1: &i32 = &i;
```

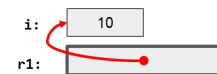


Figura 2.1: Single pointer

```
let a: [i32;3] = [1,2,3];
let r2: &[i32] = &a;
```

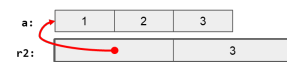


Figura 2.2: Fat pointer

```
let f: File = File::create("test.txt");
let r3: &dyn Write = &f;
```

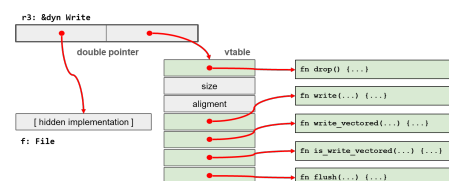


Figura 2.3: Double pointer

2.7 Tempo di vita dei riferimenti

Il BC garantisce che tutti gli accessi ad un riferimento avvengano solo in un intervallo di tempo, o righe, compreso in quello in cui il dato esiste. Serve ad impedire il fenomeno dei **dangling pointer**.

L'insieme delle righe in cui si fa accesso al riferimento costituisce il suo **lifetime**. Tale info è mantenuta dal compilatore assieme ad altre informazioni, quale il tipo, e non ha alcuna rappresentazione in fase di esecuzione.

Sebbene il tempo di vita in molte situazioni possa essere omesso, il tempo di vita di un riferimento può essere espresso nella firma del tipo. Questo aiuta sia il programmatore sia il compilatore per il mantenimento dell'informazione.

- `&'a NomeTipo`, dove *a* è un identificativo qualsiasi e il simbolo `'` serve a quantificarlo come tempo di vita
- Tale notazione è utile in quelle situazioni in cui occorre imporre vincoli sulla durata relativa dei tempo di vita di due o più riferimenti

Se un riferimento deve durare l'intera durata del programma viene indicato con `&'static NomeTipo`.

Nota: Una stringa espressa in formato letterale (`"Some string"`) ha come tipo `&'static str`, in quanto la sequenza di caratteri viene allocata dal compilatore nella sezione delle costanti e non viene mai rilasciata.

Per essere lecito, il tempo di vita di un riferimento deve essere contenuto, minore, nel tempo di vita del valore a cui punta. Sempre il BC si fa carico di garantire il vincolo. Questo richiede di esplicitare il tempo di vita quando si memorizza un riferimento all'interno di una struttura dati, come per esempio una tupla, o si usa un riferimento come valore di ritorno di una funzione.

Il vincolo (apparentemente ovvio) di esistenza in vita dei vincoli dovrebbe essere alla base anche di tutti gli usi dei puntatori in C e C++.

Poiché, però, in questi linguaggi non viene tracciato il tempo di vita, il compilatore non è in grado di identificare la presenza di dangling pointers, né identificare eventuali problemi di non rilascio o doppio rilascio, come invece avviene in Rust.

```
{
    let r;
    {
        let x = 1;
        r = &x;
    }
    assert_eq!(*r, 1);
}

//error: `x` does not live long enough
```

2.7.1 Esistenza in vita

Le regole, ovviamente, valgono anche quando si crea un riferimento ad una parte di una struttura dati più grande.

L'esistenza in vita del riferimento deve essere inclusa in quella della struttura a cui punta

```
let v = vec![1, 2, 3];
let r = &v[1]; // v deve durare più a lungo di r
```

Se, al contrario si memorizzano dei riferimenti in una struttura, tutti questi riferimenti devono avere una durata di vita maggiore della struttura dati in cui sono memorizzati.

```
{
    let mut v = Vec::new();
    {
        let a = 1;
```

```

    v.push(&a);
}
println!("{:?}", v);
}

```

\\error[E0597]: `a` does not live long enough

2.8 Riferimenti e funzioni

Se una funzione riceve un parametro di tipo riferimento (mutabile o meno), il tempo di vita del riferimento diventa parte integrante della firma della funzione. È necessario infatti che tutte le operazioni eseguite dalla funzione sul riferimento siano compatibili con la validità del dato memorizzato al suo interno.

$$\text{fn f(p: \&i32) \{...\} \Rightarrow fn f<'a>(p: \&'a i32) \{...\}}$$

Il compilatore provvede in molti casi a effettuare autonomamente la riscrittura indicata (lifetime elision).

Se la funzione riceve più riferimenti, può essere necessario indicare se il loro tempo di vita sia vincolato al più breve o se siano disgiunti.

- Nel primo caso si usa un solo identificatore per identificare l'intersezione dei loro tempi di vita

```
fn f<'a>(p1: \&'a i32, p2:\&'a i32) \{...\}
```

- Nel secondo si usano etichette diverse `fn f<'a, 'b>(p1: \&'a i32, p2:\&'b i32) \{...\}`

Uno dei casi più frequenti in cui il compilatore non riesce a dedurre il corretto tempo di vita è legato a funzioni che restituiscono un riferimento, estratto da una delle strutture dati ricevute in ingresso.

In tale caso occorre annotare il tipo restituito con la corretta etichetta

```
fn f<'a, 'b>(p1: \&'a Foo, p2:\&'b Bar) -> \&'b i32 \{ /*...*/ return \&p2.y; \}
```

Se la funzione memorizza il riferimento ricevuto in ingresso in una struttura dati, il compilatore deduce che il tempo di vita della struttura in cui il riferimento è memorizzato deve essere incluso o coincidente con il tempo di vita del riferimento.

Se questo non avviene, il compilatore identifica l'errore e impedisce alla compilazione di avere successo.

```

fn f(s: \&str, v: \&mut Vec<\&str>) \{
    v.push(s); //error: lifetime mismatch
\}

fn f<'a>(s: \&'a str, v: \&'a mut Vec<\&str>) \{
    v.push(s);
\}
//error: explicit lifetime required in the type of `v`

fn f<'a>(s: \&'a str, v: \&'a mut Vec<\&'a str>) \{
    v.push(s);
\}

fn main() \{
    let mut v: Vec<\&str> = Vec::new();
    \{
        let s= String::from("abc");
        v.push(\&s); //error: `s` does not live long enough. Il tempo di vita di v è limitata
        ↪ alla vita di vita di s.
    \}
    println!("{:?}",v);
\}

```

Se una funzione ha due o più riferimenti in ingresso e un riferimento in uscita, il compilatore non è in grado di decidere (senza eseguire il codice) da quale parametro provenga il riferimento in uscita. L'inserimento degli identificatori nella firma della funzione risolve questa ambiguità.

```
fn f<'b, 'c>(x: &'b S, y: &'c S) -> &'c u8 {...}

let v1 = S(1);
let mut v2 = S(2);

let r = f(&v1, &v2);    // Sappiamo che r è basato su v2,
                        // che rimane nello stato di prestito
                        // mentre v1 viene rilasciato e può evolvere
v2 = v1; // Questa operazione crea un problema!
print_byte(r); // Qui finisce il prestito di r (e di v2)
```

Lo scopo degli identificatori relativi al ciclo di vita è duplice:

- Per il codice che invoca la funzione (chiamante), essi indicano su quale, tra gli indirizzi in ingresso, è basato il risultato in uscita
- Per il codice all'interno della funzione (chiamato), essi garantiscono che vengano restituiti solo indirizzi cui è lecito accedere per (almeno) il tempo di vita indicato

All'atto dell'invocazione, gli identificatori forniti dal programmatore (o inseriti automaticamente dal compilatore, quando possibile) sono legati all'effettivo intervallo minimo (espresso come insieme di linee di codice) nel quale il valore da cui il prestito è stato preso debba restare bloccato per non violare le assunzioni su cui la funzione è basata.

Tentativi di modificare il valore originale da cui il prestito è preso prima che il tempo di vita sia trascorso portano ad errori di compilazione che costituiscono la base della robustezza del sistema di possesso e prestito offerto da Rust

2.9 Riferimenti e strutture dati

Lo stesso tipo di ragionamento vale se il dato viene salvato all'interno di una qualsiasi struttura dati

- Rust verifica che il valore a cui si punta abbia un tempo di vita maggiore o uguale del tempo di vita della struttura dati
- Questo richiede di esplicitare il tempo di vita della struttura rispetto al tempo di vita dei riferimenti in essa contenuti

Se una struttura che contiene riferimenti è contenuta, a sua volta, in un'altra struttura, anche quest'ultima deve avere il tempo di vita specificato in modo esplicito.

```
struct User<'a> {
    id: u32,
    name: &'a str,
}

struct Data<'a> {
    user: User<'a>,
    password: String,
}
```

2.10 Elisione dei tempi di vita

Rust, per default, provvede ad assegnare, ad ogni riferimento presente in una struttura dati o tra i parametri di una funzione, un tempo di vita distinto. Tali tempi di vita si propagano al codice che fa uso di tale struttura dati e, in assenza di ambiguità, non richiede di esplicitare gli identificatori dei tempi di vita.

```
struct Point {
    x: i32,
    y: i32,
}

fun scale(r: i32, p: Point) -> i32 {
    r * (p.x * p.x + p.y * p.y)
}

struct Point {
    x: &'a i32,
    y: &'b i32,
}

fun scale<'a,'b,'c> (r: &'a i32, p: Point<'b, 'c>)
    ↪ -> i32 {
    r * (p.x * p.x + p.y * p.y)
}
```

Se una funzione restituisce un riferimento o un tipo che contiene - direttamente o indirettamente - un riferimento, occorre disambiguare il tempo di vita del valore ritornato

- Se c'è un solo parametro in ingresso dotato di tempo di vita, Rust assume che quello debba essere il tempo di vita del risultato
- Se sono presenti più parametri in ingresso con tempo di vita, tocca al programmatore esplicitare quale debba essere il tempo di vita da associare al risultato

Nel caso di metodi che accedono a self tramite un riferimento, Rust assume che il tempo di vita da associare al risultato sia quello del riferimento a self.

Questo parte dal presupposto che se un metodo di un oggetto restituisce un dato preso a prestito (borrow), questo sia stato preso dai dati posseduti dall'oggetto stesso.

2.11 Possesso - riassunto delle regole

- Ciascun valore ha un unico possessore (variabile o campo di una struttura). Il valore viene rilasciato (drop) quando il possessore esce dal proprio scope o quando al possessore viene assegnato un nuovo valore.
- Può esistere, al più, un singolo riferimento mutabile ad un dato valore
- **Oppure**, possono esistere molti riferimenti immutabili al medesimo valore. Ma fintanto che ne esiste almeno uno, il valore non può essere mutato. Principio del *multiple reader and single writer* presente in molti ambiti dell'informatica.
- Tutti i riferimenti, che sono pestiti del valore, devono avere una durata di vita inferiore a quella del valore a cui fanno riferimento

2.12 Puntatori in Rust

Proprietà	&T	Box<T>
Possesso	No: è un prestito	SI: Possessore unico, risolve il doppio rilascio del C/C++
Rilascio	A carico del possessore	Automatico al termine dello scope
Validità	Garantita dal compilatore, il borrow checker	Garantita in fase di esecuzione
Nullabilità	Mai nullo	Mai nullo
Aliasing e mutabilità	Controllata: o molti riferimenti in sola lettura o un solo riferimento mutabile	Soggetta alla regole del possesso (no aliasing)

2.13 Slice

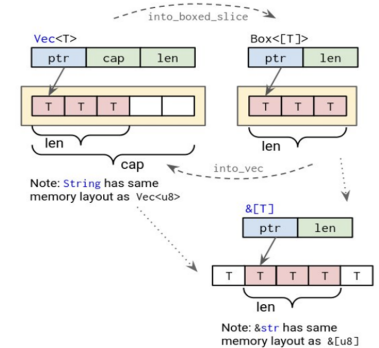
Una slice (fetta) è una VISTA di una sequenza contigua di elementi, la cui lunghezza NON è nota in fase di compilazione, ma disponibile durante l'esecuzione.

Internamente viene rappresentata come **fat pointer** una tupla di due elementi, i primi 64 bits puntano al valore della sequenza, mentre i secondi 64 bits indicano il numero di elementi consecutivi. Una slice di elementi T ha tipo [T].

Una slice **non possiede**, è un prestito a tutti gli effetti, i dati cui fa riferimento. Questi appartengono sempre ad un'altra variabile, da cui la slice viene derivata. Tutti i valori contenuti sono garantiti essere inizializzati, gli accessi sono verificati in fase di esecuzione.

È possibile ricavare una slice a partire da un array, ma anche da altri tipi di contenitori quali `std::Vec<T>`, `String`, `Box<[T]>`, etc.

```
let a: [i32; 5] = [1, 2, 3, 4, 5];
// a è un array di 5 interi
let s = &a[1..3];
// s è una slice formata da 2 elementi [2,3]
let two = s[0];
// two contiene il valore 2
```



2.14 Vantaggi introdotti dal concetto di possesso

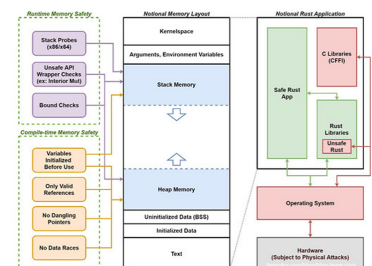
Un programma Rust, pur non avendo un garbage collector, offre molteplici garanzie relative alla correttezza degli accessi in memoria e del rilascio delle risorse.

- Non esiste il concetto di **riferimento nullo** e, di conseguenza, non c'è il rischio di dereferenziarlo
- Poiché il borrow checker vigila sulle assegnazioni dei riferimenti e sugli intervalli temporali in cui essi sono effettivamente usati impedendo accessi illeciti, non c'è il rischio di originare errori di segmentazione o accesso illegale ad aree ristrette di memoria, né la possibilità di avere riferimenti ad aree già rilasciate (dangling pointer)
- Poiché in ogni istante il borrow checker è in grado di determinare la dimensione di un blocco di memoria cui un riferimento punta, vedi le slice, non possono verificarsi **buffer overflow** né **buffer underflow**
- Per lo stesso motivo, gli iteratori offerti da Rust **non eccedono** mai i loro limiti

Tutte le variabili sono **immutabili** per default e occorre una dichiarazione esplicita per renderle mutabili. Questo obbliga il programmatore a riflettere attentamente sul come e dove i dati debbano essere modificati e su quale sia il ciclo di vita di ciascun valore.

Il modello di possesso non riguarda solo la gestione della memoria, ma anche la **gestione delle risorse** contenute in un valore come socket di rete, handle di file e database, descrittori dei dispositivi.

L'assenza di un garbage collector impedisce **comportamenti non deterministici**. In particolare, la sospensione totale del funzionamento ogni qual volta occorra ricompattare la memoria.



Capitolo 3

Tipi composti

3.1 Struct in Rust

Spesso occorre mantenere unite informazioni tra loro eterogenee. Sebbene sia possibile utilizzare una tupla, questa tende a nascondere la semantica del dato: una tupla (i32, i32) potrebbe rappresentare una frazione così come le coordinate di un pixel, rendendo il codice ambiguo.

```
struct Player {  
    name: String, // nickname  
    health: i32, // punti vita  
    level: u8, // livello corr.  
}
```

Quando si intende associare una semantica particolare o legare ad una struttura dati un insieme di comportamenti, è possibile introdurre una **struct**.

Una struct è un costrutto che permette di rappresentare un blocco di memoria in cui sono disposti, consecutivamente, una serie di campi il cui nome e tipo sono indicati dal programmatore.

Il compilatore capisce quanti byte deve allocare in memoria. Essendo che la memoria è organizzata a parole (word), per un processore a 64 bit, gli x86, è molto più efficiente leggere dati che iniziano a indirizzi multipli di 8 byte. Per questo motivo il compilatore Rust utilizza la tecnica di allineamento e padding.

Se abbiamo una struct composta da un u8 (1 byte) e un u32 (4 byte), il processore preferirebbe che il u32 iniziasse su un indirizzo divisibile per 4. Per ottenere questo, il compilatore inserisce dei “byte morti” (padding) tra i due campi.

3.1.1 Come utilizzare una struct

Si istanzia una struct tramite un blocco preceduto dal nome della struttura, contenente un valore per ciascun campo.

```
let mut s = Player { name: "Mario".to_string(), health: 25, level: 1 };
```

Quando il nome del valore coincide con quello del campo è possibile abbreviare la notazione stile TypeScript:

```
let p = Player { name, health, level };
```

Nota: A differenza del TypeScript, il quale utilizza l'equivalenza strutturale, Rust si basa sull'equivalenza nominale. Il tipo Player con 3 valori non ha nulla a che vedere con Giocatore con i medesimi campi.

Operatore di spreading

Esiste la notazione di *spreading*, parziale o totale, anche in Rust.

Se è parziale la si può utilizzare solo alla fine: `let s1 = Player {name: "Paolo".to_string(), ..s}`

Se è totale: `let s1 = Player {...s}`

Nota: La copia dipende dai tipi di dati, se posseggono il tratto `copy`, come `interi`, `bool`, etc, vengo copiatati altrimenti per gli altri dati come le stringhe il valore viene mosso e il possesso del dato viene passato alla struct e la struct originale se non è più utilizzabile nel suo insieme perché è stata parzialmente svuotata.

3.1.2 Accesso alla struct

Si accede ai singoli campi con la notazione puntata.

```
println!("Player {} has health {}", s.name, s.health );
```

Ogni struct introduce un nuovo tipo, il cui nome coincide con il nome della struct, basato sui tipi che la compongono. Questo permette di disambiguare l'uso che si intende fare delle singole informazioni contenute.

3.1.3 Struct fatte a tupla

Si possono definire delle struct simili a delle tuple, indicando solo il tipo del campo, senza attribuire un nome.

Le struct di questo tipo si istanziano come una tupla con l'aggiunta del nome della struct. Si accede tramite la notazione posizionale.

```
struct Playground ( String, u32, u32 );
struct Empty; // non viene allocata memoria per questo tipo di valore

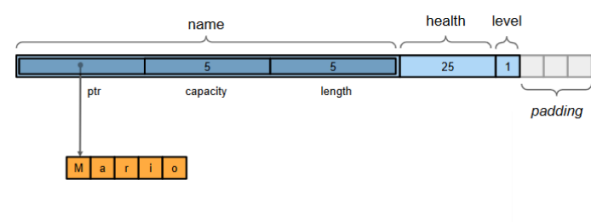
let mut f = Playground( "football".to_string(), 90, 45 );
//f.1 trovo football
//f.2 trovo 90
//f.3 trovo 45
let e = Empty;
```

La semantica a tupla è adatta in situazioni in cui la semantica del dato è ovvia e la notazione posizionale risulta più comoda, come ad esempio le coordinate di un punto. Per il resto, non vi è differenza se non la leggibilità del codice.

In Rust è possibile definire anche struct vuote, senza campi. Questi tipi di struct non occupano spazio in memoria e vengono utilizzati principalmente come marcatori o per implementare comportamenti specifici tramite il tratto `asseganto`.

3.1.4 Probabile rappresentazione in memoria

```
fn main() {
    let p1 = Player {
        name: "Mario".to_string(),
        health: 25,
        level: 1u8,
    };
    println!("{:?}", p1);
}
```



Nota: Non abbiamo garanzia che in memoria sia ordinato nel modo in cui è dichiarato perché il compilatore Rust, a differenza del compilatore del C/C++, può riordinare i campi per ottimizzare l'uso della memoria.

3.1.5 Rappresentazione in memoria

La disposizione in memoria dei singoli campi è conseguenza di vincoli ed ottimizzazioni e può essere controllata attraverso opportuni meccanismi. Ogni singolo campo, in base al proprio tipo, richiede che l'indirizzo a cui viene collocato sia multiplo di una data potenza di 2 (allineamento).

Rust mette a disposizione la notazione

```
std::mem::align_of_val(...)
```

che permette di conoscere l'allineamento richiesto da un particolare valore. Normalmente è utilizzato internamente per il dialogo con il sistema operativo.

Mentre la funzione

```
std::mem::size_of_val(...)
```

ne indica la dimensione in byte occupata in memoria.

Ovviamente i vincoli di allineamento dipendono dalla piattaforma di esecuzione, dalla CPU.

Rappresentazione stile C/C++

In alcuni casi particolare, come l'interoperabilità con codice C ed altri linguaggi, interoperabilità con l' hardware, non vogliamo che il compilatore riordini i campi.

A tal proposito viene anteposto alla definizione della struct l'attributo `#[repr(C)]`.

3.2 Visibilità

Sia la struct nel suo complesso che i singoli campi che la formano possono essere preceduti da un modificatore di visibilità. Per default, i campi sono considerati privati e quindi accessibili nel modulo corrente e nei sottomoduli, eventuali fratelli o genitori non vedono i campi.

Possono però essere resi pubblici facendo precedere il nome dalla parola chiave `pub`.

Nota: Posso definire `pub` solo la struct, ma non i campi, in questo caso i campi sono privati e non accessibili, come il `Vec` la cui `capacity` e `size` non sono accessibili.

Questo permette di implementare il meccanismo di incapsulamento (information hiding).

Per essere efficace, occorre però poter associare un insieme di comportamenti (metodi) alla struct.

A differenza di quanto avviene in altri linguaggi, in cui struttura e comportamento sono definiti contestualmente in un unico blocco, classe, in Rust la definizione dei metodi associati ad una struct avviene separatamente, in un blocco di tipo `impl` ...

3.3 Metodi

Rust non è un linguaggio ad oggetti nel senso tradizionale del termine, anche se implementa incapsulamento e polimorfismo, conseguentemente, non ha il concetto di classe. Dato che questo concetto tende a creare vincoli non voluti.

Sebbene le struct possano apparire simili alle classi di altri linguaggi, il parallelismo è limitato, in particolare, le struct NON sono organizzate in una gerarchia di ereditarietà. Il concetto di metodo si applica invece a tutti i tipi, compresi quelli primitivi.

Il metodo ha accesso ha a TUTTI i campi della struct, anche quelli privati, questo ci permette di implementare l'incapsulamento.

3.3.1 Definizione di un metodo

Si definiscono i metodi collegati ad un tipo in un blocco racchiuso tra parentesi graffe, preceduto dalla parola chiave `impl` seguita dal nome del tipo.

Le funzioni presenti in tale blocco il cui primo parametro sia `self`, `&self` o `&mut self` diventano metodi.

`Self`¹ è una parola chiave che rappresenta l'istanza del tipo di cui si sta facendo l'implementazione.

Le funzioni che non hanno come primo parametro `self` sono dette funzioni associate e svolgono il ruolo giocato dai costruttori e dai metodi statici nei linguaggi ad oggetti.

¹Corrisponde grosso modo a quello che in altri linguaggi ad oggetti viene chiamato `this`

```

class Something {
  int i;
  String s;

  void process() {...}
  int increment() {...}
};

struct Something {
  i: i32,
  s: String
}
impl Something {
  fn process(&self) {...}
  fn increment(&mut self) {...}
}

```

Se in JavaScript il *this* non compare nella lista dei parametri, dato che è implicito, in Rust viene esplicitato perché dobbiamo dire che operazione facciamo su quel *self*.

- *self*: voglio prendere possesso del ricevitore il quale non sarà più accessibile poi, movimento.
- *&self*: chiedo un prestito un sola lettura del ricevitore, posso leggere ma non posso modificarli, riferimento condiviso.
- *&mut self*: chiedo l'accesso in lettura e scrittura, riferimento esclusivo.

```

pub struct Point{
  x: i32, //privati
  y: i32,
}

impl Point {
  pub fn new(x: i32, y: i32) -> Self {
    Point { x, y }
  }

  pub fn length(&self) -> i32{
    (self.x * self.x + self.y * self.y).isqrt()
  }
}

let p1 = Point::new(3, 4);

// possibile solo nello stesso modulo
// o in un sottomodulo di quello in cui
// è stato definito Point
let p2 = Point::new(x:3, y:4); //potenzialmente
↳ non scrivibile dato che se non sono nello
↳ stesso modulo non va bene

let m1 = p1.length(); //5 quando scrivo questo
↳ il compilatore traduce in
↳ Point::length(&p1)

let m2 = Point::length(&p1); // 5

```

I metodi sono funzioni legate ad un'istanza di un dato tipo. Il legame si manifesta sia a livello sintattico, che a livello semantico.

Sintatticamente un metodo viene invocato a partire da un'istanza del tipo a cui è legato utilizzando la notazione *instance.method(...)*

Semanticamente il codice del metodo ha accesso al contenuto (pubblico e privato) del ricevitore attraverso la parola chiave *self*.

```

struct Point {
  x: i32,
  y: i32,
}

impl Point {
  fn mirror(self) -> Self {
    Self{ x: self.y, y: self.x }
  }
  fn length(&self) -> i32 {
    (self.x*self.x + self.y*self.y).isqrt()
  }
  fn scale(&mut self, s: i32) {
    self.x *= s;
    self.y *= s;
  }
}

fn main() {
  let p1 = Point{ x: 3, y: 4 };
  let mut p2 = p1.mirror();
  let l1 = p2.length(); // l1: 5
  p2.scale(2);
  let l2 = p2.length();
  // l2: 10
}

```

3.4 Costruttori

A differenza di quanto capita in altri linguaggi, in Rust non esiste il concetto di costruttore.

In Rust, invece, qualunque metodo nel blocco *impl*, che ritorna *Self*, può fare le veci del costruttore. Questo garantisce che il programmatore sia consapevole delle informazioni contenute al suo interno.

Per convenzione, si sceglie di chiamarlo *new*, ma non è obbligatorio.

```
pub fn new() -> Self {...}
```

Poiché Rust non supporta l'overloading delle funzioni, se servono più funzioni di inizializzazione, ciascuna di esse avrà un nome differente: in questo caso la convenzione è utilizzare un pattern come

```
pub fn with_details(...) -> Self {...}
```

3.5 Gestione delle risorse

Se una struttura contiene al proprio interno risorse la cui semantica richiede una esplicita gestione del ciclo di vita, occorre fare in modo che, quando la struttura cessa di esistere, tali risorse siano liberate in modo appropriato. (E.g. file handle, socket, etc.)

Questo compito è affidato ad una funzione speciale detta *Drop*, il distruttore, il quale se presente viene chiamato in automatico dal compilatore quando l'oggetto cessa di esistere per liberare le risorse che contiene.

Rust segue il paradigma RAI - Resource Acquisition Is Initialization - l'acquisizione delle risorse avviene alla creazione, il rilascio alla distruzione.

3.6 RAI - Resource Acquisition Is Initialization

Le risorse sono incapsulate in una classe, struttura, in cui:

- il costruttore acquisisce le risorse e stabilisce eventuali invarianti, oppure lancia un'eccezione se non può essere fatto
- il distruttore rilascia le risorse e NON lancia mai eccezioni

Si usano le risorse attraverso l'istanza di una classe RAI-compatibile che ha una gestione automatica delle durata di tutte le risorse, **oppure**, ha un ciclo di vita connesso al ciclo di vita di un altro oggetto (ad es., è parte di esso).

In questo contesto, la presenza della semantica del movimento, garantisce il corretto trasferimento delle risorse, mantenendo la sicurezza del rilascio.

3.7 Distruttori

Rust gestisce il rilascio di risorse contenute in un'istanza attraverso il tratto *Drop*, costituito dalla funzione

```
fn drop(&mut self) -> ()
```

Chiamato in automatico dal compilatore. Si può forzare il rilascio delle risorse contenute in un oggetto usando la funzione `drop(some_object)`; che ne acquisisce il contenuto e determina l'uscita dallo scope.

```
pub struct Shape {
    pub position: (f64, f64),
    pub size: (f64, f64),
    pub type: String
}

impl Drop for Shape {
    fn drop(&mut self) {
        println!("Dropping shape!");
    }
}
```

Nota: Il tratto *Drop* è mutuamente esclusivo con il tratto *Copy*. Se un tipo implementa il primo non può implementare l'altro, e viceversa.

3.8 Metodi statici

In C++/Java è lecito inserire all'interno del costrutto `class ....{ }` la dichiarazione di metodi preceduti dalla parola chiave `static`. Essi non sono legati ad una specifica istanza, ma possono operare sulle istanze della classe (se ne conoscono l'indirizzo) avendo accesso anche alle componenti private.

In Rust, è possibile implementare metodi analoghi semplicemente non indicando, come primo parametro, né `self` né un suo derivato. Questo permette la creazione di funzioni per la costruzione di un'istanza, metodi per la conversione di istanze di altri tipi nel tipo corrente o, semplicemente, l'accesso a funzionalità statiche.

L'esempio più comune è il metodo `new`.

```
<Tipo>::metodo(...)
```

3.9 Enum

In Rust, è possibile introdurre tipi enumerativi composti da un semplice valore scalare. Ma anche incapsulare, in ciascuna alternativa, una tupla o una struct volta a fornire ulteriori informazioni relative allo specifico valore.

Anche in questo caso è possibile legare metodi ad un'enumerazione, aggiungendo il blocco `impl ...` come nel caso delle struct.

```
enum HttpStatusCode {
    Ok,
    NotFound(String),
    InternalError {
        desc: String,
        data: Vec<u8>
    },
}

enum HttpStatusCode {
    Ok = 200,
    NotFound = 404,
    InternalError = 500
}
```

Se le struct sono un tipo prodotto perché l'insieme dei valori che può contenere è il prodotto cartesiano degli insiemi legati ai singoli campi, le enum sono un tipo somma perché il loro dominio è la somma, l'unione, dei domini dei loro campi. (E.g. o c'è `Ok` o c'è `NotFound` o c'è `InternalError`)

Nelle enum posso legare, oltre che un'etichetta, anche degli altri valori etichetta e non solo valori numerici associati al valore.

3.9.1 Rappresentazione in memoria

In memoria gli enum occupano lo spazio di un intero da 1 byte più lo spazio necessario a contenere la variante più grande.



3.9.2 Enumerazioni e clausole match

Il costrutto `match` si presta particolarmente per gestire in modo differenziato valori enumerativi.

Il comportamento offerto dal pattern matching e la possibilità di legare variabili temporanee in base alla struttura del valore che viene analizzato offre una sintassi compatta ed efficiente per esprimere comportamenti alternativi (destructuring assignment).

Il fatto che i pattern confrontati debbano essere esaustivi, garantisce che il codice resti coerente anche se il numero di possibili alternative presenti nell'enumerazione cambia nel tempo.

```
enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}

fn compute_area(shape: Shape) -> f64 {
    match shape {
        Square { s } => s * s,
        Circle { r } => r * r * std::f64::PI,
        Rectangle { w, h } => w * h,
    }
}
```

3.10 Destrutturazione

L'utilizzo del pattern matching e la possibilità di destrutturare un valore complesso non sono limitate al costrutto *match*. È possibile usare la stessa tecnica anche all'interno di costrutti di controllo del flusso come:

```

        if let <pattern> = <value> ... e while let <pattern> = <value> ...

pub fn make_filter (filter: Option<String>) -> anyhow::Result<String> {
    if let None = filter {
        return Ok("default_column_name".to_string());
    } else if let Some(column) = filter {
        return Ok(column.to_string());
    }
}

```

Tali costrutti verificano se il valore fornito corrisponda o meno al pattern indicato e, nel caso, eseguono le necessarie assegnazioni alle variabili contenute nel pattern.

```

enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}

fn process(shape: Shape) {
    // stampa solo se shape è Square...
    if let Square { s } = shape {
        println!("Square side {}", s);
    }
}

```

La destrutturazione è anche utile per ottenere un “parsing” di una struct, così da gestirne più facilmente i campi, estraendoli in contenitori singoli da trattare separatamente. I campi della struct sono “estratti” con il loro nome, direttamente con l’assegnazione.

```

...
let p = Point { x: 5, y: 10 };
...
// la destrutturazione deve rispettare i nomi dei campi
let Point { x, y } = p;

println!("The original point was: ({} , {})", x, y);

```

Il processo di destrutturazione utilizza la semantica delle assegnazioni. Se il valore implementa il tratto copy, le variabili introdotte nel pattern conterranno una copia dell’elemento corrispondente. In caso contrario, verrà eseguito un movimento, invalidando il valore originale.

```

enum Shape{
    Square{s:f64},
    Circle{r:f64},
    Rect{w:f64, h:f64}
}

impl Shape{
    fn area(&self) -> f64{
        match self{
            Shape::Square{s: &f64} => s * s,
            Shape::Circle{r: &f64} => std::f64::PI * r * r,
            Shape::Rect{w: &f64, h: &f64} => w * h,
        }
    }
}

fn main() {
    let s = Shape::Square { s: 3.0 };
    println!("Area: {}", s.area());
}

fn shrink_if_circle(shape: &mut Shape) {
    if let Shape::Circle { r } = shape { *r *= 0.5 }; //r ha tipo &mut f64
}

```

Capitolo 4

Gestione degli errori

4.1 Errori ed eccezioni

Tutte le computazioni, le funzioni chiamate, possono fallire prematuramente impedendo che possa essere restituito il valore atteso o che vengano eseguiti tutti, o parzialmente, gli effetti collaterali richiesti come la scrittura su file.

Questi fallimenti possono essere dovuti a cause differenti

- Alcune facilmente prevedibili come argomenti illeciti, conversione da testo a numero, etc.
- Altre dovute a limiti del sistema di elaborazione come memoria/spazio su disco esauriti, malfunzionamento della rete o di altre periferiche, etc.

Indipendentemente dalle cause che li hanno prodotti, i fallimenti possono essere catalogati in due gruppi principali

- **Recuperabili:** non hanno compromesso lo stato del programma, per cui è possibile attivare una strategia di ripristino
- **Non recuperabili:** causano un'alterazione imprevedibile dello stato o che indicano l'impossibilità di procedere ulteriormente nella computazione.

Nei casi irrecuperabili, occorre terminare il processo, eventualmente eseguendo qualche operazione di pulizia sull'ambiente esterno.

Negli altri, occorre mettere in atto una strategia di ripristino dello stato come ritentare l'operazione, saltare l'operazione, chiedere l'intervento dell'utente o amministratore, utilizzare strategia alternativa, etc.

Non è detto che il punto in cui si verifica il fallimento possieda abbastanza contesto per discriminare come comportarsi. Occorre fare in modo che, in caso di fallimento della computazione all'interno di una funzione, il controllo torni al suo chiamante, corredato di una opportuna descrizione di quanto successo

Questo requisito tende ad aumentare la complessità del codice, introducendo rami diversi, if/else, switch/case, match, etc., lungo i quali la computazione può procedere, favorendo l'introduzione di errori logici legati alla quantità di dettagli cui occorre badare.

Per questo motivo i linguaggi moderni introducono dei costrutti sintattici a supporto della gestione degli errori.

4.1.1 Spazio di scelta

Quando un'operazione non ha successo si deve decidere come comportarsi nello spazio di scelta che abbiamo a disposizione.

È possibile / sensato ritentare l'esecuzione?

Se l'errore è frutto di condizioni transitorie come ad esempio congestione di rete, file in uso, potrebbe avere senso, a patto di evitare cicli infiniti e introdurre opportune pause tra un tentativo e l'altro, ritentare l'esecuzione.

Un'altra domanda sensata da porsi è se sia possibile continuare l'esecuzione utilizzando un valore di default o attivando una strategia alternativa per calcolare il valore mancante.

È possibile coinvolgere l'utente nella risoluzione del problema? Se l'applicazione è interattiva e l'errore deriva dal comportamento dell'utente questo potrebbe funzionare.

È possibile isolare l'errore e mantenere il resto del sistema in funzione? È un bug o è stato violato un invariante, per cui occorre terminare il programma?

4.1.2 Supporto sintattico alla gestione degli errori

Il linguaggio C non fornisce alcun supporto sintattico alla modellazione degli errori. Lasciando completamente al programmatore la responsabilità di gestire la situazione.

In altri linguaggi come Java, Python, C++, C#, etc., viene introdotto il concetto di eccezione, tipo di dato che descrive un errore che viene lanciato, *throw*, e viene restituito al chiamante.

L'eccezione risale fino al primo *try-catch* che lo gestirà o fino al main dove incontrerà la morte.

Questa idea crea molti problemi anche solo nella conoscenza di tutte le classi di eccezione che possono essere lanciate, l'esempio lampante è Java che descrive le classi `Error` ed `Exception` con le varie differenze.

Inoltre, si creano problemi per la gestione dello stack nel quale si deve tenere traccia del contesto in cui è stata lanciata l'eccezione.

Rust non utilizza il concetto di eccezione, ma offre i tipi algebrici `Result<T, E>` e `Option<T>` per esprimere gli esiti delle computazioni.

Offre inoltre la macro `panic!(...)` per forzare l'interruzione del thread corrente producendo una descrizione testuale di quanto successo.

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

4.1.3 I limiti della gestione delle eccezioni

Il compilatore non è in grado di identificare dove vengano restituite eccezioni, conseguentemente, non forza l'utilizzo di costrutti in grado di gestirle.

La generazione di un'eccezione blocca l'esecuzione del codice seguente e riporta tipicamente un singolo tipo di errore, rendendo onerosa la validazione contemporanea di più criteri.

La possibilità che un'eccezione dello stesso tipo possa essere generata in parti diverse della computazione, ma gestita in un unico punto a monte, rende complessa la scelta delle contromisure da applicare, richiedendo l'ispezione manuale della catena delle chiamate fallite per identificare il punto di rottura.

Per permettere la corretta contrazione dello stack e il ritorno al blocco *try-catch* più recente occorre imporre una certa sovrastruttura allo stack ed al contesto di esecuzione. Tale sovrastruttura non si adatta particolarmente alle assunzioni che vengono fatte nel kernel di Linux e questo è uno dei principali motivi per cui non è possibile scrivere moduli kernel in C++ per quel sistema operativo.

4.1.4 Gestioni delle eccezioni in Rust

Rust offre una risposta funzionale al problema della modellazione degli errori basata sul tipo algebrico generico `Result<T, E>`.

Questo costrutto modella l'unione di tutti i possibili risultati con successo `T` e tutti gli errori `E` che possono verificarsi nell'esecuzione di una funzione.

```
fn read_file(name: &str) -> Result<String, io::Error> {
    let r1 = File::open(name);
    let mut file = match r1 {
        Err(why) => return Err(why),
        Ok(file) => file,
    };
    let mut s = String::new();
    let r2 = file.read_to_string(&mut s);
    match (r2) {
        Err(why) => Err(why),
        Ok(_) => Ok(s),
    }
}
```

4.1.5 Elaborare i risultati

`Result<T, E>` mette a disposizione svariati metodi che consentono di accedere ai dati contenuti al suo interno:

- I metodi `is_ok(&self)` e `is_err(&self)` permettono, rispettivamente, di determinare se l'esito di un'operazione ha avuto successo o meno
- I metodi `ok(self)` e `err(self)` consumano il risultato trasformandolo in un oggetto di tipo `Option<T>` piuttosto che `Option<E>`
- Il metodo `map(self, op: F) -> Result<U, E>` applica la funzione al valore contenuto nel risultato, se questo è `ok`, altrimenti lascia l'errore invariato
- Il metodo `contains(&self, x: &U)` restituisce vero se il risultato è valido e contiene un valore che equivale all'argomento
- Il metodo `unwrap(self) -> T` restituisce il valore contenuto, se è valido, ma invoca la macro `panic!(...)` se il risultato contiene un errore

4.1.6 Gestire gli errori

Se l'invocazione di una funzione restituisce un valore di tipo *Result* contenente un errore, occorre mettere in atto una strategia di gestione:

- Ritentare l'esecuzione, evitando di entrare in un loop infinito
- Ricostruire uno stato precedente all'errore (rollback)
- Utilizzare una strategia alternativa per la computazione fallita
- Registrare un messaggio nel log e propagare l'errore al chiamante
- Propagare tout court l'errore al chiamante, delegando ad esso la corrispondente responsabilità
- Terminare, in modo ordinato, il programma

Sebbene terminare il programma possa apparire facile, è sufficiente invocare la funzione

```
std::process::exit(code: i32) -> !, farlo in modo pulito può essere complesso.
```

Occorre infatti garantire che non vengano lasciati oggetti persistenti, come file o altre risorse di sistema, in stati non coerenti. Per questo motivo Rust mette a disposizione la macro `panic!(...)` che, oltre a terminare il thread corrente, esegue un processo di pulizia.

4.1.7 Panic

In alcune situazioni, lo stato del programma risulta così compromesso che non è pensabile eseguire azioni di ripristino: questo spesso è dovuto alla presenza di errori logici nel programma stesso, come l'accesso ad array al di fuori dei relativi limiti, divisioni intere per zero, fallimenti di assegnazione, etc.

In questi casi Rust offre la macro `panic!(...)` che accetta argomenti simili a quelli offerti dalla macro `println!(...)` per formulare un messaggio di errore.

L'effetto dell'invocazione di questa macro è la contrazione dello stack, come nel caso delle eccezioni C++, la distruzione via `.drop()` delle variabili che implementano il tratto `Drop` fino alla terminazione del thread corrente.

Se il thread in cui è stato invocato è il thread principale dell'applicazione, il processo termina con un codice di errore non nullo, altrimenti **continua**.

4.1.8 Ignorare gli errori

In alcune situazioni, può capitare che l'errore, che potenzialmente viene restituito da una funzione, non possa di fatto succedere, a conseguenza di quanto si è appena verificato nel corso dell'esecuzione.

Oppure che il programmatore assuma che non sia necessario mettere in atto una strategia di contenimento e gestione dell'errore, lasciando semplicemente terminare il processo.

Il tipo `Result<T, E>` mette a disposizione i metodi `unwrap(...)` ed `expect(...)` che restituisce il valore *T*, se presente, ed invocano la macro `panic!(...)` in caso di errore.

Nota: `expect(...)` permette di indicare una stringa che sarà usata al posto del messaggio standard generato da `unwrap(...)`, nel caso si sia verificato un errore

4.1.9 Propagare gli errori

Nella maggior parte delle funzioni in cui si verifica un errore, non si sa quale strategia mettere in atto per ripristinare lo stato del programma dato il contesto insufficiente in cui ci si trova.

Si può ovviare a questo problema restituendo un valore di tipo *Result*. Questo porta, tuttavia, a costrutti alquanto complessi, con espressioni di controllo difficili da decodificare.

Per semplificare la sintassi, Rust offre l'operatore `?` che può essere applicato a qualunque espressione che produce un valore di tipo *Result* $\langle T, E \rangle$.

Se il risultato è di tipo *Ok*(*v*), l'operatore restituisce il valore *v*, altrimenti, se il risultato è di tipo *Err*(*e*) la funzione corrente termina, ritornando il valore che incapsula l'errore.

Questo prevede che l'errore di compatibile con l'errore *E* che proviene dalla funzione chiamata.

```
fn read_file(name: &str) -> Result<String,io::Error> {
    let mut file = File::open(name)?;
    let mut s = String::new();
    file.read_to_string(&mut s)?;
    Ok(s)
}
```

4.1.10 Altri modi di esprimere il fallimento

In alcune situazioni, può essere sufficiente distinguere se la computazione ha prodotto il proprio risultato, oppure indicare che non è stato possibile completarla.

Per questi casi esiste il tipo *Option* $\langle T \rangle$ che racchiude due alternative:

- **Some**(*T*): usata per descrivere il risultato atteso
- **None**: he indica l'assenza di risultato, senza descriverne le ragioni

L'operatore `?` può essere applicato anche al tipo *Option* $\langle T \rangle$, a condizione che la funzione in cui viene usato restituisca un *Option* $\langle U \rangle$, con *U* qualsiasi.

Result $\langle T, E \rangle$ e *Option* $\langle U \rangle$

Result $\langle T, E \rangle$ e *Option* $\langle U \rangle$ sono correlati:

Result offre il metodo `ok(self)` che restituisce *Option* $\langle T \rangle$, valorizzato con il dato in caso di successo (il dato risulta mosso da *Result*, che quindi diventa inaccessibile).

Offre anche il metodo `err(self)` che restituisce *Option* $\langle E \rangle$, valorizzato con l'errore in caso di fallimento (anche in questo caso, con movimento)

4.1.11 Propagare errori eterogenei

Quando una funzione produce diverse tipologie di errore, è necessario propagare i diversi errori così da poter specializzare le contromisure. Rust offre diversi modi per propagare errori eterogenei, la scelta spetta al programmatore sulla base delle sue esigenze

La libreria standard mette a disposizione l'implementazione generica del tratto *From*, che converte qualsiasi valore che implementi il tratto *Error* in *Box* $\langle \text{dyn error}::\text{Error} \rangle$.

Gli oggetti-tratto richiedono l'utilizzo di fat pointer e vtable con il conseguente costo in termini di memoria. Durante la conversione vengono perse le informazione sul tipo dell'errore.

Si può risalire allo specifico errore tramite l'utilizzo del `downcast_ref` a patto che si conosca l'implementazione della funzione che genera gli errori. La conversione può avvenire in maniera implicita attraverso l'utilizzo dell'operatore `?`.

```
impl<E: error::Error> From<E> for Box<dyn error::Error>;
```

```

fn sum_file(path: &Path) -> Result<i32, Box<dyn error::Error>> {
    let mut file = File::open(path) ? ; // io::Error -> Box<dyn error::Error>
    let mut contents = String::new();
    file.read_to_string(&mut contents) ? ; // io::Error -> Box<dyn error::Error>
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>() ? ; // ParseIntError -> Box<dyn error::Error>
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>() {...} //tratto io::Error
            else if let Some(e) = err.downcast_ref::<ParseIntError>() {...} //tratto ParseIntError
            else { unreachable!(); } //non può capitare
        }
    }
}

```

Errori custom

Per propagare errori eterogenei senza forzare il sistema dei tipi è possibile implementare degli errori custom.

Tutti gli errori custom devono implementare il tratto *Error* e conseguentemente anche i tratti *Debug* e *Display*.

L'utilizzo di un *enum* permette di racchiudere i diversi tipi di errore da gestire successivamente tramite l'utilizzo del costrutto *match*. È necessario implementare il tratto *From* per convertire i diversi errori nel tipo custom da propagare.

```

#[derive(Debug)]
enum SumFileError {
    Io(io::Error),
    Parse(ParseIntError),
}

impl From<io::Error> for SumFileError {
    fn from(err: io::Error) -> Self {
        SumFileError::Io(err)
    }
}

impl From<ParseIntError> for SumFileError {
    fn from(err: ParseIntError) -> Self {
        SumFileError::Parse(err)
    }
}

impl fmt::Display for SumFileError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) ->
        <= fmt::Result {
        match self {
            SumFileError::Io(err) =>
                write!(f, "IO error: {}", err),
            SumFileError::Parse(err) =>
                write!(f, "Parse error: {}", err),
        }
    }
}

impl error::Error for SumFileError {
    fn source(&self) -> Option<&(dyn error::Error +
        'static)> {
        Some(match self {
            SumFileError::Io(err) => err,
            SumFileError::Parse(err) => err,
        })
    }
}

fn sum_file(path: &Path) -> Result<i32,
    <= SumFileError> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>()?;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("the sum is {}", sum),
        Err(SumFileError::Io(err)) => {...},
        Err(SumFileError::Parse(err)) => {...},
    }
}

```

Create thiserror

Un grosso aiuto all'implementazione di tipi che implementano il tratto *Error* viene dal crate **thiserror**. Esso offre una implementazione della macro *derive* specializzata per il tratto *Error*.

Definendo un tipo (enum, struct, unit) preceduto dall'attributo `#[derive(Error, Debug)]` si ottiene l'implementazione automatica di questi due tratti:

- L'attributo `#[error("Messaggio con formato")]` posta di fronte alle singole voci enumerative o all'intera struct genera l'implementazione del tratto *Display*
- L'attributo `#[from]` posto di fronte ad un campo il cui tipo implementa il tratto *Error* genera l'implementazione del tratto *From* a partire dal tipo del campo

Questo approccio è particolarmente adatto nella creazione di librerie, dove si vogliono rendere i possibili errori parte della definizione del contratto (API).

```
[dependencies]
thiserror = "1.0"

#[derive(Error, Debug)]
enum SumFileError {

    #[error("IO error {0}")]
    Io(#[from] io::Error),

    #[error("Parse error {0}")]
    Parse(#[from] ParseIntError),
}
```

Create anyhow

Il crate **anyhow** definisce l'oggetto-tratto *anyhow::Error* che semplifica la gestione idiomatica degli errori. Si può usare il tipo *anyhow::Error* per incapsulare il valore di ritorno di una funzione che può fallire.

Questo tipo offre un'implementazione automatica del tratto `From<T: Error>`, il che permette di utilizzare la notazione basata sull'operatore `?` per propagare l'errore ritornato.

Quando viene generato un errore è possibile aggiungere una descrizione che contestualizza ciò che è successo tramite i metodi `context(...)` e `with_context(...)`.

Il messaggio di errore associato verrà sostituito dalla stringa indicata seguita dalla causa originale.

Questo crate interopera correttamente con *thiserror* e risulta adatto nella scrittura di codice applicativo, piuttosto che di librerie. In questo caso, la semplicità di scrittura del codice domina rispetto al controllo dell'informazione contenuta nell'errore generato, tenuto conto che sarà interpretato da persone.

```
fn sum_file(path: &Path) -> anyhow::Result<i32> {
    let mut file = File::open(path).with_context(|| format!("Missing path {}", path)) ? ;
    let mut contents = String::new();
    file.read_to_string(&mut contents).context("File read error") ? ;
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>().with_context(|| format!("Not a number: {}", line)) ? ;
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>() {...} //tratto io::Error
            else if let Some(e) = err.downcast_ref::<ParseIntError>() {...} //tratto ParseIntError
            else { unreachable!(); } //non può capitare
        }
    }
}
```

Capitolo 5

Moduli

5.1 Modularità

L'obiettivo di questa sezione è di comprendere la differenza tra **package**, **crate** e **moduli** e come organizzare un progetto Rust mediante una **struttura gerarchica mantenibile** controllandone la **visibilità** e l'**incapsulamento** delle strutture dati.

5.1.1 Perché rendere modulare un progetto?

- Per accrescere la **manutenibilità**: suddividere il codice in moduli rende più facile comprendere, modificare ed estendere nel tempo il progetto, riducendo il rischio di introdurre effetti collaterali non voluti
- Per favorire l'**incapsulamento**: controllando quali funzioni, tipi e campi siano esposti, pub, o tenuti privati. È possibile nascondere i dettagli implementativi ed assicurarsi che le altre parti del programma possano interagire solo con interfacce ben definite
- Per garantire una **sicurezza**, safety, all'atto della compilazione: il sistema di moduli e visibilità offerto da Rust permette di rilevare in fase di compilazione la maggior parte degli errori concettuali legati all'accesso ai dati, aumentando l'affidabilità del prodotto
- Per costruire librerie **riutilizzabili**: strutturare il codice in progetti, create, e moduli ben congegnati permette il riutilizzo di componenti, riducendo la duplicazione del codice e promuovendo l'uso consistente di funzionalità comuni

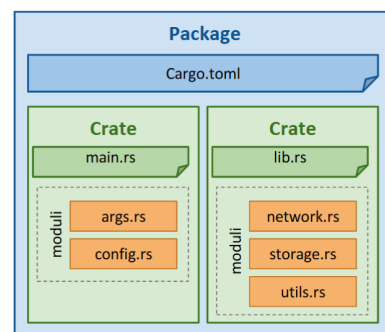
5.2 Struttura di un progetto Rust: package, crate, moduli

Un progetto Rust è organizzato, chiamato, **package** eè rappresentato da una cartella con al suo interno un file `Cargo.toml` che gestisce la compilazione.

Il package contiene la cartella `src` la quale contiene i vari **crate**. Se il package è semplice con un solo crate, il main, il quale verrà compilato aggiungendo, se necessario, i moduli aggiuntivi che abbiamo inserito. In questa maniera verrà creato un file eseguibile nella cartella `target/debug` con lo stesso nome del package.

Dentro questa cartella possono esserci più **crate**, la quale rappresenta l'unità di compilazione e può dare origine ad un eseguibile `main.rs` e/o ad una libreria `lib.rs`.

Il **modulo** è una suddivisione logica del codice, che può essere organizzata in file e cartelle. Organizza il codice e controlla la visibilità. Rappresenta lo spazio dei nomi all'interno di un crate.



5.3 Crates

Il processo di compilazione di un progetto Rust è basato sul concetto di **crate**. A livello sorgente, un crate è una unità di compilazione che ha una radice, `main.rs` o `lib.rs`, e può essere composto da più moduli e file. Un crate è l'unità di compilazione: Cargo compila ogni crate e poi li collega tra loro.

Un crate binario costituisce un'unità di versionamento e di caricamento in fase di esecuzione. Quando un crate fa riferimento a codice, funzioni, strutture, tratti, etc., esportato da un altro crate, di tipo libreria, quest'ultimo deve essere combinato con il primo per mettere a disposizione quanto richiesto. Questa operazione è detta di **linking** e dipende da come la libreria è stata compilata, staticamente o dinamicamente.

- **Linking statico:** il codice della libreria viene incorporato direttamente nell'eseguibile durante la compilazione. Il risultato è un eseguibile più grande, ma indipendente da librerie esterne al momento dell'esecuzione.
- **Linking dinamico:** il codice presenta dei marcatori, stub, che indicano al sistema operativo di cercare la libreria dinamica (DLL su Windows, .so su Linux) al momento dell'esecuzione e di caricarla nello spazio del processo. Il risultato è un eseguibile più piccolo, ma dipendente dalla presenza della libreria dinamica al momento dell'esecuzione rendendone più difficile la distribuzione.

Oltre a quanto detto fino ad ora, in Rust abbiamo la possibilità di versionare il nostro crate. Nel file `Cargo.toml` è possibile specificarne la versione.

Oltre alla versione definita nel codice sorgente Rust, ogni progetto Cargo espone la propria versione nel file `Cargo.toml`, seguendo il formato SemVer:

```
version = "MAJOR.MINOR.PATCH"
```

Le tre componenti hanno significati precisi:

- **MAJOR:** viene incrementato quando si introducono breaking changes, cioè modifiche incompatibili con le versioni precedenti. Esempio: da 0.1.2 a 1.1.2 se il programma smette di leggere CSV e inizia a leggere solo PDF.
- **MINOR:** viene incrementato quando si aggiungono nuove funzionalità retrocompatibili. Esempio: da 0.1.2 a 0.2.2 se, oltre ai CSV già supportati, il programma acquisisce la capacità di leggere anche i PDF.
- **PATCH:** viene incrementato quando si rilasciano sole correzioni di bug, senza aggiungere né rimuovere funzionalità. Esempio: da 0.1.2 a 0.1.3.

5.4 Organizzazione del codice

Spesso i package contengono due crate. Un crate binario, facente capo a `main.rs`, ed un crate libreria, facente capo a `lib.rs`. Questo permette di separare l'interfaccia eseguibile dalla logica riutilizzabile del programma.

5.4.1 Create binario

Il main, minimale, si occupa solo dell'orchestrazione generale del programma come:

- Parsing degli argomenti da linea di comando
- Configurazione dell'ambiente di esecuzione
- Chiamata alle funzioni della libreria per eseguire la logica

5.4.2 Le librerie

Le librerie si occupano della logica dell'applicazione, funzioni, strutture dati, moduli, etc., che può essere riutilizzata

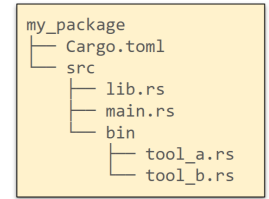
- Da più crate binari dello stesso package
- Da altri progetti Rust
- Da altri linguaggi di programmazione, se esposte con interfacce compatibili (ad esempio, C)

5.4.3 Package con più file binari

Come abbiamo appena accennato un progetto può avere più file binari che utilizzano la stessa libreria. Se presenti essi devono essere collocati nella cartella `src/bin`. A questo punto il comando `cargo build` darà origine a più eseguibili.

Questo permette di creare applicativi correlati, ad esempio un client e un server, che condividono dipendenze, hanno la stessa versione e vengono compilati insieme.

Nota: Un package è un contenitore di progetto che può includere uno o più crate binari, permettendo di sviluppare più programmi correlati che condividono codice comune.



Un package è il contenitore del progetto. Può esporre una sola libreria, la `lib.rs`, ma può dipendere da quante librerie esterne si vuole, dichiarandole come dipendenze nel file `Cargo.toml`

5.5 Moduli e visibilità

Il codice contenuto in un crate è costituito da un albero di **moduli** che oltre a dividere il sorgente in più piccoli, ne definisce anche la visibilità. Un modulo è un costrutto sintattico, introdotto dalla parola chiave `mod` e dal relativo nome che racchiude funzioni, tipi definiti dall'utente (`struct`, `enum`, `union`), tratti, relative implementazioni ed altri moduli.

Tutto il codice che non è racchiuso in un modulo di tipo `mod nome_modulo...` fa parte del modulo radice.

I moduli formano una gerarchia ad albero, la cui radice è l'unità di compilazione corrente (rappresentata dalla parola chiave `crate`).

5.5.1 Visibilità di default - privata

Di base, tutti gli elementi definiti all'interno di un modulo sono considerati privati, e sono accessibili solo al codice presente nel modulo stesso o nei suoi sotto-moduli.

5.5.2 Visibilità pubblica - pub

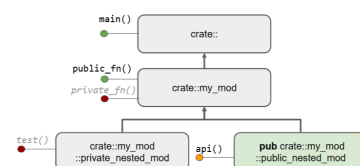
Se un elemento definito all'interno di un modulo `m1` è preceduto dalla parola chiave `pub`, diventa pubblico e può essere utilizzato dal codice presente in un modulo `m2` che non sia un suo discendente, a condizione che tutti gli antenati del modulo `m1` siano a loro volta accessibili al modulo `m2` (perché pubblici o perché il modulo `m2` è contenuto al loro interno).

```

mod my_mod {
  fn private_fn() {} // visibile solo nel modulo
  ↪ corrente
  pub fn public_fn() {} // visibile nel contesto
  ↪ dell'unità di compilazione:
  // - codice presente in questo stesso file
  // - codice che ha accesso al percorso my_mod;
  mod private_nested_mod { // sotto-modulo privato
    fn test() {} // può accedere a my_mod::private_fn()
  }
  pub mod public_nested_mod {
    // accessibile a chi ha accesso a my_mod
    pub fn api() {}
  }
}
  
```

```

fn main() {
  my_mod::public_fn();
  my_mod::public_nested_mod::api();
}
  
```



Per poter usare un simbolo (funzione, tipo, tratto, etc.) definito in un modulo diverso da quello corrente occorre indicare al compilatore dove andare a reperirlo.

Lo si può fare indicando il path del modulo. Questo path può essere assoluto o relativo.

5.5.3 Cammino assoluto

Il cammino assoluto comincia con il nome del crate in cui si trova il simbolo seguito dalla sequenza gerarchica di di moduli che occorre attraversare per giungere al simbolo.

```
let size = std::mem::size_of::<i32>(&f);
```

si utilizza il separatore `::` per connettere i nomi dei (sotto-)moduli.

Nel caso il simbolo appartenga al crate corrente, si può indicare come elemento iniziale la parola chiave `crate`.

5.5.4 Cammino relativo

Il cammino relativo comincia con le parole chiave `self` o `super`. Un parallelismo potrebbe essere, in ordine, il concetto di `./` e `../` nei file system.

```
self::my_function
super::my_function
```

5.5.5 Cammino e *

Per evitare di dover ripetere l'intero cammino per ogni occorrenza del simbolo, è possibile usare il costrutto.

```
use path::del::modulo::*;
```

Questo rende disponibile, nel file corrente, la definizione di tutti i simboli pubblici del modulo indicato.

Per poter accedere ad un simbolo occorre che tutti gli elementi del cammino siano accessibili a chi ne fa richiesta, ovvero essere dichiarati come pubblici o contenere il modulo corrente.

Un modulo può esportare tutti i simboli pubblici di un proprio sottomodulo come fossero propri con il costrutto `pub use submodule::*;`. Questo facilita la costruzione di librerie in cui viene semplificata l'API pubblica, nascondendo i dettagli della struttura interna.

Inoltre facilita l'evoluzione del codice, permettendo di modificare la struttura interna dei moduli senza modificare il codice che dipende da essi, mantenendo invariata l'interfaccia pubblica.

5.6 Moduli e file sorgente

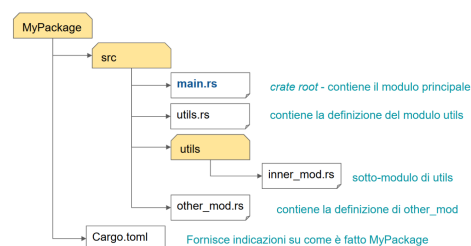
Il codice relativo ad un sotto-modulo può essere inserito:

- Nello stesso file sorgente del modulo genitore, con la notazione `mod nome_modulo{...}`
- In un file sorgente a sé, presente nella stessa cartella, chiamato `nome_modulo.rs`, in questo caso occorre che il modulo genitore sia presente la dichiarazione del modulo, nel formato `mod nome_modulo;`
- Nella sotto-cartella chiamata `nome_modulo` possono essere collocati i file sorgente di eventuali sotto-moduli di `nome_modulo`

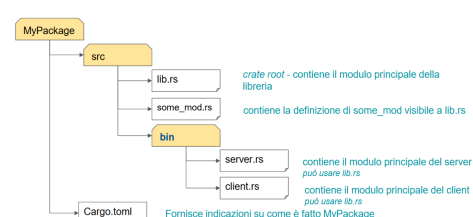
Eventuali crate esterni, non facenti parte della gerarchia di moduli definiti dal package corrente, vengono elencati nel file `cargo.toml` che descrive la struttura del package stesso.

Nella sezione `[dependencies]` sono presenti, uno per riga, i nomi dei crate da importare con l'indicazione della relativa versione

5.6.1 Struttura di un progetto eseguibile Rust



5.6.2 Struttura di un progetto con due o più eseguibili



5.7 Librerie

Una libreria è un crate che non produce un programma eseguibile, ma fornisce funzionalità riutilizzabili sotto forma di funzioni, strutture dati, tratti e moduli che possono essere utilizzati da altri crate.

Le librerie permettono di:

- **Riutilizzare** codice in più programmi
- **Separare** la logica dell'applicazione dall'interfaccia eseguibile
- **Organizzare** meglio progetti complessi
- **Condividere** funzionalità tra progetti con linguaggi diversi

Una libreria espone una API (Application Programming Interface), cioè un insieme di elementi pubblici che possono essere utilizzati dall'esterno. Gli elementi `pub` fanno parte dell'API, mentre gli elementi privati sono dettagli di implementazione.

5.8 Librerie interne ed esterne

Le librerie possono essere interne al package o esterne ad esso.

Nota: Un package può contenere una sola libreria interna (`src/lib.rs`), ma fare riferimento a molte librerie esterne

Le librerie esterne sono elencate nella sezione `[dependencies]` del file `Cargo.toml`. Ciascuna riga indica il nome della libreria, la versione desiderata ed eventuali opzioni di configurazione. All'atto della costruzione di un progetto, le librerie esterne vengono automaticamente scaricate dal sito <https://www.crates.io> e compilate a partire dai loro sorgenti. Solo dopo che tutte le dipendenze sono state compilate viene compilato il crate dell'applicazione.

Una libreria esterna viene ricompilata solo se necessario, ad esempio quando:

- Cambia la versione della dipendenza
- Cambiano le opzioni di compilazione
- Cambia il target o il profilo di build (`debug` / `release`)

5.8.1 Librerie ed interoperabilità

Le librerie permettono a programmi scritti in linguaggi diversi di riutilizzare lo stesso codice compilato. In Rust è possibile creare librerie compatibili con altri linguaggi, come C, C++ o Python, esportando funzioni tramite un'interfaccia compatibile con l'ABI del C.

Il meccanismo funziona come segue:

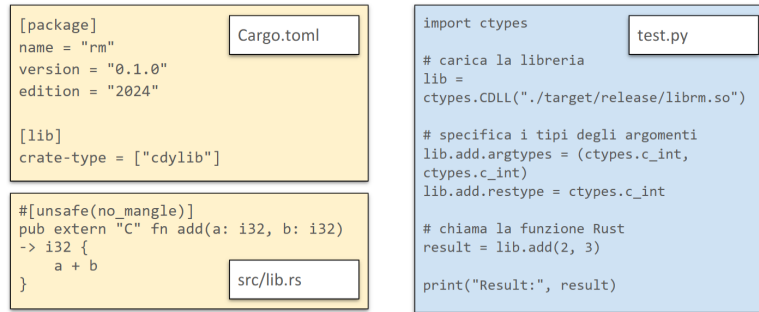
- Il codice Rust viene compilato come libreria condivisa (`.so` / `.dll`) o statica (`.a` / `.lib`)
- Le funzioni esportate seguono la convenzione di chiamata C (*C ABI*)
- Programmi scritti in altri linguaggi possono caricare e invocare queste funzioni

Una libreria può essere collegata in due modalità:

- **Linking statico** — il codice della libreria viene incorporato direttamente nell'eseguibile durante la compilazione. Il risultato è un singolo `.exe` autosufficiente, ma di dimensioni maggiori.
- **Linking dinamico** — durante la compilazione viene inserito un riferimento (*stub*) alla libreria esterna. Al momento dell'avvio del programma, linking, il sistema operativo individua e carica in memoria la libreria richiesta (`.so` o `.dll`).

Le applicazioni tipiche sono:

- Usare Rust come motore ad alte prestazioni per componenti critici
- Integrare Rust in software esistente scritto in C/C++
- Creare binding per linguaggi di scripting come Python o JavaScript



Nota: `#![no_mangle]` dice al compilatore Rust di non modificare il nome della funzione durante la compilazione, rendendola così accessibile da altri linguaggi con il nome esatto specificato nel codice sorgente. Senza questo attributo, il compilatore applica il `name mangling`, una tecnica che trasforma i nomi delle funzioni includendo informazioni aggiuntive come il percorso del modulo e un hash univoco, producendo nomi come `_ZN7mycrate3add17h3f6a1c2b8e4d5f9eE`, illeggibili e non invocabili direttamente da altri linguaggi.

5.9 Collegamento delle librerie

La sezione `[lib]` del file `Cargo.toml` che descrive il progetto contiene dettagli relativi al tipo di libreria che si intende creare. La chiave `crate-type` può assumere i seguenti valori:

- **rlib**: libreria statica Rust, valore di default, il file risultante può solo essere usato da altri eseguibili Rust che ne includeranno le parti importate direttamente nel proprio codice binario
- **dylib**: libreria dinamica Rust; viene trasformata in un file `.dll` in windows, `.so` in linux, `.dylib` in macOS; può essere utilizzata solo da codice Rust, ma sarà caricata in fase di esecuzione
- **cdylib**: libreria dinamica conforme alle specifiche C; viene trasformata in un file `.dll` in windows, `.so` in linux, `.dylib` in macOS; può essere usata da eseguibili scritti in altri linguaggi; se viene usata da un programma Rust deve essere inclusa sotto forma di funzioni esterne
- **staticlib**: libreria statica conforme alle specifiche C; viene trasformata in un file `.lib` in windows e `.a` in linux e macOS; può essere usata da eseguibili scritti in altri linguaggi; se viene usata da un programma Rust deve essere inclusa sotto forma di funzioni esterne

5.10 Preludio

Come in molti linguaggi di programmazione, anche in Rust esiste un insieme di simboli, funzioni, tipi e tratti, come `Vec` e `String`, che sono sempre disponibili in ogni modulo senza includere esplicitamente alcuna dichiarazione anche se facenti parte di crate diversi dal crate corrente.

Questi sono elencati all'interno di un modulo chiamato `std::prelude` che viene importato automaticamente all'atto della compilazione di un file sorgente. In questo modo si evita di rendere verboso il codice sorgente.

Ogni versione di Rust viene con un proprio prelude. Cargo provvede ad includere quello corretto in funzione del valore dell'attributo `package/edition` nel file `Cargo.toml`.

5.11 Clap - libreria per la gestione dei parametri da linea di comando

La libreria clap gestisce in modo dichiarativo i parametri passati attraverso la linea di comando.

La si include in un crate aggiungendo, nel file Cargo.toml, questa dipendenza:

```
[dependencies]
clap = { version = "4.5", features = ["derive"] }
```

Questo mette a disposizione un insieme di macro e di strutture dati che permettono di descrivere una struttura dati in cui verranno depositati i valori estratti dalla riga di comando, così come di derivare automaticamente una funzione di analisi che provvede a valorizzare i campi di tale struttura.

Questa libreria permette anche di esprimere programmaticamente l'insieme dei parametri, la tipologia di valori associati e gli eventuali vincoli associati, basata sul pattern builder.

```
use clap::Parser;
/// Simple program to greet a person
#[derive(Parser, Debug)]
#[command(version, long_about = None)]
struct Args {
    /// Name of the person to greet
    #[arg(short, long)]
    name: String,
    /// Number of times to greet
    #[arg(short, long, default_value_t = 1)]
    count: u8,

    //Può essere specificata più volte
    #[arg(long)]
    filter: Option<Vec<String>>,
}

fn main() {
    let args = Args::parse();
    for _ in 0..args.count {
        println!("Hello {}", args.name)
    }
}
```

```
$ demo --help
Simple program to greet a person

Usage: demo[EXE] [OPTIONS] --name <NAME>

Options:
  -n, --name <NAME> Name of the person to greet
  -c, --count <COUNT> Number of times to greet [default:
    ↪ 1]
  -h, --help Print help
  -V, --version Print version

$ demo --name Me
Hello Me!
```

Capitolo 6

Polimorfismo

6.1 Polimorfismo: cos'è e perché serve

Sebbene il processo di analisi di un dominio applicativo tenda a favorire la sua suddivisione in una molteplicità di tipi distinti (classi, strutture, funzioni, ecc.), la necessità di minimizzare il codice scritto spinge verso l'identificazione di comportamenti comuni, che possano essere condivisi tra tali tipi per unificare la struttura del codice ¹

La soluzione individuata è il **polimorfismo**: capacità di associare comportamenti comuni ad un insieme di tipi differenti, così da rendere più compatta e razionale la struttura del codice

6.2 Implementare il polimorfismo

Linguaggi di programmazione diversi hanno introdotto vari modi per implementare il polimorfismo:

- Può basarsi sul paradigma della **programmazione generica**, dove è possibile formulare funzioni o strutture dati in cui uno o più tipi non sono specificati per nome, ma tramite simboli astratti, abilitando così la scrittura di codice in grado di operare con una molteplicità di tipi
- può sfruttare la definizione di **interfacce** comuni, che possono essere implementate dai singoli tipi. Questo consente di fare riferimento all'interfaccia invece che al tipo concreto
- Nei linguaggi che implementano il concetto di **ereditarietà**, il polimorfismo si può realizzare derivando più classi concrete da una super-classe in cui è definito il comportamento comune

6.3 Il polimorfismo in Rust si basa sui tratti

In Rust un tratto, *trait*, descrive un comportamento:

- Un valore può essere stampato
- Due valori omogenei possono essere confrontati tra loro
- Il contenuto di una collezione può essere esaminato un elemento alla volta

Un tipo, predefinito o definito dall'utente, può implementare zero, uno o più tratti acquisendo la capacità di eseguire i corrispondenti comportamenti e rendendo nota questa capacità al compilatore che ne potrà usufruire per generare e ottimizzare il codice.

I tratti in Rust costituiscono l'equivalente concettuale delle interfacce di Java e C#, infatti definiscono un **insieme di metodi**, eventualmente associando loro un'implementazione di default.

¹Ciò sta alla base del principio DRY - Don't Repeat Yourself.

Esempio I tratti abilitano costrutti reali:

- `println!("{}", x)` possibile se il tipo di `x` implementa il tratto `Display`
- `println!("{:?}", x)` possibile se il tipo di `x` implementa il tratto `Debug`
- `for x in v { ... }` possibile se il tipo di `x` implementa il tratto `IntoIterator`
- `a == b` possibile se il tipo di `a` e `b` implementa il tratto `PartialEq`

Una porzione significativa della sintassi di Rust è basata sul concetto di tratto e su un insieme di tratti che fanno parte della libreria standard. Oltre a questi tratti predefiniti, possono essere affiancati tratti definiti dall'utente.

6.3.1 Definire un tratto

```
trait Validatable {
    fn validate(&self) -> bool;
}
```

Un tratto definisce un **comportamento**, non un tipo concreto. In questo caso, stiamo dicendo che i tipi che implementeranno il tratto `Validatable` dovranno avere un meccanismo - la funzione `validate(...)` - volto a stabilire se il loro contenuto è valido, secondo una qualche interpretazione legata al dominio che modellano, o meno.

6.3.2 Implementare un tratto per un dato tipo

Il costrutto `impl <trait> for <type> { ... }` e specifica, al tempo stesso, l'implementazione dei metodi corrispondenti.

Tale costrutto è separato da quello relativo all'implementazione dei metodi peculiari al tipo:

```
impl <type> { ... }
```

```
struct User {
    name: String,
    age: u8;
}
impl Validatable for User {
    fn validate(&self) -> bool {
        self.name.trim().len() > 0 &&
        self.age >= 18
    }
}

impl User { /* metodi di User */ }
```

6.3.3 Un tratto può essere implementato da molti tipi

Lo stesso comportamento viene realizzato in modi differenti in base al tipo specifico, ciò che resta comune è l'insieme dei metodi offerti dal tratto e la loro firma.

```
struct Order {
    total: f64,
    quantity: u32
}
impl Validatable for Order {
    fn validate(&self) -> bool {
        self.total > 0 &&
        self.quantity > 0
    }
}
impl Order { /* ... */ }
```

6.3.4 Un tratto può essere implementato dai tipi esistenti

È lecito che un tipo esistente implementi un tratto definito dall'utente.

Rust impone regole precise su dove sia lecito scrivere un'implementazione, per evitare conflitti

```
impl Validatable for String {
    fn validate(&self) -> bool {
        self.trim().len() > 0
    }
}
impl Validatable for i32 {
    fn validate(&self) -> bool {
        *self > 0
    }
}
```

6.3.5 Invocare i metodi di un tratto

Se il tipo concreto di un valore è noto, chiamare i metodi di un tratto non ha differenze rispetto a chiamare i metodi specifici del tipo.

Non solo sul piano sintattico, ma anche su quello del codice generato.

La chiamata viene risolta staticamente e non introduce penalizzazioni

```
let u: User = User {
  name: "Mario".to_string(),
  age: 32
};
let o: Order = Order {
  total: 1200.0,
  quantity: 8
};
u.validate();
o.validate();
```

6.4 I tratti come vincoli

```
fn check<T: Validatable>(item: &T) {
  println!("{}", item.validate());
}
```

Questa funzione accetta come parametro solo valori il cui tipo implementa il tratto *Validatable*.

Tentativi di passare a questa funzione tipi privi del tratto danno origine ad un errore in fase di compilazione.

La conoscenza specifica del tipo, rappresentato dalla meta-variabile T, permette al compilatore di ottimizzare il codice generato

6.4.1 Due modi di usare un tratto

Come vincolo su un tipo generico. Quando il tipo concreto di un valore è noto al compilatore, la chiamata viene risolta staticamente. Questo è il caso più comune

```
fn process1<T: Validatable>(x: &T) {
  println!("{}", x.validate());
}
```

Come oggetto-tratto con invocazione dinamica. Se il tipo concreto è nascosto dietro l'interfaccia, la scelta del metodo avviene in fase di esecuzione. La parola chiave *dyn* indica un oggetto-tratto, ovvero ad un puntatore di cui si sa solo che punta ad un dato che implementa il tratto

```
fn process2(x: &dyn Validatable) {
  println!("{}", x.validate());
}
```

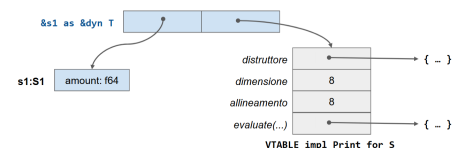
6.5 Invocazione dinamica

La parola chiave *dyn* segnala l'uso del tratto come oggetto-tratto. I metodi vengono risolti in fase di esecuzione. Questo mette in evidenza al programmatore che il programma dovrà utilizzare il meccanismo di invocazione dinamica (dynamic dispatch)

```
fn process(x: &dyn Validatable) {
  println!("{}", x.validate());
}
fn main() {
  let u = User { ... };
  let o = Order { ... };
  process(&u);
  process(&o);
}
```

6.6 Oggetti-tratto

I riferimenti/puntatori ai tipi tratto vengono detti oggetti-tratto. Possono essere condivisi o mutabili e devono rispettare le regole dell'esistenza in vita del valore a cui fanno riferimento. Gli oggetti-tratto vengono implementati tramite fat pointer.



6.7 Vincoli degli oggetti-tratto

Non tutti i tratti possono essere referenziati dinamicamente. Il compilatore deve, infatti, essere in grado di costruire la vtable associata al tratto.

Un tratto è compatibile con l'invocazione dinamica se i suoi metodi soddisfano i seguenti vincoli:

- Operano su `&self` o `&mut self` sono metodi a livello istanza che non consumano il ricevitore
- Non restituiscono `Self`
- Non contengono parametri generici

6.8 Tratti con implementazioni di default

Nella definizione di un tratto è lecito indicare, per una data funzione, un'implementazione di default.

Le funzioni che implementano il tratto saranno libere di adottarla o potranno sovrascriverla con altro codice, purché venga rispettata la firma delle funzione, tipo dei parametri e del valore di ritorno.

```
trait T4 {
    fn f() { println!("default"); }
}
```

```
struct OtherType;
impl T4 for OtherType {
    fn f() { println!("Other"); }
}
```

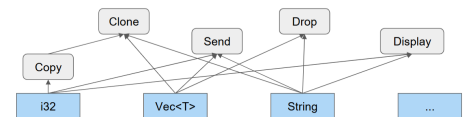
```
struct SomeType;
impl T4 for SomeType { }
//uso dell'implementazione di default
```

```
fn main() {
    SomeType::f(); // default
    OtherType::f(); // Other
}
```

6.9 Tipi e tratti

Poiché tipi diversi possono implementare tratti comuni, si viene a creare una forma di “parentela” alquanto articolata tra tipi.

Rust introduce una ventina di tratti predefiniti, cui il compilatore associa un particolare significato, e permette al programmatore di aggiungerne altri a piacere, al fine di estendere tale comportamento.



Nota: In Rust i tratti non classificano i tipi in una gerarchia unica, ma descrivono proprietà e comportamenti trasversali

6.10 Programmazione generica

Notazione che permette di scrivere funzioni, strutture dati e tratti che utilizzano diversi tipi di dati senza duplicare il codice, tramite parametri generici. Consente di mantenere il codice flessibile, riutilizzabile e sicuro grazie al controllo dei tipi in fase di compilazione.

In Rust la programmazione generica si combina naturalmente con i tratti, che permettono di vincolare i tipi generici a comportamenti specifici.

```
fn first<T>(slice: &[T]) -> Option<T> {
    slice.get(0)
}
```

Nota: T parametro generico che rappresenta un tipo

6.11 Funzioni generiche

Si può definire una funzione in modo che operi su un tipo di dato non ancora precisato. Rappresentato da un segnaposto racchiuso tra `<` e `>`. T è generico ma non arbitrario: deve implementare il tratto `PartialOrd`

```
fn max<T>(t1: T, t2: T) -> T
where T: PartialOrd {
    if t1 > t2 { t1 }
    else { t2 }
}
```

6.12 Dall'astrazione al codice concreto

Al momento della chiamata, il compilatore:

- Deduce il tipo di `T`
- Controlla che soddisfi tutti i vincoli cui è soggetto
- Genera una versione specializzata della funzione per quel tipo

Se la funzione generica viene invocata in più punti del programma con tipi diversi. Il compilatore genererà tante versioni della funzione quanti sono i tipi con cui viene utilizzata.

Questo processo è detto **monomorfizzazione**.

6.13 Tipi generici

Si possono usare costrutti generici anche per definire un tipo composto, struct, tupla, enum, basato su parti variabili.

Ciascuna meta-variabile può essere soggetta ad eventuali restrizioni, indicate come l'insieme dei tratti che deve implementare e/o come il tempo di vita che deve garantire.

Se una struttura generica implementa dei metodi, i nomi delle meta-variabili ed i vincoli cui sono soggette vengono ripetuti nel blocco `impl`.

```
struct MyStruct<T>
  where T: SomeTrait {
    value: T
  }

impl<T> MyStruct<T>
  where T: SomeTrait {
    fn process(&self) { /.../ }
```

6.14 Sintassi dei tipi generici

Rust offre due modi per specificare la sintassi dei vincoli sui tipi generici:

- Una versione compatta `<T: SomeTrait>`
- La versione estesa `<T> ... where T: SomeTrait`

In entrambi i casi, se è necessario indicare che il tipo deve implementare più tratti, questi possono essere combinati con il segno `+`.

```
struct MyStruct<T>
  where T: SomeTrait {
    value: T
  }

impl<T> MyStruct<T>
  where T: SomeTrait {
    fn process(&self) { /.../ }
```

6.15 Tratti con tipi associati

Un tratto può associare uno o più tipi al comportamento che descrive. Il tipo associato fa parte della definizione del tratto. Chi implementa il tratto deve specificarne il valore concreto. Questo è utile quando un comportamento dipende anche da un tipo collegato.

```
trait Container {
  type Item;

  fn get(&self) -> &Self::Item;
}

struct IntBox(i32);

impl Container for IntBox {
  type Item = i32;

  fn get(&self) -> &Self::Item {
    &self.0
  }
}
```

Nota: Un tratto non descrive solo metodi, ma anche tipi associati al comportamento

6.16 Tipi associati e generics

I tipi associati possono dipendere da un parametro generico del tipo che implementa il tratto:

- Il tratto definisce il tipo associato
- L'implementazione concreta ne specifica il valore
- Se il tipo che implementa il tratto è generico, anche il tipo associato può dipendere da quel parametro

```
trait Container {
    type Item;

    fn get(&self) -> &Self::Item;
}

struct Boxed<T>(T);

impl<T> Container for Boxed<T> {
    type Item = T;

    fn get(&self) -> &Self::Item {
        &self.0
    }
}
```

Note: Il tipo associato in questo caso coincide con il parametro generico T

6.17 Super-tratti

Un tratto può richiedere che chi lo implementa implementi anche un altro tratto

Se `TrattoB: TrattoA`, allora ogni tipo che implementa `TrattoB` deve implementare anche `TrattoA`. Questo permette di costruire tratti più specifici a partire da tratti più generali

```
trait Printable{
    fn print(&self);
}

trait AdvancedPrintable: Printable {
    fn print_twice(&self) {
        self.print();
        self.print();
    }
}
```

6.18 &dyn Trait e T: Trait a confronto

& dyn Trait	T: Trait
Il tipo concreto è nascosto dietro l'oggetto-tratto	Il tipo T è noto al compilatore
Invocazione dinamica	Invocazione statica
Nessuna monomorfizzazione	Codice specializzato tramite monomorfizzazione
Richiede tratti object-safe	Possibile combinare più vincoli con +
Adatto quando si vuole un'unica interfaccia per tipi diversi	Adatto quando conta l'efficienza e l'eterogeneità del codice generato non è un problema

Nota: In Rust il polimorfismo dinamico non è implicito: si usa `dyn` solo quando serve davvero. Nel caso normale, i tratti vengono sfruttati tramite tipi generici e invocazione statica.

6.19 Approfondimento: tratti nella libreria standard

Rust definisce un insieme di tratti la cui implementazione abilita l'uso degli operatori presenti nel linguaggio con i tipi definiti dall'utente.

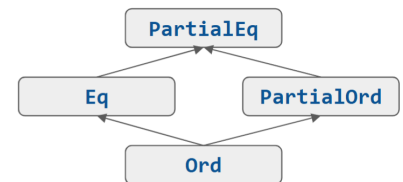
L'implementazione di alcuni di questi tratti può essere aggiunta automaticamente ai tipi definiti dall'utente attraverso la macro `#[derive]`.

```
#[derive(PartialEq)]
struct Foo<T> {
    a: i32,
    b: T,
}

impl<T: PartialEq> PartialEq for Foo<T> {
    fn eq(&self, other: &Foo<T>) -> bool {
        self.a == other.a && self.b == other.b
    }
}
```

6.20 Confronto tra valori

- **PartialEq**: Tratto che permette di verificare l'uguaglianza, `==` e `!=` tra valori. Non richiede che tutti i valori siano confrontabili come ad esempio il NaN nei float
- **Eq**: Estende **PartialEq**. Indica che l'uguaglianza è totale. Ogni valore è confrontabile con sé stesso.
- **PartialOrd**: Permette confronti di ordine, `<`, `>`, `<=`, `>=`. Può essere parziale, alcuni confronti non sono definiti
- **Ord**: Estende **PartialOrd** ed **Eq**. Definisce un ordinamento totale, tutti i valori sono confrontabili.



6.21 Visualizzare i contenuti

Il tratto **Display** crea una visualizzazione comprensibile ad un utente finale. Questo tratto deve essere implementato manualmente e da un punto di vista pratico, la sua implementazione richiede la definizione di un solo metodo.

```
trait Display {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result;
}
```

Il tratto **Debug** ha la stessa firma di **Display**. Il suo scopo, tuttavia, è differente: creare una rappresentazione comprensibile al programmatore del valore.

- Viene attivato usando il segnaposto `{:?}` nelle macro di stampa
- Può essere sintetizzato automaticamente attraverso l'annotazione `#[derive(Debug)]`

6.22 Copia e duplicazione

6.22.1 Clone

Il tratto **Clone** indica la capacità di un tipo di creare un duplicato di un valore dato. Può essere usato per trasformare un riferimento, `&T` in un valore posseduto, `T`.

Tale trasformazione, ovviamente, può essere molto costosa in termini di tempo e memoria, tuttavia è sempre sotto il controllo del programmatore perché tale funzionalità non è mai attivata automaticamente.

6.22.2 Copy

Il tratto **Copy** è un sotto-tratto di **Clone**.

- Esso implica il fatto che la copia ottenuta sia - a tutti gli effetti - il solo duplicato dei bit presenti nel valore originale, senza nessuna trasformazione
- La sua implementazione può essere veloce ed efficiente, infatti si basa sulla duplicazione bit-a-bit del valore originale
- La presenza del tratto **Copy** trasforma la semantica delle operazioni di assegnazione. Quello che normalmente determina un movimento, che rende inaccessibile il valore originale, diventa una copia.

6.23 Rilasciare le risorse

Il tratto **Drop** permette di definire il metodo `drop(&mut self)` che fa le veci del distruttore nel linguaggio C++.

Il compilatore Rust garantisce che tale metodo sarà chiamato nel momento in cui la variabile che contiene un valore di questo tipo uscirà dal proprio scope sintattico o subito prima che le venga assegnato un nuovo valore.

Questo permette di rilasciare eventuali risorse possedute dalla struttura dati o abilitare comportamenti duali a quelli della creazione dell'oggetto, tipici del pattern *RAII - Resource Acquisition Is Initialization*.

Questo tratto è mutuamente esclusivo con il tratto **Copy**. Questo vincolo elimina, in assenza di blocchi `unsafe { ... }`, la possibilità di avere un doppio rilascio delle risorse presenti sullo heap.

Questo tratto si accompagna alla funzione globale `fn drop<T>(_x:T) {}`. Essa forza il passaggio di possesso di un valore ad una nuova variabile (`_x`) che uscirà di scena subito, provocandone la distruzione.

6.24 Conversione tra tipi

I tratti **From** e **Into** permettono di effettuare conversioni di tipo, prendono possesso del valore lo convertono e ritornano il possesso al chiamante.

I tratti sono perfettamente duali, scrivere `T: From<i32>` equivale a scrivere `i32: Into<T>`.

Se si implementa **From**, il tratto **Into** viene generato automaticamente.

L'implementazione del tratto **From** NON È SIMMETRICA: se è possibile passare dal tipo **T** al tipo **U**, non è affatto detto che sia possibile tornare indietro, da **U** a **T**.

```
trait From<T>: Sized {
    fn from(other: T) -> Self;
}
```

```
trait Into<T>: Sized {
    fn into(self) -> T;
}
```

Il tratto **FromStr** permette di gestire la conversione da stringa e l'eventuale fallimento. Il metodo `from_str()` viene implicitamente richiamato tutte le volte che si utilizza il metodo `parse`

```
pub trait FromStr{
    type Err;
    fn from_str(s: &str) -> Result<Self, Self::Err>;
}
```

Capitolo 7

Test

7.1 Test

La parte di testing può solo confermare la presenza di difetti, bug, ma non la loro assenza. Perché testare allora?

- Un test verifica automaticamente il comportamento del codice
- Non dimostra l'assenza di errori
- Aumenta la confidenza nel sistema
- Previene regressioni nel tempo

I test sono fondamentali per mantenere codice di qualità.

Esempio: problema reale - divisione per zero

```
fn divide(a: i32, b: i32) -> i32 {  
    a / b  
}
```

Funziona fino a che $b \neq 0$. Questo caso limite, a run-time, farà andare in panic il programma. Per accorgercene automaticamente dobbiamo introdurre il concetto di *unit test*, test di unità.

Ci sono molte tipologie di test. La più semplice, fornita direttamente dall'IDE di sviluppo, è l'analisi statica del programma, che ci avverte in caso di variabili non usate, errori di sintassi, etc.

7.2 Test di unità

Un test di unità, o **unit test**, valuta, in maniera isolata, il comportamento di un singolo componente software, funzione, struttura dati, etc, indipendentemente dal resto del sistema. Ne sollecita il comportamento tipico, ovvero conforme all'uso per cui è stato progettato, sia quello agli estremi o oltre il suo previsto capo di utilizzo.

Tale sollecitazione viene esercitata tramite frammenti di codice che invocano il componente e verificano che esso risponda in modo atteso, confrontando i valori ritornati dalla funzione/contenuti nella struttura dati dopo la sollecitazione con valori/condizioni di essere attesi sulla base delle specifiche.

Nel caso di Rust, troviamo la macro **assert** e **assert_eq**, che verificano che l'aspettativa sia soddisfatta o meno.

Nell'ambiente Cargo, i test vengono eseguiti con il comando **cargo test**.

```
#[test]  
fn test_divide() {  
    assert_eq!(divide(6,3), 2);  
}
```

Nota: Verifica un caso tipico ma non copre errori, un test singolo non basta!

Proprio per questo motivo dobbiamo andare a testare i casi limite.

Gli unit test sono considerati test with box, ovvero test che si basano sulla conoscenza del codice sorgente, e normalmente sono scritti ed eseguiti direttamente dal programmatore. In questo caso siamo in grado di predire che determinanti parametri in ingresso portano a un certo risultato in uscita.

- Presuppongono la conoscenza dei meccanismi interni al componente, allo scopo di verificare come questi reagiscono in presenza di casi limite
- Il loro numero dipende dalla complessità ciclomatica del modulo testato
- La loro qualità è alla base del processo di refactoring cui un artefatto software tipicamente è soggetto nel corso del suo naturale ciclo di vita

Il costo delle modifiche che vengono evidenziate da questo tipo di test è normalmente basso. Tuttavia i benefici che la scrittura di test ben fatti portano alla qualità del codice sono alti e compensano abbondantemente nel medio termine i maggiori costi dovuti alla scrittura dei test.

7.2.1 Quanti test fare

Il loro numero dipende dalla complessità ciclomatica ¹ del modulo testato.

Meglio due funzioni semplici in cascata che una unica complessa. Spesso si utilizza il refactoring per semplificare il codice.

Nota: I test sono importanti averli dall'inizio così da specificare la funzionalità del software e arricchire la documentazione.

7.3 Test dei casi limite

È fondamentale testare i casi limite. Per questo Rust offre la possibilità di scrivere `#[should_panic]` avvertendo il compilatore che ci aspettiamo che la funzione vada in panic. Verifica che il codice fallisca quando deve effettivamente fallire.

```
#[test]
#[should_panic]
fn test_divide_by_zero() {
    divide(10,0);
}
```

I test devono verificare:

- I casi normali
- I casi limite come divisione per 0, accesso al primo e all'ultimo elemento di un contenitore, etc.
- La corretta identificazione/gestione/propagazione degli errori

Note: I test servono anche a testare errori!

7.4 Refactoring: miglioriamo il codice

Nel momento in cui dobbiamo testare una funzione vi sono due possibilità

- Le specifiche non sono chiare, non sappiamo come gestire un errore. Con il linguaggio naturale sarebbe difficile esprimere tutte le casistiche

¹Complessità di un programma. Misura direttamente il numero di cammini linearmente indipendenti attraverso il grafo di controllo di flusso.

- Quando mi rendo conto che il comportamento non può essere determinato dal cliente finale, dato che è una scelta progettuale. A questo punto si è costretti a riflettere come migliorare il codice (ritornare un Option o un Result al posto di un intero, etc.)

```
fn divide(a: i32, b: i32) -> Option<i32> {
    if b == 0 { None }
    else { Some(a / b) }
}

#[test]
fn test_divide_safe() {
    assert_eq!(divide(2,1), Some(2));
    assert_eq!(divide(1,0), None);
}
```

Questo processo ha reso il nostro codice più safe.

Note: Niente panic e questo implica un'API più sicura

7.5 Test di integrazione

Fino ad ora abbiamo visto i singoli test, ma il nostro sistema è composto da varie funzioni che ne chiamano altre e che interagiscono tra loro.

Un test di integrazione valuta il comportamento all'interfaccia di due moduli software. Si focalizza sul determinare la correttezza di tale interfaccia, evidenziando le incongruenze tra le parti che stanno interagendo.

Come nel caso dei test di unità, è costituito da frammenti di codice che sollecitano l'interazione tra due parti:

- A differenza dei test di unità, tuttavia, ignora volutamente la struttura interna dei moduli che stanno interagendo, assumendo il loro corretto comportamento - sul piano individuale - alle specifiche definite. Infatti parliamo di test a black box.
- Per questo motivo viene eseguito solo su oggetti che hanno correttamente superato il test di unità

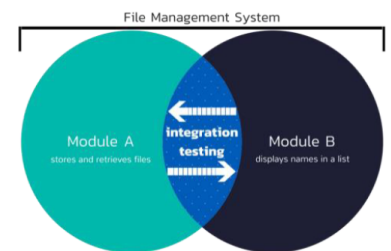
I costi delle modifiche che occorre introdurre se un test di integrazione fallisce sono generalmente più elevati. In quanto richiedono il ridisegno di una o entrambe le parti che interagiscono.

Questa tipologia di test sono co-progettati da sviluppatori e tester, ma eseguiti solo dagli ultimi.

Esempio: Il modulo A rappresenta la persistenza con l'accesso al database; mentre il modulo B rappresenta la logica applicativa.

I test di integrazione possono verificare che:

- I dati presenti nel modulo di persistenza siano correttamente interpretati dalla logica applicativa
- Gli errori del livello di persistenza siano propagati correttamente
- Le transazioni siano coerenti



7.5.1 Test di integrazione: modulo Repository

```
pub struct OrderRepository {
    data: HashMap<u32, f64>;
}

impl OrderRepository {
    pub fn new() -> Self {
        Self {data: HashMap::new()}
    }
    pub fn save(&mut self, id: u32, v: f64) {
        self.data.insert(id, v);
    }
    pub fn find(&self, id: u32) -> Option<f64>{
        self.data.get(&id).copied()
    }
}
```

Modulo base che offre la capacità di memorizzare copie chiave/valore. E di reperire il valore data la chiave

7.5.2 Test di integrazione: modulo Service

```
pub struct OrderService<'a> {
    repo: &'a mut OrderRepository
}
impl<'a> OrderService<'a> {
    pub fn new(repo: &'a mut OrderRepository) -> Self { Self { repo } }
    pub fn apply_discount(&mut self, id: u32) -> Result<f64, String> {
        let t = self.repo.find(id).ok_or("Order not found")?;
        let discounted = if t > 100.0 { t * 0.9 }
                        else { t };
        self.repo.save(id, discounted);
        Ok(discounted)
    }
}
```

Contiene la logica applicativa. Si appoggia sulle funzionalità del modulo precedenti.

7.5.3 Test di integrazione: esempio di test

```
// caso normale
#[test]
fn test_apply_discount_integration() {
    // Arrange
    let mut repo = OrderRepository::new();
    repo.save(1, 200.0);
    let mut s = OrderService::new(&mut repo);
    // Act
    let result = s.apply_discount(1);
    // Assert
    assert!(result.is_ok());
    let saved = repo.find(1);
    assert_eq!(saved, Some(180.0));
}

// caso limite
#[test]
fn test_apply_discount_not_found() {
    // Arrange
    let mut repo = OrderRepository::new();
    let mut s = OrderService::new(&mut repo);
    // Act
    let result = s.apply_discount(42);
    // Assert
    assert!(result.is_err());
}
```

7.5.4 Test di integrazione: razionale

- Coinvolgono più elementi
- Verificano il flusso completo, lo stato condiviso e la gestione degli errori
- Non verificano dettagli interni alle strutture, black box test, bensì verificano il comportamento globale

Unit test: Verifica una singola funzione

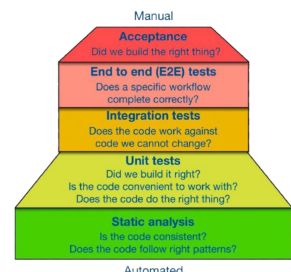
Integration test: Verifica che il sistema funzioni davvero insieme

7.6 Test complessivi -end-to-end test

I test end-to-end valuta il comportamento del prodotto finito, sia nei suoi aspetti, cosa deve/non deve fare, che non funzionale come aspetti quali sicurezza, affidabilità, etc.

È basato su modelli di comportamento tratti da casi d'uso reali. Progettato ed eseguito da tester, allo scopo di non farsi influenzare dalle assunzioni fatte durante lo sviluppo.

Questi test verificano che il sistema sia conforme alle aspettative del committente/cliente il quale è lui ad eseguire i test.



7.7 Cosa fanno i test

- I test trovano i bug ma non possono dimostrare l'assenza di essi
- Documentano il comportamento del software, anche prima che il software stesso sia stato scritto, perché permettono di specificare il comportamento atteso

- Migliorano il design, in quanto portano a riflettere sulla struttura del codice. Riflettendo sul comportamento di un dato modulo, è possibile identificare debolezze strutturali e mancate specifiche

Nota: I test aiutano ad aumentare la confidenza nel codice

7.8 Il flusso di lavoro con i test

1. Si scrive una funzione
2. Si scrivono i test che ne verificano il comportamento
3. Si esegue cargo test
4. Si correggono gli errori che vengono riportati
5. Si ripete fino a che il programma è completo

7.9 Test in Rust

I **test di unità** possono essere scritti nel modulo da testare o, meglio, in un suo sotto-modulo chiamato convenzionalmente 'tests'.

Il sotto-modulo contenente i test è preceduto dall'annotazione `#[cfg(test)]`. Questo informa il compilatore che il codice contenuto al suo interno deve essere incluso solo quando la compilazione avviene con il comando `cargo test`.

I **test di integrazione** sono invece contenuti in una cartella separata denominata `tests`, a lato della cartella `src` del progetto.

- Sono scritti come se i singoli frammenti di codice fossero i consumatori del crate che deve essere testato
- I file sorgente all'interno della cartella di test importano i simboli pubblici oggetto di valutazione tramite istruzioni di tipo `use...`
- Solo le librerie possono avere test di integrazione: per questo si creano programmi formati da un eseguibile contenente un `main` banale e da una libreria che contiene la logica applicativa

7.9.1 Sintassi dei test

```
// funzione da testare
fn sum(a: i32, b: i32) -> i32
{
    a + b
}

#[cfg(test)]
mod tests {
    fn sum_inputs_outputs() -> Vec<((i32, i32), i32)> {
        vec![(1, 1), (0, 0), (2, -2)]
    }
    #[test]
    fn test_sums() {
        for (input, output) in sum_inputs_outputs() {
            assert_eq!(crate::sum(input.0, input.1), output);
        }
    }
}
```

La struttura generale di una funzione etichettata con `#[test]` è la seguente **struttura AAA**:

- **Arrange**: Preparazione dei valori su cui operare
- **Act**: Esecuzione del codice da testare
- **Assert**: Verifica, mediante asserzioni, che i risultati ottenuti siano

```
#[test]
fn test_divide_aaa() {
    // Arrange
    let a = 6;
    let b = 2;
    // Act
    let result = divide(a, b);
    // Assert
    assert_eq!(result, 3);
}
```

7.9.2 Asserzioni

Rust offre varie macro procedurali per verificare le asserzioni, tra cui

- `assert!( boolean_condition )` - impone che la condizione sia vera
- `assert_eq!( expected, actual )` - impone l'eguaglianza tra i due valori
- `assert_ne!( expected, actual )` - impone la disuguaglianza tra i due valori

Per verificare che un'espressione generi la condizione di panico, è possibile annotare il test anche con l'annotazione `#[should_panic(expected = "msg")]`. In questo caso, viene verificato che venga generata una condizione di panico riportante "msg".

Se una funzione di test restituisce un valore di tipo `Result<T, Error>`, è possibile utilizzare la condizione di errore che indica che il test è fallito.

7.10 Eseguire i test

Il comando `cargo test` ricompila il programma con la direttiva "test" ed esegue tutti i test contenuti nel package. Compresi quelli eventualmente presenti all'interno della documentazione. Viene riportato un report dettagliato che indica ogni singolo fallimento, dove viene evidenziato il risultato atteso e quello realmente ottenuto seguito da un riassunto con il nome di tutte le funzioni di test che sono fallite.

È possibile eseguire solo alcuni test, specificando il nome della funzione, o una sua sottoparte, come ulteriore parametro del comando.

```
cargo test add_ esegue solo i test che contengono "add_" nel nome
```

Se uno o più test è decorato con l'attributo `#[ignore]`, viene normalmente tralasciato dall'esecuzione, a meno che non vengano inclusi esplicitamente con

```
cargo test -- --ignored esegue solo i test ignorati
```

7.11 Automatizzare la creazione di test

Alcui sistemi devono essere provati con molti dati diversi, una lib può essere `rtest` dando la possibilità di scrivere funzioni con parametri parametrici invocando la funzione `test` più volte tante quante sono i case definiti.

7.11.1 Test parametrici con `rtest`

Ogni singolo test è basato sul paradigma AAA, Arrange, Act, Assert. La fase di Arrangement porta spesso a creare codice ripetitivo, volto a popolare strutture dati con i necessari valori iniziali necessari all'esecuzione del test. Normalmente, le funzioni di test sono prive di parametri, dovendo poter essere invocate direttamente dal motore di test.

La libreria `rtest` permette di creare test parametrici, costituiti cioè da funzioni dotate parametri in ingresso. Tali funzioni devono essere precedute dall'attributo `#[rtest]`

I parametri possono essere forniti in due modi alternativi:

- Tramite funzioni annotate con l'attributo `#[fixture]`, il cui nome deve coincidere con il nome del parametro di ingresso presente nei test
- Tramite un elenco di tuple alternative, descritte tramite l'attributo `#[case(...)]`

```
use rtest::rtest;
#[rtest]
#[case(1, 2, 3)]
#[case(0, 0, 0)]
#[case(-1, 1, 0)]
fn test_sum(#[case] a: i32,
            #[case] b: i32,
            #[case] expected: i32) {
    assert_eq!(sum(a, b), expected);
}
```

7.12 Riassunto

- Verificano la correttezza dell'interazione tra due o più moduli
- Validano il contratto dell'interfaccia (tipi, semantica, protocollo di chiamata)
- Assumono che i singoli moduli abbiano già superato i test di unità
- Individuano:
 - Incongruenze semantiche
 - Errori di orchestrazione
 - Problemi di gestione degli errori
 - Violazioni di invarianti condivise
- In Rust:
 - Collocati nella cartella `tests/`
 - Compilati come crate separati
 - Testano esclusivamente l'API pubblica
- Costi:
 - Setup più complesso
 - Correzioni architetturali potenzialmente costose

Capitolo 8

Collezione di dati

Tutti i linguaggi offrono, nella propria libreria standard, un insieme di strutture dati volte a semplificare la vita ai programmatori implementando quelli che sono i migliori algoritmi noti per gestire problemi comuni:

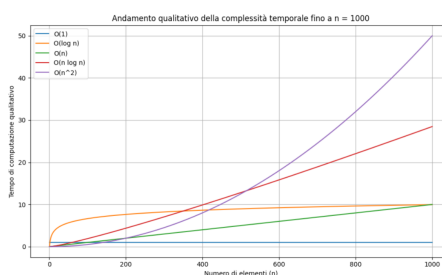
- Liste ordinate
- Insiemi di elementi univoci
- Mappe chiave-valore

Se esistono strategie diverse di implementazione, spesso sono presenti versioni alternative con diverse caratteristiche in termini di prestazioni.

È responsabilità del programmatore conoscere proprietà semantiche e costi computazionali delle diverse strutture dati, per scegliere quella adatta al caso d'uso. La scelta del contenitore deve tener conto anche di vincoli legati al possesso, località della memoria e pattern di accesso

8.1 Complessità

Descrizione	Accesso	Ricerca	Inserimento	Cancellazione
Vec<T>	$O(1)$	$O(n)$	Fine: $O(1)$, posiz: $O(n)$	Fine: $O(1)$, posiz: $O(n)$
VecDeque<T>	$O(n)$	$O(n)$	Estremi: $O(1)$, interno: $O(n)$	Estremi: $O(1)$, interno: $O(n)$
LinkedList<T>	$O(n)$	$O(n)$	dato il nodo: $O(1)$	dato il nodo: $O(1)$
BinaryHeap<T>	$O(1)$	$O(1)$	$O(\log(n))$	Pop: $O(\log(n))$
HashMap<T>	-	avg: $O(1)$	avg: $O(1)$	avg: $O(1)$
BTreeMap<T>	-	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
HashSet<T>	-	avg: $O(1)$	avg: $O(1)$	avg: $O(1)$
BTreeSet<T>	-	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$



Descrizione	Rust	C++	Java	Python
Array dinamico	<code>std::Vec<T></code>	<code>std::vector<T></code>	<code>java.util.ArrayList<T></code>	<code>list</code>
Coda a doppia entrata	<code>std::VecDeque<T></code>	<code>std::deque<T></code>	<code>java.util.ArrayDeque<T></code>	<code>collections.deque</code>
Lista doppiamente concatenata	<code>std::LinkedList<T></code>	<code>std::list<T></code> ^a	<code>java.util.LinkedList<T></code>	—
Coda a priorità	<code>std::BinaryHeap<T></code>	<code>std::priority_queue<T></code>	<code>java.util.PriorityQueue<T></code>	<code>heapq</code>
Tabella hash	<code>std::HashMap<K, V></code>	<code>std::unordered_map<K, V></code>	<code>java.util.HashMap<K, V></code>	<code>dict</code>
Mappa ordinata	<code>std::BTreeMap<K, V></code>	<code>std::map<K, V></code>	<code>java.util.TreeMap<K, V></code>	—
Insieme hash	<code>std::HashSet<T></code>	<code>std::unordered_set<T></code>	<code>java.util.HashSet<T></code>	<code>set</code>
Insieme ordinato	<code>std::BTreeSet<T></code>	<code>std::set<T></code>	<code>java.util.TreeSet<T></code>	—

^aEsiste anche singola: `std::forward_list<T>`

8.2 Metodi comuni a tutte le collezioni

Molte collezioni della standard library di Rust espongono un insieme di metodi e tratti comuni:

- `new()`: alloca una nuova collezione
- `with_capacity(capacity: usize)`: alloca una nuova collezione con capacità iniziale fissata
- `len()`: permette di conoscere l'attuale dimensione della collezione
- `clear()`: rimuove tutti gli elementi della collezione
- `is_empty()`: ritorna true se la collezione è vuota
- `iter()`: per iterare sui valori della collezione

Inoltre, molte collezioni implementano i tratti `IntoIterator` e `FromIterator`:

- `IntoIterator::into_iter()`: permette di convertire qualsiasi collezione in un iteratore
- `FromIterator::collect()`: permette di ottenere una collezione partendo da un iteratore

8.3 Vec<T>

Il tipo `Vec<T>` rappresenta una sequenza ridimensionabile di elementi di tipo `T`, allocati sullo heap. Questa collezione offre una serie di metodi per accedere al suo contenuto e per inserire/togliere valori al suo interno.

Un valore di tipo `Vec<T>` è rappresentato internamente da tre campi logici:

- **Un puntatore** ad un buffer allocato sullo heap nel quale sono memorizzati gli elementi
- **Un intero** privo di segno che indica la dimensione complessiva del buffer
- **Un intero** privo di segno che indica quanti elementi sono valorizzati nel buffer

Se si richiede ad un oggetto di tipo `Vec<T>` di inserire un nuovo elemento, questo verrà memorizzato nel buffer nella prima posizione libera e verrà incrementato il contatore che indica il numero di elementi effettivamente presenti.

Nel caso in cui il buffer fosse già completo, verrà allocato un nuovo buffer di dimensioni maggiori. Il contenuto del buffer precedente sarà riversato in quello nuovo, dove verrà poi anche inserito il nuovo elemento. Dopodiché il buffer precedente sarà de-allocato.

8.3.1 Metodi di Vec<T>

Offre una vasta serie di metodi per accedere al suo contenuto e per inserire/togliere valori al suo interno, tra cui:

- `Vec::with_capacity(n)` alloca un vettore con capacità `n`
- `capacity()` ritorna la capacità attualmente allocata
- `len()` ritorna il numero di elementi effettivamente presenti nel vettore
- `push(value)` aggiunge un elemento in coda
- `pop()` rimuove e ritorna un `std::Option` contenente l'ultimo elemento del vettore, se esistente
- `insert(index, value)` aggiunge un elemento alla posizione ricevuta in argomento spostando gli elementi
- `remove(index)` rimuove, compatta e ritorna l'elemento alla posizione ricevuta in argomento
- ... molti altri

I dati contenuti in un vettore devono essere omogenei. Se occorre memorizzare dati di tipo differente, è possibile utilizzare un tipo enumerativo somma disgiunta dei casi ammessi

```
enum SpreadsheetCell {
    Int(i32),
    Float(f64),
    Text(String),
}

let row = vec![
    SpreadsheetCell::Int(3),
    SpreadsheetCell::Text(String::from("blue")),
    SpreadsheetCell::Float(10.12),
];
```

Quando usarlo

- Serve tenere i dati in una sequenza ordinata con accesso casuale veloce
- Serve prevalentemente inserire/rimuovere in coda
- Serve ordinare, deduplicare, suddividere in blocchi contigui i dati
- Occorre un buffer elastico per gestire volumi di dati variabili

Punti di forza

Eccellente località dei dati che favorisce l'uso della cache infatti accedere ad un elemento ha costo unitario, inserire al fondo ha costo unitario ammortizzato

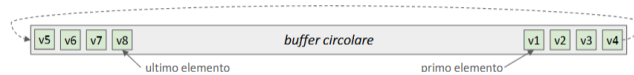
Punti di debolezza

Inserire/rimuovere all'inizio o al centro richiede lo spostamento di tutti gli elementi successivi.

8.4 VecDeque<T>

Il tipo `VecDeque<T>` modella una coda a doppia entrata: esso alloca sullo heap una serie di elementi di tipo `T`.

A differenza di `Vec<T>`, permette l'inserimento e la rimozione, con costo unitario, sia all'inizio che alla fine del vettore tramite i metodi `push_front()`, `push_back()`, `pop_front()` e `pop_back()`.



Viene implementato come un buffer circolare e non garantisce che gli elementi siano contigui in memoria, per rendere gli elementi contigui in memoria utilizzando il metodo `make_contiguous()` e ora si possono applicare gli algoritmi di sorting.

Nota: `VecDeque<T>` risulta più efficiente di `Vec<T>` se se si eseguono molte inserimenti o rimozioni ad entrambi gli estremi. In tutti gli altri casi è preferibile utilizzare `Vec<T>`.

Quando usarlo

- Occorre una coda con comportamento FIFO
- Serve frequentemente inserire in coda e estrarre dalla testa
- Occorre implementare una finestra scorrevole, sliding window, o mantenere la lista di N elementi recenti
- Serve estrarre/inserire sia in testa che in coda

Punti di forza

Le operazioni in testa e in coda sono efficienti e gli elementi sono memorizzati in un array, con accessi efficienti all'i-esimo elemento

Punti di debolezza

Non tutti gli elementi sono contigui, talora occorre renderli tali invocando `make_contiguous()`. Inserire nel mezzo ha un costo proporzionale al numero di elementi da spostare.

8.5 LinkedList<T>

Il tipo `LinkedList<T>` permette di rappresentare in memoria una lista doppiamente concatenata. L'accesso per posizione ha complessità $O(n)$, mentre inserimento e rimozione agli estremi o dato il nodo corretto ha complessità $O(1)$.

Il tempo di join è costante, utile per applicazioni real-time.

Come per `VecDeque<T>`, permette di inserire e rimuovere elementi da entrambe le estremità della lista.

È possibile inizializzare una `LinkedList<T>` a partire da un array

```
let ll = LinkedList::from([1, 2, 3]);
```

I metodi attualmente offerti da `LinkedList<T>` sono un ristretto sottoinsieme dei metodi offerti da `VecDeque<T>`.

Tuttavia, è quasi sempre preferibile utilizzare `Vec<T>` o `VecDeque<T>` poiché superiori in termini di prestazioni ed uso della memoria.

Quando usarla

- Occorre suddividere una sequenza o concatenare due sequenze in tempo costante
- Non è tollerabile il comportamento di riallocazione/movimento di elementi a fronte di inserimenti/rimozioni in quanto potrebbe introdurre ritardi variabili (`VecDeque<T>`)

Punti di forza

Concatenazione e suddivisione molto efficienti. Le operazioni in testa e coda hanno tempo costante

Punti di debolezza

Località della cache molto limitata rispetto a `Vec<T>` o `VecDeque<T>`. L'accesso casuale ha costo lineare. Nella maggior parte dei casi reali ha comportamento più lento rispetto a `Vec<T>` o `VecDeque<T>`

Metodi

I metodi attualmente offerti da `LinkedList<T>` sono un ristretto sottoinsieme dei metodi offerti da `VecDeque<T>`.

Tuttavia, è quasi sempre preferibile utilizzare `Vec<T>` o `VecDeque<T>` poiché superiori in termini di prestazioni ed uso della memoria.

8.6 Mappe

8.6.1 HashMap<K,V>

Una `HashMap<K,V>` è una collezione di coppie composte da una **chiave** di tipo `K`: `Eq+Hash` e un **valore** di tipo `V`: i valori sono salvati nello heap usando una struttura dati hash.

È indicata quando non serve un ordinamento totale delle chiavi e si vogliono operazioni in tempo medio costante. Massimizza la rapidità media di accesso.

Il tempo di accesso ad un elemento è in media $O(1)$, ma può diventare $O(n)$ nel caso peggiore, quando tutte le chiavi collidono allo stesso valore. Per questo caso la funzione hash deve essere ben progettata.

È preferibile utilizzare una `HashMap<K,V>` quando le chiavi **NON** hanno un ordine. L'inserimento di una nuova entry nella `HashMap<K,V>` può causare la riallocazione ed il movimento dei dati.

Quando usarla

- Occorre gestire conteggi, indici, cache, raggruppamenti di elementi, unioni basate su chiave
- Serve accumulare dati etichettati per poterli elaborare successivamente
- Serve una ricerca con complessità attesa $O(1)$ e non serve garantire un ordine di iterazione

Punti di forza

Accesso e modifica con complessità attesa $O(1)$. L'API offerta da Rust è eccellente per la gestione di conteggi e inizializzazione condizionale.

Punti di debolezza

L'ordine di iterazione non è stabile né segue un criterio prevedibile. Le chiavi devono implementare i tratti **Eq** e **Hash**: se le due implementazioni non sono coerenti o ottimizzate possono nascere errori logici.

8.6.2 BTreeMap<K,V>

Una **BTreeMap**<K,V> è una collezione di coppie composte da una chiave di tipo K: **Ord** e un valore di tipo V: i valori sono salvati nello heap mantenendo le chiavi ordinate tramite una struttura di tipo B-tree.

È indicata quando servono ordinamento delle chiavi, interrogazioni su intervalli e costi prevedibili. Garantisce scansioni ordinate per chiave.

Anche in questo caso l'inserimento di una nuova entry nella **BTreeMap**<K, V> può causare la riallocazione ed il movimento dei dati.

La chiave deve essere univoca ed il tipo K deve implementare il tratto **Ord**.

Anche nel caso di **BTreeMap**<K,V>, l'inserimento di una nuova coppia può causare la riallocazione ed il movimento dei dati.

Quando usarla

- Serve iterare sugli elementi in modo ordinato deterministicamente
- Occorre reperire tutti gli elementi presenti in un **intervallo di chiavi**
- Occorre un ordinamento stabile per la creazione di report, API, test

Punti di forza

Iterazione ordinata naturalmente. Interrogazioni su intervalli di chiavi

Punti di debolezza

Il costo di accesso ad un elemento è più costoso, $O(\log(n))$. Le chiavi devono implementare il tratto **Ord** e devono garantire immutabilità di confronto (stabili): un eventuale cambio di ordinamento mentre una chiave è parte di una mappa è un errore logico.

8.6.3 Mappe e possesso

Poiché una mappa nella maggior parte dei casi possiede sia le chiavi che i valori presenti al suo interno, le operazioni relative all'inserimento, cancellazione, accesso ai singoli elementi richiedono il rispetto di tale possesso.

Inoltre, Rust cerca di ottimizzare l'accesso ai dati contenuti evitando di dover navigare più volte la struttura interna alla ricerca del valore associato ad un chiave.

La conseguenza di ciò è che l'interfaccia applicativa offerta è alquanto differente da quella tipicamente offerta da altri linguaggi, pur avendo lo stesso potere espressivo.

In particolare:

- **Accesso**: i metodi di accesso, `get(...)`, `get_key_value(...)`, `keys(...)`, etc., usano chiavi di tipo referenziato, `&K`, e ritornano valori di tipo referenziato, `&V`.
- **Modifica**: i metodi che modificano la mappa tendono ad usare riferimenti a chiavi e valori, `remove(...)`, `retain(...)`. Passando una funzione si decide se rimuovere o mantenere un elemento nella mappa.
- **Inserimento**: i metodi di inserimento richiedono invece il movimento di valori posseduti
- **Modifica o inserimento condizionale**: L'aggiornamento o l'inserimento condizionale di un valore associato a una chiave usa spesso la enum **Entry**, che evita ricerche duplicate.

8.6.4 Entry<'a,K,V>

Rust offre la possibilità di ottimizzare l'utilizzo delle mappe: in particolare attraverso il metodo `entry` che permette di cercare una chiave all'interno di una mappa e ritorna un enum in base al risultato della ricerca.

```
entry(&mut self, key: K) -> Entry<'a, K, V>
```

L'enum `Entry<'a, K, V>` permette di effettuare una sola ricerca della chiave e poi decidere se modificare il valore esistente oppure inserire un valore di default:

- `and_modify<F>(self, f: F)` in caso di successo consente di modificare il valore se la chiave è presente
- `or_insert(self, default: V)` in caso di fallimento inserisce un valore di default se la chiave è assente senza costi aggiuntivi poiché il puntatore sarà già indirizzato verso una zona di memoria libera

```
let mut animals: HashMap<&str, u32> = HashMap::new();

animals.entry("dog")           //hai la chiave "dog"?
    .and_modify(|e| { *e += 1 }) // se si, la modifichi
    .or_insert(1);              // altrimenti la aggiungi e inserisci 1

//nota il tempo di vita implicito in questo caso è 'static, si potrebbe esplicitare
```

Nota: Questo permette di abbassare la complessità ciclomatica permettendo di non inserire blocchi `if` o `for` espliciti

8.6.5 Operare con le mappe

Nelle mappe non vi è la macro per l'inizializzazione rapida.

```
use std::collections::HashMap;

fn main() {
    // inizializzo una mappa a partire da un insieme di valori
    let mut scores = HashMap::from([("Alice",80), ("Bob", 90), ("Carol",70)]);
    // modifico il valore associato ad una chiave, se esiste
    scores.entry("Carol").and_modify(|v| *v = 75 );
    // trasferisco il contenuto della mappa in un Vec<(K,V)>
    let mut v: Vec<&'static str, i32> = scores
        .into_iter() //smonta la mappa e prendo il possesso
        .collect(); //colleziono i valori in un vettore

    // ordino per il valore
    v.sort_by_key(|(_, val)| *val);
    println!("{:?}",v);
}
```

8.7 Insiemi

8.7.1 HashSet<T>

Un `HashSet<T>` è un insieme di elementi univoci di tipo `T` i cui valori sono salvati nello heap come singole hash table. Sono delle mappe con solo la chiave, senza un valore associato.

- L'inserimento di una nuova entry nell'`HashSet<T>` può causare la riallocazione e il movimento dei dati.
- Un `HashSet<T>` può essere visto come una struttura analoga a `HashMap<T, ()>`, dove conta solo la presenza dell'elemento

Quando usarlo

- Serve tenere traccia della presenza di un elemento
- Serve implementare meccanismi di deduplicazione, controllo di appartenenza, unicità
- Serve un'implementazione efficiente dell'algebra insiemistica, somma, differenza o intersezioni, senza esigenze di ordinamento

Punti di forza

Verifica di appartenenza e inserimento hanno costo atteso $O(1)$. Offre operazioni espressive quali unione, intersezione e differenza

Punti di debolezza

Nessuna garanzia di ordinamento nell'iterazione. I valori devono implementare i tratti **Eq** e **Hash**: se le due implementazioni non sono coerenti o ottimizzate possono nascere errori logici.

8.7.2 BTreeSet<T>

Un **BTreeSet<T>** è un insieme di elementi univoci di tipo **T** i cui valori sono salvati nello heap come un singolo albero dove ogni entry rappresenta un nodo.

- L'inserimento di una nuova entry nell'**BTreeSet<T>** può causare la riallocazione e il movimento dei dati.
- Un **BTreeSet<T>** può essere visto come una struttura analoga a **BTreeMap<T, ()>**

Quando usarlo

- Serve iterare su valori **ordinati** e **privi di duplicati**
- Serve creare report ordinati, esplorare contenuti sulla base del prefisso o effettuare asserzioni stabili per i test
- Occorre estrarre gli elementi contenuti in un intervallo

Punti di forza

Le iterazioni ordinate e la ricerca per intervallo non richiedono operazioni ulteriori. Comportamento eccellente per indici privi di duplicati e ordinati

Punti di debolezza

Verificare l'appartenenza di un elemento ha costo logaritmico. Gli elementi devono avere un ordinamento totale stabile

8.7.3 BinaryHeap<T>

Un **BinaryHeap<T>** è una collezione di elementi di tipo **T** i cui valori sono salvati nello heap mantenendo il massimo elemento accessibile in cima alla struttura

Il tipo **T** deve implementare il tratto **Ord** per poter essere ordinato. La struttura non è ordinata globalmente: garantisce solo la proprietà di heap.

Il metodo **peek()** restituisce un riferimento all'elemento più grande con complessità $O(1)$. L'inserimento e la modifica di un elemento tramite il metodo **peek_mut()** hanno complessità $O(\log(n))$.

Quando usarlo

- Serve implementare uno **scheduler**, una coda a priorità o mantenere l'elenco dei migliori **N** elementi
- Occorre elaborare prima di tutto gli elementi a costo minore o a priorità maggiore
- Si vuole evitare di dover ordinare l'intero insieme di valori dopo ciascun inserimento

Punti di forza

Osservare l'elemento maggiore ha costo $O(1)$, push/pop hanno costo logaritmico. Ideale per algoritmi di tipo *top-N*.

Punti di debolezza

L'iterazione non è ordinata. Gli elementi devono implementare il tratto **Ord**: talora occorre un'implementazione custom o racchiudere gli elementi all'interno di `std::cmp::Reverse<T>`.

Nota: Permette di non ordinare l'intero insieme di valori rispetto ad utilizzare un `Vec<T>` ed ordinare prima di prendere i primi N elementi.

Capitolo 9

File I/O

9.1 File e file system

Un file è un'astrazione del sistema operativo per accedere in modo persistente a una sequenza di byte, identificata tramite una voce nel file system. I nomi sono organizzati in una struttura gerarchica fatta di cartelle o directory che permette di identificare uno specifico file attraverso una concatenazione dei nomi di cartelle e nome del file che ne esprime il cammino (path) a partire da una radice nota

Ad ogni file sono associati metadati e permessi di accesso infatti il sistema operativo media le operazioni di lettura, scrittura ed esecuzione secondo le proprie regole di sicurezza

Le librerie dei linguaggi di programmazione offrono meccanismi indipendenti dal sistema operativo per accedere al contenuto di un file.

Nel caso di Rust, l'astrazione principale è offerta da `std::fs::File` che modella un file aperto in lettura e/o scrittura.

Il C++, dalla versione 17, ha introdotto la libreria `std::filesystem`.

9.2 Percorsi

Ogni sistema operativo ha regole proprie per la definizione di cosa sia un percorso lecito per indicare un file, quali siano le radici note dei percorsi, come combinare segmenti parziali in un percorso complessivo.

In Rust le struct `std::path::Path` e `std::path::PathBuf` nascondono tali differenze offrendo un meccanismo portabile per comporre e scomporre un cammino e ricavare indicazioni sul file eventualmente referenziato.

- **Path** è un tipo **borrowed**, analoga a **str**: rappresenta una vista immutabile su un percorso
- **PathBuf**, analogamente a **String**, possiede il proprio contenuto e può essere modificato

Attraverso i metodi offerti da questi tipi, è possibile ricavare informazioni sulla esistenza del file, sulla sua natura come file semplici, directory, collegamenti simbolici, ecc., sui metadati associati come dimensioni, data di creazione e di ultima modifica e permessi, ecc.

ATTENZIONE: le verifiche sul file system non sono atomiche rispetto a operazioni successive: fra controllo e uso, lo stato può cambiare!

9.3 Navigare il file system

La funzione `std::fs::read_dir(dir: &Path) -> Result<ReadDir>` restituisce, se ha successo, un iteratore al contenuto della cartella dir. Le singole voci restituite sono di tipo `std::fs::DirEntry` e descrivono gli elementi contenuti nella cartella in termini di nome, tipo - file, cartella, collegamento simbolico -, metadati e cammino.

La funzione `std::fs::create_dir(dir: &Path) -> Result<()>` crea una nuova cartella. Fallisce se non si dispone delle necessarie autorizzazioni, se la cartella esiste già o se la cartella genitrice del cammino indicato non esiste.

La funzione `std::fs::remove_dir(dir: &Path) -> Result<()>` rimuove una cartella, a condizione che **esista**, si disponga dei necessari **permessi** e che sia **vuota**.

9.4 Manipolare i file nel file system

La funzione `std::fs::copy(from: &Path, to: &Path) -> Result<i64>` copia il contenuto di un file in un secondo file. Restituisce in caso di successo il numero di byte copiati.

La funzione `std::fs::rename(from: &Path, to: &Path) -> Result<()>` rinomina (o sposta) una voce del file system. Il comportamento in presenza di destinazione esistente o di file system differenti dipende dal sistema operativo sottostante.

La funzione `std::fs::remove_file(path: &Path) -> Result<()>` elimina un file, a patto che non sia una directory. L'effettiva rimozione di un file aperto dipende dalla semantica del sistema operativo.

9.5 Operazioni con i file

L'accesso al blocco di byte legato ad un file è totalmente mediato dal sistema operativo. Infatti per poter leggere o scrivere tale blocco occorre aprire il file. Il sistema operativo offre apposite funzioni che restituiscono un riferimento opaco al file sotto forma di *handle* o *file descriptor*, di fatto un numero intero.

La struct `std::fs::File` di Rust offre due metodi di base per aprire un file:

`open(path: P) -> Result<File> where P: AsRef<Path>` - apre il file in lettura, a condizione che esista

`create(path: P) -> Result<File> where P: AsRef<Path>` - tronca il file a 0 byte, se esiste, o lo crea, se non esiste ancora, dopodiché lo apre in scrittura

Maggiori opportunità sono offerte dalla struct `std::fs::OpenOptions`. Essa permette di combinare in modo esplicito flag come `read`, `write`, `append`, `truncate`, `create_new`.

9.6 Leggere e scrivere file

La funzione `std::fs::read_to_string(path: &Path)` legge l'intero file in memoria e lo interpreta come testo UTF-8. Questa funzione non deve essere usata per la lettura di file binari né per file troppo grossi.

La funzione `std::fs::write(path: &Path, contents: &[u8])` scrive l'intero buffer sul file, creandolo se non esiste o sovrascrivendolo.

Poiché un file può avere dimensioni molto maggiori del massimo blocco di memoria allocabile, occorre utilizzare tali funzioni quando si è certi che il contenuto può essere ospitato nella memoria del processo.

Nota: Queste funzioni sono adatte per file di dimensioni moderate. Per gestire contenuti di dimensioni arbitrariamente grandi, occorre utilizzare le API di streaming

```
use std::fs;
let contents = fs::read_to_string("file.txt"?);
fs::write("Text is;", contents)?;
```

9.6.1 Aprire un file

```
use std::fs::File;
use std::io::{Write, BufReader, BufRead, Error};
fn main() -> io::Result<()> {
    let path = "lines.txt";

    let mut output = File::create(path)?;
    write!(output, "Rust\n <3 \nFun"?);

    let input = File::open(path)?;
    let buffered = BufReader::new(input);

    for line in buffered.lines() {
        println!("{}", line?);
    }

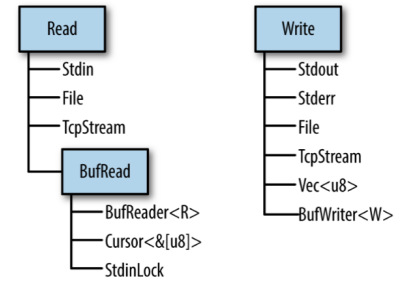
    Ok(())
}
```

9.7 I tratti relativi a I/O

Rust gestisce le operazioni di I/O attraverso l'utilizzo di alcuni tratti che implementano i metodi di base per le attività di lettura e scrittura. L'utilizzo di tratti favorisce la scrittura di codice generico, indipendente dal tipo specifico su cui viene eseguito.

I tratti principali offerti da Rust sono: `Read`, `BufRead`, `Write` e `Seek`. Tutti e quattro i tratti possono essere importati in maniera concisa attraverso il costrutto `use std::io:: prelude::*`.

Molte operazioni offerte da questi tratti restituiscono valori di tipo `io::Result<T>`, che è l'alias di `Result<T, io::Error>`, occorre pertanto gestire opportunamente i risultati.



9.7.1 std::io::Read

Tratto che indica la capacità di **leggere un flusso di byte**. `File`, `Stdin` e `TcpStream` sono alcuni dei tipi che implementano questo tratto.

Per implementare il tratto `Read` è sufficiente fornire l'implementazione del metodo `read(buf: &mut [u8]) -> Result<usize>`. Questo metodo tenta di leggere fino a `buf.len()` byte, ma può generarne di meno senza generare errori.

Rust genererà tutti gli altri metodi sulla base dell'implementazione di `read(buf: &mut [u8])` fornita dal programmatore.

- In caso di successo, il metodo `read(buf: &mut [u8])` deve ritornare `Ok(n)`. Dove n è compreso tra 0 e `buf.len()`
- Il valore `Ok(0)` può indicare che il flusso è terminato oppure che il buffer passato ha lunghezza 0.

Ogni chiamata al metodo `read(buf: &mut [u8])` può causare l'invocazione di una chiamata di sistema, con il conseguente costo legato al cambio di contesto.

```

use std::io::{self, Read};

struct MyReader {
    data: Vec<u8>,
    pos: usize,
}

impl Read for MyReader {
    fn read(&mut self, buf: &mut [u8]) -> io::Result<usize> {
        let remaining = &self.data[self.pos..];
        let n = remaining.len().min(buf.len());
        buf[..n].copy_from_slice(&remaining[..n]);
        self.pos += n;
        Ok(n)
    }
}

fn main() {
    let mut reader = MyReader { data: b"ciao".to_vec(), pos: 0 };
    let mut out = String::new();
    reader.read_to_string(&mut out).unwrap();
    println!("{}", out); // "ciao"
}
  
```

9.7.2 Metodi del tratto Read

L'implementazione del tratto `Read` mette a disposizione diversi metodi per gestire le operazioni più comuni di I/O

`read_to_end(buf: &mut Vec<u8>) -> Result<usize>` - continua a leggere fino all'EOF: si limita a richiamare il metodo `read()` fino a quando quest'ultimo non ritorna un valore `Ok(0)` o un errore fatale.

`read_to_string(buf: &mut String) -> Result<usize>` - continua a leggere fino all'EOF e riceve come parametro una `&mut String`. Assume che il contenuto sia UTF-8

`read_exact(buf: &mut [u8]) -> Result<()>` - prova a leggere l'esatto numero di byte contenuti nel buffer; adatto per record a dimensione fissa.

`bytes() -> Bytes<Self>` - ritorna un iteratore sui byte, gli elementi dell'iteratore sono dei `Result<u8, io::Error>`

`chain<R: Read>(next: R) -> Chain<Self, R>` - permette di concatenare due reader

`take(limit: u64) -> Take<Self>` - limita il numero massimo di byte che sarà possibile leggere

9.8 std::io::BufRead

Il tratto `BufRead` offre una serie di metodi che permettono di migliorare le prestazioni dell'I/O appoggiandosi ad un buffer in memoria.

- Il buffering riduce il numero di accessi al livello sottostante ed è particolarmente utile per letture frequenti e di piccola dimensione
- Non è utile quando si legge da elementi già presenti in memoria

L'implementazione del tratto richiede i metodi `fill_buf()` e `consume(amt: usize)`. I due metodi devono sempre essere utilizzati insieme: `fill_buf()` espone il contenuto disponibile nel buffer senza avanzare la posizione logica; dopo averne usato una parte, occorre chiamare `consume(...)`.

Offre i metodi `read_line(&mut self, buf: &mut String)` e `lines(self)` per accedere al contenuto testuale di un file.

```
use std::io;
use std::io::prelude::*;

let stdin = io::stdin(); // stdin non implementa il tratto BufRead
let mut handle = stdin.lock(); // lock permette di ottenere uno StdinLock che implementa BufRead

let buffer = handle.fill_buf()?;
println!("{:?}", buffer);

let length = buffer.len();
handle.consume(length); // garantisce che i byte letti non vengano
```

9.9 std::io::Write

Tratto che indica la capacità di scrivere un flusso di dati. Questo tratto è implementato, tra gli altri, dalle struct `File`, `Stdout` e `TcpStream`.

Il tratto `Write` richiede l'implementazione dei metodi `write` e `flush`.

`write(buf: &[u8]) -> Result<usize>` - prova a scrivere una parte del buffer ricevuto come argomento e ritorna il numero di byte effettivamente scritti

`flush() -> Result<()>` - inasizza l'output garantendo che tutti gli eventuali buffer transitori siano correttamente svuotati

`write_all(buf: &[u8]) -> Result<()>` - ripete le scritture fino a che tutti i dati sono stati trasferiti o si è verificato un errore

9.10 std::io::Seek

Tratto che permette di ri-posizionare il cursore di lettura/scrittura in un flusso di byte. Quando il flusso attinge ad un dato di dimensione nota, è possibile posizionare il cursore in modo relativo rispetto all'inizio del flusso, `SeekFrom::Start(n: u64)`, alla sua fine `SeekFrom::End(n: i64)` o alla posizione corrente `SeekFrom::Current(n: i64)`.

Il tratto offre i seguenti metodi:

- `fn seek(&mut self, pos: SeekFrom) -> Result<u64>`: posiziona il cursore alla posizione (in byte) indicata dal parametro `pos` e ritorna la nuova posizione del cursore
- `fn rewind(&mut self) -> Result<()>`: posiziona il cursore all'inizio del flusso
- `fn stream_position(&mut self) -> Result<u64>`: restituisce la posizione corrente del cursore rispetto all'inizio del flusso

`Seek` è disponibile solo per sorgenti/destinazioni che supportano il riposizionamento, come i file. Non è generalmente appropriato per stream sequenziali come socket o pipe.

Esempio: lettura di un file contenente dati binari

```
use std::fs::File;
use std::io::Read;
fn main() -> std::io::Result<()> {
    // "/dev/urandom": sorgente di byte casuali provenienti da una sorgente sicura
    let mut f = File::open("/dev/urandom")?;

    for _ in 0..10 {
        let mut buff = [0;4];
        let r = f.read_exact(&mut buff);
        if r.is_err() { return r; }
        if buff.iter().any(|b| *b==0) { return Ok(()); }
        let i = i32::from_be_bytes(buff);
        println!("{:x}", i);
    }

    for _ in 0..10 {
        let mut buff = [0;4];
        f.read_exact(&mut buff)?;
        let i = i32::from_be_bytes(buff);
        println!("0x{:08x}", i);
    }
    Ok(())
}
```

9.11 Gestione degli errori di I/O

Le operazioni di I/O restituiscono normalmente un valore di tipo `io::Result<T>`, che è l'alias di `Result<T, io::Error>`

`io::Error` contiene:

- Un valore `ErrorKind`, utile per classificare l'errore
- Informazioni aggiuntive dipendenti dalla piattaforma

In molti casi, il comportamento corretto è propagare l'errore con `?`. In alternativa, occorre gestire esplicitamente i casi recuperabili.

Un errore di I/O non implica sempre una condizione fatale:

- Alcune operazioni possono essere semplicemente **ritentate**
- Altre indicano fine input, **permessi insufficienti o dati non validi**

9.11.1 ErrorKind: casi comuni e strategie

- `ErrorKind::Interrupted`: Operazione interrotta, generalmente ritentabile
- `ErrorKind::NotFound`: File o percorso assente
- `ErrorKind::PermissionDenied`: Accesso negato dal sistema operativo
- `ErrorKind::UnexpectedEof`: Fine del flusso raggiunta prima del previsto, tipicamente restituito da `read_exact()`
- `ErrorKind::InvalidData`: Dati letti non validi per l'interpretazione richiesta, ad esempio testo non in formato valido UTF-8
- `ErrorKind::AlreadyExists`: Risorsa già esistente quando era richiesta creazione esclusiva

```
use std::io::ErrorKind::Interrupted;

match reader.read(&mut buf) {
    Err(e)
        if e.kind() == Interrupted => {
        /* ritenta */
    },
    Err(e) => return Err(e),
    Ok(n) => {
        /* usa i primi n byte */
    }
}
```

9.11.2 Buone pratiche

- Usare `read_to_string()` solo per file testuali e non troppo grandi
- Usare `BufReader` per letture piccole/frequenti
- Ricordare che `read()` e `write()` possono essere parziali
- Usare `write_all()` quando si vuole trasferire tutto il buffer
- Usare `OpenOptions` per semantiche di apertura esplicite
- Non assumere che comportamento di cambio nome/rimozione sia identico su tutte le piattaforme

9.12 Lettura e scrittura di contenuti strutturati

Sebbene sia possibile operare a livello di singoli byte/caratteri e ricostruire un dato strutturato a partire dai suoi componenti elementari, spesso è poco conveniente farlo.

Rust mette a disposizione il framework `Serde`, *Serialization-Deserialization*, il cui scopo è generare implementazioni efficienti di funzioni di serializzazione e deserializzazione di strutture dati arbitrarie verso formati standard quali JSON, YAML, CSV, BSON, Avro, XML e tanti altri.

Tale framework definisce i tratti `serde::Serialize` e `serde::Deserialize` che possono essere implementati dai tipi definiti all'interno di un programma.

`Serde` offre un'implementazione predefinita per la serializzazione dei tipi elementari e di alcuni tipi della libreria standard.

`String`, `&str`, `Vec<T>`, `HashMap<K, V>`, etc.

Supporta inoltre la macro `#[derive(Serialize, Deserialize)]` per generare, in fase di compilazione, l'implementazione dei tratti corrispondenti, a condizione che la struttura dati cui tale macro è applicata non contenga elementi “patologici”

9.12.1 Uso del framework Serde

Si inseriscono, nel file Cargo.toml, le dipendenze

```
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

La seconda indica il tipo di formato da generare/leggere. In alternativa, è possibile indicare un altro sotto-progetto compatibile come `csv = "1.3"` o `bson = "2.9"`.

Si decora la struttura dati da leggere con la macro `#[derive(...)]`

```
#[derive(Serialize, Deserialize, Debug)]
struct Data {
    name: String,
    data: Vec<u8>,
    attributes: HashMap<String, String>,
}

fn save(data: &Data, path: &str) -> Result<()> {
    let mut f = File::options()
        .write(true)
        .create(true)
        .truncate(true)
        .open(path)?;

    f.write_all(
        serde_json::to_string(data)?
        .as_bytes()
    )?;
    Ok(())
}

fn load(path: &str) -> Result<Data> {
    let mut f = File::open(path)?;

    let mut s = String::new();

    f.read_to_string(&mut s)?;

    return Ok(serde_json::from_str(&s)?);
}
```

Capitolo 10

Chiusure

10.1 Funzioni e chiusure

La programmazione in stile funzionale introduce il concetto di **funzioni di ordine superiore**. Si tratta di funzioni i cui parametri e/o il cui tipo di ritorno sono a loro volta una funzione. Questo richiede, ad un linguaggio, la possibilità di trattare una funzione come un tipo dati qualsiasi, consentendo di memorizzarla in una variabile.

Rust offre vari modi di trattare il comportamento eseguibile come valore:

- Puntatori a funzione di tipo `fn(T1, ..., Tn) -> U`
- Chiusure, introdotte dalla sintassi lambda `p1: T1, ..., pn: Tn -> R ... |`
- Parametri generici vincolati da tratti funzionali `FnOnce`, `FnMut`, `Fn`.

La differenza chiave non è sintattica, ma semantica. I valori di tipo `fn(...)` non catturano stato. Le chiusure e i tipi derivati possono includere uno stato catturato

La scelta del tipo di valore invocabile influenza vari aspetti: La rappresentazione del valore, prestiti e possesso, modalità di invocazione, possibilità di riuso.

10.1.1 Puntatori a funzione - `fn(T1, ..., Tn) -> U`

Un valore di tipo `fn(A, B) -> R` è un puntatore a funzione con firma compatibile. Può essere invocato come funzione normale: `f(x, y)`.

È adatto quando:

- Il comportamento da eseguire è già definito
- Non serve catturare stato

```
fn f1(i: i32, d: f64) -> f64 {  
    return i as f64 * d;  
}  
  
let ptr: fn(i32, f64) -> f64;  
ptr = f1; // assegno il puntatore  
ptr(2, 3.14); // chiamo la funzione
```

```
fn add(a: i32, b: i32) -> i32 { a + b }  
fn multiply(a: i32, b: i32) -> i32 { a * b }  
fn choose(c: bool) -> fn(i32, i32) -> i32 {  
    if c { add } else { multiply }  
}  
  
fn main() {  
    let f1 = choose(true);  
    let f2 = choose(false);  
    println!("f1 => {}", f1(2, 3)); // 5  
    println!("f2 => {}", f2(2, 3)); // 6  
}
```

Nota: `fn(...)` rappresenta codice invocabile senza stato catturato

10.2 Chiusure: codice + ambiente catturato

Un puntatore a funzione invoca solo codice; non incorpora un ambiente lessicale. Una chiusura è un valore invocabile che può catturare variabili visibili nel punto in cui è definita. Le variabili catturate diventano parte dello stato della chiusura. Ogni chiusura ha un tipo proprio, unico e anonimo generato dal compilatore.

Tale tipo implementa almeno il tratto `FnOnce`, ed eventualmente anche i tratti `FnMut` e `Fn`.

Note: Una chiusura è simile ad una struct anonima con alcuni campi catturati e con un'operazione di chiamata definita dal compilatore

10.2.1 Differenza essenziale: una chiusura cattura l'ambiente

- `f_ptr` non dispone di ambiente catturato, il suo risultato dipende solo dagli argomenti e da ciò che la funzione può già raggiungere senza cattura
- `closure(...)` usa anche `offset`, che appartiene al contesto lessicale esterno
- `offset` è una variabile libera nel corpo della chiusura
- Il compilatore decide come catturarla: per riferimento, come riferimento mutabile o per valore

```
fn add_one(x: i32) -> i32 {
    x + 1
}

fn main() {
    let offset = 10;
    let f_ptr: fn(i32)->i32 = add_one;
    let closure = |x| { x + offset };
    println!("{}", f_ptr(5)); // 6
    println!("{}", closure(5)); // 15
}
```

Nota: Una chiusura è una funzione accompagnata dal proprio ambiente

10.2.2 Chiusure che non catturano

Una chiusura viene invocata come una funzione: `f(5)`. Se **non cattura variabili**, può essere convertita a puntatore a funzione. Se invece cattura un ambiente, la si descrive tramite:

- `impl Fn`
- `impl FnMut`
- `impl FnOnce`

Come oggetto tratto, `Box<dyn Fn(...)>` quando occorre implementare il dispatch dinamico.

```
fn ret_fun() -> fn(i32) -> i32 {
    |x| { x + 1 } //Funziona perché
    ↳ la funzione ritornata non
    ↳ cattura valori
}

fn ret_closure(i: i32) -> impl Fn(i32) -> i32 {
    move |x| { x + i } //viene catturata la variabile i
}
```

10.2.3 Modalità di cattura in Rust

Una chiusura cattura ogni variabile libera nel modo meno restrittivo compatibile con il suo utilizzo nel corpo:

- Se la variabile è solo letta, la cattura avviene tramite riferimento condiviso `&`
- Se nel corpo richiede modifica dello stato catturato, la chiusura richiede accesso mutabile al proprio, `let mut f = ...| ...|` e implementa tipicamente il tratto `FnMut`
- Se il corpo consuma un valore catturato, la chiusura implementa il tratto `FnOnce`
- Il borrow checker verifica la coerenza tra catture, prestiti e il tempo di vita

La parola chiave `move` forza la cattura per valore: `let f = move |...| {...};`. `move` non implica necessariamente `FnOnce`. Il tratto implementato dalla chiusura dipende da come il valore viene usato.

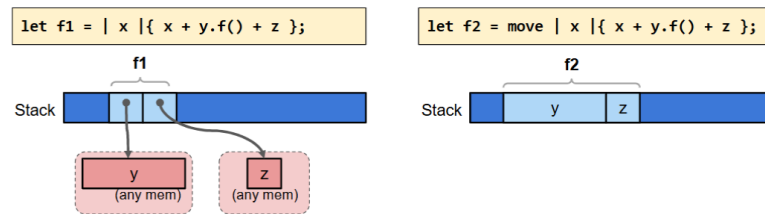
10.2.4 Modello mentale: le chiusure come struct anonime

Il compilatore materializza l'ambiente catturato come campi di un tipo anonimo

- Senza `move`, la cattura può avvenire per riferimento
- Con `move`, i campi dell'ambiente vengono catturati per valore
- Il corpo della chiusura opera su questi campi

```
let y: Y = ...;
let z: Z = ...;
let f = |x| x + y.m() + z;
//is equivalent to
struct __Closure<'a> {
    y: &'a Y, // o Y o &mut Y in base all'uso
           ↪ nel corpo
    z: &'a Z,
}
// impl Fn / FnMut / FnOnce for __Closure
```

10.2.5 Cattura delle variabili in Rust



```
// Stack referencia la memoria
let f1 = |x| { x + y.f() + z };
```

```
// Stack possiede/copia la memoria
let f2 = move |x| { x + y.f() + z };
```

Nota: Il diagramma è qualitativo: in assenza di `move`, la modalità di cattura dipende dall'uso; `move` forza invece la cattura per valore

10.2.6 Gerarchia dei tratti funzionali

$$\text{Fn} \subset \text{FnMut} \subset \text{FnOnce}$$

Tratto	Ricevitore concettuale	Ambiente catturato	Invocazioni
<code>FnOnce</code>	<code>self</code>	Può essere consumato	al più una
<code>FnMut</code>	<code>&mut self</code>	Può essere modificato	zero o più
<code>Fn</code>	<code>&self</code>	È in sola lettura	zero o più

10.3 FnOnce vs FnMut

```
let range = 1..10;
let f = || range.count();
let n1 = f();
// let n2 = f();
// errore: range è stato consumato
```

`range.count()` consuma `range`, il valore catturato viene tolto dall'ambiente quindi la chiusura può essere chiamata una sola volta. → `f: FnOnce() -> usize`

```
let mut sum = 0;
let mut f = |x| sum += x;
f(5);
f(7);
// sum vale 12
```

La chiusura modifica il proprio ambiente. Occorre invocarla tramite accesso mutabile → `f: FnMut(i32) -> ()`

10.4 Fn

Una chiusura di tipo `Fn` accede all'ambiente catturato senza consumarlo né mutarlo, dunque può essere invocata più volte e produrrà sempre il medesimo risultato.

Tipicamente, la cattura avviene tramite accesso in sola lettura al dato esterno. Questo è il caso più restrittivo ma anche quello più riusabile.

```
let s = "hello";
let f = |v| v < s;
f("world"); // false
f("bye"); // true
```

10.5 Chiusure come argomenti

Ogni chiusura ha un tipo anonimo distinto. Per accettare una chiusura come parametro si usa spesso un **tipo generico**.

Il vincolo posto su tale tipo dipende dall'uso previsto:

- `FnOnce` se basta una chiamata
- `FnMut` se si vuole modificare l'ambiente
- `Fn` se vengono effettuate solo letture

La presenza del parametro generico dà origine a **monomorfizzazione**. Se serve eterogeneità in fase di esecuzione, si può usare `Box<dyn Fn(...) -> R>`

10.5.1 Restituire una chiusura in sicurezza

Se una funzione restituisce una chiusura, il valore restituito deve essere slegato dallo stack frame locale.

- `move` trasferisce il possesso dei valori alla chiusura
- `impl Fn() -> T` nasconde il tipo anonimo concreto della chiusura
- L'invocazione del metodo `clone` permette di restituire ogni volta un nuovo valore senza consumare quello catturato

```
fn value_source<T>(v: T) -> impl Fn() -> T where T: Clone {
    return move || { v.clone() };
}
```

Note: `move` serve spesso a rendere la chiusura restituita autosufficiente

10.5.2 Esempio di generatore

Questa chiusura cattura `b` per valore, e `i` per valore, mantenendolo come stato interno mutabile.

Ad ogni invocazione lo stato si aggiorna e produce un risultato diverso.

Il tratto associato è `FnMut`, perché l'ambiente viene modificato. Questo pattern è utile quando si vuole incapsulare uno stato senza definire esplicitamente una struct dedicata.

```
fn generator(prefix: &str) -> impl FnMut() -> String {
    let mut i = 0;
    let b = prefix.to_string();
    return move || { i += 1; format!("{}", b, i) }
}

fn main() {
    let mut f = generator("id_");
    for _ in 1..5 {
        println!("{}", f());
    }
}
```

// Output atteso:
// id_1
// id_2
// id_3
// id_4

Capitolo 11

Iteratori

11.1 Iteratori

Un iteratore è una struttura dati dotata di stato, in grado di generare una sequenza di valori. I valori possono essere estratti da un contenitore, di cui l'iteratore detiene un riferimento, o generati programmaticamente, come nel caso di un intervallo di valori.

Un iteratore offre tipicamente un modo per verificare se sono presenti ulteriori valori da generare, `hasNext()` -> `bool`, ed un altro per accedere al valore successivo, `next()` -> `E`.

Talora, come in Rust, le due operazioni sono combinate: `next()` -> `Option<E>`. Inoltre, un iteratore può offrire ulteriori metodi che consentono di derivare un nuovo iteratore, che trasforma la sequenza di valori in un'altra sequenza, accorpendo, eliminando, trasformando, ecc., i singoli elementi originali.

Pressoché tutti i linguaggi moderni offrono il concetto di iteratore come parte della propria libreria standard. Infatti essi permettono di accedere ai valori contenuti all'interno di collezioni come liste, insiemi, mappe, scorrere i caratteri presenti all'interno di una stringa o leggere il contenuto di un file di testo estraendo una riga alla volta.

11.1.1 Uso degli iteratori

L'uso degli iteratori si sovrappone concettualmente a quello dei cicli `for/while`. Tuttavia, offre svariati vantaggi in termini di compattezza del codice, leggibilità, manutenibilità ed efficienza, nascondendo i dettagli necessari a generare o accedere ai singoli elementi. I vantaggi sono maggiormente evidenti se l'operazione svolta all'interno del ciclo è complessa e richiede, ad esempio, di scartare alcuni valori e di trasformarne altri.

Approccio Imperativo

```
let v1 = vec![1, 2, 3, 4, 5, 6];
let mut v2 = Vec::<String>::new();
for i in 0..v1.len() {
    if v1[i] % 2 != 0 { continue; }
    v2.push(format!("a{}", v1[i]))
}
println!("{:?}", v2);
// ["a2", "a4", "a6"]
```

Approccio Funzionale (Iteratori)

```
let v1 = vec![1, 2, 3, 4, 5, 6];
let mut v2: Vec<String>;
v2 = v1.iter()
    .filter(|val| { *val % 2 == 0 })
    .map(|val| format!("a{}", val))
    .collect();
println!("{:?}", v2);
// ["a2", "a4", "a6"]
```

11.1.2 Caratteristiche degli iteratori

- Un iteratore offre un **modo uniforme** di accedere agli elementi, indipendentemente da come essi siano generati o da dove siano prelevati, permettendo al codice che utilizza tali valori di ignorare la fonte ed essere più generico.
- Operano in **modalità pigra (lazy)**: solo a fronte della richiesta di un valore successivo, si occupano di generarlo / prelevarlo.
- Possono abilitare l'**elaborazione parallela** dei dati ospitati in una collezione, permettendo di sfruttare la presenza di più core nell'elaboratore.

- La possibilità di **derivare un iteratore** da un altro iteratore aumenta la flessibilità del codice. È facile inserire all'interno di una catena di elaborazione passi ulteriori senza dover intervenire sul codice che utilizza i valori generati.

11.2 Iteratori in Rust

In Rust, un iteratore è una qualsiasi struttura dati che implementa il tratto `std::iter::Iterator`.

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;    // ... molti altri metodi con implementazione
                                                // di default
}
```

Un tipo può segnalare la capacità di essere esplorato tramite un iteratore, implementando il tratto `std::iter::IntoIterator`.

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {
    type Item;
    type IntoIter: Iterator;
    fn into_iter(self) -> Self::IntoIter;
}
```

11.2.1 Implementare un iteratore in Rust

```
struct Words<'a> {
    text: &'a str,
}
impl<'a> Words<'a> {
    fn new(text: &'a str) -> Self {
        Self { text }
    }
}
struct WordsIter<'a> {
    text: &'a str,
    pos: usize,
}

impl<'a> IntoIterator for Words<'a> {
    type Item = &'a str;
    type IntoIter = WordsIter<'a>;
    fn into_iter(self) -> Self::IntoIter {
        WordsIter {
            text: self.text,
            pos: 0,
        }
    }
}
```

Esempio di implementazione di un iteratore per una parola:

```
impl<'a> Iterator for WordsIter<'a> {
    type Item = &'a str;
    fn next(&mut self) -> Option<Self::Item> {
        let bytes = self.text.as_bytes();
        let len = bytes.len();
        // elimino gli spazi iniziali
        while self.pos < len && bytes[self.pos].is_ascii_whitespace() { self.pos += 1; }
        if self.pos >= len { return None; }
        let start = self.pos;
        // estraggo tutto il contenuto non bianco
        while self.pos < len && !bytes[self.pos].is_ascii_whitespace() { self.pos += 1; }
        Some(&self.text[start..self.pos])
    }
}
```

Esempio di implementazione di un iteratore per un range di valori personalizzato:

```
struct MyRange<const FROM: isize, const TO: isize> {}

impl<const FROM: isize, const TO: isize> IntoIterator for MyRange<FROM, TO> {
    type Item = isize;
    type IntoIter = MyRangeIterator<FROM, TO>;
    fn into_iter(self) -> Self::IntoIter {
        MyRangeIterator::<FROM, TO>::new()
    }
}
```

```

    }
}

struct MyRangeIterator<const FROM: isize, const TO: isize> { val: isize }

impl<const FROM: isize, const TO: isize> MyRangeIterator<FROM, TO> {
    fn new() -> Self {
        MyRangeIterator { val: FROM }
    }
}

impl<const FROM: isize, const TO: isize> Iterator for MyRangeIterator<FROM, TO> {
    type Item = isize;
    fn next(&mut self) -> Option<Self::Item> {
        if FROM < TO {
            if self.val < TO {
                let ret = self.val;
                self.val += 1;
                Some(ret)
            } else { None }
        } else {
            if self.val > TO {
                let ret = self.val;
                self.val -= 1;
                Some(ret)
            } else { None }
        }
    }
}

```

Caratteristiche:

- L'iteratore è pigro: viene generato un nuovo valore solo quando viene chiamato `next()`
- Non esegue allocazioni
- Il risultato ha un tempo di vita collegato al tempo di vita del valore iniziale
- **pos** rappresenta la posizione corrente nell'analisi

```

fn main() {
    let line = "INFO 2025-05-10 user=alice
    ↪ action=login";
    let words = Words::new(line);
    for w in words {
        println!("{}", w);
    }
    //INFO
    //2025-05-10
    //user=alice
    //action=login
}

```

11.3 Iteratori e possesso

I contenitori presenti nella libreria standard mettono normalmente a disposizione tre metodi per ricavare un iteratore ai dati contenuti al loro interno:

- `iter()`: restituisce oggetti di tipo `&Item` e non consuma il contenuto del contenitore.
- `iter_mut()`: restituisce oggetti di tipo `&mut Item` e permette di modificare gli elementi all'interno del contenitore.
- `into_iter()`: prende possesso del contenitore e restituisce oggetti di tipo `Item` estraendoli dal contenitore.

È comune, per tali contenitori, dichiarare tre implementazioni distinte del tratto `IntoIterator`:

- una per il tipo `Container` (`into_iter()`),
- una per il tipo `&Container` (`iter()`),
- una per il tipo `&mut Container` (`iter_mut()`).
- In alcuni casi (come `HashSet<T>`, `HashMap<T>`) la terza implementazione non è fornita perché romperebbe le astrazioni.

Non tutte le collezioni espongono tutte le forme di iterazione mutabile nello stesso modo: in particolare, la possibilità di mutare gli elementi dipende dalle garanzie che la struttura deve preservare

```
let mut v = vec![String::from("a"), String::from("b"), String::from("c")];

for s in &v { println!("{}", s); } // s ha tipo &String

for s in &mut v { s.push_str("1"); } // s ha tipo &mut String - Modifico il contenuto
                                   // del vettore

for s in v { println!("{}", s); } // s ha tipo String - invalido il contenuto del vettore
                                   // che non è più utilizzabile dopo il ciclo
```

11.3.1 Derivare un iteratore

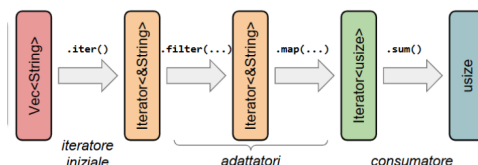
Ogni iteratore implementa `IntoIterator` restituendo sé stesso. Per questo motivo un iteratore può essere usato direttamente in un ciclo `for`.

```
let mut v = vec![String::new("a"), String::new("b"), String::new("c")];
let it = v.iter_mut();
for s in it { // it.into_iter() -> it
    // qui s ha tipo &mut String e opera sui valori contenuti in v
}
```

11.4 Adattatori

Il tratto `Iterator` definisce un molti metodi che consumano un iteratore costruiscono un nuovo iteratore derivato, in grado di offrire funzionalità ulteriori. Possono essere combinati in catene più o meno lunghe al termine delle quali occorre porre un consumatore finale. Tutti gli adattatori sono infatti pigri (*lazy*) di natura e non invocano il metodo `next()` dell'oggetto a monte se non a seguito di una richiesta proveniente da un loro consumatore.

```
let v: Vec<String> = ... ;
v.iter()
  .filter(|x| x.len()<4)
  .map(|x|x.len())
  .sum();
```



11.4.1 Adattatori: trasformare gli elementi

- `map(self, f)`: Trasforma ogni elemento in ingresso con la chiusura `f` e produce un nuovo iteratore dei valori trasformati

where `F: FnMut(Self::Item) -> V`

- `enumerate(self) -> Enumerate<Self>`: Ritorna un iteratore che restituisce una tupla formata dall'indice dell'iterazione e dal valore (`i, val`).
- `copied<'s, T(self) -> Copied<Self>`: Ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale verranno copiati quando l'iteratore sarà consumato
- `cloned<'s, T(self) -> Cloned<Self>`: Ritorna un nuovo iteratore, tutti gli elementi dell'iteratore originale verranno clonati quando l'iteratore sarà consumato
- `inspect<F>(self, f: F) -> Inspect<Self, F>`: Ogni volta che riceve una richiesta, preleva un elemento dall'iteratore a monte e la passa sia alla funzione, che ha possibilità di ispezionarlo, che al consumatore a valle

where `F: FnMut(&Self::Item)`

11.4.2 Adattatori: filtrare

- `filter<P>(self, predicate: P) -> Filter<Self, P>`: Seleziona gli elementi per i quali l'esecuzione della chiusura `predicate` ritorna `true`.

where `P: FnMut(&Self::Item) -> bool`.

- `take(self, n: usize) -> Take<Self>`: Ritorna un iteratore che contiene al più i primi `n` elementi dell'iteratore su cui viene eseguito (meno, se l'iteratore originale non contiene abbastanza elementi).
- `take_while<P>(self, predicate: P) -> TakeWhile<Self, P>`: Esegue la funzione ricevuta su tutti gli elementi dell'iteratore originale, conserva tutti gli elementi fino a quando la funzione ritorna `true`; dal momento in cui diventa `false`, scarta tutti i valori rimanenti.

where `P: FnMut(&Self::Item) -> bool`.

- `skip(self, n: usize) -> Skip<Self>`: Ritorna un iteratore che esclude i primi `n` elementi dell'iteratore su cui viene eseguito, se si raggiunge la fine ritorna un iteratore vuoto.
- `skip_while<P>(self, predicate: P) -> SkipWhile<Self, P>`: Esegue la funzione su tutti gli elementi dell'iteratore originale, esclude tutti gli elementi fino a quando la funzione ritorna `true`, dal momento in cui diventa `false` conserva tutti i valori rimanenti.

where `P: FnMut(&Self::Item) -> bool`.

11.4.3 Adattatori: combinare

- `chain<U>(self, other: U) -> Chain<Self, <U as IntoIterator>::IntoIter>`: Prende come argomento un valore iterabile e ne concatena gli elementi all'originale, ritornando uno nuovo iteratore.
- `zip<U>(self, other: U) -> Zip<Self, <U as IntoIterator>::IntoIter>`: Combina due iteratori per ritornare un nuovo iteratore che ha come elementi le coppie composte dai valori dei primi due iteratori.
- `cycle(self) -> Cycle<Self>`: Raggiunta la fine di un iteratore riparte dall'inizio, ciclando all'infinito.
- `rev(self) -> Rev<Self>`: Ritorna un iteratore con la direzione invertita.

11.4.4 Adattatori: funzionalità ulteriori

- `flatten(self) -> Flatten<Self>`: Ritorna un iteratore dal quale sono state rimosse le strutture annidate.

where `Self::Item: IntoIterator`.

- `flat_map<U, F>(self, f: F) -> FlatMap<Self, U, F>`: Concatena in maniera concisa `map` e `flatten`, eseguendo la chiusura `f` e rimuovendo le strutture annidate.

where `F: FnMut(Self::Item) -> U, U: IntoIterator`.

- `filter_map<V, F>(self, f: F) -> FilterMap<Self, F>`: Concatena in maniera concisa `filter` e `map`, l'iteratore risultante conterrà solo elementi per i quali la chiusura ritorna `Some(V)`.

where `F: FnMut(Self::Item) -> Option<V>`.

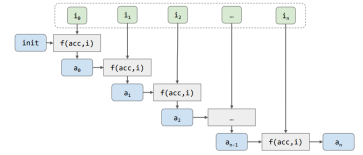
- `fuse(self) -> Fuse<Self>`: Termina dopo il primo `None`.
- `peekable(self)`: Permette di chiamare i metodi `peek()` e `peek_mut()` per accedere al valore successivo senza consumarlo.
- `by_ref(&mut self)`: Prende in prestito un iteratore senza consumarlo, lasciando intatto il possesso dell'originale.

11.4.5 fold(...): riduzione esplicita di un iteratore

`fold(init, f)` è un consumatore finale che attraversa l'iteratore mantenendo un accumulatore.

- L'accumulatore viene inizializzato con il valore `init`.
- Ad ogni passo applica `f(acc, item)` e produce un nuovo accumulatore.
- Al termine restituisce il valore finale dell'accumulatore.

Molte aggregazioni possono essere espresse con `fold(...)`, come somma, prodotto, conteggio, concatenazione, costruzione di strutture, ecc.



```
let v = vec![1, 2, 3, 4];

let sum = v.iter().fold(0, |acc, x| acc + x);
let count = v.iter().fold(0, |acc, _| acc + 1);
let product = v.iter().fold(1, |acc, x| acc * x);
```

11.4.6 Consumatori: aggregare

- `fold<B, F>(self, init: B, f: F) -> B`: Esegue la chiusura ricevuta accumulando i risultati sul primo argomento ricevuto.
- `try_fold<B, F, R>(&mut self, init: B, f: F) -> R`: Esegue la chiusura ricevuta fino a quando ritorna con successo, accumulando i risultati sul primo argomento ricevuto.
- `reduce<F>(self, f: F) -> Option<Self::Item>`: Riduce l'iteratore ad un singolo elemento eseguendo la funzione ricevuta.
- `sum<S>(self) -> S`: Somma tutti gli elementi di un iteratore e ritorna il valore ottenuto.
- `product<P>(self) -> P`: Moltiplica tutti gli elementi di un iteratore e ritorna il valore ottenuto.
- `count(self) -> usize`: Ritorna il numero di elementi dell'iteratore.
- `collect<B>(self) -> B`: Trasforma un iteratore in una collezione.

where B: FromIterator<Self::Item>.

- `partition<B, F>(self, f: F) -> (B, B)`: Consuma un iteratore e ritorna due collezioni sulla base del predicato ricevuto.
- `for_each<F>(self, f: F)`: Esegue la chiusura ricevuta su tutti gli elementi dell'iteratore.

where F: FnMut(Item).

- `try_for_each<F, R>(&mut self, f: F) -> R`: Esegue una chiusura che può fallire su tutti gli elementi dell'iteratore, si ferma dopo il primo fallimento.

where F: FnMut(Item) -> R, R: Try<Output=()>.

- `max(self) -> Option<Self::Item>`: Ritorna il massimo tra gli elementi dell'iteratore, se trova due massimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None.
- `max_by(self, compare) / min_by(self, compare)`: Ritornano il massimo o il minimo sulla base della chiusura di confronto ricevuta in argomento.
- `max_by_key(self, f) / min_by_key(self, f)`: Eseguono la chiusura ricevuta e ritornano l'elemento che produce il risultato massimo o minimo.
- `min(self) -> Option<Self::Item>`: Ritorna il minimo tra gli elementi dell'iteratore, se trova due minimi equivalenti torna l'ultimo, se l'iteratore è vuoto viene ritornato None.

11.4.7 Consumatori: cercare e verificare

- `find<P>(&mut self, predicate: P) -> Option<Self::Item>`: Cerca un elemento sulla base della chiusura ricevuta come argomento e lo ritorna.

where P: FnMut(&Item) -> bool.

- `find_map<B, F>(&mut self, f: F) -> Option<B>`: Esegue la chiusura ricevuta su tutti gli elementi e ritorna il primo risultato valido.
- `position<P>(&mut self, predicate: P) -> Option<usize>`: Cerca un elemento sulla base della chiusura ricevuta come argomento e ritorna la posizione.
- `rposition<P>(&mut self, predicate: P) -> Option<usize>`: Cerca un elemento sulla base della chiusura ricevuta come argomento, partendo da destra e ritornando la posizione.
- `last(self) -> Option<Self::Item>`: Ritorna l'ultimo elemento dell'iteratore.
- `nth(&mut self, n: usize) -> Option<Self::Item>`: Ritorna l'ennesimo elemento dell'iteratore, consumando i precedenti.
- `all<F>(&mut self, f: F) -> bool`: Verifica che la chiusura ricevuta restituisca true per tutti gli elementi restituiti dall'iteratore.

where F: FnMut(Item) -> bool.

- `any<F>(&mut self, f: F) -> bool`: Verifica che la chiusura ricevuta restituisca true per almeno un elemento restituito dall'iteratore.

where F: FnMut(Item) -> bool.

11.4.8 Consumatori: confrontare

- `cmp<I>(self, other: I) -> Ordering`: Confronta gli elementi di due iteratori.
- `eq<I>(self, other: I) -> bool`: Verifica se gli elementi di due iteratori sono uguali.
- `ne<I>(self, other: I) -> bool`: Verifica se gli elementi di due iteratori sono diversi.
- `lt<I>(self, other: I) -> bool`: Verifica se gli elementi di un iteratore sono minori rispetto a quelli di un secondo iteratore.
- `le<I>(self, other: I) -> bool`: Verifica se gli elementi di un iteratore sono minori o uguali rispetto a quelli di un secondo iteratore.
- `gt<I>(self, other: I) -> bool`: Verifica se gli elementi di un iteratore sono maggiori rispetto a quelli di un secondo iteratore.
- `ge<I>(self, other: I) -> bool`: Verifica se gli elementi di un iteratore sono maggiori o uguali rispetto a quelli di un secondo iteratore.

Capitolo 12

Smart Pointers

12.1 Smart Pointer

Ogni valore manipolato da un programma è memorizzato nello spazio di indirizzamento del processo. L'operatore `&` e `&mut` in Rust permette, in C, C++ e Rust, di ottenere l'indirizzo del primo byte in cui è memorizzato, producendo un riferimento ad un valore. In Rust, tali riferimenti non sono puntatori nativi esposti direttamente al programmatore, ma valori soggetti alle regole del borrow checker.

L'operazione duale, detta dereferenza (*dereferencing*) o risoluzione del riferimento, trasforma un indirizzo nel corrispondente valore referenziato. Si utilizza l'operatore `*` per dereferenziare esplicitamente. Nella sintassi dei metodi, il compilatore può applicare direttamente dereferenziazione e prestito quando appropriato.

Sia Rust che C++ permettono di ridefinire il comportamento degli operatori del linguaggio per tipi arbitrari e di definire tipi generici, che possono essere espansi in una moltitudine di tipo concreti, in funzione di come vengono utilizzati.

Questi meccanismi permettono di definire tipi che si usano sintatticamente in modo simile ai puntatori, ma che aggiungono proprietà come:

- **Possesso:** Garanzia di inizializzazione e rilascio.
- **Conteggio:** Conteggio dei riferimenti.
- **Mutabilità:** Mutabilità interna.
- **Sincronizzazione:** Sincronizzazione concorrente.

12.1.1 Uso dei puntatori

Tramite l'uso di puntatori è possibile costruire strutture dati dinamiche della topologia non prevedibile a priori e/o variabili nel tempo come grafi, alberi, liste. La grande libertà però espone il programmatore a problemi legati alla deduzione di correttezza del codice che lo manipola.

Le regole restrittive imposte dal borrow checker di Rust impediscono combinazioni di aliasing e mutabilità che renderebbero difficile garantire la sicurezza del programma. Con soli riferimenti diventa molto difficile rappresentare strutture cicliche persistenti. Questa minore capacità espressiva abilita, però, analisi più approfondite ed è alla base delle garanzie di sanità offerte dal linguaggio.

Smart pointer come `Rc<T>` e `Arc<T>` introducono il concetto di possesso condiviso. Altri Smart pointer come `std::rc::Weak<T>` e `std::sync::Weak<T>` introducono riferimenti privi di possesso, utili per rappresentare riferimenti privi di possesso ed evitare cicli di possesso forte.

12.2 Smart pointer in Rust

Rust offre una varietà maggiore di smart pointer rispetto al C++, allo scopo di coprire ulteriori casi e definire ottimizzazioni possibili nel caso specifico di programmi puramente sequenziali piuttosto che di programmi concorrenti. Alcuni ricalcano le astrazioni del C++ come `Box`, `Rc`, `Arc`, `Weak`, mentre altri sono peculiari di Rust come `Cell`, `RefCell`, `Cow`, `Mutex`, `RwLock`.

Molti Smart pointer sono realizzati come strutture dati che incapsulano dati e metadati aggiuntivi. Spesso implementano i tratti `Deref` e, talvolta, `DerefMut`, per consentire un accesso al contenuto, simile a quello che

offrirebbe un puntatore. Quando un tipo implementa questi tratti, il compilatore può usarli per rendere naturale l'accesso al contenuto.

- **Possesso esclusivo:** `Box<T>`,
- **Possesso condiviso:** `Rc<T>`, `Arc<T>`, `Weak<T>`
- **Mutabilità condivisa:** `Cell<T>`, `RefCell<T>`
- **Sincronizzazione concorrente:** `Mutex<T>`, `RwLock<T>`
- **Clone on write:** `Cow<'a, B>`

```
trait Deref {
    type Target: ?Sized;
    fn deref(&self) -> &Self::Target;
}

trait DerefMut: Deref {
    fn deref_mut(&mut self) -> &mut Self::Target;
}
```

12.2.1 `std::Box<T>`

Struttura che possiede un valore allocato dinamicamente sullo *heap* all'atto della sua costruzione tramite `Box::new()`.

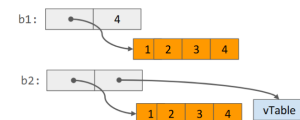
Il dato puntato è **posseduto** dal `Box`: quando esce dal proprio scope sintattico, il blocco sullo heap viene rilasciato automaticamente tramite il tratto `Drop` (anticipabile con `drop(b)`).

Se la struttura viene mossa in un'altra variabile, o ritornata da una funzione, il possesso del valore allocato sullo heap passa alla destinazione che diventa responsabile del suo rilascio. Questo rende possibile ottenere cicli di vita che si estendono oltre la durata della funzione in cui il dato è stato creato.

Il tipo `T` può avere una dimensione non nota in fase di compilazione, ovvero non implementa il tratto `Sized`. In questo caso, l'oggetto `Box<T>` diventa un *fat pointer* formato da un **puntatore** seguito da un intero di dimensione **usize** contenente la lunghezza del dato puntato.

Analogamente, se al posto del tipo concreto `T` si indica con un oggetto-tratto (`dyn Trait`), si ha un fat pointer composto da due puntatori: quello al dato sullo heap e quello a vtable del tratto.

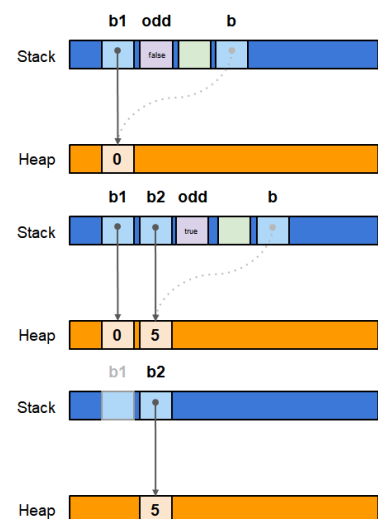
```
let b1: Box<i32> = Box::new([1, 2, 3]);
let b2: Box<dyn Debug> = Box::new([1, 2, 3]);
```



Esempio di Box

```
fn produce(odd: bool) -> Box<i32> {
    let mut b = Box::new(0);
    if odd { *b = 5; }
    return b;
}

fn main() {
    let b1 = produce(false);
    println!("b1: {}", b1);
    let b2 = produce(true);
    drop(b1);
    println!("b2: {}", b2);
}
```



Nota: `b1` diventa inaccessibile già dopo la chiamata alla macro `Drop`

12.3 Conteggio dei Riferimenti

12.3.1 `std::rc::Rc<T>`

Quando occorre disporre di più possessori di uno stesso dato immutabile, si usa `Rc<T>` - Resource Counted. Internamente mantiene il **dato** allocato e due contatori: **riferimenti forti** e **deboli**. Ad ogni **clonazione** del puntatore, il contatore viene **incrementato**; **uscendo dallo scope** viene **decrementato** e, arrivato a 0, il blocco viene rilasciato. Non è *thread-safe* per la concorrenza si usa `Arc<T>`.

Nota: `Rc<T>` si presta a realizzare alberi e grafi aciclici. Non può essere usato da **più di un thread**, bensì solo da programmi non concorrenti dato che manca di sincronizzazione interna.

```
enum List {
  Cons(i32, Rc<List>),
  Nil,
}

use crate::List::{Cons, Nil};
use std::rc::Rc;

fn main() {
  let a = Rc::new(Cons(5, Rc::new(Cons(10,
    ↪ Rc::new(Nil)))));
  let b = Rc::new(Cons(3, Rc::clone(&a)));
  let c = Rc::new(Cons(4, Rc::clone(&a)));
}
```

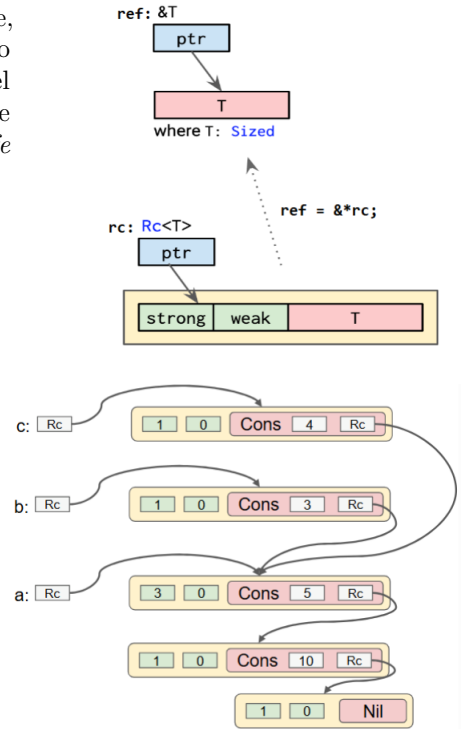


Figura 12.1: Esempio di DAG - Directed Acyclic Graph

Nota: Il grafo costruito con `Rc` deve essere aciclico.

Per evitare problemi di omonimia con i metodi contenuti nel dato incapsulato, tutti i metodi di `Rc` sono dichiarati con la sintassi estesa:

- `pub fn strong_count(this: &Rc<T>) -> usize`
- Chiamando `this`, e non `self`, il parametro che indica l'istanza, non è possibile utilizzare la notazione puntata per invocare i metodi, ma occorre richiamarli nella forma estesa
- `Rc::<T>::strong_count(&a)`

Per motivi di efficienza, l'operazione di incremento e decremento sui campi privati `strong_count` e `weak_count` non è thread-safe. Per questo motivo, non è possibile utilizzare questo smart pointer in un contesto concorrente. Esiste un altro tipo, trattato in seguito, che supera questo limite: `std::sync::Arc<T>`

Mutare il contenuto di `Rc<T>`

Un valore di tipo `Rc<T>` non offre normalmente accesso mutabile diretto al contenuto.

- Il metodo `Rc::get_mut(&mut rc) -> Option<&mut T>` restituisce un riferimento mutabile se e solo se `rc` è l'unico riferimento forte al valore, altrimenti restituisce `None`
- Il metodo `Rc::make_mut(&mut rc) -> &mut T` restituisce sempre un riferimento mutabile: tuttavia, se esistono altri riferimenti forti, il valore verrà prima clonato, così da lasciare invariato quanto conosciuto dagli altri possessori

Quando occorre mutabilità condivisa senza effettuare delle copie, si usa tipicamente `Rc<RefCell<T>>`.

```

use std::rc::Rc;

let mut a = Rc::new(String::from("abc"));

if let Some(s) = Rc::get_mut(&mut a) {s.push('!');}

assert_eq!(&*a, "abc!"); // a = rc *rc = stringa &*a = slice

let mut x = Rc::new(String::from("hello"));
let y = x.clone();

Rc::make_mut(&mut x).push('!');

assert_eq!(&*x, "hello!");
assert_eq!(&*y, "hello!");

```

12.3.2 std::rc::Weak<T>

Se si costruisce una sequenza circolare di puntatori con `Rc<T>`, la memoria non potrebbe più essere rilasciata. Come nel caso di `shared_ptr` in C++, la catena dei puntatori terrebbe in vita tutti i blocchi, garantendo che il conteggio dei riferimenti valga almeno 1.

È possibile rappresentare back-reference e dipendenze cicliche senza introdurre cicli di possesso forte utilizzando il tipo `Weak<T>`. Esso è una versione di `Rc<T>` che contiene un riferimento senza possesso al blocco allocato.

Si crea un valore di tipo `Weak<T>` a partire da un `Rc<T>` con il metodo `Rc::downgrade(&rc)`.

Se il valore originale è ancora in vita, ovvero se `strong_count() > 0`, è possibile costruire un nuovo valore di tipo `Rc<T>` invocando il metodo `upgrade()`, che restituisce un `Option<Rc<T>`.

```

use std::rc::Rc;

let five = Rc::new(5);
let weak_five = Rc::downgrade(&five);
let strong_five: Option<Rc<_>> = weak_five.upgrade();
assert!(strong_five.is_some());

// Destroy all strong pointers.
drop(strong_five);
drop(five);

assert!(weak_five.upgrade().is_none());

```

Note: Il tipo `Weak<T>` offre il supporto per la costruzione di strutture dati cicliche non possibile con `Rc<T>`

12.4 Interior Mutability

12.4.1 std::cell::Cell<T>

In condizioni normali, Rust consente o più riferimenti condivisi immutabili, oppure un unico riferimento mutabile. Esistono situazioni in cui l'analisi statica eseguita in fase di compilazione è troppo restrittiva.

Tuttavia, il modulo `std::cell` offre contenitori per *mutabilità condivisa e controllata* in contesti single-thread. È possibile cioè avere più riferimenti al valore pure essendo in grado di mutarlo. I tipi offerti possono funzionare solo in contesti non concorrenti (basati su singolo thread).

La struct `std::cell::Cell<T>` implementa la mutabilità del dato contenuto al suo interno attraverso metodi che non richiedono la mutabilità del contenitore. Consente accessi per copia/sostituzione del contenuto. Si dice che `Cell` implementa un meccanismo di **interior mutability**

```

use std::cell::Cell;
struct SomeStruct {
    a: u8,
    b: Cell<u8>,
}

```

```
let my_struct = SomeStruct { a: 0, b: Cell::new(1) };
// my_struct.a = 100; // ERRORE: `my_struct` è immutabile
my_struct.b.set(100); // OK: anche se `my_struct` è immutabile, `b` è una Cell e può essere
↳ modificata/assert_eq!(my_struct.b.get(), 100);
assert_eq!(my_struct.b.get(), 100);
```

Metodi di Cell<T>

- `get(&self) -> T` - restituisce il dato contenuto al suo interno, a condizione che T implementi il tratto `Copy`
- `take(&self) -> T` - restituisce il valore contenuto, sostituendolo con il risultato dell'invocazione di `Default::default()`. A condizione che T implementi il tratto **Default** che permette la costruzione di un valore predefinito
- `replace(&self, val: T) -> T` - sostituisce il valore contenuto nella cella con quello passato come parametro e lo restituisce come risultato. Questo metodo non pone restrizioni sul tipo di dato
- `into_inner(self) -> T` - consuma la cella e restituisce il valore contenuto. Anche in questo caso, può essere usato con ogni tipo di dato

Nota: Come `Rc<T>`, non è possibile utilizzarlo in un contesto concorrente.

12.4.2 std::cell::RefCell<T>

Simula riferimenti condivisi e mutabili verificati in fase di esecuzione (a runtime) anziché compilazione. Tentativi di violazione (es. chiamare `borrow_mut()` mentre c'è già un prestito) generano **panic**.

`RefCell<T>`



`Cell<T>` non consente di creare riferimenti al dato contenuto al suo interno, ma solo di inserire, estrarre o sostituire il valore.

La struct `std::cell::RefCell<T>` consente di ottenere oggetti (guardie) che si comportano come riferimenti (condivisi o mutabili) al contenuto, con regole di prestito verificate all'atto dell'esecuzione e non in compilazione. Eventuali tentativi di violazione delle regole generano una condizione di **panic**.

Metodi di RefCell<T>

- `borrow(&self) -> Ref<'_, T>` - restituisce una guardia che si comporta come un riferimento condiviso. Oppure provoca un **panic** se è già presente un riferimento mutabile.
- `borrow_mut(&self) -> RefMut<'_, T>` - restituisce una guardia che si comporta come un riferimento mutabile. Oppure provoca un **panic** se è già presente un riferimento semplice.

Note: Il programma può andare in **panic** se si tenta di violare le regole di prestito.

```
use std::cell::RefCell;
let c = RefCell::new(5);
{
    let mut m = c.borrow_mut();
    assert!(c.try_borrow().is_err());
    *m = 6;
}

{
    let m = c.borrow();
    assert!(c.try_borrow().is_ok());
    assert!(*m == 6);
}

assert_eq!(*c.borrow(), 6);
```

12.4.3 std::borrow::Cow<'a, B>

Smart pointer che implementa il meccanismo *clone on write* tramite `enum` (`Borrowed`, `Owned`). Se si modifica un dato condiviso, il dato viene clonato: si prende possesso della copia e si effettua la modifica, lasciando l'originale invariato; Altrimenti se il dato che si vuole modificare era già posseduto, si modifica direttamente l'originale.

Implementato sotto forma di enumerazione:

```
pub enum Cow<'a, B>
where B: 'a + ToOwned + ?Sized,
{
    Borrowed(&'a B),
    Owned(<B as ToOwned>::Owned),
}
```

Si istanzia attraverso il metodo `Cow::from(...)`

In base al dato da incapsulare, viene scelta la variante **Owned** o **Borrowed**.

```
use std::borrow::Cow;
// Riusa la slice se non occorre
// effettuare modifiche
// Altrimenti, alloca una nuova String
fn normalize_whitespace(line: &str) ->
    Cow<'_, str> {
    if line.contains('\t') {
        Cow::Owned(line.replace('\t', " "))
    } else {
        Cow::Borrowed(line)
    }
}
```

```
fn main() {
    let a = "INFO user=alice op=login";
    let b = "WARN\tuser=bob\top=logout";
    let na = normalize_whitespace(a);
    let nb = normalize_whitespace(b);
    println!("a -> {}", na);
    println!("b -> {}", nb);

    assert!(
        matches!(na, Cow::Borrowed(_));

    assert!(
        matches!(nb, Cow::Owned(_));
}
```

12.4.4 Smart pointer e metodi (`self`)

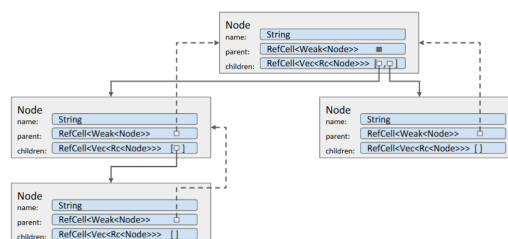
Nella firma di un metodo, è possibile utilizzare, oltre a `self` ed alle sue variazioni, anche `Box<Self>`, `Rc<Self>`, o `Arc<Self>`. In tal caso, il metodo può essere solo invocato a partire dal corrispondente tipo di puntatore. L'invocazione del metodo passa la proprietà del puntatore al metodo stesso.

A differenza di quanto accade con i riferimenti, non è disponibile una forma abbreviata per la sintassi di `self`. In questi casi il tipo del receiver deve essere dichiarato esplicitamente, ad esempio, `self: Rc<Self>`.

```
impl Node {
    fn append_to(self: Rc<Self>, parent: &mut Node) {
        parent.children.push(self);
    }
}
```

12.5 Esempio: Albero doppiamente collegato

Un albero dove i nodi figli conoscono il padre e viceversa si implementa combinando `Rc`, `RefCell` e `Weak`.



```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

impl Node {
    fn new(name: &str) -> Rc<Self> {
        Rc::new(Node {
            name: name.to_string(),
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }

    fn add_child(parent: Rc<Node>, child: Rc<Node>) {
        *child.parent.borrow_mut() = Rc::downgrade(&parent);
        parent.children.borrow_mut().push(child);
    }

    fn print_tree(node: &Rc<Node>, depth: usize) {
        println!("{}", " ".repeat(depth * 2), node.name);
        for child in node.children.borrow().iter() {
            Node::print_tree(child, depth + 1);
        }
    }

    fn get_root(node: &Rc<Node>)-> Rc<Node> {
        let mut n = node.clone();
        loop {
            let parent = n.parent.borrow().upgrade();
            match parent {
                Some(p) => n = p.clone(),
                None => break,
            }
        }
    }

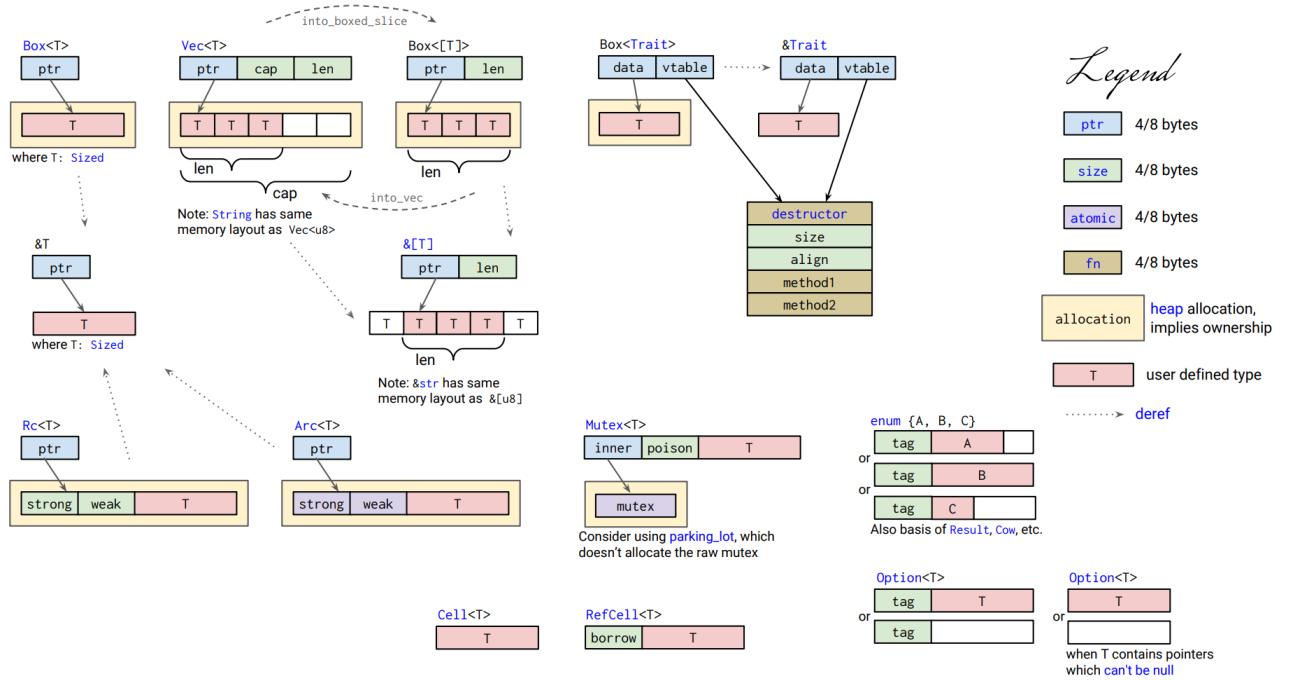
    fn main() {
        let root = Node::new("root");
        let child1 = Node::new("child1");
        let child2 = Node::new("child2");
        Node::add_child(root.clone(), child1.clone());
        Node::add_child(root.clone(), child2);
        Node::add_child(child1.clone(), Node::new("subchild"));
        Node::print_tree(&Node::get_root(&child1), 0);
    }
}

//root
//  child1
//    subchild
//  child2

```

12.6 Cheat Sheet dei Container in Rust

Per facilitare la comprensione del layout in memoria dei vari container e puntatori intelligenti:



Capitolo 13

Concorrenza

13.1 Programmazione concorrente

Un programma concorrente dispone di due o più flussi di esecuzione contemporanei, per perseguire un obiettivo comune probabilmente in un tempo inferiore.

Tali flussi possono essere eseguiti in parallelo (se il processore dispone di più core) e/o alternarsi nel tempo, sotto il controllo di uno schedulatore.

All'atto della creazione, un processo dispone di un **unico flusso di esecuzione**, il thread principale, che esegue il codice del programma, il quale può chiedere allo schedulatore di creare nuovi thread.

Un thread rappresenta una **computazione indipendente**.

Basata su un proprio stack (pre-allocato all'atto della creazione del thread) collocato nello stesso spazio di indirizzamento in cui operano gli altri thread del processo. Tale computazione si svolge fino al proprio termine, potendo dare origine ad un **risultato** o ad un errore, provocando un **abort**.

Il S.O. e/o le librerie di supporto allocano le risorse fisiche necessarie, lo scheduler ripartisce, nel tempo, l'utilizzo dei core disponibili tra i diversi thread in modo non deterministico. Tutti i thread creati sono identificati in modo univoco e viene mantenuto, per quelli in uso, l'indicazione del loro stato di esecuzione.

La gestione dei thread può essere demandata direttamente al S.O., in questo caso si parla di **thread nativi**, oppure gestita da librerie a livello utente, con il supporto parziale del sistema operativo. Questi vengono detti **green thread** o **fibre** e richiedono una certa forma di cooperazione da parte del codice che deve, in modo esplicito, invocare lo scheduler per cedere l'uso della CPU.

C++ e Rust offrono, nella propria libreria standard, supporto per i thread nativi, inoltre, entrambi offrono librerie di terze parti per il supporto di green thread nascondendo i dettagli implementativi.

13.2 Thread nativi

I diversi sistemi operativi offrono funzionalità simili, ma non identiche, per governare l'interazione di un programma con l'insieme dei thread che lo costituiscono

Le funzioni supportate da tutti i sistemi operativi sono:

- **Creazione:** di un thread, indicando la funzione che rappresenta la computazione che deve essere svolta e la dimensione dello stack richiesto: questa operazione restituisce un **handle** opaco mediante il quale fare riferimento al thread
- **Identificazione:** del thread corrente, sotto forma di valore univoco a livello di sistema (TID - Thread ID)
- **Attesa:** della terminazione di un thread, a partire dalla sua handle, e accesso al suo stato finale, successo/fallimento.

Tra le funzioni non supportate, spicca la richiesta di **cancellazione** di un thread. Questa può solo essere implementata in modo cooperativo dal thread stesso. Per farli terminare sta al programmatore a scrivere del codice che li faccia terminare.

Entità	Memoria	Gestione
Processo	Spazio di indirizzamento proprio	Sistema operativo
Thread nativo	stack proprio, heap/variabili globali/codice condivisi	Sistema operativo
Green thread / Fibra	Possono, come no, avere uno stack. Nel processo ospitante	Libreria / runtime

13.2.1 Processi, thread e modelli di esecuzione

Nota: Più thread nello stesso processo condividono i dati, ma non lo stack

Nei processi, nei sistemi Unix, la creazione di un nuovo processo con **fork**, si crea una copia completa dello spazio di indirizzamento del processo padre, rendendo difficile la comunicazione tra i processi, che avviene tramite meccanismi di IPC (Inter Process Communication).

Il concetto di thread, ovvero condividere lo spazio di indirizzamento tra più flussi di esecuzione, introduce molte insidie.

13.2.2 Cosa implica la concorrenza

Ad inizio degli anni 2000, era difficile scrivere programmi concorrenti perché non esistevano strumenti per gestire la multiplatforma, fino a che l'uscita di Java, con il suo supporto nativo per i thread e multiplatforma. Il C++ solo nel 2011, 13 anni dopo Java, ha introdotto questo supporto mancante.

Possibilità di **sovrapporre temporalmente** attività di computazione e operazioni di I/O:

- I sistemi operativi tendono ad offrire API bloccanti che arrestano, di fatto, la prosecuzione di un thread fino a che il dato richiesto non è pronto (es.: `read(fd)`, `accept(socket)`, ...)
- Suddividendo l'algoritmo in più thread, si può cercare sfruttare i tempi di attesa che un dato thread subisce a seguito delle operazioni di I/O per eseguire, in altri thread, operazioni utili al risultato
- Occorre che la complessità aggiunta dalla suddivisione sia compensata da un effettivo guadagno in termini di tempi di esecuzione

Ci sono molti motivi per utilizzare i thread, tra cui efficientare e utilizzare al meglio le risorse hardware che abbiamo a disposizione. Sfruttando i momenti morti, come l'I/O, per eseguire altre operazioni.

Riduzione del sovraccarico dovuto alla comunicazione tra processi

- Sebbene la suddetta parallelizzazione possa essere fatta anche creando processi separati, il costo di comunicazione e sincronizzazione tra processi è sensibilmente più alto di quello tra thread
- I thread condividono infatti lo spazio di indirizzamento: questo rende più efficiente la condivisione dei dati rispetto a processi distinti
- Tale efficienza **non implica sicurezza** automatica: gli accessi concorrenti ai dati condivisi richiedono **sincronizzazione**

Potrei creare due processi distinti, ma durante il merge dei dati porta ad una serializzazione di dati che potrebbe essere più costosa. Lo spazio di indirizzamento tra thread è condiviso, sia nel bene che nel male.

- Possibilità di sfruttare appieno le capacità di **elaborazione** delle CPU **multicore**. Più flussi di esecuzione possono svolgersi contemporaneamente, riducendo così il tempo totale di elaborazione: vero parallelismo
- **Aumento significativo della complessità** del programma. Nuove fonti e tipologie di errore. **Non determinismo dell'esecuzione**, non riuscendo a definire a livello di sistema operativo l'ordine di esecuzione dei thread.
- La memoria **non** può più essere pensata come un "**deposito statico**". I dati scritti al suo interno possono cambiare in conseguenza dell'attività di altri thread
- I thread devono **coordinare l'accesso** alla memoria. Tramite opportuni costrutti di sincronizzazione. La presenza di cache legate ai singoli core introduce non determinismo nell'ordinamento e nella visibilità delle azioni sulla memoria

13.2.3 Concorrente e parallelismo

Concorrente più flussi di elaborazione **attivi** nello stesso intervallo di tempo, il cui avanzamento può avvenire in alternanza su uno stesso core sotto il controllo dello scheduler, oppure in parallelo se sono disponibili più core.

Parallelismo caso particolare in cui i flussi di elaborazione sono realmente in esecuzione nello stesso istante poiché assegnati a core distinti

Nota: Ogni parallelismo è una forma di concorrenza, ma non ogni concorrenza implica parallelismo

13.2.4 Concorrenza in pratica

Se, all'interno di un processo, sono presenti due o più thread, questi possono procedere indipendentemente nella propria computazione. Sebbene sia possibile creare thread che si ignorano reciprocamente e non necessitano alcuno scambio di informazione, sul piano pratico questo avviene molto raramente.

L'utilità di suddividere la computazione globale in più sotto-computazioni nasce, per lo più, dal fatto che ciascuna di esse contribuisce in qualche modo al risultato finale. Questo richiede che esista una forma di comunicazione/sincronizzazione tra thread differenti.

I meccanismi soggiacenti alla comunicazione/sincronizzazione interferiscono con le ottimizzazioni usate dai processori per migliorare l'esecuzione. Introducendo una serie di **complessità inattese** e lontane dal pensiero comune legato al modello di esecuzione sequenziale.

In un sistema single-core, il concetto di thread è puramente una astrazione offerta dal sistema operativo. Il processore si limita ad alternare il proprio ciclo di esecuzione basato sulla successione delle microoperazioni **fetch/decode/execute**, procedendo di istruzione in istruzione secondo la logica del codice macchina.

Il sistema operativo può intervenire in questa sequenza, grazie ad un'interruzione che attiva lo scheduler. Questo salva lo stato dei registri in una qualche area di memoria dedicata alle meta-informazioni del thread corrente e li ripristina con il contenuto relativo ad un thread differente, **task switching**.

Se la CPU è dotata di due o più core, non cambia molto. Ciascun core procede indipendentemente dagli altri e lo scheduler provvede a gestire le attività di tutti, allocando di volta in volta i core disponibili ad eseguire l'uno o l'altro thread, secondo le necessità.

Poiché l'esecuzione di ciascun thread procede indipendentemente da quelle degli altri, un'eventuale necessità di comunicazione deve essere soddisfatta passando per l'utilizzo di un'area di memoria condivisa. In cui il thread T1 possa depositare le informazioni che intende comunicare al thread T2.

Sebbene i due thread utilizzino lo stesso spazio di indirizzamento e possano, in linea di principio, accedere al dato memorizzato, questa operazione risulta **più complessa** di quanto si possa pensare a prima vista. Per rendere la comunicazione utile sul piano pratico, può essere inoltre necessario avvalersi di pattern di interazione che definiscano con precisione i ruoli che le due o più parti coinvolte possono giocare.

13.3 Modello di memoria

Quando un thread legge il contenuto di una locazione di memoria, può trovare:

- Il valore iniziale contenuto nel file eseguibile che è stato mappato in memoria (es: variabile globale inizializzata)
- Il valore che questo stesso thread ha precedentemente depositato all'interno della locazione
- Il valore che è stato depositato da un altro thread

La presenza di cache hardware e il possibile riordinamento delle istruzioni da parte della CPU rendono il terzo caso problematico. In generale non è predicibile quale valore venga letto senza controllare letture e scritture da parte dei thread. Occorre usare un costrutto di sincronizzazione esplicito che permetta di definire l'ordine di esecuzione



13.3.1 Problemi aperti

La cache porta con se un problema, ogni thread agisce sulla propria cache, se vogliamo che i dati fluiscano tra una cache e l'altra occorre tempo.

Sorgono 3 problemi che in un contesto sequenziale non sono presenti:

- **Atomicità:** Solitamente vogliamo salvare più di un byte alla volta. Non è detto che quella operazione stia perfettamente in una riga di cache.
- **Visibilità:** Ogni thread scrive sulla propria cache, e solo forzando il sync della cache i dati sono resi visibili a tutti
- **Ordinamento:** La sequenza delle operazioni, dato il tempo di re-sync, non è garantita e l'ordine dei risultati non è predicibile

La cache è un fatto puramente Hardware, in nessun linguaggio di programma, se pur a basso livello come ARM, non è possibile accedere direttamente alla cache. Il processore non ha consapevolezza della sua esistenza.

Problema	Domanda chiave	Esempio intuitivo
Atomicità	L'operazione è atomica?	a += 1 consiste in una lettura seguita da una scrittura, 3 operazioni, non è atomica
Visibilità	Un altro thread vede davvero la scrittura?	Un thread legge un valore obsoleto
Ordinamento	Gli altri thread osservano operazioni distinte nello stesso ordine?	Scritture viste in un ordine diverso

13.4 La risposta dei processori

Ciascuna famiglia di processori offre una propria risposta ai problemi menzionati.

La piattaforma **x86** adotta un modello **quasi sequenzialmente consistente** ed offre le istruzioni di tipo *fence*, che forzano il completamento delle operazioni di scrittura, bloccando temporaneamente gli altri core che dovessero cercare di accedere nel frattempo allo stesso segmento di indirizzi

Per contro, sulla piattaforma **ARM** viene usato un modello molto più lasco, basato su **liste di propagazione dei cambiamenti**, ed offre le istruzioni di tipo *barrier* che consentono l'ordinamento causale rispettivamente per il calcolo degli indirizzi, delle istruzioni, dei dati.

Se tali istruzioni non vengono incluse all'interno del codice generato, **non è garantito un ordinamento predicibile**, quando due thread diversi, alle operazioni di **lettura e scrittura di dati condivisi**, in presenza di attività concorrenti. C++ e Rust condividono il modello di memoria su cui sono basati e annegano, nelle funzioni di libreria dei tipi dedicati alla concorrenza, tali istruzioni allo scopo di garantire le necessarie proprietà di funzionamento

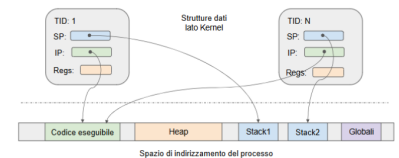
13.4.1 Errori comuni

- L'**uso superficiale** dei costrutti di sincronizzazione porta a blocchi passivi o attivi del programma. In ogni caso, fonte di guai per il programmatore
- L'**assenza** di costrutti di sincronizzazione, porta a risultati imprevedibili. Anche molto lontani da quanto sarebbe logico aspettarsi

- Si possono verificare **malfunzionamenti casuali**: Dovuti al comportamento non deterministico e asincrono dell'esecuzione concorrente. Estremamente difficili da riprodurre e da eliminare
- Gli errori possono manifestarsi cambiando la **piattaforma di esecuzione**, oppure soltanto dopo numerose esecuzioni. Tipicamente emergono nel momento meno adatto (chiaro).

13.5 Thread e memoria

Il sistema operativo mantiene, al proprio interno, una rappresentazione dei thread presenti all'interno di un dato processo. Ad ogni thread viene associato un identificativo univoco (TID - Thread ID), il suo stato di esecuzione (non schedabile, schedabile, in esecuzione sul core i, terminato con errore, terminato senza errore, ...) e le informazioni necessarie a salvare/ripristinare lo stato dei registri interni al processore. Provvede inoltre ad allocare, nello spazio di indirizzamento del processo, un blocco di indirizzi contigui destinato ad ospitare lo stack che governerà la sua esecuzione



Tutti i thread presenti in un processo condividono:

Le variabili globali - le costanti - l'area eseguibile in cui è contenuto il codice - lo heap

13.5.1 Esecuzione e non determinismo

- L'esecuzione di ogni singolo thread procede secondo le normali regole sequenziali. Per cui è possibile prevedere cosa avvenga "prima" e cosa "dopo"
- Se più thread sono in esecuzione, non è possibile fare assunzioni sulle velocità relative di avanzamento, se non ricorrendo a forme esplicite di sincronizzazione e comunicazione
- La sincronizzazione può riguardare il raggiungimento di un particolare stato da parte di un thread, abilitando di conseguenza altri a procedere, oppure l'esigenza di un thread di eseguire azioni su aree condivise. Allo scopo di impedire ad altri di accedere alle stesse aree
- In alcuni casi, all'informazione logica che abilita/impedisce la prosecuzione di altri thread, si accompagna il trasferimento di informazioni più strutturate. Che rappresentano l'esito totale o parziale di una computazione avvenuta o la richiesta di elaborazione di ulteriori dati

```
use std::thread;
use std::time::Duration;

fn run(msg: &str) {
    for i in 0..10 {
        println!("{}",msg,i);
        thread::sleep(Duration::from_nanos(1));
    }
}

fn main() {
    let t1 = thread::spawn(|| {run( "aaaa"); });
    let t2 = thread::spawn(|| {run( "bbbb"); });
    t1.join().unwrap(); //uso di unwrap perché ritengo che non fallisca
    t2.join().unwrap();
    // sicuramente saranno stampati i vari aaaa e bbbb ma non possiamo dire
    // l'ordine
}
```

```
aaaa0
aaaa1
aaaa2
bbbb0
bbbb1
bbbb2
aaaa3
aaaa4
aaaa5
aaaa6
aaaa7
bbbb3
bbbb4
bbbb5
aaaa8
aaaa9
bbbb6
bbbb7
bbbb8
bbbb9
```

13.6 Esecuzione e non determinismo

L'output dei programmi precedenti è **solo uno**

```

#include <iostream>
#include <thread>
int a = 0; //Questo non è possibile in safe RUST
void run() {
    while (a >= 0) {
        int before = a;
        a++; //il delta negativo è dato da questa operazione (non è atomica)
        int after = a;

        //in un contesto sequenziale, non passerebbe mai
        if (after - before != 1)
            std::cout << before << " -> " << after
                << "(" << after - before << ")\n";
    }
}

//creo due thread e ne attendo la terminazione
int main() {
    std::thread t1(run);
    std::thread t2(run);
    t1.join();
    t2.join();
}

```

Esaminando l'uscita del programma precedente, si vedono molti casi in cui la differenza tra **after** e **before** è superiore a 1. In alcuni casi tale valore è anche molto grande: perché? Talora capita che la differenza sia **nulla**, **negativa**. Come è possibile, se entrambi i flussi incrementano sempre la variabile **a**?

13.6.1 Interferenza

Si verifica quando più thread fanno accesso a uno stesso dato, **modificandolo**, in assenza di sincronizzazione.

La sua presenza dà origine a **malfunzionamenti casuali**, molto difficili da identificare.

13.7 Sincronizzazione

Se due thread cercano di accedere in lettura/scrittura ad una stessa variabile si verifica una corsa critica, **race condition**. In base a condizioni non controllabili dal programmatore (come la presenza di memoria cache, il momento in cui avviene un task switch, le ottimizzazioni fatte dai singoli processori con la predizione della prossima istruzione da eseguire, ...) il dato memorizzato potrebbe essere quello scritto dal primo thread, quello scritto dal secondo oppure un terzo valore **completamente arbitrario**.

L'accesso in lettura/scrittura a variabili il cui contenuto è, potenzialmente, scritto da altri thread è soggetto a diversi vincoli:

- Deve essere preceduto/seguito da istruzioni e proteggano da dati obsoleti presenti nella cache (fence/barrier)
- Deve avvenire solo quando c'è l'evidenza che il dato non sta venendo modificato da altri
- Se si sta operando una lettura in attesa di un risultato, si vuole evitare di eseguire cicli continui di polling, che consumano inutilmente cicli di CPU e batteria

Alcune delle condizioni citate sono garantite da **apposite istruzioni macchina**. Che dipendono dal processore responsabile dell'esecuzione del codice.

Altre richiedono la garanzia di **invarianti a livello sistema** che può essere fornita solo da meccanismi offerti dal sistema operativo, che, controllando la schedulazione dei thread, può farsi carico che avvenga / non avvenga una determinata condizione.

Che sono i tipi atomici, i mutex operano su una struttura grande a piacere, e se dobbiamo aspettare una codizione particolare ci sono le **condition mutex** utilizzabili assieme ad un mutex.

Ne consegue che i meccanismi di sincronizzazione dipendono dalla coppia processore/sistema operativo, che collettivamente definiscono l'interfaccia binaria dell'applicazione. ABI - Application Binary Interface.

Le librerie standard dei diversi linguaggi di programmazione si fanno, talora, carico di standardizzare tale comportamento, offrendo API comuni a livello di codice sorgente. Come nei casi di C++11 e successivi e di Rust

13.7.1 Strutture native di sincronizzazione

	Windows	Linux
Strutture dati utente	CriticalSection	pthread_mutex
	SRWLock	pthread_cond
	ConditionVariable	
Oggetti kernel	Mutex	Semaphore
	Event	Pipe
	Semaphore	Signal
	Pipe	Futex
	Mailslot	

13.8 Correttezza

Processo logico, partendo da particolari assiomi, di dimostrare che un programma concorrente è privo di errori. In questo caso ci basiamo sugli assiomi di Mutex, atomic e convar per dimostrare che un programma è privo di errori di sincronizzazione.

1. Occorre fare in modo che non capitino mai che un thread “operi” su un dato, alterandone il contenuto. Mentre un altro sta già operando sullo stesso oggetto.
2. In particolare, non devono essere visibili **stati transitori** dell’oggetto. Dovuti al meccanismo di aggiornamento in cui solo una parte dell’informazione contenuta è cambiata.
3. Tutti gli oggetti condivisi mutabili devono godere di questa proprietà. Questi mantengono al proprio interno degli “invarianti” definiti a livello applicativo. Perché gli oggetti immutabili non sono soggetti a interferenza?
4. Bisogna impedire che gli invarianti siano violati. Si effettuano le mutazioni (cambi di stato) con metodi che garantiscono la validità degli invarianti prima e dopo l’esecuzione e che bloccano l’accesso concorrente mentre la mutazione è in corso.
5. Si accede allo stato attraverso altri metodi. Che controllano che non ci sia una mutazione in corso. E che impediscono che essa inizi mentre si sta facendo accesso allo stato condiviso

In quasi tutti i linguaggi di programmazione, è compito del programmatore riconoscere quando, dove e come utilizzare la sincronizzazione. Un uso sbagliato porta a risultati disastrosi.

Occorre dimostrare la correttezza formale dell’algoritmo complessivo. Garantendo che, qualunque sia l’ordine di esecuzione scelto dal sistema operativo, la latenza introdotta dai sottosistemi di memoria, il programma resta corretto. Questo è possibile grazie alla presenza di primitive offerte dal sistema operativo / compilatore / librerie di esecuzione che garantiscono alcuni invarianti di basso livello.

Se un programma non è corretto, è inutile pensare di testarlo o ottimizzarlo. L’esito dei test è non deterministico: potrebbero passare per pura combinazione fortuita (o anche probabile) delle condizioni al contorno, ma non danno alcuna garanzia. Le ottimizzazioni non farebbero che esacerbare il problema, rendendone più difficile la comprensione

13.8.1 Safe Rust

In Rust, le limitazioni imposte dal borrow checker sulla esclusività dell’accesso in scrittura, unite all’utilizzo di tratti che modellano il comportamento che un tipo esibisce quando viene passato da un thread ad un altro, diventano garanti della correttezza degli accessi. Trasformando errori in esecuzione difficili da identificare e replicare in errori di compilazione. Questo ha portato a definire questo aspetto di Rust come *fearless concurrency*.

Resta comunque responsabilità del programmatore la comprensione dell'algoritmo e la garanzia di terminazione dello stesso, così come la verifica dell'assenza di blocchi attivi e passivi che possono impedire il procedere dell'algoritmo.

13.9 Correttezza: non solo sicurezza, ma anche progressione

Per essere corretto dal punto di vista della concorrenza un programma deve esibire due proprietà fondamentali:

- **Safety:** Dove il safe rust mi da garanzia
- **Liveness:** Il programma prima o poi deve finire

Si ha sicurezza quando non sono presenti corse critiche né stati inconsistenti.

Si ha garanzia di progressione quando sono assenti i seguenti fenomeni:

- **Deadlock:** due o più thread restano bloccati in attesa reciproca
- **Livelock:** i thread continuano a reagire senza fare progresso (essere troppo educati non va bene)
- **Starvation:** un thread resta escluso troppo a lungo dall'accesso ad una risorsa data la loro più alta priorità

Note: Rust aiuta sulla safety, ma non elimina automaticamente i problemi di liveness

13.9.1 Accesso condiviso: i possibili problemi

- **Atomicità:** quali operazioni di memoria hanno effetti indivisibili? Se due thread **fanno accesso alla stessa struttura dati**, rispettivamente in lettura e scrittura, non c'è nessuna garanzia su quale delle due operazioni sia **eseguita per prima**.
- **Visibilità:** la scrittura di una variabile può essere osservata da una lettura eseguita da un altro thread? Se un thread legge un dato che un altro thread sta modificando, il valore letto può essere **diverso** sia dal valore **iniziale** che da quello **finale**.
- **Ordinamento:** sotto quali condizioni, sequenze di operazioni effettuate da un thread sono visibili nello stesso ordine da parte di altri thread? Se, **quando osservato** dall'esterno, il comportamento di un singolo thread appare indistinguibile a seguito di una modifica alla sequenza delle istruzioni. Sia il compilatore che la CPU possono **invertire l'ordine di esecuzione** delle singole istruzioni

13.9.2 Accesso condiviso: le possibili soluzioni

Atomic

Alla base di tutti i meccanismi di accesso condiviso ci sono istruzioni apposite, offerte dai singoli processori, volte a garantire operazioni di tipo Read-Modify-Write di tipo atomico (cioè, non interrompibili e non osservabili nei loro stati intermedi), su CPU single- e multi-core. Tali operazioni sono limitate a tipi semplici (booleani, interi, puntatori) e sono esposte dalle librerie standard di C++ e Rust attraverso opportune astrazioni, che incapsulano l'utilizzo di barriere di memoria

13.9.3 Mutex

Per estendere le garanzie di atomicità e dipendenza causale a strutture dati più complesse, occorre introdurre il concetto di Mutex. Essi estendono il principio della mutua esclusione a thread differenti. Un mutex può essere libero o posseduto da un singolo thread. Se un secondo thread cerca di ottenere il possesso del mutex mentre è in uso da parte di un altro thread, rimane in attesa (senza consumare cicli di CPU) fino a che esso non viene rilasciato

Condition variable

In alcuni casi, occorre attendere - senza consumare cicli di CPU - che si verifichi una condizione più complessa del semplice rilascio di un mutex da parte di un thread. Una condition variable permette di realizzare tale attesa, a condizione che il thread che causa l'avverarsi della condizione si occupi di segnalarlo, generando una notifica tramite appositi metodi. Una condition variable può essere usata solo in coppia con un mutex

13.9.4 Quale strumento utilizzare?

Caso	Strumento
Valore semplice e condiviso	Tipo atomico
Stato mutabile condiviso	Mutex<T>
Molte letture, poche scritture	RwLock<T> (letture parallele)
Attesa di una condizione	Condvar + Mutex<T>
Trasferimento di dati tra thread	Canale, sincronizza attraverso la comunicazione

13.10 Uso dei thread

Le API dei S.O. permettono la gestione del ciclo di vita dei thread

- Creazione e terminazione di thread
- Meccanismi di sincronizzazione
- Aree private di memoria

I dettagli relativi a ciascuna piattaforma differiscono alquanto. Rendendo complessa la portabilità delle applicazioni.

La versione 2011 del linguaggio C++ ha introdotto una standardizzazione nella creazione e gestione dei thread. Tale standardizzazione, tuttavia, nello sforzo di uniformare i comportamenti, nasconde le peculiarità offerte dai singoli sistemi operativi per gestire i casi particolari connessi alla computazione, come cancellazione e fallimento.

Una standardizzazione analoga è offerta dalla libreria standard di Rust, con una maggiore attenzione alla gestione del fallimento.

13.11 Thread in Rust

Si crea un thread nativo in Rust attraverso la funzione `std::thread::spawn(...)`

- Accetta una funzione lambda che rappresenta la computazione che il thread deve svolgere
- Ritorna una struct di tipo `std::thread::JoinHandle<T>`, dove T rappresenta il tipo restituito dalla computazione del thread, ovvero il tipo ritornato dalla funzione lambda.

Per sapere quando la computazione del thread è terminata e quale valore abbia prodotto, occorre utilizzare il metodo `join()` offerto dalla handle. Tale metodo restituisce un'enumerazione di tipo `std::thread::Result<T>` che contiene, nell'operazione **Ok**, il valore finale e nell'opzione **Err** il valore eventualmente passato nella macro `panic!()`, nel caso in cui sia stata invocata nel corso della computazione del thread stesso.

Non si crea **nessun rapporto di parentela** tra thread creatore, quello in cui si invoca `spawn()`, e thread creato, né occorre che l'uno sopravviva all'altro.

Quando l'handle di un thread esce dello scope e viene rilasciata, non c'è più modo di avere notizie, dirette, sull'esito del thread creato, che acquisisce lo stato `detached`.

```
use std::thread;

//move trasferisce alla funzione il possesso di quanto catturato
//la durata del thread è indipendente da quella del thread che lo ha creato
let let thread_join_handle = thread::spawn(move || {

    //computazione da eseguire
});

//altre attività da eseguire

match thread_join_handle.join() {
    Ok(result) => println!("Il thread ha restituito: {:?}", result),
    Err(why) => println!("Il thread è terminato con un panic: {:?}", why),
}
```

13.11.1 Configurare un thread

È possibile configurare un thread prima di lanciare la sua esecuzione tramite la struct `std::thread::Builder`. Questo permette di assegnare un nome ad un thread e di definire la dimensione dello stack da allocare per il thread stesso.

Il metodo `spawn()` consuma l'oggetto `Builder`, crea il thread corrispondente e restituisce un enum di tipo `io::Result<JoinHandle>`

```
use std::thread;
let builder = thread::Builder::new()
    .name("t1".into())
    .stack_size(100_000);
let handler = builder.spawn(|| { /* codice */}).unwrap();
handler.join().unwrap();
//unwrap(): certamente in questo caso non fallirà
```

13.12 I tratti della concorrenza

Nell'ambito del proprio sforzo di garantire la correttezza degli accessi alla memoria e l'assenza di comportamenti non definiti, Rust introduce due tratti **marcatori**, senza metodi, il cui scopo è fornire indicazioni sul comportamento di un tipo in un contesto multi-thread.

- **Send**: il tratto `std::marker::Send` è applicato automaticamente a tutti i tipi che possono essere trasferiti in sicurezza da un thread ad un altro, ovvero in grado di garantire che non è possibile avere accessi al loro contenuto **contemporaneamente** come ad esempio numeri, stringhe, etc.
- **Sync**: il tratto `std::marker::Sync` è applicato automaticamente a tutti i tipi **T** tali che **&T** risulta avere il tratto **Send**, ovvero che **possono essere condivisi** in sicurezza tra thread differenti, senza creare problemi di comportamenti non definiti.

Puntatori e riferimenti **non hanno** non hanno il tratto **Send**, infatti l'esecuzione indipendente dei thread non consente infatti al borrow checker di fornire le proprie garanzie di correttezza. Sono entrambi unsafe.

Tratto send

`pub unsafe auto trait Send { }`. Se un tipo dispone del tratto **Send**, è lecito passarlo per **valore** ad altri thread. L'uso del movimento (o della copia, là dove possibile) garantisce la non contemporaneità degli accessi.

I tipi composti, struct, tuple, enum, array, godono del tratto **Send** se tutti i loro campi lo posseggono. È possibile forzare l'assegnazione/rimozione di tale tratto solo all'interno di un blocco unsafe: il programmatore deve essere conscio della scelta adottata e farsi carico della relativa responsabilità

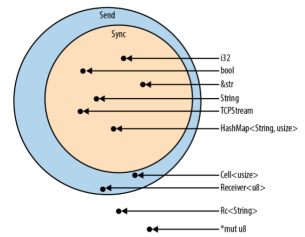
Tratto Sync

`pub unsafe auto trait Sync { }`. Se un tipo dispone del tratto **Sync**, è lecito passarlo **come riferimento non mutabile** ad altri thread o, in altre parole, un tipo è **Sync** se è lecito accedervi in modo concorrente a partire da un riferimento non mutabile.

I tipi che implementano una mutabilità interna, come `Cell` e `RefCell`, non dispongono di questo tratto.

13.12.1 I tratti della concorrenza: riassunto

- **Send**: Il valore può essere spostato in sicurezza in un altro thread
- **Sync**: Un riferimento condiviso **&T** può essere usato in sicurezza in un altro thread



Note: **Send** riguarda il trasferimento, **Sync** la condivisione.

È possibile creare thread solo se i dati catturati dalla funzione lambda che ne descrive la computazione e il suo tipo di ritorno hanno il tratto **Send**, altrimenti il compilatore genererà un errore.

In mancanza di questa garanzia, il borrow checker genera un errore di compilazione, rilevando la propria impossibilità a garantire la correttezza di quanto si sta chiedendo di eseguire.

```
fn main() {
    let data1 = Rc::new(1);
    let data2 = data1.clone();

    println!("t0: {}", *data1);
    let jh = spawn(move || {
        println!("t1: {}", *data2);
    });

    jh.join().unwrap();
}
```

```
error[E0277]: `Rc<i32>` cannot be sent
between threads safely
--> src/main.rs:7:12
7 |         let jh = spawn(move || {
  |         ^^^^^^
  |         |
  |         | `Rc<i32>` cannot be
  |         | sent between threads safely
  |         the trait `Send` is not implemented for
  |         `Rc<i32>`
```

Tipo	Send	Sync	Note
Rc<T>	No	No	Conteggio dei riferimenti non thread-safe
Arc<T>	Sì	Sì, se lo è T	Conteggio dei riferimenti thread-safe
Cell<T>	Dipende da T	No	Mutabilità interna
Mutex<T>	Sì	Sì, se lo è T	Accesso protetto

13.13 Modello di concorrenza

La libreria standard di Rust supporta **due modelli** base per la realizzazione di programmi concorrenti

1. La condivisione di dati basata su sincronizzazione degli accessi ad una **struttura dati condivisa**, a cui tutti i thread interessati possono accedere in lettura e scrittura. Tutti i thread possono leggere e scrivere in questa struttura dati.
2. La condivisione di dati basata sullo **scambio di messaggi** che prevede uno o più mittenti ed un solo destinatario. Questa soluzione è più semplice e si adatta a meno situazioni rispetto alla prima. L'implementazione base è many-to-one, dove molti leggono e solo uno scrive.

Sono inoltre disponibili librerie esterne che supportano ulteriori modelli:

- La libreria **actix** supporta il modello degli attori
- La libreria **rayon** supporta il modello work stealing
- La libreria **crossbeam** permette la condivisione di dati memorizzati nello stack del thread genitore con i thread figli

13.14 Condivisione dello stato

Per poter condividere dati modificabili tra thread differenti, occorre disporre di un qualche meccanismo che permetta, ad un solo thread alla volta, di acquisire il permesso di modifica, questo lo svolgimento degli altri thread che dovessero richiedere accesso alla stessa risorsa.

Mutex Il modo più semplice di ottenere questo comportamento è attraverso l'uso di un **mutex** - MUTual EXclusion lock. Costrutti di questo tipo sono offerti nativamente dai sistemi operativi e sono riesportati in modo indipendente dalla piattaforma dalle librerie standard C++ e Rust.

Lock e Unlock Gli oggetti nativi offerti dai sistemi operativi offrono, come minimo, due metodi: `lock()` e `unlock()`.

- Invocando, `lock()`, un thread richiede il possesso del mutex: se questo non può essere garantito al momento, perché il mutex è in uso ad un altro thread, l'invocazione si blocca fino a che il mutex non è stato rilasciato dall'attuale possessore
- È lecito invocare `unlock()` solo se il thread che lo esegue è l'attuale possessore del mutex

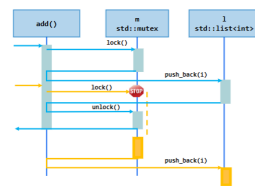
Entrambi i metodi `lock()` e `unlock()` includono una barriera di memoria. Essa garantisce la visibilità delle operazioni eseguite fino a quel punto dagli altri thread. Permette di imporre la dipendenza causale.

Sul piano pratico, occorre associare ad ogni risorsa condivisa un mutex che deve essere **sempre** acquisito prima di fare accesso, sia in lettura che in scrittura, alla risorsa.

Nelle astrazioni base offerte dai sistemi operativi e nell'implementazione offerta in C++, non c'è una corrispondenza sintattica tra un mutex e la struttura dati che questo protegge. La relazione, in questi ambienti, è nella mente, e nelle intenzioni, del programmatore.

Un mutex può, in linea di principio, proteggere molte strutture diverse, ma riduce il grado di parallelismo complessivo del programma.

```
std::list<int> l;
std::mutex m;
void add(int i){
    m.lock();
    l.push_back(i);
    m.unlock();
}
```



13.15 Rilasciare i mutex

Se un thread che è in possesso di un mutex termina, anche, ma non solo, per via di un errore, senza rilasciare il mutex, si crea un problema.

- I sistemi operativi tendono a liberare il mutex
- Però, quale stato hanno le risorse che il mutex protegge?

Per evitare la situazione, nei linguaggi che lo supportano si ricorre al paradigma RAIL. Si incapsula l'acquisizione del lock nel costruttore di un oggetto "guardia" e il suo rilascio nel distruttore. Si lega il ciclo di vita dell'oggetto guardia al blocco di codice in cui occorre fare accesso al dato protetto.

13.16 Mutex in Rust

L'accesso ad uno stato condiviso in Rust richiede l'utilizzo di **due blocchi** in cascata:

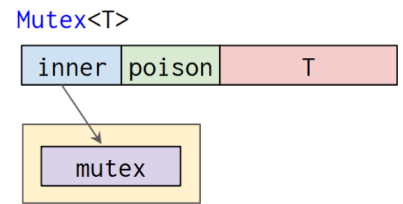
- Il primo volto a permettere il possesso multiplo di una struttura dati in sola lettura da parte di più thread, realizzato mediante il costrutto `std::sync::Arc<T>`
- Il secondo che consente l'acquisizione in lettura/scrittura della struttura dati, realizzato alternativamente mediante il costrutto `std::sync::Mutex<T>`, con `std::sync::RwLock<T>` oppure ricorrendo ai **tipi atomici**

Questa combinazione che prende spunto dal pattern RAIL, permette di rendere esplicito nella struttura del codice e nei pattern di accesso ai dati cosa sia condiviso e impedisce di fatto l'accesso senza il corretto possesso del lock relativo, permettendo al compilatore di bloccare ogni tentativo di accesso non conforme.

13.16.1 std::sync::Mutex<T>

Un oggetto di tipo `Mutex<T>` incapsula un dato di tipo `T` oltre al riferimento ad un mutex nativo del sistema operativo.

L'unico modo per accedere al dato è invocare il metodo `lock()`. Questo metodo restituisce un oggetto di tipo `LockResult<MutexGuard<T>>` e resta bloccato fino a che non è stato possibile acquisire il mutex nativo. Se l'ultimo thread che ha acquisito il mutex fosse terminato prima di averlo rilasciato, il mutex si troverebbe nello stato avvelenato, e la risposta conterrebbe un errore.



Se il metodo `lock()` ha successo, la risposta contiene un `MutexGuard<T>`, uno smart pointer. Tale oggetto implementa il tratto `Deref<T>` e si comporta come uno smart-pointer. Dereferenziandolo, si ottiene un riferimento mutabile al dato `T`. Quando il `MutexGuard<T>` esce dallo scope, il mutex nativo viene rilasciato, permettendo ad altri thread di chiederne il possesso.

Un valore di tipo `MutexGuard<T>` deve restare in vita solo per il tempo strettamente necessario. Evitando I/O e computazioni lunghe mentre si possiede il lock.

Note: Se un thread panicasse, **poisoned** verrebbe settato.

```
use std::sync::Mutex;
use std::sync::Arc;
use std::thread;

fn main(){
    let mut v: Vec<i32> = vec![];

    // la coda è all'italiana, onAvg è più efficeinte perché non
    // c'è una sovrastruttura per mantenere l'ordine
    // ma questo può causare starvation
    let m = Mutex::new(v);

    // arc rende accessibile il dato a più proprietari, essendo thread safe
    // solitamente si passano strutture
    let a = Arc::new(m);

    let a1 = a.clone();
    let a2 = a.clone();

    //due puntatori che conoscono il mutex

    let t1 = thread::spawn(move || {
        // il lock può essere avvelenato, devo gestire l'errore
        if let Ok(mut g) = a1.lock() {
            g.push(1);
            g.push(2);
            g.push(3);
        } // g esce di scope e rilascia il lock (unlock)
    });

    let t2 = thread::spawn(move || {
        if let Ok(mut g) = a2.lock() {
            g.push(4);
            g.push(5);
            //panic!("Errore irreversibile"); // il lock è avvelenato
            g.push(6);
        }
    });

    // t1.join().unwrap();
    // t2.join().unwrap();

    // riprendo il possesso del lock
```

```
// println!("{:?}", a.lock().unwrap());

if let Ok(_) = t1.join() {}
if let Ok(_) = t2.join() {}

if let Ok(v) = a.lock() {
    println!("{:?}", v);
} else{
    println!("Il lock è avvelenato");
}

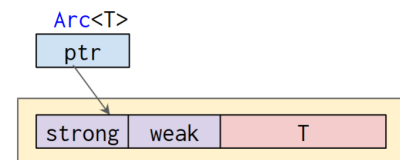
// possibili output: [1,2,3,4,5,6] oppure [4,5,6,1,2,3]
}
```

Note: Il mutex è utilizzato se almeno uno deve scrivere, altrimenti è sufficiente utilizzare `Arc<T>` da solo. Se si vuole principalmente leggere, è meglio utilizzare `RwLock<T>` al posto di `Mutex<T>`.

13.16.2 `std::sync::Arc<T>`

Un oggetto di tipo **Mutex** può avere un solo possessore. Per superare questo vincolo, lo si incapsula all'interno di un oggetto di tipo `std::sync::Arc<T>`.

`Arc<T>` permette di condividere il possesso di un dato, allocandolo nello heap e mantenendo un conteggio dei riferimenti esistenti di tipo *thread-safe*.



È possibile duplicare un oggetto di questo tipo attraverso il metodo `clone()`. Tale metodo si limita a duplicare il puntatore al blocco sullo heap, avendo cura di incrementare, in modo atomico, il contatore dei riferimenti associati al dato. Il dato clonato viene ceduto ad un thread specificando la parola-chiave `move` di fronte alla funzione lambda che ne descrive la computazione

Condivisione dello stato

```
let shared_data = Arc::new(Mutex::new(Vec::new()));
let mut threads = vec![];
for (i in 1..10) {
    let mut data = shared_data.clone(); // duplicazione del possesso
    threads.push( thread::spawn( move || { // data è ceduto al thread
        let mut v = data.lock().unwrap(); // v è di tipo MutexGuard<T>
        v.push(i); // quando v esce dall scope
    } )); // il lock viene rilasciato
}
for t in threads { t.join().unwrap(); } //v contiene i numeri da 1 a 9
```

Note: `Arc<Mutex<T>` = possesso condiviso + accesso mutabile protetto

Se un thread viene creato mediante la primitiva `std::thread::spawn()`, il compilatore non può fare assunzioni sulla sua durata, di conseguenza, impedisce l'utilizzo di riferimenti condivisi tra la funzione lambda del thread e la funzione all'interno della quale il thread viene creato.

La libreria standard offre un ulteriore modo di creare un thread, tramite la funzione `std::thread::scope(s: std::thread::Scope| ...)|`.

Posso creare dei thread che possono durare meno della funzione `scope` stessa, a differenza della primitiva `spawn` che si sa quando parte ma non quando finisce. Questo permette di non aspettare le handle dei thread creati e di aspettare se ogni thread ha finito la sua esecuzione.

Essa accetta come parametro una funzione lambda il cui compito è racchiudere l'intero ciclo di vita dei thread creati al suo interno. Il parametro `s` passato a tale funzione offre il metodo `.spawn(...)` mediante il quale è possibile creare nuovi thread.

Terminata l'esecuzione della funzione lambda, la funzione `scope(...)` non ritorna fino a che tutti i thread creati al suo interno non sono terminati. Questo permette al borrow checker di considerare corretto l'uso di

riferimenti a variabili locali, dato che la loro durata sarà almeno pari a quella della funzione `scope(...)` e, di conseguenza, a quella dei thread creati al suo interno.

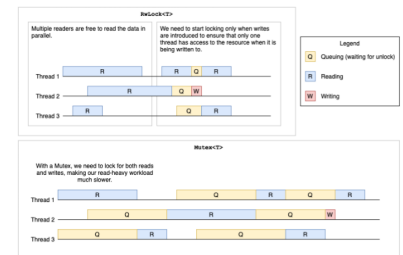
Differenza chiave rispetto a `thread::spawn()`: Con `thread::scope(...)` il compilatore sa che i thread terminano prima dell'uscita dallo scope

```
let mut v = vec![1, 2, 3];
let mut x = 0;
thread::scope(|s| {
    s.spawn(|| { // è lecito creare un riferimento a v
        println!("length: {}", v.len());
    });

    s.spawn(|| { // anche qui viene catturato &v
        for n in &v {println!("{n}"); }
        x += v[0]+v[2]; // x è catturata come &mut
    });
});
// Solo quando entrambi i thread saranno terminati si proseguirà
v.push(4); // non ci sono più riferimenti, si può modificare
assert_eq!(x, v.len());
```

13.16.3 `std::sync::RwLock<T>`

Se gli accessi in lettura e in scrittura sono sbilanciati, può essere conveniente sostituire, alla `struct Mutex<T>`, la `struct RwLock<T>`. Questa offre il metodo `read()` per accedere, in modo condiviso, in lettura ed il metodo `write()` per accedere in modo esclusivo in scrittura



13.17 Tipi atomici

È il tipo più elementare di `Mutex`, è pure meno costoso rispetto ad richiedere un lock.

Il modulo `std::sync::atomic` mette a disposizione alcune strutture dati che costituiscono primitive di comunicazione tra thread basate sul principio della memoria condivisa. Esso offre versioni atomiche di valori booleani, numeri interi con e senza segno e puntatori nativi. Ciascun tipo è associato ad operazioni che, se usate correttamente, permettono di sincronizzare gli aggiornamenti di tali valori tra thread differenti.

Accanto alle operazioni di lettura e scrittura con associata barriera di memoria, questi tipi offrono funzionalità di tipo Read-Modify-Write:

`swap(...)`, `compare_exchange(...)`, `fetch_add(...)`, `fetch_update(...)`

Ciascuna di queste operazioni riceve come parametro esplicito il tipo di barriera di memoria da applicare per la fase di lettura e per quella di scrittura.

Note: prima di utilizzare i tipi atomici, bisogna mangiarne di pasta, pensa ad imparare gli altri.

Sebbene siano tutti thread-safe, implementano il tratto *Sync*, non offrono meccanismi di condivisione esplicita. Come tutti i valori in Rust, sono soggetti alla regola del possessore unico. Per permettere a più thread di accedere al loro valore, è comune incapsularli all'interno di un elemento di tipo `Arc<T>` oppure dichiararli come variabili globali, attraverso la parola chiave `static`.

In modo analogo a `Cell<T>`, implementano il meccanismo di mutabilità interna. Poiché le operazioni di modifica sono garantite essere thread-safe, i metodi che ne modificano il contenuto richiedono solo un accesso condiviso (`&self`) e non un accesso esclusivo (`&mut self`)

Note: I tipi atomici sono potenti ma più difficili da usare correttamente rispetto ai lock. Richiedono particolare attenzione all'ordinamento della memoria.

```
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::{hint, thread};

fn main() {
    let spinlock = Arc::new(AtomicUsize::new(1));
    let spinlock_clone = Arc::clone(&spinlock);

    let thread = thread::spawn(move || {
        spinlock_clone.store(0, Ordering::Release);
    });
    // Attendi
    while spinlock.load(Ordering::Acquire) != 0 {
        hint::spin_loop();
    }

    thread.join().unwrap();
}
```

13.18 std::sync::Weak<T>

Analogamente a quanto succede con gli smart pointer di tipo `Rc<T>`, anche nel caso di `Arc<T>` la creazione di catene circolari impedisce il rilascio delle strutture.

Per questo motivo è disponibile la `struct std::sync::Weak<T>` che permette - sulla falsariga di quanto avviene con `std::rc::Weak<T>` - di realizzare dipendenze circolari con riferimenti che non partecipano al conteggio, garantendo così la possibilità di rilascio.

Per fare accesso al dato puntato, occorre invocare il metodo `upgrade()`, che restituisce un valore di tipo `Option<Arc<T>>`.

Si crea un oggetto di tipo `Weak<T>` a partire da un riferimento di tipo `Arc<T>` invocato su quest'ultimo il metodo `downgrade()`.

13.19 Attese condizionate

Spesso un thread deve aspettare uno o più risultati intermedi prodotti da altri thread. Per motivi di efficienza, l'attesa non deve consumare risorse e deve terminare non appena un dato è disponibile.

La presenza di dati condivisi richiede come minimo l'utilizzo di un mutex, questo per garantire l'assenza di interferenze tra i due thread che devono fare accesso ai dati.

Il polling ha due limiti:

- Consuma capacità di calcolo e batteria in cicli inutili
- Introduce una latenza tra il momento in cui il dato è disponibile e il momento in cui il secondo thread si sblocca

Per gestire queste situazioni, i sistemi operativi offrono il concetto di *condition variable*. Strutture dati di sincronizzazione che permettono di bloccare l'esecuzione di un thread, così da evitare il consumo di CPU, nell'attesa che qualcosa succeda.

L'uso di `Condvar`, che va usata con `Mutex`, - struttura dati gestita dal sistema operativo che permette di sospendere e mettere nella coda dei non runnable per non sprecare risorse, e poi ci deve essere un altro meccanismo che ripesci questo thread e lo inserisca nuovamente nella coda dei runnable quando il dato è disponibile. Diverso da `sleep()` nel quale devo specificare il tempo di attesa.

L'uso di *condvar* è basata sulla cooperazione all'interno del sistema: se un thread si sospende in attesa di una condizione, è necessario che tutti i thread che eseguono azioni che potrebbero provocare il verificarsi della condizione si facciano carico di inviare una notifica alla condition variable.

Il pattern di utilizzo prevede che esista una espressione booleana il cui valore possa essere usato per determinare se occorre attendere o meno.

La valutazione di tale espressione deve avvenire mentre si possiede un mutex, per garantire l'assenza di **corse critiche**. In Rust, questo vuol dire che le variabili che consentono la valutazione della condizione sono incapsulate nel **mutex**.

Ogni condition variable deve essere usata in coppia con un singolo mutex. Eventuali tentativi di usare mutex diversi per una stessa condition variable può determinare un fallimento in fase di esecuzione.

Nota: Condvar, utilizzata con un mutex, interfaccia con il SO per aspettare i risultati di altri thread, permette di aspettare il dato contenuto nel mutex. Bisogna capire cosa stiamo aspettando, solitamente con una lambda function.

13.19.1 Condvar in C++

Il linguaggio C++ offre la classe `std::condition_variable`, essa viene usata in coppia con un oggetto di tipo `std::mutex` racchiuso all'interno di un oggetto di tipo `std::unique_lock<std::mutex>`.

13.19.2 Condvar in Rust

Rust offre la struct `std::sync::Condvar`. La sua semantica è totalmente allineata con la corrispondente classe C++. La struttura dei suoi metodi facilita il collegamento con il dato protetto dal mutex, rendendo più naturale il suo utilizzo.

13.19.3 Condvar - metodi principali

- `pub fn new() -> Condvar` crea una nuova istanza
- `pub fn notify_one(&self)` Sveglia un thread **a caso** tra quelli in attesa sulla condition variable
- `pub fn notify_all(&self)` Sveglia tutti i thread in attesa sulla condition variable, che usciranno, **uno alla volta**, dal metodo wait possedendo il lock (altrimenti ritorna a dormire fino a che il lock è disponibile, in questo caso è in attesa del mutex e non di una notifica)
- `pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> LockResult<MutexGuard<'a, T>>` Sospende il thread corrente fino alla ricezione di una notifica: durante la sospensione, rilascia il lock; al ricevere della notifica, riacquisisce il lock e restituisce una nuova guardia. Ha il mutex, gli altri non possono modificare nulla.

Quando si chiama `wait` succedono due cose:

- Viene distrutto il lock
- Il thread corrente viene sospeso dal SO

13.19.4 Pattern Condvar

```
let pair = Arc::new((Mutex::new(false), Condvar::new()));
let pair2 = Arc::clone(&pair);
// Inside of our lock, spawn a new thread, and then wait for it to start.
// guarda la handle se è partito altrimenti non ha senso aspettarlo nel while dopo
thread::spawn(move|| {
    let (lock, cvar) = &*pair2;
    //ricorda di gestire e non usare unwrap, se errore? panico e non arriverebbe mai a notify!
    let mut started = lock.lock().unwrap();
    *started = true;
    // We notify the condvar that the value has changed.
    cvar.notify_one();
});
// Wait for the thread to start up.
let (lock, cvar) = &*pair;
// started, il lock viene rilasciato e resituito un nuovo lock dopo la notifica
let mut started = lock.lock().unwrap();

// se qui faccio operazioni stampalate, vado in panico e quello sopra panica perché ho usato unwrap!
```

```
// prevenzione nel caso il primo thread abbia già inviato la notifica (partito prima del thread
↳ principale) rimarrei sempre ibernato.
while !*started {
    started = cvar.wait(started).unwrap();
}
```

13.20 Meccanismo di funzionamento

Concettualmente, una condition variable mantiene una collezione di thread in attesa che si verifichi la condizione attesa. Inizialmente la lista è vuota, quando un thread esegue il metodo `wait(...)`, viene sospeso e aggiunto alla lista e marcato come non eseguibile dall'SO.

Quando sono eseguiti i metodi `notify_one(...)` o `notify_all(...)`, uno o tutti i thread presenti nella collezione sono risvegliati. Si basa sul S.O. per sospendere/risvegliare i thread

La presenza di un unico lock fa sì che, se più thread ricevono la notifica, il risveglio sia progressivo. Non appena un thread rilascia il lock, un altro può acquisirlo e proseguire.

La relazione tra l'evento e la notifica è solo nella testa del programmatore. Per questo, si rende esplicito l'evento che si è verificato appoggiandosi ad una o più variabili condivise (sotto il controllo del mutex).

13.21 Notifiche spurie

Altri thread mandano notifica sulla condvar e sempre nell'while se è la mia notifica o meno (sveglio in anticipo).

È possibile che un thread in attesa su una condition variable sia risvegliato in assenza di un'esplicita notifica. Problema delle cosiddette **notifiche spurie**.

Occorre, al ritorno dal metodo `wait(...)`, controllare se la condizione attesa è verificata. Per semplificare tale verifica, esiste una versione del metodo di attesa che riceve come argomento una funzione volta a valutare il predicato richiesto.

Per questo Rust offre il metodo:

```
pub fn wait_while<'a, T, F>(&self, guard: MutexGuard<'a, T>, condition: F ) ->
↳ LockResult<MutexGuard<'a, T>>
where F: FnMut(&mut T) -> bool{
    ...
}
```

Al risveglio, ri-acquisisce il lock e valuta la funzione condition: se questa restituisce true, si riaddormenta, altrimenti esce dall'attesa

13.22 Notifiche perse

Il while mi protegge anche dalla prima iterazione, se sono partito in ritardo rispetto all'altro thread, se la notifica è già arrivata non rimango ibernato per sempre. (partenza in ritardo).

Analogamente, se un thread ha eseguito una qualche azione che può abilitare la prosecuzione di un altro thread, ed invoca il metodo `notify_one(...)` / `notify_all(...)` per segnalare tale fatto, è possibile che la notifica vada persa. Succede se l'altro thread **non ha ancora eseguito** la corrispondente istruzione di attesa.

Per questo motivo, occorre sempre racchiudere l'istruzione di attesa in un ciclo che verifica se occorre o meno addormentarsi e, al risveglio, se ci siano le condizioni o meno per continuare a dormire. In entrambi i casi, il metodo `wait_while(...)` ci viene in aiuto e ci protegge. Esso è equivalente a:

```
while condition(&mut *guard) {
    guard = self.wait(guard)?;
}
Ok(guard)
```

13.23 Attesa temporizzata

Altri metodi permettono di limitare il tempo massimo di attesa, permettendo al thread di risvegliarsi anche in assenza del verificarsi della condizione.

```
pub fn wait_timeout<'a, T>(<br>    &self,<br>    guard: MutexGuard<'a, T>,<br>    dur: Duration<br>) -> LockResult<(MutexGuard<'a, T>,<br>    ↳ WaitTimeoutResult)> {<br>    ...<br>}
```

Attende per un tempo massimo pari a `dur`

```
pub fn wait_timeout_while<'a, T, F>(<br>    &self,<br>    guard: MutexGuard<'a, T>,<br>    dur: Duration,<br>    condition: F<br>) -> LockResult<(MutexGuard<'a, T>,<br>    ↳ WaitTimeoutResult)><br>where F: FnMut(&mut T) -> bool {<br>    ...<br>}<br>Attende per un tempo massimo pari a dur;<br>↳ eventuali notifiche ricevute portano a<br>↳ ri-addormentarsi se la funzione condition<br>restituisce false
```

13.24 Stato condiviso o scambio di messaggi?

Modello	Idea	Vantaggio	Costo
Stato condiviso	Più thread accedono agli stessi dati	Efficiente e flessibile	Richiede sincronizzazione esplicita
Scambio di messaggi	I dati vengono trasferiti tra thread	Possesso più chiaro, meno interferenza diretta	Protocollo di comunicazione da progettare

13.25 Condivisione di messaggi

In alternativa alla condivisione dello stato, Rust offre un meccanismo di comunicazione e sincronizzazione tra thread basato sulla condivisione di messaggi

- La funzione `std::sync::mpsc::channel<T>()` (multiple producer, single consumer) restituisce una coppia ordinata formata da una `struct Sender<T>` e una `struct Receiver<T>`.
- Tutti i dati inviati tramite il metodo `send(...)` della prima possono essere consumati attraverso il metodo `recv(...)` della seconda, nello stesso ordine in cui sono stati inviati
- Il metodo `send(...)` offre la garanzia che chi lo invoca non sarà bloccato, ovvero il canale di comunicazione ha una capacità infinita di memorizzazione temporanea dei messaggi. Il canale funziona come un buffer, limitata alla memoria del mio pc, questo può provocare errori
- Il metodo `recv(...)`, invece, si blocca senza consumare cicli macchina in attesa di un messaggio o della terminazione dell'oggetto `Sender` e di tutti i suoi eventuali cloni

L'implementazione fornita gode della proprietà *multiple producer, single consumer*, ovvero, permette di creare più cloni dell'oggetto `sender`. Mentre obbliga ad avere una singola copia dell'oggetto `receiver`.

In questo modello di comunicazione, il singolo dato prodotto da un thread viene ceduto al canale e da questo al thread ricevente che diventa il possessore finale del valore.

Questa operazione agisce al tempo stesso da sincronizzazione (la ricezione è necessariamente successiva all'invio) e da comunicazione (il dato passato rappresenta l'unità di messaggio).

Nota: I messaggi funzionano molto bene in una pipeline, dove ogni stage ha un proprio thread. Facili da usare ma non sempre possibile utilizzabili.

Un numero arbitrario di messaggi può essere scambiato sul canale

- A condizione che il ricevitore sia attivo
- Se il ricevitore viene deallocato, eventuali tentativi di invio falliscono con la generazione di un valore di tipo `SendError<T>`,

- Se tutti i trasmettitori vengono deallocati, tentativi di lettura sul ricevitore falliscono con la generazione di un valore di tipo `RecvError`

```
use std::sync::mpsc::sync_channel;
use std::thread;

let (tx, rx) = channel();

for _ in 0..3 {
    let tx = tx.clone();
    // cloned tx dropped within thread
    thread::spawn(move || tx.send("ok").unwrap());
}
// Drop the last sender to stop `rx` waiting for message.
// The program will not complete if we comment this out.
// **All** `tx` needs to be dropped for `rx` to have `Err`.
drop(tx);
// Unbounded receiver waiting for all senders to complete.
while let Ok(msg) = rx.recv() {
    println!("{}", msg);
}
```

13.26 Canali sincroni

La funzione `std::sync::mpsc::sync_channel<T>(bound: usize)` restituisce invece una coppia di valori di tipo `SyncSender<T>`, `Receiver<T>`.

A differenza di un canale semplice, questo è limitato: se il numero di messaggi giacenti nel canale raggiunge il limite definito (`bound`) le invocazioni del metodo `send(...)` diventano bloccanti fino a che non si libera un posto eseguendo una lettura con successo.

Se viene costruito un canale sincrono di dimensione 0, diventa un canale di tipo *rendezvous*: ogni operazione di lettura deve sovrapporsi temporalmente ad una di scrittura. Le restanti operazioni offerte da `SyncSender<T>` hanno semantica simile alle corrispondenti offerte da `Sender<T>`.

```
use std::sync::mpsc::sync_channel;
use std::thread;
let (sender, receiver) = sync_channel(1);
// this returns immediately
sender.send(1).unwrap();
thread::spawn(move || {
    // this will block until the previous message has been received
    sender.send(2).unwrap();
});
assert_eq!(receiver.recv().unwrap(), 1);
assert_eq!(receiver.recv().unwrap(), 2);
```

13.27 Condvar esempi

Esempio di un buffer limitato, bounded budffer con dimensione fissata e logica fifo (vec no, vecdec si). Non sono stati considerati i casi a buffer lunghezza zero.

Se cerco di leggere e non c'è nulla, devo aspettare (attesa indeterminata, non voglio usare cicli CPU), usiamo una condvar

src/lib.rs (mod bounded_buffer) src/bounded_buffer.rs

Note: Il flag si deve mettere dentro allo stesso Mutex. No a 1 condvar e 2 mutex, massimo 1 mutex e 2 condvar.

```
struct BoundedBuffer<T: Send> {
    //proteggerò il buffer con mutex, e un flag
    data: Mutex<(VecDeque<T>, bool)>, // (data, is_open)
    // vogliamo un'attesa indeterminata, non vogliamo consumare CPU
    cv: Condvar,
```

```

    // read-only non mi serve il mutex
    capacity: usize,
}

// per il passaggio di T tra i Thread deve implementare il tratto Send
impl<T: Send> BoundedBuffer<T> {

    // no problema di concorrenza
    pub fn new(capacity: usize) -> Self {
        Self {
            data: Mutex::new((VecDeque::with_capacity(capacity), true)),
            cv: Condvar::new(),
            capacity,
        }
    }

    // i metodi devono essere &Self e non &mut Self!
    // non aspetto se c'è spazio e se è aperto il canale
    pub fn push(&self, item: T) -> bool {
        //primo passo: prendere il possesso
        let mut data = self.data.lock().unwrap();

        //aspetto fino a che c'è spazio nel buffer
        data = self.cv.wait_while(data, |data| data.0.len() >= self.capacity && data.1).unwrap();

        // mi sveglio, ma il canale è chiuso esco
        if !data.1 {
            return false;
        }

        //ora c'è spazio, posso inserire
        data.0.push_back(item);

        // finisco di fare i cambiamenti, per evitare problemi con il lock,
        drop(data);
        // notifico che c'è un nuovo elemento
        // preferisci notificare quando non si ha il lock! Più efficiente
        self.cv.notify_all();
        true
    }

    // se la coda è vuota aspetto, anche per sempre
    pub fn pop(&self) -> Option<T> {
        let mut data = self.data.lock().unwrap();

        // aspetto fino a che c'è almeno un elemento
        data = self.cv.wait_while(data, |data| data.0.is_empty() && data.1).unwrap();

        // se c'è qualcosa da leggere lo ritorno, altrimenti il canale è chiuso
        if data.0.len() > 0 {

            // ora c'è almeno un elemento, posso rimuovere
            let item = data.0.pop_front();

            // finisco di fare i cambiamenti, per evitare problemi con il lock,
            drop(data);
            // notifico che c'è posto
            // preferisci notificare quando non si ha il lock! Più efficiente
            self.cv.notify_all();

            Some(item)
        } else {
            // il canale è chiuso e non ci sono più elementi
            None
        }
    }
}

```

```

    }

    pub fn close(&self) {
        let mut data = self.data.lock().unwrap();
        data.1 = false; // chiudo il canale
        drop(data);
        self.cv.notify_all(); // sveglio tutti i thread in attesa
    }
}

impl<T: Send> Drop for BoundedBuffer<T> {
    fn drop(&mut self) {
        self.close();
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let buffer = BoundedBuffer::new(10);
        buffer.push(1);
        buffer.push(2);
        assert_eq!(buffer.pop(), Some(1));
        assert_eq!(buffer.pop(), Some(2));
    }

    #[test]
    fn it_is_thread_safe(){
        let buffer = Arc::new(BoundedBuffer::new(10));
        let buffer1 = Arc::clone(&buffer);

        let mut handles = std::thread::spawn(move || {
            for i in 0..10 {
                buffer1.push(i);
            }
            thread::sleep(std::time::Duration::from_millis(1000));
            buffer1.close();
        });

        for i in 0..10 {
            assert_eq!(buffer.pop(), Some(i));
            std::thread::sleep(std::time::Duration::from_millis(10));
        }

        assert_eq!(buffer.pop(), None); // il canale è chiuso e non ci sono più elementi

        handles.join().unwrap();
    }

    #[test]
    fn it_closes(){
        let buffer = BoundedBuffer::new(10);
        buffer.close();
        assert_eq!(buffer.pop(), None);
    }
}

```

13.28 Gestione dell'attesa

La struct `Receiver<T>` offre il metodo `recv_timeout(d: Duration)`. Esso consente di limitare il tempo massimo di attesa alla durata passata come parametro.

Questo metodo può restituire due tipi di errore

- `mpsc::RecvTimeoutError::Timeout` per indicare che non è stato ricevuto alcun messaggio entro il tempo indicato
- `mpsc::RecvTimeoutError::Disconnected` per indicare che il canale è vuoto e non sono più disponibili in *Sender* in grado di spedire messaggi

Quando occorre non bloccarsi nell'attesa di nuovi messaggi, è possibile utilizzare il metodo `try_recv()`.

Se nessun messaggio è presente, il metodo torna subito con un errore di tipo `mpsc::TryRecvError::Empty`. Se non sono più disponibili *Sender*, il metodo restituisce un errore di tipo `mpsc::TryRecvError::Disconnected`

13.29 Canali: regole pratiche

- `send()` trasferisce il valore al destinatario
- `recv()` attende un messaggio oppure la richiesta di chiusura del canale

Finché esiste almeno un *Sender*, il *Receiver* resta in attesa. Chiudere gli estremi inutilizzati è parte del protocollo

13.30 La libreria Crossbeam

Libreria ben documentata e attivamente mantenuta che offre una serie di costrutti a supporto dell'elaborazione concorrente.

- **Costrutti atomici**
- **Strutture dati concorrenti**
- **Canali MPMC:** le funzioni `crossbeam::channel::{bounded(...), unbounded()}` creano canali unidirezionali con capacità limitata o illimitata basati sul paradigma MPMC i cui estremi possono essere condivisi per semplice clonazione; le funzioni `crossbeam::channel::{after(...), tick(...)}` creano il solo estremo di ricezione che consegnerà un messaggio dopo il tempo indicato o periodicamente (una sorta di sveglia)

Nota: Noi ci concentreremo principalmente su "Canali MPMC" - Multiple Producer, Multiple Consumer, creando delle pipeline competitive. Il consumer più veloce pesca e gestisce.

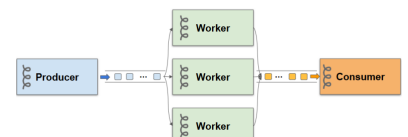
13.31 Uso della libreria Crossbeam

Il paradigma MPMC offre un meccanismo potente per l'implementazione di pattern concorrenti in Rust:

13.31.1 Fan-out / Fan-in

Permette di distribuire attività a più thread indipendenti e raccogliere i risultati prodotti in un singolo punto; usa una coppia di canali per distribuire e raccogliere i dati.

Esempio: mi entrano tanti pdf e il producer crea tanti worker che li processa separatamente per poi raccogliere i risultati in un unico punto.



```
fn worker(id: usize, rx: Receiver<i32>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("W{} ({})", id, value)).unwrap();
    }
}
fn main() {
```

```

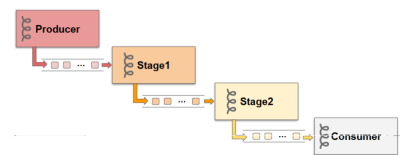
let (tx_input, rx_input) = bounded::<i32>(10);
let (tx_output, rx_output) = bounded::<String>(10);
let mut worker_handles = Vec::new();
for i in 0..3 {
    let rx = rx_input.clone();
    let tx = tx_output.clone();
    worker_handles.push( thread::spawn(move || worker(i, rx, tx)) );
}
for i in 1..=10 { tx_input.send(i).unwrap(); }
drop(tx_input);
drop(tx_output);
while let Ok(result) = rx_output.recv() {
    println!("Received result: {}", result);
}
for handle in worker_handles { handle.join().unwrap(); }
}

```

13.31.2 Pipeline

Computazioni se i singoli passi sono onerose ma deve essere garantita la sequenza di esecuzione.

Crea una serie di fasi di lavorazione, ciascuna delle quali è eseguita da un singolo thread e utilizza un canale per inoltrare i semi-lavorati tra due fasi successive.



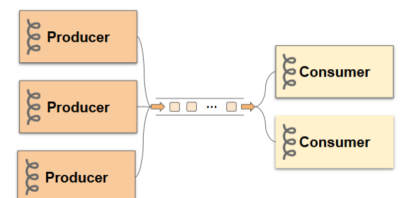
```

fn stage_one(rx: Receiver<i32>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S1({})", value)).unwrap();
    }
}
fn stage_two(rx: Receiver<String>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S2( {} )", value)).unwrap();
    }
}
fn main() {
    let (tx_input, rx_input) = bounded::<i32>(10);
    let (tx_stage_one, rx_stage_one) = bounded::<String>(10);
    let (tx_output, rx_output) = bounded::<String>(10);
    let stage_one_handle = thread::spawn(move || stage_one(rx_input, tx_stage_one));
    let stage_two_handle = thread::spawn(move || stage_two(rx_stage_one, tx_output));
    for i in 1..=10 { tx_input.send(i).unwrap(); }
    drop(tx_input);
    while let Ok(result) = rx_output.recv() { println!("Received result: {}", result); }
    stage_one_handle.join().unwrap();
    stage_two_handle.join().unwrap();
}

```

13.32 Producer/consumer - Thread pool

Consente ad uno o più thread produttori di generare valori che saranno elaborati dal primo thread consumatore disponibile; usa un singolo canale per la comunicazione



```

fn producer(id: usize, tx: Sender<(usize,i32)>) {
    for i in 1..=5 { tx.send((id,i)).unwrap(); }
}
fn consumer(id: usize, rx: Receiver<(usize,i32)>) {
    while let Ok((sender_id, val)) = rx.recv() {
        println!("Consumer {} received {} from {}", id, val, sender_id);
    }
}

```

```
    }  
}  
fn main() {  
    let (tx, rx) = bounded::<(usize, i32)>(10);  
    let mut handles = Vec::new();  
    for i in 0..3 {  
        let tx = tx.clone();  
        handles.push( thread::spawn(move || producer(i, tx)) );  
    }  
    for i in 0..2 {  
        let rx = rx.clone();  
        handles.push(thread::spawn(move || consumer(i, rx)));  
    }  
    drop(tx);  
    for handle in handles { handle.join().unwrap(); }  
}
```

13.33 Checklist finale

Prima di introdurre concorrenza, chiedersi sempre:

- I thread devono condividere uno stato o scambiarsi dei messaggi? Più semplice con i canali, ma non sempre è possibile utilizzarlo (mutex + condvar è la più generale)
- Serve mutua esclusione, lettura condivisa o sincronizzazione su un evento?
- Il programma è corretto solo per alcuni modi con cui i thread possono alternarsi o per tutti?
- Ci sono rischi di deadlock, livelock o starvation?
- Esiste una soluzione più semplice con meno condivisione?

Capitolo 14

Programmazione asincrona

14.1 Programmazione Asincrona

Se la necessità di eseguire più operazioni, indipendenti tra loro, in **parallelo** porta alla creazione di più thread, altre esigenze di programmazione possono beneficiare di un approccio differente.

L'uso dei thread trova piena giustificazione quando occorre eseguire algoritmi complessi, basati principalmente sull'uso intenso della CPU e si dispone di un hardware multicore.

In queste situazioni, infatti, è possibile ridurre il tempo complessivo di elaborazione sfruttando il fatto che più computazioni procedono parallelamente.

Se il programma, invece, deve coordinare molteplici attività che si svolgono **al di fuori** di esso, il problema dominante diventa l'attesa dei risultati. Sebbene sia possibile gestire queste situazioni creando più thread, questo, in generale, non è conveniente.

14.1.1 Concorrenza non è parallelismo

- **Parallelismo:** più computazioni procedono fisicamente nello stesso istante
 - Richiede più unità di calcolo (core): permette di ridurre il tempo di elaborazione per attività CPU-bound
 - Lo strumento naturale in Rust è il thread
- **Concorrenza:** più compiti progrediscono in modo interlacciato, non necessariamente simultaneo
 - Un singolo core può gestire migliaia di compiti concorrenti se ognuno passa gran parte del tempo in attesa
 - Il vantaggio non è la velocità di calcolo, ma la capacità di sovrapporre molte attese: un unico thread può coordinare migliaia di attese

14.2 Operazioni bloccanti attese

Molte operazioni richiedono di ottenere informazioni da un **sottosistema separato**, come ad esempio il file system, la rete, un timer, etc.

Per queste operazioni l'esecuzione non può proseguire finché la risposta non è disponibile. Il sistema operativo rileva la situazione e sposta il thread corrente nello stato "*NotRunnable*", sospendendolo, e la CPU viene liberata, ma lo stack dei thread **resta allocato** per tutta la durata dell'attesa.

La **domanda di progetto**: : il collo di bottiglia è la CPU o l'attesa (I/O, rete, timer)?

- CPU-bound → parallelismo con thread
- I/O-bound → concorrenza con esecuzione asincrona

Nota: Si usano i **thread** quando **occorre elaborare** in parallelo, mentre si usa l'**esecuzione asincrona** quando **occorre aspettare** in parallelo

14.2.1 Il costo dei thread

Ogni thread del sistema operativo ha un costo fisso, indipendente dal lavoro che svolge:

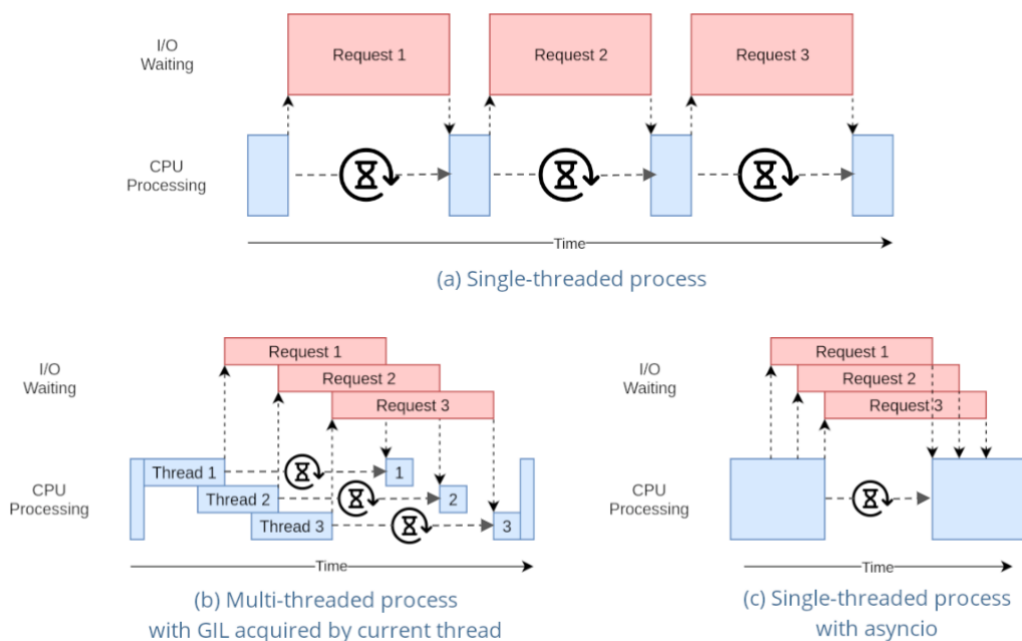
- **Stack pre-allocato:** tipicamente da 1 a 8 MB di spazio di indirizzamento riservato per thread
- **Strutture del kernel:** descrittore del thread, slot nello schedulatore, contabilità
- **Cambio di contesto:** salvataggio/ripristino dei registri, invalidazione di cache e TLB ($\sim 1 - 2 \mu s$)

Finché i thread sono poche decine, questi costi sono trascurabili, diventano critici quando se ne vogliono creare molti (1000+), come in un server di rete. Esempio: 10.000 connessioni simultanee con un thread ciascuna → diversi GB solo di stack. Si paga il costo di un thread completo per fargli fare, di fatto, nulla

14.3 Tre strategie di esecuzione

Se il programma ha altro da fare durante l'attesa, ci sono tre strategie possibili:

Strategia	Idea	Vantaggi	Limiti
Sequenziale (bloccante)	Aspetto e poi continuo	Semplice Facile da ragionare	Spreca tempo di CPU Nessuna sovrapposizione
Thread multipli (preemptive)	Delego l'operazione bloccante ad altri thread	Parallelismo reale Sfrutta multicore	Costo elevato (stack, scheduling) Sincronizzazione complessa
Asincronia (cooperativo)	Separo richiesta e continuazione	Non blocca il thread Scala a migliaia di operazioni	Più complesso concettualmente Richiede supporto di esecuzione



14.4 Separare richiesta e continuazione (callback)

Nell'approccio asincrono : avvio un'operazione, NON attendo il risultato, specifico cosa fare quando il risultato arriva (continuazione)

Il modo più semplice per indicare la continuazione è una callback. Pseudo-codice:

```
read_async(file, |result| { process(result); });
```

Timeline

t_0 : invoco `read_async(...)`

t_1 : il programma continua

t_2 : arriva il risultato

t_3 : viene eseguita la callback

In Javascript è semplice perché funziona single-threaded by design con una message queue. In Rust non abbiamo una message queue, il programmatore si sceglie cosa fare utilizzando tre librerie diverse come Tokio, smol o async-std.

Vantaggio: il thread NON resta bloccato dall'attesa e può svolgere altro lavoro

14.5 I limiti delle callback

Ci sono tre limiti importanti legati all'uso delle callback:

- **Annidamento:** illeggibile al crescere dei passi
- **Errori:** in punti diversi: all'invocazione e nell'elaborazione asincrona
- Cicli e tentativi ripetuti richiedono una **macchina a stati** scritta a mano

I costrutti `async/await` automatizzano questa trasformazione

Nota: Le callback funzionano, ma non scalano in complessità

```
let handle1 = open_file_async("f1", FileMode::read)?;
let handle2 = open_file_async("f2", FileMode::write)?;
let mut buffer = vec![];
read_async(handle1, &mut buffer, |res1|{
    if (res1.is_ok()) {
        write_async(handle2, res1.unwrap(), |res2| {
            if (res2.is_ok()) {
                //scrittura completata con successo
            } else {
                //scrittura fallita
            }
        });
    } else {
        //lettura fallita
    }
})?; // impossibilità di leggere
//Quando il programma arriva qui, non è ancora
//stato fatto nulla!
```

14.5.1 Async/await: stessa idea, forma diversa

Callback

```
read_async(..., |r1| {
    write_async(..., |r2| {
        ...
    });
});
// Quando arriva qui è già successo qualcosa?
```

Async/await

```
sync {
    let r1 = read(...).await;
    let r2 = write(r1).await;
}
// Il compilatore trasforma il codice nella
// stessa macchina a stati delle callback
```

Se `async/await` genera una macchina a stati → chi la esegue?

14.6 Funzioni asincrone

Il compilatore Rust supporta esplicitamente la programmazione asincrona grazie all'introduzione di due parole chiave nel linguaggio: **async** e **await**

14.6.1 La parola chiave *async*

Anteporre **async** a una funzione, o a un blocco, ne cambia radicalmente la semantica

- Il compilatore analizza il corpo e lo trasforma in una macchina a stati finiti
- Il tipo di ritorno passa da **T** a un tipo anonimo che implementa il tratto **Future<Output=T>**
- Il corpo della funzione viene ridotto alla sola inizializzazione di quella macchina a stati

Conseguenza cruciale: **chiamare una funzione async NON ne esegue il corpo**

- Restituisce solo un **Future** inerte, nel suo stato iniziale
- È un valore che descrive una computazione da fare, non la computazione già avviata

```
// quello che si scrive:
async fn copy(f1: String, f2: String) -> Result<> { /* codice */ }

// quello che il compilatore genera, concettualmente
fn copy(f1: String, f2: String) -> impl Future<Output = Result<>> { /* SM init */ }
```

14.6.2 L'operatore *.await*

Per ottenere il risultato di un **Future**, occorre attenderlo esplicitamente con l'operatore **.await**. Senza **.await**, il Future resta inerte e non viene mai eseguito (il compilatore lo segnala con un warning).

Semantica di *.await*

- Se il valore non è ancora disponibile, la funzione corrente si **sospende** e restituisce il controllo
- Il thread non resta bloccato: è libero di eseguire altri task pronti
- Quando il valore arriva, l'esecuzione riprende esattamente dal punto di sospensione

Ogni **.await** introduce un nuovo stato nella macchina a stati della funzione. L'operatore **?** si combina con **.await** per preparare gli errori in modo lineare.

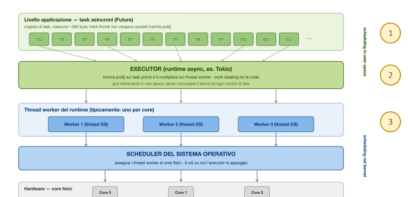
.await NON esegue il **Future** → segnala solo dove sospendere e riprendere.

14.6.3 Un future è inerte: serve un esecutore

Il compilatore genera i tipi della macchina a stati e riscrive le funzioni, ma il linguaggio si ferma qui. Nessun supporto preconstituito per **eseguire** effettivamente i **Future**

Affinché una computazione asincrona proceda, qualcuno deve farne avanzare la macchina a stati. Se viene chiamata una funzione **async** dentro un'altra funzione **async**, il suo Future viene automaticamente a far parte della macchina a stati del chiamante. Se viene chiamata da una funzione normale, occorre gestire il Future esplicitamente con un executor

L'executor è uno **schedulatore cooperativo** a livello applicazione. Mantiene i task pronti e ne fa avanzare la macchina a stati invocando il metodo **poll()** del **Future**. A differenza dello scheduler del S.O. non può sottrarre il controllo ad un task, che deve cederlo spontaneamente ad un punto di attesa (**.await**)



14.7 Il tratto Future

Tratto che viene implementato dagli oggetti che descrivono una computazione asincrona. Tale computazione può essere ancora in corso o essere terminata.

Il tratto offre un singolo metodo, `poll(...)`, che implementa la logica della macchina a stati associata alla funzione. L'oggetto che implementa il tratto mantiene al suo interno lo stato corrente.

I metodo `poll(...)` restituisce `enum` che può assumere due soli valori:

- `Poll::Pending` - per indicare che la computazione è ancora in corso
- `Poll::Ready(T)` - per indicare che la computazione è terminata e ha come risultato `val`

Un oggetto che implementa questo tratto è **inerte**. Affinché la computazione proceda, occorre che qualcuno ne invochi il metodo `poll(...)`.

```
use std::pin::Pin;
use std::task::{Context, Poll};

pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context)
        -> Poll<Self::Output>;
}

pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

`Pin<T>` è un particolare smart pointer che impedisce che la struttura venga mossa. Permettendo l'uso di riferimenti relativi.

`Context` incapsula un oggetto di tipo **Waker**, mediante il quale è possibile notificare all'esecutore che il metodo `poll(...)` può essere richiamato.

14.7.1 Chi invoca `poll()`: executor, reactor, Waker

L'**executor** mantiene la coda dei task pronti e ne invoca il metodo `poll()` in un ciclo. Se `poll()` restituisce `Pending`, l'executormette il task da parte e passa al successivo.

Comportamento cooperativo, non preemptive: un task che non raggiunge mai `.await` monopolizza il worker

Il problema: come sa l'executor quando riprovare, senza fare polling attivo a vuoto?

Risposta Il **Waker** contenuti nel **Context**.

- Prima di restituire **Pending**, il **Future** registra il proprio **Waker** presso il sottosistema (es, il reactor I/O)
- Il reactor osserva gli eventi del sistema operativo (epoll, kqueue, IOCP via mio)
- Quando l'evento atteso si verifica, il reactor risveglia il task, chiamando `wake()` sul **Waker**
- Il task torna nella coda dei pronti e l'executor lo ri-campiona: la macchina a stati avanza

È un modello ad eventi: non vengono sprecati cicli di CPU durante le attese

14.8 Esecutori

Per far funzionare un programma Rust asincrono occorre aggiungere una libreria di supporto all'esecuzione, a scelta del programmatore

- **Tokio** - la più diffusa: rete, file, timer, sincronizzazione, canali; basata sul crate
- **smol** - leggera, adatta a sistemi embedded
- **async-std** - controparte asincrona della libreria standard

Un runtime può basarsi su un singolo ciclo reattivo e/o utilizzare un thread-pool cui delegare l'esecuzione di più **Future** in parallelo. In questo caso, è possibile che l'elaborazione di una funzione asincrona inizi in un thread ma sia continuata in un thread differente. Questo implica che tutti i valori utilizzati nella funzione asincrona il cui uso si estende in più stati devono implementare il tratto **Send**, mentre i riferimenti devono implementare il tratto **Sync**.

14.9 Possesso nel tempo

Nel codice sincrono, il possesso è legato prevalentemente allo stack corrente. Nel codice asincrono, una computazione può sospendersi e riprendere più tardi.

Le variabili usate dopo `.await` diventano parte dello stato del **Future**. Se l'esecutore è multithread, quel Future può essere spostato tra worker.

Nei programmi asincroni, il possesso attraversa il tempo, non solo lo spazio:

- I dati usati dopo `.await` devono restare validi
- I task spostabili tra thread devono avere il tratto **Send**
- I riferimenti condivisi tra thread devono avere il tratto **Sync**
- La creazione di task indipendenti spesso richiede la proprietà dei dati

14.10 Tokio: i costrutti di base

14.10.1 Impostare un progetto con Tokio

Si aggiunge **tokio** come dipendenza nel file Cargo.toml scegliendo quali funzionalità attivare.

Si annota il punto di ingresso del programma con `#[tokio::main]`, che avvia la libreria di esecuzione e rende la funzione `main()` **async**

La proprietà **flavor** determina la struttura dell'esecutore:

- **multi_threaded** (default): un pool di thread worker, tipicamente uno per core
- **current_thread**: un solo thread esegue tutti i task; utile per test e ambienti vincolati

```
# Cargo.toml
[dependencies]
tokio = { version = "1", features = ["full"] }

#[tokio::main(flavor = "multi_thread", worker_threads = 4)] //o altro...
async fn main() {
    //... creazione di future e attesa relativa
}
```

14.10.2 Attendere un Future: esecuzione sequenziale

Il modo più semplice di usare un **Future** è attenderlo con `.await` dentro un blocco **async**.

Due `.await` consecutivi creano una sequenza: il secondo parte solo quando il primo è completo. Durante ogni attesa il thread è libero di servire altri task, ma questa funzione non avanza. È il comportamento corretto quando il secondo passo dipende dal risultato del primo

```
async fn fetch_user(id:u64) -> User { /* ... */ }
async fn fetch_orders(user: &User) -> Vec<Order> { /* ... */ }

#[tokio::main]
async fn main() {
    let user = fetch_user(42).await; // prima questo...
    let orders = fetch_orders(&user).await; // ...poi questo (dipende da user)
    println!("{}", ha {} ordini", user.name, orders.len());
}
```

14.10.3 tokio::spawn - avviare task concorrenti

`tokio::spawn(future)` consegna un **Future** all'esecutore come task indipendente.

A differenza di `.await`, l'esecuzione inizia subito e in modo concorrente, senza sospendere il chiamante.

Restituisce un **JoinHandle**, che rappresenta il task e permette di ricuperarne il risultato

Vincoli sul tipo Il **Future** deve essere **Send** e **'static**:

- **Send** perché il task può migrare tra i thread worker (work stealing)
- **'static** non vuol dire che i dati vivono per sempre, ma che non dipendono dallo stack corrente
- Si usa **move** per trasferire la proprietà dei dati al task

.await = "Aspetta questo $\Leftrightarrow$ = **spawn(...)** "fa' questo mentre faccio altro"

```
async fn expensive_io() -> Result<()> { /* ... */ }
async fn do_other_work() { /* ... */ }

#[tokio::main]
async fn main() {
    let h = tokio::spawn( async move { // parte immediatamente
        expensive_io().await // eseguito in background
    });

    do_other_work().await; // nel frattempo faccio altro
    let result = h.await.unwrap(); // quando ho finito, attendo il risultato
}
```

14.10.4 Non perdere il controllo dei task

spawn(...) crea un task con un ciclo di vita separato. Un task non osservato può:

- Fallire senza che il chiamante se ne accorga
- Continuare oltre la richiesta che lo ha creato
- Complicare la terminazione e la cancellazione

Occorre conservare il valore di tipo **JoinHandle** restituito da **spawn(...)** quando servono risultato, errore o arresto controllato. Quando si hanno molti task omogenei è possibile usare strutture dedicate come **JoinSet<T>**

- Offre il metodo **spawn(...)** che automaticamente memorizza la **JoinHandle** corrispondente
- Permette di iterare su tutte le handle per attenderne la terminazione

14.10.5 join! - attendere più Future insieme

tokio::join!(f1, f2, ...) attende il completamento di **tutti i Future** passati.

- Li fa progredire in modo interlacciato sullo stesso task: non viene eseguito **spawn()**, non viene richiesto il tratto **Send**
- Restituisce una tupla contenente i risultati, nell'ordine dei parametri passati

È il meccanismo corretto quando i compiti sono indipendenti e servono tutti i risultati

try_join!(...) è la variante per **Future** che restituiscono **Result<T,E>**. Ritorna **Ok((r1, r2, ...))** se tutti hanno successo, oppure si interrompe al **primo errore** restituendo

```
// Le due richieste non dipendono l'una dall'altra → attesa in parallelo
let (user, config) = tokio::join!(
    fetch_user(42),
    fetch_config()
);
// Variante con propagazione dell'errore
let (user, config) = tokio::try_join!(fetch_user(42), fetch_config())?;
```

14.10.6 select! - il primo che si completa

`tokio::select!(f1, f2, ...)` attende su più rami asincroni e procede con il primo che termina. I rami non selezionati vengono **cancellati** (viene invocato **Drop** sul loro **Future**)

La sintassi di ogni ramo simile a quella di **match**:

`<pattern> = <espressione async> (, if <guard>)? => <gestore>`

La guardia opzionale disattiva un ramo quando una preconditione è falsa

Un ramo **else** finale viene valutato se nessun ramo corrisponde al pattern indicato.

```
tokio::select! {
  data = fetch_from_network() => {
    process(data);
  },
  _ = tokio::time::sleep(Duration::from_secs(5)) => {
    eprintln!("timeout: nessuna risposta");
  }
}
```

14.10.7 Gestione del tempo

`tokio::time::sleep(d).await` sospende il task corrente per la durata indicata

- Non consuma né CPU né un thread: occupa solo la memoria del **Future** (300 byte)
- Allo scadere, l'esecutore risveglia il task e l'esecuzione riprende

`tokio::time::timeout(d, future).await` impone un limite di tempo a qualsiasi **Future**

- Restituisce `Result<T, Elapsed>`: `Ok(t)` se l'operazione completa in tempo, `Err` se scade prima
- È implementato tramite una `select!` tra il **Future** e un timer

14.10.8 Cancellare un task

La struct **JoinHandle** restituita da `spawn(...)` offre il metodo `abort()` per cancellare il task.

La cancellazione ha effetto quando il task è sospeso su `.await`. Se il task è dentro un ciclo attivo, l'interruzione avviene al prossimo `.await`.

Invoca `abort()` si attende la **JoinHandle** per verificare l'effettivo arresto. Si ottiene un **JoinError** con `is_cancelled() == true`

```
let handle = tokio::spawn(async {
  tokio::time::sleep(Duration::from_secs(10)).await;
});
// richiede la cancellazione
handle.abort();
// verifica che sia avvenuta
assert!(handle.await.unwrap_err().is_cancelled());
```

14.10.9 Esempio

```
async fn count(){
  for i in 1..10 {
    tokio::time::sleep(Duration::from_secs(1)).await;
    println!("Count: {}", i);
  }
}

#[tokio::main]
async fn main() {
  let f1 = count();
  let f2 = async {
    for i in 'a'..'z' {
      tokio::time::sleep(Duration::from_millis(200)).await;
      print!("Main: {}", i);
    }
  };
}
```

```

        std::io::stdout().flush().unwrap();
    }
};

// f1 e f2 procedono in parallelo
// AND
tokio::join!(f1, f2);

// se voglio capire quale dei due termina per primo
// interrompendo l'altro!
// OR
tokio::select! {
    _ = f1 => println!("f1 ha terminato"),
    _ = f2 => println!("f2 ha terminato"),
};
}

```

14.11 Guida alla progettazione

Quale costruito, in quale situazione

Il lavoro è CPU intensive ($> \sim 100\mu s$) ?	→ Thread o Rayon
Passi di I/O che dipendono l'uno dall'altro?	→ <code>.await</code> in sequenza
Avviare un'attività indipendente / cancellabile?	→ <code>tokio::spawn</code> → <code>JoinHandle</code>
Più attività indipendenti, servono tutti i risultati?	→ <code>tokio::join!</code> / <code>try_join!</code>
Reagire alla prima tra N condizioni?	→ <code>tokio::select!</code>
Passare dati tra task?	→ canali (vedi oltre)
Stato mutabile condiviso tra task?	→ <code>Arc<tokio::sync::Mutex<T></code>

14.11.1 Esempio: sequenza o concorrenza?

Lo stesso problema, due strutture diversa, in base alle **dipendenze tra i passi**

<i>// Dipendenti -> sequenza</i>	<i>// Indipendenti -> concorrenza</i>
<i>// orders dipende da user</i>	<i>// nessuna dipendenza tra i due</i>
<code>let user = fetch_user(id).await;</code>	<code>let (user, cfg) = tokio::join!(</code>
<code>let orders = fetch_orders(&user).await;</code>	<code>fetch_user(id),</code>
<i>// Tempo: t_user + t_orders</i>	<code>fetch_cfg()</code>
	<code>);</code>
	<i>//Tempo: max(t_user, t_cfg)</i>

Regola pratica: si introducono `.await` sequenziali solo quando un passo usa un risultato precedente. Se due operazioni sono indipendenti, `join!` le sovrappone senza creare task né richiedere **Send**.

14.11.2 Il caso speciale: attività che richiedono molta CPU

La schedulazione implementata dall'esecutore è cooperativa

- Un task async non deve occupare il thread soggiacente a lungo senza effettuare `.await`
- Un calcolo lungo ($> \sim 100\mu s$) senza punti di sospensione blocca il thread e ritarda tutti gli altri task su quella coda
- È l'errore più comune: "il mio server async è lento" spesso significa che un task non cede mai il controllo

`tokio::task::spawn_blocking(f)` sposta una closure sincrona su un pool dedicato

Pool separato e limitato, distinto dai thread worker async. Tokio mette a disposizione due thread pool: uno a dimensione fissa per gli executor, uno limitato per le chiamate bloccanti. Per eseguire operazioni che richiedono un uso massiccio della CPU, si delega a librerie dedicate come Rayon, eventualmente regolando la concorrenza con un semaforo

14.11.3 Quattro modi di coordinate i Future

Caso	Strumento
Dipendenza	<code>.await</code>
Indipendenti, stessi scope	<code>join!</code>
Indipendenti, lifecycle separato	<code>spawn</code>
Reagire al primo evento	<code>select!</code>

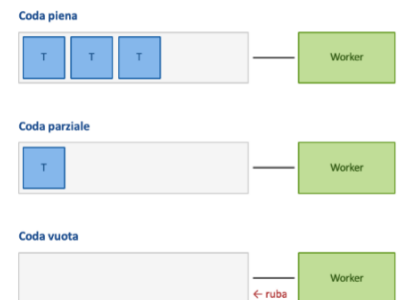
14.12 Condividere e comunicare

14.12.1 Architettura di elaborazione

tokio opera utilizzando un certo numero di code di esecuzione (una per core).

Ogni coda è gestita da un ciclo reattivo (Processor) che estrae ed esegue i task presenti. Quando un Processor esaurisce i task della propria coda, prova a rubarne alcuni ad altre code, su base euristica. Sincronizzazione minimizzata con operazioni atomiche

Conseguenza: un task può iniziare e proseguire su thread diversi → serve coordinamento. I valori che attraversano un `.await` devono essere **Send** I riferimenti condivisi devono essere **Sync** Stessa disciplina del multithread: i dati condivisi vanno protetti



14.13 Condividere dati tra task

Tokio mette a disposizione una ricca serie di primitive asincrone nel modulo `tokio::sync`.

Due approcci:

- Condivisione dello **stato** tramite **Barrier**, **Mutex**, **Notify**, **RwLock**, **Semaphore**
- Comunicazione tramite **messaggi** con canali **oneshot**, **mpsc**, **broadcast**, **watch**

Sebbene assomiglino alle primitive della libreria standard, c'è una differenza fondamentale:

- Le operazioni di sincronizzazione sono **asincrone** e non bloccanti
- Se la risorsa non è disponibile, il task corrente viene sospeso fino a che non diventa possibile proseguire, senza impedire al thread soggiacente di elaborare altri task.

14.13.1 Stato condiviso: `Arc<tokio::sync::Mutex<T>`

Per condividere uno stato mutabile tra task, si combina **Arc** (possesso condiviso) con **Mutex** (accesso esclusivo).

Si usa `tokio::sync::Mutex`, non `std::sync::Mutex`: il metodo `lock()` è asincrono. Attendere il lock con `.await` non blocca il thread: cede il controllo fino a che il lock è libero. Non mantenere un lock attraverso `.await`

```
use tokio::sync::Mutex;
use std::sync::Arc;
#[tokio::main]
async fn main() {
    let data = Arc::new(Mutex::new(0));
    let mut v = vec![];
    for _ in 0..4 {
        let data = Arc::clone(&data);
        v.push(tokio::spawn(async move {
            let mut lock = data.lock().await;
            *lock += 1;
        }));
    }
    for h in v { let _ = join!(h); }
    assert_eq!(*(data.lock().await), 4);
}
```

14.13.2 Stato condiviso vs canali

Usare **Arc<Mutex<T>** è semplice, ma spesso NON è la soluzione migliore

Può diventare:

- Un collo di bottiglia
- Fonte di deadlock
- Difficile da far scalare

Stato condiviso $\Leftrightarrow$ sincronizzazione esplicita

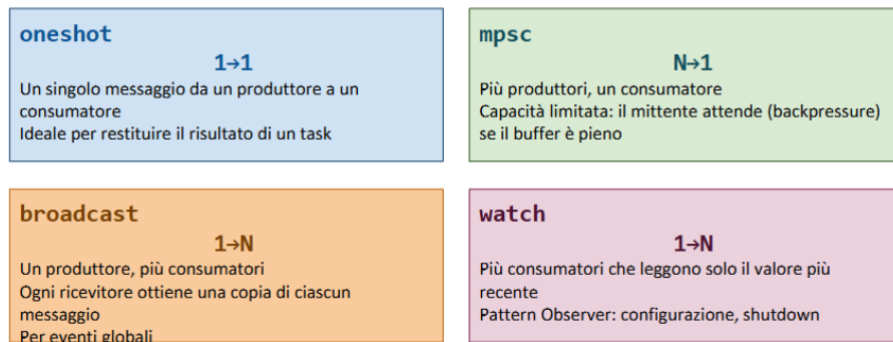
Spesso è più semplice passare messaggi che condividere memoria

- Elimina l'uso di lock
- Elimina corse critiche
- Struttura il flusso di comunicazione e di sincronizzazione allo stesso tempo

Stato condiviso	Canali
Accesso diretto ai dati	Comunicazione esplicita
Richiede lock	Nessun lock
Più flessibile	Più strutturato
Rischio deadlock	Più sicuro
Più difficile da scalare	Più naturale per sistemi concorrenti

Note: Preferire l'uso dei canali quando possibile

14.13.3 Comunicare invece di condividere: i canali



Canali oneshot - Invio di un solo messaggio

```
async fn some_computation() -> String { "Some result".to_string() }

#[tokio::main]
async fn main() {
    let (tx, rx) = oneshot::channel();
    tokio::spawn(async move {
        let res = some_computation().await;
        tx.send(res).unwrap(); // il canale viene chiuso automaticamente
    });
    // altro lavoro

    // attende il risultato
    let res = rx.await.unwrap();
}
```

Canali mpsc - Multiple Producer Single Consumer

```
async fn some_computation(i: u32) -> String { format!("Value {}", i) }

#[tokio::main]
async fn main() {
    // capacità 10 → backpressure
    let (tx, mut rx) = mpsc::channel(10);

    tokio::spawn(async move {
        for i in 0..10 {
            let res = some_computation(i).await;
            tx.send(res).await.unwrap();
        }
    });
    while let Some(res) = rx.await.unwrap() { println!("{}", res); }
}
```

Canali broadcast - comunicazione multi-molti

```
#[tokio::main]
async fn main() {
    let (tx, mut rx1) = broadcast::channel(16);
    let mut rx2 = tx.subscribe();

    tokio::spawn(async move {
        assert_eq!(rx1.recv().await.unwrap(), 10);
        assert_eq!(rx1.recv().await.unwrap(), 20);
    });
    tokio::spawn(async move {
        assert_eq!(rx2.recv().await.unwrap(), 10);
    });
}
```

```

    assert_eq!(rx2.recv().await.unwrap(), 20);
  });
  tx.send(10).unwrap();
  tx.send(20).unwrap();
}

```

Canali watch - pattern Observer

```

#[tokio::main]
async fn main() {
    let (tx, mut rx) = watch::channel("value 0");
    for i in 0..2 {
        let mut rx = rx.clone();
        tokio::spawn(async move {
            while rx.changed().await.is_ok() {
                println!("received: {:?}", *rx.borrow());
            }
        });
    }
    let d = Duration::from_secs(1);
    tx.send("value 1").unwrap(); tokio::time::sleep(d).await;
    tx.send("value 2").unwrap(); tokio::time::sleep(d).await;
}

```

14.13.4 Backpressure

Se i produttori sono più veloci dei consumatori, la coda cresce. Una coda illimitata trasforma un problema di throughput in un problema di memoria. Un canale limitato rende esplicita la capacità di un sistema.

Quando il buffer del canale è pieno, il produttore deve:

- Attendere
- Scartare
- Degradare
- Fallire esplicitamente

Note: La capacità non va nascosta: va progettata

14.13.5 Quando usare cosa

Caso	Strumento
Stato condiviso semplice	<code>Arc<Mutex<T>></code>
Pipeline / flusso dati	<code>mpsc</code>
Notifica singola	<code>oneshot</code>
Eventi globali	<code>broadcast</code>
Stato osservabile	<code>watch</code>

14.13.6 Errori tipici in async Rust

- Usare `async` per lavoro CPU-bound
- Fare `spawn` e ignorare il `JoinHandle`
- Tenere il possesso di un lock mentre si esegue `.await`

- Usare `Arc<Mutex<T>` come soluzione predefinita
- Usare code non limitate senza politica di backpressure
- Non prevedere timeout, cancellazione e terminazione del programma

Implementare un semplice server Http

```
use tokio::io::AsyncWriteExt;
use tokio::net::{TcpListener, TcpStream};
use tokio::task;

#[tokio::main]
async fn main() {
    let listener = TcpListener::bind("127.0.0.1:8181").await.unwrap();

    loop {
        let (stream, _) = listener.accept().await.unwrap();
        // ogni connessione -> un task indipendente
        tokio::spawn(handle_connection(stream));
    }
}

async fn handle_connection(mut stream: TcpStream) {
    let contents = "{\"message\": \"Hello, Tokio!\"}";

    let response = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nContent-Length: {} \r\n\r\n{}",
        contents.len(),
        contents
    );

    stream.write(response.as_bytes()).await.unwrap();
    stream.flush().await.unwrap();
}
```

14.14 Come funziona internamente

14.14.1 Esecuzione parziale

La macchina a stati finiti può essere implementata da una chiusura. Essa racchiude il proprio stato e tutte le variabili locali di cui l'esecuzione ha bisogno. Quando viene eseguita ritorna un valore che indica se ha raggiunto uno degli stati finali o se si trova ancora in uno stato intermedio. Questo permette di non bloccare il thread corrente e procedere con l'esecuzione di altri task

I punti di blocco sono tutti in corrispondenza degli stati intermedi. Questi sono immediatamente preceduti dall'invocazione di una funzione asincrona. Quando uno stato intermedio viene raggiunto, la chiusura ritorna

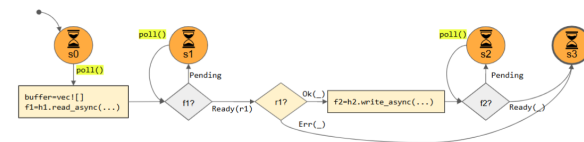
Perché la macchina a stati possa progredire occorrono due condizioni:

- Che la funzione asincrona invocata scateni qualche attività (in che thread avviene?) che possa portare all'evoluzione dello stato
- Che ci sia qualcuno che indichi che vi sono le condizioni affinché lo stato possa evolvere

14.14.2 Generare la macchina a stati

```
async fn copy(h1: FileHandle, h2: FileHandle) -> std::io::Result<()> {
    let mut buffer = vec![];
    h1.read_async(&mut buffer).await?;
    h2.write_async(&buffer).await
}
```

Per ciascuno stato individuato, il compilatore sintetizza una struct in cui memorizzare le variabili locali necessarie per consentire l'esecuzione.



```
struct S0 {
    h1: FileHandle,
    h2: FileHandle,
}
```

```
struct S2 {
    f2: impl Future<Output=Result<usize>>,
}
```

```
struct S1 {
    h2: FileHandle,
    buffer: Vec<u8>,
    f1: impl Future<Output=Result<usize>>,
}
```

```
struct S3 {}
```

Inoltre genera una enum che racchiude i possibili stati e implementa il tratto Future

```
enum CopySM {
    s0(S0),
    s1(S1),
    s2(S2),
    s3(S3)
}

impl Future for CopySM {
    type Output = std::io::Result<>
    fn poll(self: Pin<&mut Self>, cx: &mut Context)
        -> Poll<Self::Output> {
        loop { match self {
            CopySM::s0(state) => { ... },
            CopySM::s1(state) => { ... },
            CopySM::s2(state) => { ... },
            CopySM::s3(state) => { ... },
        } }
    }
}
```

```
CopySM::s0(state) => {
    let mut buffer = vec![];
    let f1 = state.h1.read_async(&mut buffer);
    let state = S1 {h2: state.h2, buffer, f1 };
    *self = CopySM::S1(state);
}
```

```
CopySM::s1(state) => {
    match (state.f1.poll(cx) {
        Poll::Pending => return Poll::Pending,
        Poll::Ready(r1) =>
            if r1.is_ok() {
                let f2 = state.h2.write_async(&mut state.buffer);
                let state = S2{ f2 };
                *self = CopySM::s2(state);
            } else {
                *self = CopySM::s3(S3);
                return Poll::Ready(r1);
            }
    })
}
```

```
CopySM::s2(state) => {
    match (state.f2.poll(cx) {
        Poll::Pending => return Poll::Pending,
        Poll::Ready(r2) =>
            *self = CopySM::s3(S3);
            return Poll::Ready(r2);
    })
}
```

```
CopySM::s3(_) => {
    panic!("poll() was invoked again after Poll::Ready has been returned");
}
```

14.14.3 Implementare la funzione

Il codice generato dal compilatore per la funzione si riduce all'inizializzazione della macchina a stati.

```
fn copy(h1: FileHandle, h2: FileHandle) ->
    impl Future<Output = std::io::Result<>> > {
    CopySM::s0(
    S0 { h1, h2 }
    )
}
```

14.15 Prestazioni a confronto

I costi di creazione e cambiamento di contesto tra attività asincrone e thread sono analizzati nel sito

<https://github.com/jimblandy/context-switch>. Il sito riporta la metodologia, il codice e i risultati ottenuti su un elaboratore specifico, con il relativo sistema operativo. Gli ordini di grandezza sono comunque interessanti.

Operazione	async	thread
Creazione di task	0.3 μ s	17 μ s
Cambio di contesto	0.2 μ s	1.7 μ s
Uso di memoria	300 – 500 Bytes	> 9.5 KBytes

Dall'analisi di questi numeri, emerge che l'uso di costrutti asincroni può ridurre l'utilizzo della memoria. Specialmente in quei contesti in cui si tenderebbe a preallocare un elevato numero di thread per poter fare fronte a carichi di lavoro improvvisi (server di rete).

È meno oneroso creare task asincroni rispetto a thread. Inoltre il tempo necessario a cambiare contesto è minore, ma solo nei casi migliori: se il cambio di contesto è dovuto alla disponibilità di dati su una periferica su cui si intende eseguire I/O, il costo è analogo Operazioni di I/O asincrone, che richiedano un'attesa, dovranno essere ritentate in seguito. Questo richiede l'esecuzione di (almeno) due system-call, invece che una sola come nel caso sincrono

14.15.1 Quando l'esecuzione asincrona non conviene?

L'esecuzione asincrona non è universalmente migliore, ha un dominio di applicazione preciso:

- **Compiti strettamente legati alla CPU:** se i task non si sospendono quasi mai, async aggiunge complessità senza benefici; si usano i thread
- **Pochi compiti, di breve durata:** il costo dei thread è trascurabile e il codice sincrono è più semplice da scrivere e correggere
- **Codice fortemente sequenziale:** se non c'è nulla da sovrapporre, l'asincronicità è solo un appesantimento concettuale

L'esecuzione asincrona dà il meglio di sé quando si gestiscono **molte attese simultanee** come server di rete con migliaia di connessioni, client che interrogano molti servizi in parallelo, pipeline I/O- intensive.

Si possono combinare i due mondi: una componente asincrona per I/O, `spawn_blocking` o `Rayon` per il calcolo puro.

Parte II

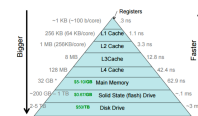
OS Programming

Capitolo 15

Main memory

15.1 Memoria principale

Questa sezione illustra un modo efficace per sfruttare al meglio le potenzialità dell'hardware acquistato; il focus principale è sulla **memoria principale**. Abbiamo visto all'inizio del corso che un programma, memorizzato su disco, diventa un processo quando viene caricato dal disco nella memoria principale.



La memoria principale e i registri sono gli unici supporti di memorizzazione a cui la CPU può accedere direttamente. L'unità di memoria vede solo un flusso di:

- indirizzi + richieste di lettura, oppure
- indirizzo + dati e richieste di scrittura

La CPU può accedere ai registri in uno, o meno, cicli di clock. Tuttavia, quando la CPU preleva dati dalla memoria principale utilizza molti cicli di clock, causando uno **stall**.

La cache si trova tra la memoria principale e i registri e funge da buffer per i dati necessari; non vogliamo un collo di bottiglia.

Tutti questi livelli contengono una copia dello stesso sottoinsieme di dati.

15.1.1 Località spaziale e località temporale

Località spaziale: La località spaziale significa che se leggo una variabile o un'istruzione in memoria, tutti i dati vicini hanno un'alta probabilità di essere prelevati; pertanto quando leggo, leggo in blocchi e memorizzo tutti i dati in una memoria più veloce.

Località temporale: La località temporale significa che se leggo una variabile, la probabilità di riutilizzarla è alta; pertanto la mantengo nella memoria cache.

S.No.	Spatial Locality	Temporal Locality
1.	In Spatial Locality, nearby instructions to recently executed instruction are likely to be executed soon.	In Temporal Locality, a recently executed instruction is likely to be executed again very soon.
2.	It refers to the tendency of execution which involve a number of memory locations .	It refers to the tendency of execution where memory location that have been used recently have a access.
3.	It is also known as locality in space.	It is also known as locality in time.
4.	It only refers to data item which are closed together in memory.	It repeatedly refers to same data in short time span.
5.	Each time new data comes into execution.	Each time same useful data comes into execution.
6.	Example : Data elements accessed in array (where each time different (or just next) element is being accessing).	Example : Data elements accessed in loops (where same data elements are accessed multiple times).

15.2 Problema della riallocazione

Si presenta il problema di riallocazione: l'indirizzo deve essere scalato a quello reale. Come si può risolvere?

- Si guardando i riferimenti e calcolo l'indirizzo. Difficile
- Carico gli indirizzi così come sono. Fattibile utilizzando l'hardware.

Utilizzando l'hardware, base e limit, prima di arrivare al bus inseriamo un sommatore e in assembly non cambia nulla.

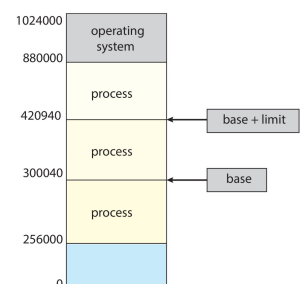
15.3 Protezione

Quando un programma è diventato un processo, tutti o gran parte dei suoi dati sono memorizzati nella memoria principale. Abbiamo bisogno di un modo per:

- Suddividere la memoria tra tutti i processi;
- Garantire che un processo possa accedere solo agli indirizzi del proprio spazio di indirizzamento^a.

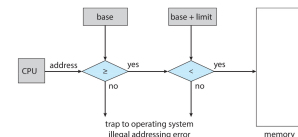
Possiamo risolvere questi problemi utilizzando una coppia di registri **base** e **limite** che definiscono lo spazio di indirizzamento logico di un processo.

^aLo spazio di indirizzamento, o address space, è una porzione di memoria cui un programma può utilizzare



15.3.1 Protezione degli indirizzi hardware

La CPU/OS deve verificare ogni accesso alla memoria generato in modalità utente per assicurarsi che sia compreso tra base e limite per quell'utente. Le istruzioni per caricare i registri base e limite sono privilegiate.



15.4 Binding degli indirizzi

Quando scriviamo un programma C, vogliamo utilizzare indirizzi simbolici e vogliamo che sia il compilatore a trasformarlo in indirizzi relativi. Il linker decide di riallocare a valori assoluti.

Inoltre non ci vogliamo preoccupare di dove si trovi la variabile in memoria, sappiamo solo che è in memoria.

- Gli indirizzi nel codice sorgente sono tipicamente simbolici;
- Gli indirizzi nel codice compilato vengono legati a indirizzi rilocabili, es. «14 byte dall'inizio di questo modulo» invece di «byte 14»;
- Il linker o il loader leggerà gli indirizzi rilocabili ad indirizzi assoluti;
- Ogni binding mappa uno spazio di indirizzamento in un altro.

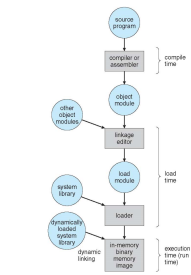
Nel codice compilato vediamo quanta memoria è necessaria e non l'indirizzo codificato in modo fisso. Vogliamo un binding dinamico e non statico.

Binding di istruzioni e dati in memoria

Quando un programma viene avviato, il SO assegna una quantità di spazio al programma.

Il binding degli indirizzi di istruzioni e dati agli indirizzi di memoria può avvenire in tre momenti diversi:

- **Tempo di compilazione:** Se la posizione in memoria è nota a priori, può essere generato codice assoluto; è necessario ricompilare il codice se la posizione iniziale cambia. Utilizzato nei sistemi embedded o nei sistemi in cui i task sono noti a priori e sono in numero ridotto. Sto generando l'eseguibile;
- **Tempo di caricamento:** Deve essere generato codice rilocabile se la posizione in memoria non è nota al momento della compilazione. Strategia ampiamente utilizzata nei vecchi SO o in sistemi semplici come le lavastoviglie...
- **Tempo di esecuzione:** Il binding viene ritardato al runtime se il processo può essere spostato durante l'esecuzione da un segmento di memoria a un altro; richiede supporto hardware (es. registri base e limite) ed è lo standard de facto per i SO moderni.



15.5 Memory Management Unit - MMU

Ho bisogno di un modo per passare dalla memoria virtuale alla memoria hardware e viceversa, soprattutto quando gli indirizzi sono diversi in fase di esecuzione. Introduciamo il concetto di **processo di binding**.

15.5.1 Spazio di indirizzamento logico vs. fisico

Il concetto di spazio di indirizzamento logico legato a un separato **spazio di indirizzamento fisico** è centrale per una corretta gestione della memoria.

- **Indirizzo logico** - generato dalla CPU; detto anche **indirizzo virtuale**;
- **Indirizzo fisico** - indirizzo visto dall'unità di memoria.

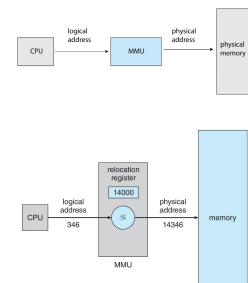
Gli indirizzi logici e fisici coincidono negli schemi di binding al tempo di compilazione e di caricamento, ma differiscono nello schema di binding al tempo di esecuzione.

15.6 MMU

Dispositivo hardware, all'interno della CPU, che al runtime mappa gli indirizzi virtuali, che troviamo nel codice assembler, in indirizzi fisici. Vi sono diverse tipologie di MMU, anche in base a quanto vogliamo far pagare il chip.

Il registro base ora si chiama registro di rilocalizzazione. Il valore nel registro di rilocalizzazione viene aggiunto a ogni indirizzo generato da un processo utente nel momento in cui viene inviato alla memoria.

Questo componente è composto da un sommatore ed un comparatore per bloccare il programma se vuole utilizzare spazi non facenti parti della sua address space.



Il programma utente lavora con indirizzi logici; **non vede mai i veri indirizzi fisici**. Se si stampa l'indirizzo di una variabile in un programma C, viene stampato il valore logico e non il valore fisico reale!

15.6.1 Registro di riallocazione

Se abbiamo una MMU basta su Relocation register, gli indirizzi logici sono calcolati al volo. Viene utilizzato quando si vuole ritardare il binding fino alla fase di esecuzione.

Oltre che al relocation register ci sono anche altri modi per caricare nuovi processi in memoria: **Dynamic loading** e **dynamic linking**

15.6.2 Caricamento dinamico

L'intero programma non deve necessariamente essere tutto in memoria per essere eseguito. Una routine, parte del programma, non vorremmo venisse caricata finché non viene chiamata. Questo migliora l'utilizzo dello spazio in memoria. Il loader perciò deve intervenire quando vogliamo caricare nuove porzioni di codice. Questo fa sì che il programma deve essere organizzato ed avere un formato specifico.

Il SO può aiutare fornendo librerie per implementare il caricamento dinamico.

15.6.3 Linking Statico vs Dinamico

Linking statico - librerie di sistema e codice di programma sono tutti combinati dal linker nell'immagine del programma binario in fase di compilazione. Problema: grandi eseguibili.

Linking dinamico - il linking è rinviato fino alla fase di esecuzione del programma.

Nota: Le DLL possono avere le due varianti: load-time dynamic linking (caricamento all'avvio) o run-time dynamic linking (caricamento su richiesta durante l'esecuzione)

Non tutti gli indirizzi sono risolti prima dell'esecuzione, pensiamo ad esempio alla printf, possiamo assumere che molti programmi in esecuzione possono volerla utilizzare. Per questo motivo teniamo ancora indirizzi non risolti e inseriamo piccolo frammento di codice, chiamati *stub*, utilizzato per individuare la routine di libreria residente in memoria. A runtime lo stub, che faceva finta di essere la routine corretta, viene sostituita con l'indirizzo della routine corretta e la esegue a runtime.

Il sistema operativo verifica se la routine si trova nello spazio di indirizzamento del processo e, in caso contrario, la aggiunge.

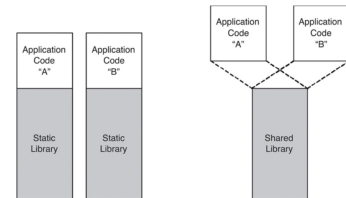


Figura 15.1: File .dll

Vantaggio: Il dynamic linking è particolarmente utile per le librerie, il sistema è anche visto come “shared libraries” (librerie condivise); questo consente di ottenere programmi di dimensioni ridotte poiché le librerie sono condivise tra più programmi. Tutti i programmi, tranne il primo che la caricherà, la troveranno già in memoria.

Questo metodo offre inoltre un ulteriore vantaggio: quando Windows aggiorna una libreria, è necessario ricompilare solamente quella e non tutte le altre applicazioni che la utilizzano.

Problemi: Spazio condiviso e versioning delle librerie, normalmente viene risolto tramite l'aggiornamento di queste ultime.

15.7 Allocazione dei programmi

A questo punto dobbiamo parlare di dove effettivamente vengono allocati i programmi in memoria, e di come viene creato lo spazio di indirizzamento. Nella memoria centrale va inserito tutto, dal sistema operativo ai processi utente. Solitamente il SO si prende una parte di RAM in alto o in basso dato che lui non parteciperà al gioco dell'allocazione e deallocazione della memoria

15.7.1 Allocazione contigua

Vogliamo gestire efficientemente la memoria principale perché deve supportare sia il SO che i processi utente ed è una risorsa limitata; deve essere allocata in modo efficiente.

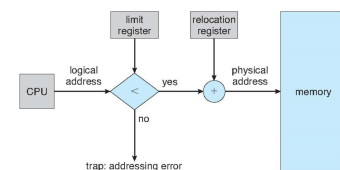
L'allocazione contigua è uno dei primi metodi: array e liste concatenate.

La memoria principale è solitamente suddivisa in due partizioni:

- Sistema operativo residente, solitamente collocato in memoria bassa con il vettore degli interrupt;
- I processi utente sono collocati in memoria alta

Ogni processo è contenuto in una singola sezione contigua di memoria.

Il relocation register viene utilizzato per proteggere i processi utente l'uno dall'altro e per impedire modifiche al codice e ai dati del sistema operativo.



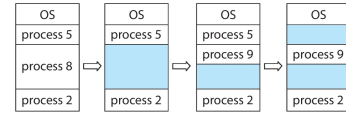
- Il base register contiene il valore del più piccolo indirizzo fisico
- Il limit register contiene l'intervallo degli indirizzi logici - ogni indirizzo logico deve essere inferiore al registro limite
- La MMU mappa dinamicamente gli indirizzi logici

15.7.2 Partizione variabile

Il SO può decidere l'allocazione a partizioni multiple. Il grado di multiprogrammazione è limitato dal numero di partizioni.

- **Partizione variabile** - dimensioni per l'efficienza (dimensionate in base alle necessità del processo).
- **Buco** - blocco di memoria disponibile; buchi di varie dimensioni sono dispersi nella memoria. Indesiderato; idealmente si vuole un blocco contiguo di memoria disponibile.

Quando un processo arriva, gli viene allocata memoria da un buco sufficientemente grande da contenerlo; quando un processo termina e libera la sua partizione, le partizioni libere adiacenti vengono unite. Tutto ciò è gestito dal sistema operativo che mantiene informazioni su: partizioni allocate e partizioni libere (buchi)



15.7.3 Problema dell'allocazione dinamica della memoria

Come soddisfare una richiesta di dimensione n da una lista di buchi liberi?

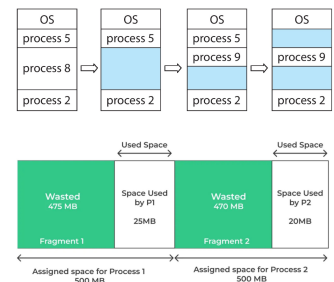
- **First-fit**: Alloca il primo buco sufficientemente grande
Può produrre molti piccoli buchi residui
- **Best-fit**: Alloca il buco più piccolo sufficientemente grande; deve scorrere l'intera lista, salvo che sia ordinata per dimensione
Produce il buco residuo più piccolo, che potrebbe non essere più riempito.
- **Worst-fit**: Alloca il buco più grande; deve anch'esso scorrere l'intera lista
Produce il buco residuo più grande, che può essere riempito da altri processi.

First-fit e best-fit sono migliori di worst-fit in termini di velocità e utilizzo della memoria. Per decidere tra Best e Worst fit andrebbero fatti test sul campo.

15.7.4 Frammentazione

Frammentazione esterna - esiste sufficiente spazio di memoria totale per soddisfare una richiesta, ma non è contiguo. Sia la strategia first-fit che best-fit causano frammentazione esterna.

Frammentazione interna - la memoria allocata può essere leggermente più grande di quella richiesta; la dimensione minima è definita dal SO e la differenza tra la dimensione del programma C e questa dimensione minima è detta frammentazione interna. L'uso della partizione dinamica per assegnare spazio al processo può risolvere questo problema.



È sensato pensare che allocando N blocchi, la metà sono frammentati. E.g. 100 blocchi, 50 frammentati, 50 utilizzati, frammentazione del 50% (thumb rule).

15.7.5 Compattazione

Si riduce la frammentazione esterna tramite la **compattazione**: il contenuto della memoria viene riorganizzato in modo da raccogliere tutta la memoria libera in un unico blocco contiguo. La compactazione è possibile solo se la rilocalizzazione è dinamica e viene eseguita al tempo di esecuzione.

Esempio: La cosa migliore sarebbe trasferire tutti i processi da un'altra parte, questo implica avere altra RAM la quale avrei utilizzato se l'avessi avuta. Posso utilizzare il disco ma questo significa che non potrei utilizzare il pc.

Inoltre si presenta il problema dell'I/O: se un processo è "waiting for IO", non posso spostarlo perché altrimenti l'I/O non saprebbe dove inviare i dati.

Soluzioni: 1. Evitare che il processo venga spostato, la cui conseguenza è una deframmentazione non ottimale; 2. Spostare comunque il programma utilizzando un buffer, anche se questo comporta un tempo complessivo di spostamento più lungo. Spostare la memoria da utente a kernel fatto all'inizio di "waiting for IO".

Sebbene la tecnica di compattazione sia molto utile per migliorare l'utilizzo della memoria e ridurre la frammentazione esterna, il problema è che durante il processo viene sprecata una grande quantità di tempo e la CPU rimane inattiva, riducendo l'efficienza complessiva del sistema.

15.8 Paginazione

Supponiamo di consentire che lo spazio di indirizzamento fisico di un processo possa essere non contiguo: il vantaggio è maggiore. In questo modo, al processo viene allocata memoria fisica ogni volta che la memoria principale è disponibile. Evitiamo la frammentazione esterna e blocchi di memoria di dimensioni variabili, sono della stessa dimensione.

Il processo di paginazione consiste nel dividere la memoria fisica in blocchi di dimensione fissa chiamati **frame**; la dimensione è una potenza di 2, compresa tra 512 byte e 16 MiB. Anche la memoria logica viene divisa in blocchi della stessa dimensione chiamati **pagine**.

Nota: a questo punto è indifferente parlare di SSD o RAM, è la stessa cosa; cambiano solo la latenza e la volatilità.

Questo permette di tenere traccia di tutti i frame liberi e consente di eseguire un programma di dimensione N pagine, trovando N frame liberi e caricando il programma. Per fare ciò dobbiamo impostare una tabella delle pagine per tradurre gli indirizzi logici in fisici.

La frammentazione interna è ancora presente.

Per trovare i frame liberi possiamo utilizzare una free-list oppure un vettore chiamato bitmap.

15.8.1 Schema di traduzione degli indirizzi

L'indirizzo (virtuale) generato dalla CPU è suddiviso in:

- **Numero di pagina (p)** - usato come indice in una tabella delle pagine che contiene l'indirizzo base di ogni pagina nella memoria fisica;
- **Offset di pagina (d)** - combinato con l'indirizzo base definisce l'indirizzo di memoria fisica inviato all'unità di memoria.

Per un dato spazio di indirizzamento logico 2^m e dimensione di pagina 2^n .

page number	page offset
p	d
m - n	n

15.9 Hardware di paginazione

Abbiamo quasi annullato il problema della frammentazione dato che tutti i processi ora possono prendere la stessa dimensione della pagina, e non importa se sono contigue o no.

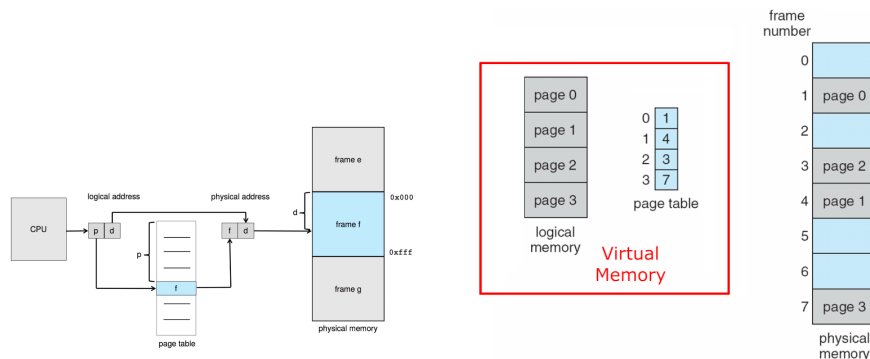
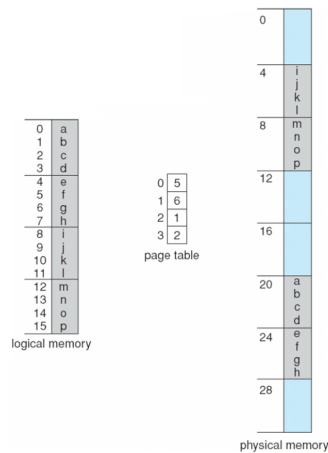


Figura 15.2: Modello di paginazione della memoria logica e fisica

Esempio di paginazione

Indirizzo logico: $n = 2$ (bit di offset), $n-m = 3$ (bit di pagina). Con una dimensione di pagina di 4 byte e una memoria fisica di 32 bit ($2^m = 8$ pagine).



Paginazione - Calcolo della frammentazione interna

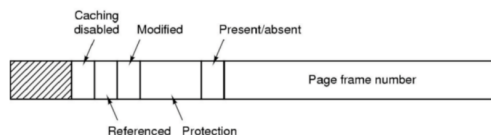
Dimensione pagina = 2.048 byte; dimensione processo = 72.766 byte; si usano 35 pagine + 1.086 byte, la frammentazione interna è $2.048 - 1.086 = 962$ byte.

Caso peggiore = dimensione frame - 1 byte; in media frammentazione = $1/2$ dimensione frame.

Quindi, frame di dimensione ridotta consentono una piccola frammentazione interna... ma la dimensione della tabella delle pagine aumenta perché ogni tabella delle pagine, per processo, occupa memoria: dimensione pagina piccola = moltissime pagine = molta memoria allocata per tracciare le pagine e non disponibile per i task utente/SO.

Le dimensioni delle pagine sono cresciute nel tempo:

- Solaris supporta due dimensioni di pagina - 8 KB e 4 MB
- Win10 supporta due dimensioni di pagina - 4 KB e 2 MB
- Linux supporta due dimensioni di pagina - 4 KB e dimensione huge page personalizzata



Sappiamo, dato l'architettura e il funzionamento della memoria e dei processori, che non possiamo allocare vettori di dimensioni arbitrarie, ma dobbiamo utilizzare potenze di 2.

A questo punto, essendo che abbiamo celle disponibili, possiamo memorizzare dati aggiuntivi. Ad esempio, la protection bit per indicare se il frame è read-only o se è anche permesso fare accessi read-write.

15.9.1 Quante pagine?

Supponiamo una dimensione di pagina di 4KB (212 byte) e una CPU hardware a 32 bit. La voce della tabella delle pagine è pari a 32 bit. 2^{32} pagine, ciascuna da 4KiB, per un totale di memoria indirizzabile pari a 16 TiB ($2^{12} * 2^{10}$, dove il primo termine rappresenta gli indirizzi di paginazione e il secondo l'offset).

Altro esempio:

$m = \text{lunghezza indirizzo} = 8 \text{ bit.}$

$n = \log_2(\text{dimensione pagina}) = \log_2(32 \text{ byte}) = 5 \text{ bit di offset.}$

$m - n = 3 = \text{numero di pagine che posso indirizzare.}$

Riducendo la dimensione della pagina si possono indirizzare più pagine e l'offset diminuisce. Normalmente $m = 32$ e la dimensione della pagina è 4KiB, come nell'esempio precedente.

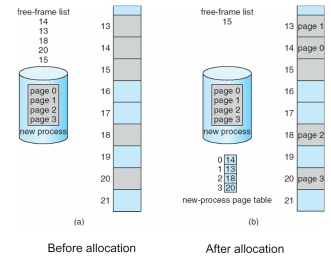


Figura 15.3: Frame liberi

15.9.2 Implementazione della page table

La tabella delle pagine, come detto in precedenza, è mantenuta nella memoria principale. È il principale "artefatto" della gestione della memoria virtuale:

Il registro base della tabella delle pagine (Page-table base register - PTBR) punta alla tabella delle pagine

Il registro di lunghezza della tabella delle pagine (Page-table length register - PTLR) indica la dimensione della tabella delle pagine

In questo schema ogni accesso a dati o istruzioni richiede **due accessi alla memoria!** Uno per la tabella delle pagine e uno per i dati o l'istruzione.

Il problema del doppio accesso alla memoria può essere risolto tramite l'uso di una speciale cache hardware ad accesso rapido chiamata *translation look-aside buffer*, **TLB**.

Sfrutta la **località temporale**: la tendenza dei programmi a utilizzare ripetutamente gli stessi dati nel corso della loro esecuzione.

15.9.3 Soluzione hardware - Translation Look-Aside Buffer

La TLB è una cache univoca per tutte le page table. Il primo approccio potrebbe essere quello di svuotare la TLB ad ogni context switch, ma questo è inefficiente.

Invece di svuotare la TLB, memorizziamo anche **identificatori dello spazio di indirizzamento di un dato processo (ASID)** in ogni voce del TLB.

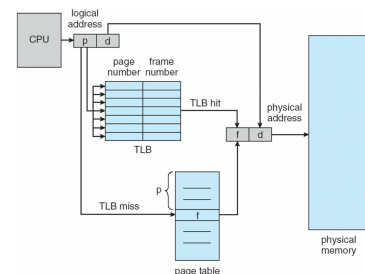
I TLB sono tipicamente piccoli, da 64 a 1024 voci, ma più veloci; sono quindi una cache completamente associativa.

In caso di TLB miss, il valore viene caricato nel TLB per un accesso più rapido la volta successiva; le politiche di sostituzione devono essere considerate; alcune voci possono essere fissate per un accesso rapido permanente.

Non accelera tutto:

- Caso medio: accesso nel TLB + accesso in memoria per il dato;
- Caso peggiore: nessun dato nel TLB, quindi accesso in memoria per la tabella delle pagine + accesso in memoria per il dato.

Una volta riempita la TLB, dobbiamo utilizzare algoritmi per decidere quale voce sostituire in caso di TLB miss. Sicuramente l'utilizzo di un valid-invalid bit per ogni voce del TLB è utile per identificare le voci che possono essere sovrascritte.



Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

15.9.4 Tempo di accesso effettivo

Hit ratio - percentuale delle volte in cui un numero di pagina viene trovato nel TLB. Un hit ratio dell'80% significa che troviamo il numero di pagina desiderato nel TLB l'80% delle volte.

Supponiamo che siano necessari 10 nanosecondi per accedere alla memoria. Se troviamo la pagina desiderata nel TLB, un accesso alla memoria mappata richiede 10 ns; altrimenti servono due accessi alla memoria, quindi 20 ns.

Tempo di accesso effettivo (EAT) = $0,80 \times 10 + 0,20 \times 20 = 12$ nanosecondi, il che implica un rallentamento del 20% nel tempo di accesso.

Considerando un hit ratio più realistico del 99%, $EAT = 0,99 \times 10 + 0,01 \times 20 = 10,1$ ns, il che implica solo l'1% di rallentamento nel tempo di accesso.

15.10 Codice C e page fault

Immagina di avere la RAM piena, con solo una pagina libera, e in questa pagina vogliamo fare questo:

```
int main(){
    int[128,128] data;

    ...

    for (j = 0; j < 128; j++)
        for (i = 0; i < 128; i++)
            data[i,j] = 0;

    //quanti page fault? 128 * 128 = 16384

    ...

    for (i = 0; i < 128; i++)
        for (j = 0; j < 128; j++)
            data[i,j] = 0;

    //quanti page fault? solo 128
}
```

Page fault - Un page fault si verifica quando un programma tenta di accedere a dati o codice che si trovano nel suo spazio di indirizzamento, ma non sono attualmente nella RAM del sistema. Quando si accede a qualcosa nella RAM, il SO carica un blocco di dati dall'SSD nella RAM. Nel primo caso il SO carica un blocco ma il codice C usa solo la prima colonna; nel secondo caso il SO carica un blocco di dati e ne usa tutta la colonna.

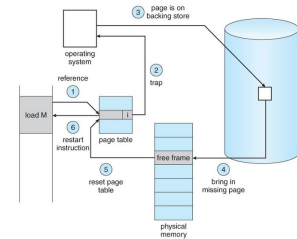


Figura 15.4: Page fault

15.11 Page fault

Cos'è un page fault? I dati sono tipicamente memorizzati in memorie non volatili, grandi e lente, e poi caricati nella RAM; ciò significa che i dati necessari per l'esecuzione (una pagina) possono esistere solo nella memoria secondaria a causa del caricamento dinamico. Devono quindi essere caricati nella RAM, poi nella cache, poi nei registri: questo è un **page fault**!

Non possiamo eliminarlo, ma cerchiamo di mitigare questo evento lento.

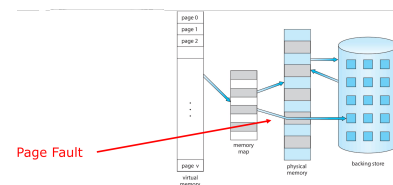


Figura 15.5: Page fault

Contesto

Il codice deve essere in memoria per essere eseguito, ma l'intero programma è raramente utilizzato interamente per via della gestione delle eccezioni, funzioni non utilizzate e larghe strutture dati.

Tutto il programma non è necessario allo stesso tempo.

Considera la possibilità di eseguire un programma parzialmente caricato: il programma non è più vincolato dai limiti della memoria fisica, ogni programma occupa meno memoria durante l'esecuzione, il che consente a più programmi di essere eseguiti contemporaneamente, aumentando l'utilizzo della CPU e il throughput senza aumentare i tempi di risposta o turnaround. Inoltre, è necessario meno I/O per caricare o scambiare programmi

in memoria, il che significa che ogni programma utente viene eseguito più velocemente.

Stiamo parlando di loading dinamico e non abbiamo bisogno che tutto il programma sia tutto in RAM. La ram deve essere capiente abbastanza da contenere il kernel + la parte dei processi che stiamo eseguendo.

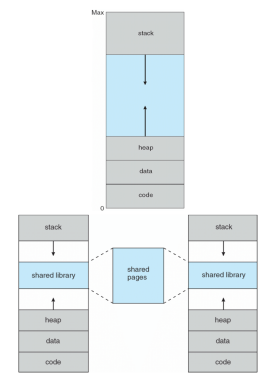
Memoria virtuale

Virtual memory: separazione tra memoria logica utente e memoria fisica; solo una parte del programma deve essere in memoria per l'esecuzione; lo spazio di indirizzamento logico può quindi essere molto più grande dello spazio di indirizzamento fisico (RAM); consente la condivisione degli spazi di indirizzamento da parte di più processi; consente una creazione di processi più efficiente; più programmi in esecuzione contemporaneamente; meno I/O necessario per caricare o scambiare processi in memoria.

Virtual address space: molto maggiore della memoria fisica; visualizzazione logica di come il processo è memorizzato in memoria; di solito inizia all'indirizzo 0 ma non è detto, con indirizzi contigui fino alla fine dello spazio; nel frattempo, la memoria fisica è organizzata in frame di pagina; la MMU deve mappare gli indirizzi logici a quelli fisici.

La virtual memory può essere implementata tramite: demand paging o demand segmentation.

Normalmente lo spazio di indirizzamento logico è progettato in modo che lo stack inizi all'indirizzo logico massimo e cresca "verso il basso", mentre l'heap cresce "verso l'alto". Questo massimizza l'utilizzo dello spazio degli indirizzi, lasciando uno spazio inutilizzato tra i due, noto come "buco". Non è necessaria memoria fisica fino a quando l'heap o lo stack non crescono fino a una nuova pagina. Ciò consente spazi di indirizzamento sparsi con buchi lasciati per la crescita, librerie dinamicamente collegate, ecc. Le librerie di sistema possono essere condivise tramite mappatura nello spazio di indirizzamento virtuale. La memoria condivisa può essere implementata mappando pagine in lettura-scrittura nello spazio di indirizzamento virtuale. Le pagine possono essere condivise durante fork(), accelerando la creazione dei processi.

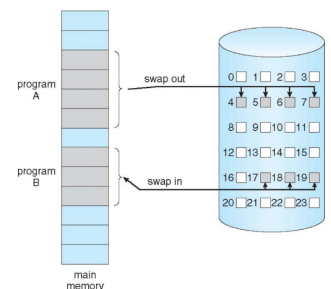


Paginazione su Richiesta - Demand Paging

Posso caricare un processo interamente in memoria al momento del caricamento, oppure posso caricare una pagina solo quando è necessaria. Il demand paging richiede meno I/O, non esegue I/O non necessario, richiede meno memoria, offre una risposta più rapida e consente più utenti. È simile a un sistema di paging con swapping.

Il demand paging consiste nel caricare la prima pagina del processo e di comportarsi come abbiamo fatto prima per i processi, swap in e swap out, ma lo fai a livello di pagina page in e page out.

Lazy swapper non swappa mai pagine nella memoria fino a che quella pagina non è necessaria.



Nota: Uno swapper trasferisce in memoria/disco l'intero processo. Quando invece si trasferiscono solo le singole pagine (e non l'intero processo), si parla di pager e l'operazione si chiama paging.

Concetti di Base

Con lo swapping, il pager indovina quali pagine saranno utilizzate prima di essere nuovamente scambiate. Invece, il pager porta in memoria solo quelle pagine necessarie. Come determinare quel set di pagine? È necessaria una nuova funzionalità della MMU per implementare il demand paging. Se le pagine necessarie sono già residenti in memoria (**Memory resident**), non c'è differenza rispetto al non demanding paging.

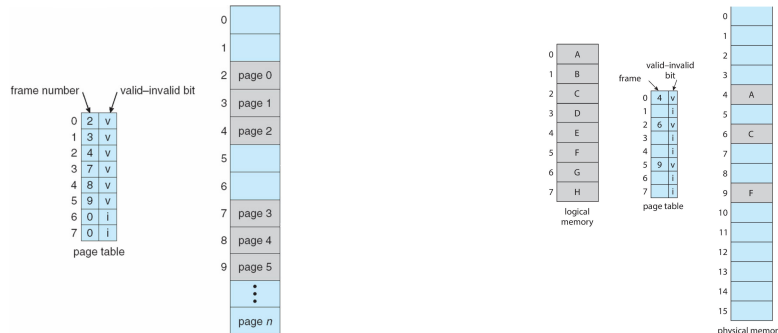
Se la pagina necessaria NON è residente in memoria, è necessario rilevare e caricare la pagina in memoria dallo storage senza modificare il comportamento del programma e senza che il programmatore debba modificare il codice.

15.11.1 Come rilevare i page fault?

Possiamo usare un bit, **valid-invalid**, per tracciare se l'indirizzo è nella RAM o nella memoria secondaria.

Nota: Vi è un problema dell'allineamento tra la memoria primaria e fisica.

- valido (1): indica che la pagina associata è nello spazio di indirizzamento logico del processo ed esiste già nella RAM;
- non valido (0): indica che la pagina non è nello spazio di indirizzamento logico del processo e non è nella RAM, il che significa che si verificherà un **page fault/trap nel kernel**. Inizialmente, sono tutte non valide.



15.11.2 Passi nella gestione dei page fault

1. Se c'è un riferimento a una pagina (invalida), il primo riferimento a quella pagina genera un trap al sistema operativo = page fault;
2. Il Sistema Operativo guarda un'altra tabella per decidere se:

Riferimento non valido → abort

Semplicemente non in memoria, deve essere caricata dal disco

3. Trovare un frame libero in memoria;
4. Caricare la pagina nel frame tramite un'operazione disco pianificata, la parte più lenta del processo;
5. Aggiornare le tabelle per indicare che la pagina è ora in memoria; impostare il bit valido = v;
6. Riavviare l'istruzione che ha causato il page fault;

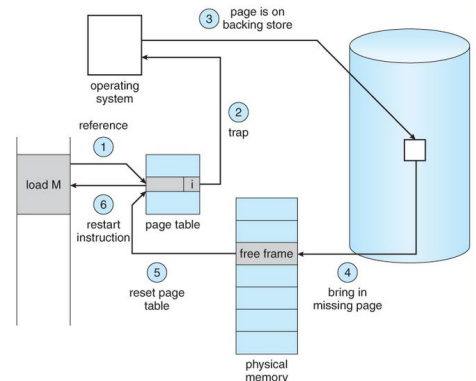


Figura 15.6: Page fault

15.11.3 Costo di un page fault

Supponiamo un tasso di page fault $0 < p < 1$; se $p = 0$ nessun page fault, se $p = 1$ ogni riferimento è un fault. Tempo di accesso effettivo:

$$EAT = (1 - p) \times \text{accesso in memoria} + p (\text{overhead page fault} + \text{swap page out} + \text{swap page in})$$

Tempo di accesso in memoria = 200 nanosecondi; tempo medio di gestione del page fault = 8 millisecondi; un accesso su 1.000 causa un page fault. Sembra un numero piccolo, ma non lo è!

Ipotizzando un tempo di accesso RAM = 200 nanosecondi, tempo medio di gestione del page fault = 8 millisecondi:

$$EAT = (1 - p)200 + p (8 \text{ millisecondi}) = 200 + p7.999.800$$

Se un accesso su 1.000 causa un page fault, allora $EAT = 8,2$ microsecondi.

Questo è un rallentamento di un fattore $8,2 / 0,2 = 41!!$ Non è per niente buono! Peggio della TLB.

Questa è probabilmente la fonte principale di rallentamenti quando si elabora qualcosa.

15.12 Aspetti nel richiedere paging

Caso estremo - avviare un processo senza pagine in memoria. La prima fetch genererà subito un page fault.

Il sistema operativo imposta il puntatore dell'istruzione alla prima istruzione del processo, che non è residente in memoria, causando un page fault. E questo accade per ogni pagina del processo al primo accesso, il che è noto come **pure demand paging**.

In realtà, un certa istruzione potrebbe accedere a più pagine → più page fault. Considera il fetch e decode di un'istruzione che aggiunge 2 numeri dalla memoria e memorizza il risultato nuovamente in memoria. Il dolore (pain) è diminuito a causa della località di riferimento, Locality of reference.

Il supporto hardware necessario per il demand paging include:

Tabella delle pagine con bit valido/invalido

Memoria secondaria (dispositivo di swap con spazio di swap)

Riavvio dell'istruzione

15.13 Riavvio dell'Istruzione

Considera un istruzione che può accedere a più locazioni, indirizzi, di memoria. La sorgente e la destinazione in alcuni caso possono sovrapporsi.

- Back move
- Auto incremento/decremento della locazione
- Riavviare l'intera operazione? Cosa succede se l'istruzione destinataria è overlapped?

15.14 Strategia di sostituzione delle pagine

Quando si verifica un page fault, il sistema operativo deve portare la pagina desiderata dalla memoria secondaria nella memoria principale.

La maggior parte dei sistemi operativi mantiene una lista di frame liberi – un pool di frame liberi per soddisfare tali richieste.

Il puntatore è inserito proprio nel frame e quando un frame è libero utilizzala per non allocare un'altra struttura per contenere la lista di frame.

head → 7 → 97 → 15 → 126 ... → 75

I sistemi operativi tipicamente allocano i frame liberi tramite una tecnica nota come zero-fill-on-demand – il contenuto dei frame viene azzerato prima di essere allocato. All'avvio del sistema, tutta la memoria disponibile viene inserita nella lista dei frame liberi.

15.14.1 Passi nel richiedere Paging - Caso Peggior

1. Trap all'operating system con conseguente context switch
2. Salva i registri utente e lo stato del processo
3. Determina che l'interruzione è stata causata da un page fault
4. Controlla che il riferimento alla pagina sia legale, altrimenti abort, e determina la posizione della pagina sul disco
5. Emetti una lettura dal disco a un frame libero:

Attendi in una coda per questo dispositivo fino a quando la richiesta di lettura non viene soddisfatta
Attendi il tempo di seek e/o latenza del dispositivo

Inizia il trasferimento della pagina a un frame libero

6. Mentre aspetti, assegna la CPU a qualche altro utente
7. Ricevi un'interruzione dal sottosistema I/O del disco (I/O completato)
8. Salva i registri e lo stato del processo per l'altro utente
9. Determina che l'interruzione è stata causata dal disco
10. Correggi la tabella delle pagine e altre tabelle per mostrare che la pagina è ora in memoria
11. Attendi che la CPU venga assegnata a questo processo di nuovo
12. Ripristina i registri utente, lo stato del processo e la nuova tabella delle pagine, quindi riprendi l'istruzione interrotta

15.14.2 Ottimizzazioni della Paginazione su Richiesta

Lo swap space è più veloce dell'I/O del file system anche se sullo stesso dispositivo; lo swap viene allocato in blocchi più grandi, richiedendo meno gestione rispetto al file system; copia l'intera immagine del processo nello spazio di swap (backing store) al momento del caricamento del processo, quindi pagina dentro e fuori dallo spazio di swap; usato in BSD Unix più vecchio; demand page dal programma binario su disco, ma scarta piuttosto che paginare quando si libera un frame; usato in Solaris e BSD attuale; è ancora necessario scrivere nello spazio di swap per le pagine non associate a un file (come stack e heap) - memoria anonima - o per le pagine modificate in memoria ma non ancora scritte indietro al file system; i sistemi mobili tipicamente non supportano lo swapping, invece demand page dal file system e reclamano le pagine di sola lettura (come il codice).

15.14.3 Copia su Scrittura (Copy-on-Write)

Copy-on-Write (COW) consente sia al processo padre che a quello figlio di condividere inizialmente le stesse pagine in memoria. Se uno dei processi modifica una pagina condivisa, solo allora la pagina viene copiata. COW consente una creazione di processi più efficiente poiché vengono copiate solo le pagine modificate. In generale, le pagine libere vengono allocate da un pool di pagine zero-fill-on-demand. Il pool dovrebbe sempre avere frame liberi per un'esecuzione rapida del demand page; non si vuole dover liberare un frame oltre ad altre elaborazioni su un page fault. Perché azzerare una pagina prima di allocarla? La variazione `vfork()` della system call `fork()` sospende il processo padre e il figlio utilizza lo spazio degli indirizzi copy-on-write del padre. Progettato per far sì che il figlio chiami `exec()`, molto efficiente.



15.14.4 Cosa succede se non ci sono frame liberi?

Come per gli algoritmi di scheduling, possiamo implementare la soluzione più semplice usando la casualità: prendere un frame in memoria, metterlo nell'HD e caricare la pagina necessaria nella RAM; questa soluzione non è ottimale!

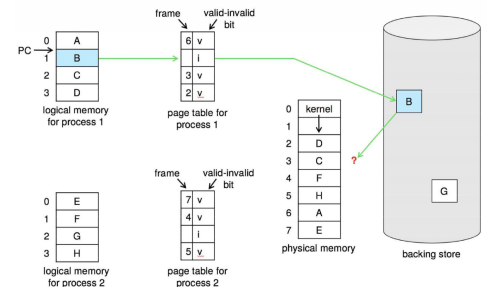
Questa tecnica, indipendentemente dall'algoritmo, si chiama **sostituzione di pagina** - trovare una pagina in memoria non realmente in uso, rimuoverla e caricare la pagina necessaria.

Si previene la sovra-allocazione della memoria modificando la routine di gestione dei page fault in modo da includere la sostituzione delle pagine.

Possiamo farlo usando il **bit di modifica (dirty bit)** per ridurre l'overhead dei trasferimenti di pagina - solo le pagine modificate vengono scritte su disco.

Se il dirty bit è impostato significa che il valore nella RAM è cambiato e prima di caricare la nuova pagina devo scrivere il valore nell'HD.

La sostituzione delle pagine completa la separazione tra memoria logica e fisica - una grande memoria virtuale può essere fornita con una memoria fisica più piccola.



Questo processo significa accedere all'HD **due volte**.

15.14.5 Sostituzione di pagina di base

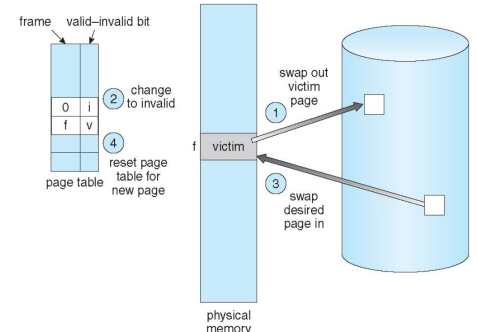
1. Trovare la posizione della pagina desiderata su disco
2. Trovare un frame libero:

Se c'è un frame libero, usarlo

Se non c'è un frame libero, usare un algoritmo di sostituzione delle pagine per selezionare un frame vittima

Scrivere il frame vittima su disco se è sporco (= modificato mentre era in RAM)

3. Caricare la pagina desiderata nel frame (appena) libero; aggiornare le tabelle delle pagine e dei frame
4. Continuare il processo riavviando l'istruzione che ha causato il trap



15.15 Algoritmi di sostituzione di pagine e frame

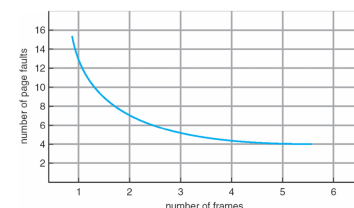
L'algoritmo di allocazione dei frame determina:

1. Quanti frame assegnare a ciascun processo
2. Quali frame sostituire; idealmente sostituisco solo la pagina che non mi servirà presto (ma non conosco il futuro)

Algoritmo di sostituzione delle pagine:

Si vuole che il tasso di page fault più basso sia al primo accesso che ai successivi

In questa immagine si può vedere che più frame slot sono associati a ogni processo, minore è la probabilità di avere page fault.

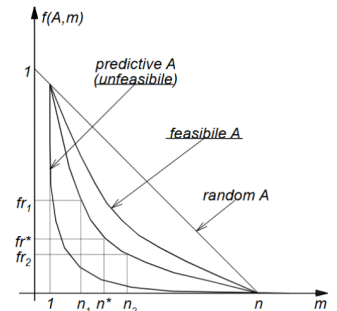


15.15.1 Frequenza di Page Fault - probability empirica

dove

$$f(A, m) = \sum_{\forall w} p(w) \frac{F(A, w, m)}{\text{len}(w)}$$

- **A** un algoritmo di sostituzione delle pagine in valutazione
- **w** una data stringa di riferimento
- **p(w)** probabilità di riferimento alla stringa **w**
- **len(w)** lunghezza della stringa **w**
- **m** numero di page frame liberi
- **F(A, w, m)** numero di page fault causati data una certa stringa di riferimento **w** usando l'algoritmo **A** sul sistema di un dato sistema con **m** page frame



Ci aspettiamo che un dato algoritmo casuale abbia una curva lineare decrescente. Mentre il caso migliore, ovvero l'algoritmo che prevede il futuro, abbia una curva decrescente più ripida.

Quindi dato un algoritmo e il numero n di frame e troviamo la frequenza di page fault.

15.15.2 Selezione delle vittime usando il modify bit

Quando si sceglie la vittima, la pagina da buttare fuori per far posto alla nuova, si deve guardare il *reference bit* (se una pagina è stata usata nel passato recente) e il *modify bit* (se è stata modificata) i quali indicano se la pagina è attiva o meno e possiamo utilizzare questa informazione per scegliere la vittima negli algoritmi di sostituzione delle pagine.

Nota: Il caso migliore è 00, il caso peggiore è 11.

Ref. bit	Mod. bit
0	0
0	1
1	0
1	1

15.15.3 Algoritmo FIFO

Si assume che una pagina che è entrata in memoria prima delle altre sia stata già utilizzata e l'algoritmo FIFO selezionerà quella pagina.

Stringa di riferimento: 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1;

$m = 3$ frame, 3 pagine possono essere in memoria contemporaneamente per processo. Dentro i frame troviamo il **residence set** di 3 pagine.

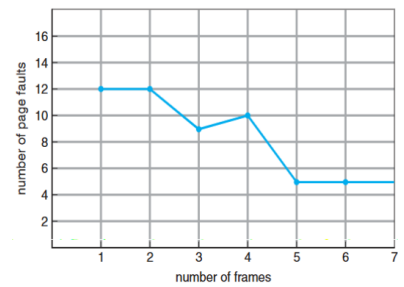
reference string																					
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1		
7	7	7	2		2	2	4	4	4	0		0	0		7	7	7				
		0	0	0	3	3	3	2	2	2		1	1		1	0	0				
			1	1	1	0	0	0	3	3		3	2		2	2	1				
page frames																					

Dopo i primi 3 page fault, dato il pure demand paging, dopo vi inizierà l'algoritmo di sostituzione delle pagine.

Possiamo realizzare l'algoritmo: con una lista concatenata con inserimento in coda e rimozione in testa (illimitato); altrimenti con una coda circolare (limitato). In questo ultimo caso siamo davanti ad un FIFO circolare semplificato: inizialmente 2 puntatori per head e tail, una volta riempito tutto mi basta ricordare solo il puntatore a head e portarlo avanti.

Con questo metodo si hanno 15 page fault; si può fare meglio? FIFO veloce, $O(1)$, ma non è un buon algoritmo!

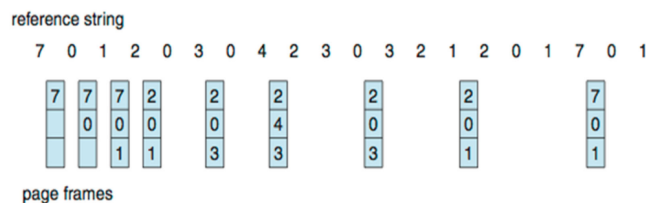
Nota: Non è sempre vero che, sullo specifico caso, aumentando il numero di frame si riduca il numero di page fault; questo fenomeno è noto come **anomalie di Belady**.



15.15.4 Algoritmo ottimale

L'algoritmo ottimale sceglie la pagina che non verrà utilizzata per il periodo più lungo nel futuro e questo diminuisce, per questo specifico esempio, il numero di page fault a 9.

Ma come detto non possiamo prevedere il futuro; TUTTAVIA questa è la soluzione ottimale e dunque possiamo utilizzarla per misurare le prestazioni degli altri algoritmi.



Nota: Anche in questo caso non eliminiamo completamente i page fault dato che i frame sono limitati.

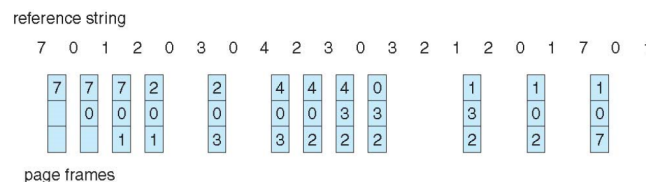
15.15.5 Algoritmo LRU - Least Recently Used

Abbiamo visto che il programma non può conoscere il futuro, ma possiamo usare la conoscenza del passato!

Sostituisce la pagina che non è stata utilizzata per il periodo di tempo più lungo; sfrutta il **principio di località temporale**, associando ad ogni pagina il tempo dell'ultimo utilizzo.

L'assunzione di questo algoritmo è che una pagina che è stata utilizzata poco nel passato sarà utilizzata poco anche nel futuro.

Associamo ad ogni pagina il numero di accessi alla pagina, ma di difficile da implementare senza consumare memoria aggiuntiva e rallentare il sistema. Inoltre dovrà essere implementata una ricerca lineare e suppone che una pagina si mantenga traccia dell'accesso.



12 page fault - meglio di FIFO ma peggio di OPT; generalmente un buon algoritmo e ampiamente usato. Ma come si implementa?

$$f = \frac{10}{12} = 0.83 \rightarrow 83\%$$

Time	1	2	3	4	5	6	7	8	9	10	11	12
String	4	3	2	1	4	3	5	4	3	2	1	5
Fault	*	*	*	*	*	*	*	*	*	*	*	*
Frame 0	4	4	4	1	1	1	5	5	5	2	2	2
Frame 1	-	3	3	3	4	4	4	4	4	4	1	1
Frame 2	-	-	2	2	2	3	3	3	3	3	3	5

victim

Implementazione con contatore: Ogni voce di pagina ha un contatore; ogni volta che la pagina viene referenziata, sia in lettura o scrittura, si copia il clock nel contatore, il tempo di accesso. Quando una pagina deve essere sostituita, si cerca il contatore con il valore minore; è necessaria una ricerca nella tabella.

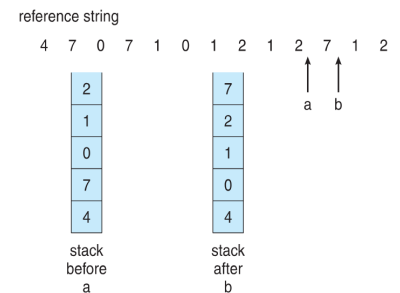
Il clock può essere nella TLB (costoso deve mantenere un int) che si aggiorna automaticamente oppure nella tabella delle pagine. In ogni caso è di difficile implementazione.

Implementazione con stack (lista concatenata): Si mantiene uno stack di numeri di pagina in forma di lista doppiamente concatenata. Quando una pagina viene referenziata, viene spostata in cima. È necessario aggiornare 6 puntatori per ogni accesso alla pagina ^a.

Estrazione $O(1)$ ma inserimento costoso.

LRU e OPT sono casi di algoritmi a stack che non hanno l'anomalia di Belady. Si usa uno stack per registrare i riferimenti alle pagine più recenti.

^aSi deve rimuovere la pagina dalla lista e inserirla in cima e i riferimenti devono essere aggiornati



15.15.6 Algoritmi LRU approssimati

LRU richiede hardware speciale ed è comunque lento e costoso in termini di memoria (4 byte per ogni voce).

Possiamo semplificare LRU utilizzando un solo bit: il **bit di riferimento**. Inizialmente ad ogni pagina viene associato il bit con valore 0; quando la pagina viene referenziata il bit viene impostato a 1. Se il SO necessita di memoria per un processo, può sostituire qualsiasi pagina con il bit a 0, se ne esiste una.

Prima o poi tutte le pagine avranno il bit di riferimento a 1, perciò dobbiamo trovare un modo per azzerarlo.

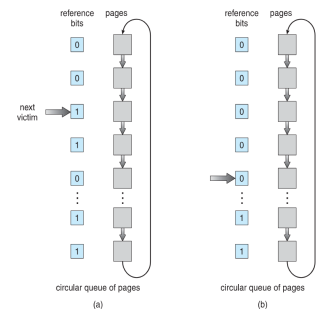
Algoritmo second-chance:

Generalmente FIFO, più il bit di riferimento fornito dall'hardware con sostituzione **Clock** - buffer circolare. Nel caso peggiore, l'algoritmo scorre l'intera lista. Se la pagina da sostituire ha:

- Bit di riferimento = 0 → sostituirla
- Bit di riferimento = 1 allora:

impostare il bit di riferimento a 0, lasciare la pagina in memoria, così che al secondo passaggio la trovo a 0

sostituire la pagina successiva, con le stesse regole



Messo ad 1 dalla TLB in hw, messo a zero dal software dato che si ha molto tempo a disposizione causa il page fault.

Miglioramento dell'Algoritmo della Seconda Chance

Possiamo migliorare l'algoritmo usando il bit di riferimento e il bit di modifica (se disponibile) in concerto. Prendiamo la coppia ordinata (riferimento, modifica):

- (0, 0) né recentemente usata né modificata — la migliore pagina da sostituire, minimizza il tempo di gestione del page fault
- (0, 1) non recentemente usata ma modificata — non così buona, deve essere scritta prima della sostituzione
- (1, 0) recentemente usata ma pulita — probabilmente sarà usata di nuovo presto

- (1, 1) recentemente usata e modificata — probabilmente sarà usata di nuovo presto e deve essere scritta prima della sostituzione

Quando viene chiamata la sostituzione di pagina, si utilizza lo schema clock ma si usano le quattro classi per sostituire la pagina nella classe non vuota più bassa. Potrebbe essere necessario cercare più volte nella coda circolare.

Prima di cercare 0,1 devo rifare tutto il giro, e via così fino a 1,1 quindi al massimo faccio almeno 3 giri. Ovviamente si può fare in un giro solo ma devo occupare memoria.

Questo si basa sempre dell'LRU, che misura un'inattività della pagina.

15.15.7 Algoritmi basati sul conteggio

Si mantiene un contatore del numero di riferimenti effettuati a ogni pagina; non comune.

Nota: Anche qui prima o poi si deve azzerare tutti i contatori.

Algoritmo LFU (Least Frequently Used):

- Sostituisce la pagina con il contatore più basso
- Problema: può capitare che venga sempre sostituita la pagina appena inserita dato che hanno il contatore impostato a 1

Algoritmo MFU (Most Frequently Used):

Si basa sull'argomento che la pagina con il contatore più basso è stata probabilmente appena caricata e non è ancora stata utilizzata

15.15.8 Page-Buffering Algorithms

Prendiamo un pool di frame liberi, sempre disponibile quando necessario, non trovato al momento del page fault. Leggiamo la pagina in un frame libero e selezioniamo una vittima da espellere e aggiungiamo al pool dei frame liberi. Quando è conveniente, espelliamo la vittima, possibilmente tenendo una lista di pagine modificate. Quando lo storage secondario è altrimenti inattivo, scriviamo le pagine lì e impostiamo a non sporco. Possibilmente, manteniamo intatti i contenuti del frame libero e annotiamo cosa c'è dentro; se viene referenziato di nuovo prima di essere riutilizzato, non c'è bisogno di caricare nuovamente i contenuti da disco. Generalmente utile per ridurre la penalità se viene selezionato il frame vittima sbagliato.

Soluzioni aggiuntive rispetto ai precedenti, vogliamo ridurre il **tempo** di gestione del page fault.

Quando cerco un frame libero lo ottendo, dato che mi sono tenuto qualche frame libero. Quando si verifica un page fault cerco una vittima, la quale verrà espulsa solo quando è conveniente.

Le vittime sono lasciate ancora con il contenuto e nel caso se fai riferimento ad una vittima prima che venga espulsa, non c'è bisogno di caricarla da disco.

Mantengo le vittime fino a che non ho proprio bisogno di liberare frame, nel mentre le mantengo in memoria anche se sono segnate come vittima.

Nota: Immagina il cestino di un sistema operativo.

15.15.9 Applicazioni e Sostituzione delle Pagine

Tutti questi algoritmi di sostituzione delle pagine sono basati su ipotesi e non possono prevedere con precisione il futuro accesso alle pagine. Tuttavia, alcune applicazioni, come i database, hanno una migliore conoscenza dei loro modelli di accesso alla memoria e possono utilizzare questa conoscenza per ottimizzare le prestazioni. Le applicazioni che richiedono molta memoria possono causare un doppio buffering, in cui il sistema operativo mantiene una copia della pagina in memoria come buffer I/O, mentre l'applicazione mantiene la pagina in memoria per il proprio lavoro. In questi casi, il sistema operativo può fornire un accesso diretto al disco, bypassando il buffering e altre funzionalità del sistema operativo, consentendo all'applicazione di gestire direttamente l'I/O del disco.

15.16 Allocazione dei frame ai processi

Quanto deve valere m e come facciamo a deciderlo?

Ogni processo necessita di un numero minimo di frame; il massimo è ovviamente il numero totale di frame nel sistema.

Due principali schemi di allocazione: **allocazione fissa** e **allocazione a priorità**, oltre a molte varianti.

15.16.1 Allocazione fissa

L'allocazione data ad un dato processo è fissa

Allocazione eguale se ci sono 100 frame (dopo aver allocato i frame per il SO) e 5 processi, si assegnano 20 frame a ciascun processo. Si mantiene anche una parte come pool di frame liberi.

Allocazione proporzionale Si alloca in base alla dimensione del processo; dinamica in funzione del grado di multiprogrammazione, le dimensioni dei processi cambiano.

- s_i = dimensione del processo p_i in pagine
- $S = \sum s_i$
- m = numero totale di frame disponibili
- a_i = numero di frame allocati a $p_i = \frac{s_i}{S} \cdot m$

$$m = 64 \quad s_1 = 10 \quad s_2 = 127 \quad a_1 = \frac{10}{137} \cdot 62 \approx 4 \quad a_2 = \frac{127}{137} \cdot 62 \approx 57$$

In entrambi i tipi di allocazione non è possibile tener conto dei task ad alta/bassa priorità; idealmente si vorrebbe che quelli ad alta priorità terminassero prima, assegnando loro più memoria/CPU.

15.16.2 Allocazione globale vs. locale

Sostituzione globale - il processo seleziona un frame di sostituzione dall'insieme di tutti i frame.

- un processo può sottrarre un frame a un altro
- il tempo di esecuzione del processo può variare notevolmente dato che dipende da ciò che stanno facendo gli altri processi
- throughput maggiore, quindi più comune

Sostituzione locale - ogni processo seleziona solo dal proprio insieme di frame allocati

- Prestazioni per processo più consistenti
- possibile sottoutilizzo della memoria
- Quindi, throughput minore

15.16.3 Recupero delle pagine

Una strategia per implementare la politica di sostituzione globale è: tutte le richieste di memoria vengono soddisfatte dalla lista dei frame liberi, invece di aspettare che la lista scenda a zero prima di iniziare a selezionare le pagine da sostituire. Il recupero delle pagine viene attivato dal kernel quando la lista scende sotto una certa soglia.

Il processo kernel che lo gestisce è chiamato **reaper**.

Questa strategia cerca di garantire che ci sia sempre sufficiente memoria libera per soddisfare le nuove richieste, ovvero che la lista dei frame liberi non si svuoti mai.

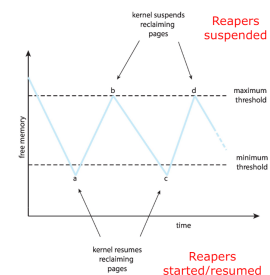
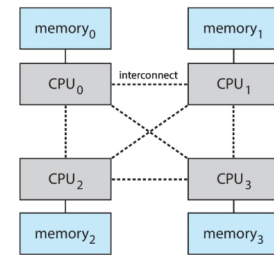


Figura 15.7: Esempio di recupero delle pagine

15.16.4 Non-Uniform Memory Access

Fino ad ora tutta la memoria è stata considerata uguale, ma molti sistemi sono NUMA (Non-Uniform Memory Access) - la velocità di accesso alla memoria varia. Considera le schede di sistema contenenti CPU e memoria, interconnesse tramite un bus di sistema - architettura di multiprocessing NUMA.

La memoria condivisa fa sì che tutti i core rallentano e relativi problemi di consistenza. Sarebbe auspicabile che un processo che è stato allocato alla CPU1 usa la memoria locale alla CPU1. Tutto questo diventa ancora più complicato quando un processo viene migrato da una CPU a un'altra.



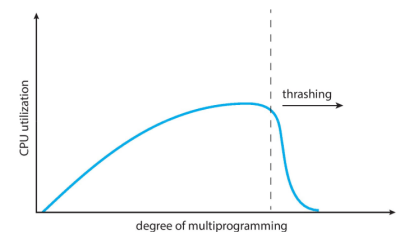
L'ottima prestazione deriva dall'allocare la memoria "vicino" alla CPU su cui è programmato il thread e modificare lo scheduler per programmare il thread sulla stessa scheda di sistema quando possibile. Soluzione adottata da Solaris creando lgroups, una struttura per tracciare i gruppi a bassa latenza CPU/memoria, usata dallo scheduler e dal pager. Quando possibile, schedula tutti i thread di un processo e allocare tutta la memoria per quel processo all'interno dell'lgroup.

In un multicore lo scheduler cerca di preferire la stessa CPU per un dato processo e idem per l'allocazione della memoria.

15.16.5 Thrashing

Thrashing - un processo è occupato a scambiare continuamente pagine in entrata e in uscita.

Se un processo non ha un numero "sufficiente" di pagine, il tasso di page fault è molto alto: **page fault** per ottenere la pagina, **sostituzione** del frame esistente, ma rapidamente si ha bisogno di nuovo del **frame sostituito**.



Tutti questi cicli portano a:

- Basso utilizzo della CPU
- Il sistema operativo pensa di dover aumentare il grado di multiprogrammazione
- Un altro processo viene aggiunto al sistema

15.16.6 Paging su richiesta e thrashing

Vorrei che tutte le località a cui accedo siano in memoria.

Perché il demand paging funziona? Modello di località degli accessi:

Il processo migra da una località all'altra

Le località possono sovrapporsi

Perché si verifica il thrashing? La somma delle dimensioni delle località è maggiore della dimensione totale della memoria. Limitare gli effetti usando la sostituzione di pagina locale o a priorità.

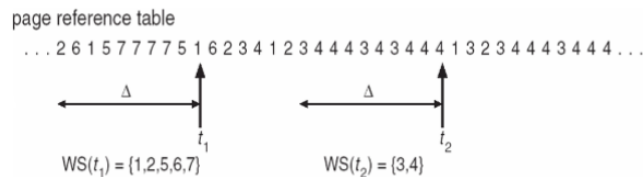
15.16.7 Working-Set Model

Chiamiamo Working-Set le pagine su cui sta lavorando un processo. Come lo definiamo? Le pagine su cui un processo sta lavorando cambia nel tempo, bisogna definire una finestra di lavoro, Δ , l'intervallo a cui faccio riferimento e.g. 10 000.

Δ è la finestra di lavoro, un numero fisso di riferimenti. WSS_i (Working Set Size) è il working set del processo P_i , ovvero il numero totale di pagine referenziate nelle più recenti Δ istruzioni. D è la somma di tutti i WSS_i , ovvero il numero totale di frame richiesti da tutti i processi.

- se Δ è troppo piccolo, non coprirà l'intera località. Fotografo un passato troppo recente e nel futuro probabilmente avrò bisogno di altre pagine che non sono nel working set. (Eg $\Delta = 1$ la prossima volta faccio sicuramente page fault)
- se Δ è troppo grande, coprirà diverse località.
- se $\Delta = \infty$, coprirà l'intero programma. Tutto quello che ho usato nel passato lo considero parte del working e gli unici page fault sono solo le pagine a cui NON ho mai fatto accesso. Impossibile perché la memoria è finita.

Se $D > m \implies$ thrashing. Policy se $D > m$, allora sospendi o swap out uno dei processi.



Più il programma è locale più richiede pochi frame, migliorando la gestione della memoria. Permette dinamicità globale tra i vari processi.

Caso ideale avere tutto il Working Set in memoria, e mi va meglio se è piccolo (WSS i piccolo).

15.16.8 Tenere traccia del Working Set

Essendo una sliding window, rischio di perdersi l'ultimo riferimento. Prima attivo il page fault solo quando faccio riferimento ad una pagina ora invece devo buttare via pagine (page out) quando faccio riferimento solo alle stesse pagine perché il programma è molto locale.

A rigore dovrei ricordarmi il periodo dell'ultimo accesso, simile a LRU.

Approssimativamente con un timer a intervalli + un bit di riferimento. Esempio: $\Delta = 10.000$; l'interruzione del timer avviene ogni 5000 unità di tempo; mantenere in memoria 2 bit per ogni pagina; ogni volta che il timer interrompe, copiare e impostare i valori di tutti i bit di riferimento a 0; se uno dei bit in memoria = 1 $\implies$ pagina nel working set. Perché non è completamente accurato? Miglioramento = 10 bit e interruzione ogni 1000 unità di tempo.

Approssimiamo l'intervallo e il working set reale sarà più grande, riallineandolo solo ogni tanto.

I due bit sono **Reference bit** e **Reference bit precedente**.

Se ho una pagina ha il bit di riferimento a 1 allora ci ho fatto riferimento. Uno fotografa i 5000 istanti prima e l'altro fotografa i 5000 istanti prima di quello.

Se allo scatto del timer, allino esattamente al working set altrimenti tra 10k e 15k dato che interrompo ogni 5000 unità di tempo.

Ogni 5 istanti prima di riavere guardo tutte le pagine: guardo i 2 bit: butto via le pagine con 00 le altre le tengo. mentre faccio il giro, tutti i reference bit (gli ultimi) vengono azzerati e il loro valore viene copiato nel bit che conteneva il reference bit precedente.

Working set : cerchiamo di tenere traccia delle pagine in cui sto lavorando e solo ed almeno queste le tengo in memoria. Utilizziamo un'approssimazione, rispetto al cambiamento di tutti i riferimenti, di 5000 di istanti di tempo, sovapprossimando del 50% (15000) istanti rispetto al teorico 10k.

Idea - usare il tempo per dimensionare le pagine di lavoro, si può utilizzare la frequenza rispetto al tempo

15.16.9 Frequenza dei page fault

Invece di rendere dinamica la sliding window, guardiamo l'effetto che otteniamo allocando più o meno frame.

Si stabilisce un tasso di page fault (PFF) "accettabile" e si usa la politica di sostituzione locale.

- Se il tasso effettivo è troppo basso, il processo perde un frame
- Se il tasso effettivo è troppo alto, il processo guadagna un frame

Troviamo 2 soglie per evitare oscillazioni. Aumentando il numero di frame, m , si abbassa il tasso di page fault.

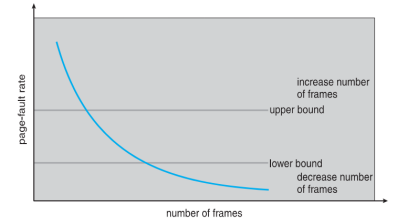


Figura 15.8: PFF

15.17 Page Fault Frequency Algorithm

Al differenza del Working Set, o Working Set Size, al posto di avere un timer qui l'algoritmo si attiva soltanto quando ci sono page fault. Dobbiamo ricordare quanto tempo intercorre tra due page fault. Frequenza bassa se il tempo che intercorre è grande, e viceversa.

È attivato ad ogni evento di page fault, non al momento del riferimento alla pagina.

È basato sull'intervallo τ , tempo che intercorre tra due page fault.

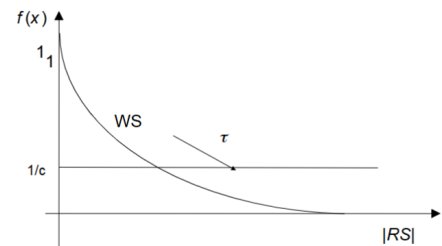
- $\tau < C$ se il page fault è più grande di quella desiderata, aggiungi un frame al Resident Set del processo
- $\tau \geq C$ se la frequenza dei page fault è OK perché τ è grande, rimuovi un frame dal Resident Set del processo con bit a 0. Poi pulisci, setta a 0, il bit di riferimento di tutte le altre pagine nel Resident Set.

Supponiamo che τ vada bene, ora abbiamo 2 modi per prendere decisioni: 2 azioni prendo quelli a zero e metto tutto a 1 il reference bit.

In rosso le vittime. In Verde la pagina referenziata con residence set a 1.

Tempo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Stringa	6	4	4	3	2	4	4	4	1	3	3	2	4	4	5	1	2	2	6	1	2	5	4	
Frame 0	6	6	6	6	6	6	6	6	6	6	6	6	6	6	5	5	5	5	5	6	6	6	6	6
Frame 1	-	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	2	2	2	2	2
Frame 2	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	1	1	1	1	1
Frame 3	-	-	-	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	-	-	-	5	5
Frame 4	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	4	
Fault	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
Ref Bit									0						0					0				

Figura 15.9: Esempio di frequenza di page fault con $C = 3$



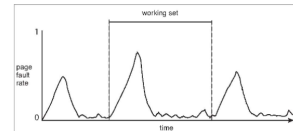
Spiegazione L'inizio di questo esempio è da considerarsi come un transitorio in questo esempio.

- **Tempo 1-5** - si verificano page fault. Al tempo 4 faccio riferimento a 3, genero un page fault, e devo guardare quanto tempo è passato dall'ultimo page fault, ovvero 2 (tempo 4 - tempo 2) che è minore di C (3) quindi aggiungo una pagina al residence set.
- **Tempo 6-8** - non ci sono page fault essendo tutte le pagine referenziate in memoria.
- **Tempo 9** - si verifica un page fault, si attiva l'algoritmo nel caso che va bene, guardo quanto tempo è passato dall'ultimo page fault, ovvero 4 (tempo 9 - tempo 5) che è maggiore di C (3), non rimuovo nessuna pagina perché hanno tutti il reference bit a 1 e a questo punto azzerò tutti i reference bit. A questo punto però dobbiamo fare assunzioni se anche il valore appena aggiunto gli venga azzerato o meno il reference bit, in questo caso viene azzerato anche questo.
- **Tempo 10-13** - sono colorate in verde le pagine referenziate e il relativo reference bit è impostato ad 1.
- **Tempo 14** - a questo punto le pagine non referenziate se avviene page fault vengono eliminate
- **Tempo 15** - si fa riferimento a 5, non presente nel residence set, a questo punto avviene un page fault e si eliminano le pagine con residence bit a 0 e si aggiunge il 5 al residence set. A questo punto il residence set è diminuito (da 5 frame $\rightarrow$ 4 frame), e si azzerano tutti i reference bit, riportando la situazione all'inizio.
- **Tempo 16** - si fa riferimento nuovamente ad 1, si aggiunge al residence set e si setta a 1 il reference bit, azzerando gli altri

- **Working set:** pagine di un dato processo
- **WSS:** algoritmo per allocare dinamicamente il working set usando il tempo
- **Page Fault Frequency:** Usa la frequenza dei page fault per allocare dinamicamente il working set.

Working Sets and Page Fault Rates

Vi è una relazione diretta tra il working set di un processo e il suo tasso di page fault. Il working set cambia nel tempo.



15.18 Allocazione in memoria del kernel

È trattata in maniera diversa rispetto alla memoria utente ed è spesso è allocata dal free-memory pool.

Il kernel richiede memoria per strutture di dimensioni variabili. Parte della memoria del kernel deve essere contigua, vedi I/O device, dato che lavorano con indirizzi fisici.

15.19 Buddy System

Alloca memoria da segmenti di dimensioni fisse costituiti da pagine fisicamente contigue, solitamente allocate usando dimensioni di potenza di 2, si riduce il problema della frammentazione esterna, ma quella interna rimane la quale sarà al massimo il 50% (uso 65Kb al posto di 128Kb):

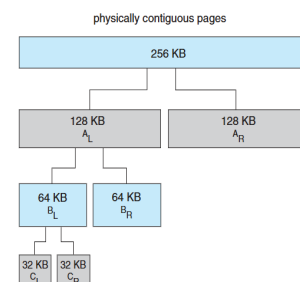
- Soddisfa le richieste in unità di dimensioni potenza di 2
- La richiesta viene arrotondata alla potenza di 2 più vicina (10Kb $\rightarrow$ 16Kb)
- Quando è necessaria un'allocazione più piccola di quella disponibile, il chunk corrente viene diviso in due buddy della potenza di 2 successiva più bassa. Genero due parti eg. 128Kb $\rightarrow$ 2x64Kb $\rightarrow$ 4x32Kb $\rightarrow$ 8x16Kb

Questo processo continua fino a quando una dimensione adatta è trovata.

Vantaggio: unisce rapidamente i chunk inutilizzati in chunk più grandi.

Svantaggio: frammentazione esterna, ma effetto ridotto dato che si possono tentare di unire blocchi vicini, sono potenze di 2

Esempio: assumiamo che sia disponibile un chunk di 256KB, il kernel richiede 21KB. Viene diviso in A_L e A_R di 128KB ciascuno. Uno viene ulteriormente diviso in B_L e B_R di 64KB. Uno viene ulteriormente diviso in C_L e C_R di 32KB ciascuno - uno viene usato per soddisfare la richiesta e 11KB di frammentazione interna.

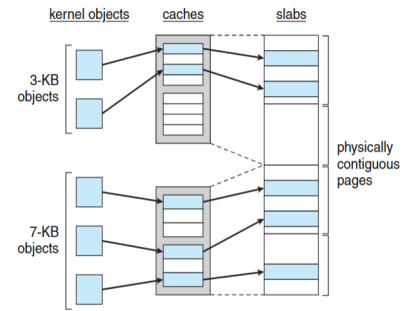


15.20 Slab Allocator

Una strategia più flessibile utilizzata in ambiente linux in alternativa utilizza gli **slab** - composti da una o più pagine fisicamente contigue - e la **Cache** - costituita da uno o più slab.

Una singola cache per ogni struttura dati univoca del kernel. Ogni cache è riempita con oggetti - istanziazioni della struttura dati. Quando la cache viene creata, viene riempita con oggetti marcati come **free**, altrimenti quando le strutture vengono memorizzate, gli oggetti sono marcati come **used**.

Se uno slab è pieno di oggetti usati, il prossimo oggetto viene allocato da uno slab vuoto. Se non ci sono slab vuoti, viene allocato un nuovo slab. I vantaggi includono l'assenza di frammentazione e la soddisfazione rapida delle richieste di memoria



Esempio - Allocatore Slab in Linux Per esempio, il descrittore di processo è di tipo `struct task_struct` con approssimativamente 1.7KB di memoria.

Nuovo task → alloca una nuova struttura dalla cache. Utilizzerà una `struct task_struct` libera esistente.

Lo slab può essere in tre possibili stati:

1. Full - tutto usato
2. Empty - tutto libero
3. Partial - alcuni liberi, alcuni usati

Su richiesta, l'allocatore slab

1. Usa uno struct libero in uno slab parziale
2. Se nessuno, ne prende uno da uno slab vuoto
3. Se non c'è uno slab vuoto, ne crea uno nuovo vuoto

Lo slab è nato in Solaris, ora è diffuso sia per il kernel mode che per la memoria utente in vari SO. Linux 2.2 aveva SLAB, ora ha sia gli allocatori SLOB che SLUB

15.21 Prepaging

Per caricare un processo, si deve leggere l'eseguibile. Siamo passati da caricarlo tutto fino al caricarlo durante i page fault (pure/partial demangin page).

Per ridurre il grande numero di page fault che si verificano all'avvio del processo, il **prepaging** può essere utilizzato per caricare più della semplice pagina richiesta. Si precaricano tutte o alcune delle pagine che un processo necessiterà, prima che vengano referenziate, ma se le pagine precaricate non vengono utilizzate, l'I/O e la memoria sono stati sprecati.

Esempio: Assumiamo che s pagine sono precaricate e α delle pagine viene utilizzato

- s costo di $s \cdot \alpha$ risparmia page fault $> 0 <$ del costo del prepaging. $s \cdot (1 - \alpha)$ pagine non necessarie?
- α vicino a zero → il prepaging perde

15.22 Dimensione della Pagina

Talvolta i progettisti di SO hanno una scelta, specialmente se operano su CPU custom-built. Per dimensionare correttamente le pagine dobbiamo tenere conto di diversi fattori, tra cui:

- Frammentazione interna → più grande è la pagina e più frammentazione interna avremo. Meglio piccole.
- Dimensione della tabella delle pagine → più grande è la pagina e meno righe nella page table, quindi meno memoria per la page table. Meglio grandi.

- Risoluzione
- Overhead dell'I/O → più grande è la pagina e più ammortizzo il costo di I/O. Meglio grandi.
- Numero di page fault → più grande è la pagina e meno page fault
- Località → più grande è la pagina e più probabilmente avrò località, ma se è troppo grande rischio di avere un tasso di page fault più alto dato che una data funzione è presente in una pagina non caricata.
- Dimensione ed efficacia del TLB -> meglio pagine più grandi così con lo stesso hw "vede" più memoria (TLB reach)

Dire che aumentare la dimensione della pagina riduce il numero di page fault non è sempre vero, dato che il page fault è decisa da:

$$\text{Page Fault} = f(m, A, w)$$

Dove: m è il numero di frame, A è l'algoritmo di sostituzione e w è la stringa di riferimento

Sempre potenze di 2, solitamente nell'intervallo 2^{12} (4096byte) a 2^{22} (4194304byte). In media, crescente nel tempo.

15.23 TLB Reach

TLB Reach - La quantità di memoria accessibile dal TLB.

$$\text{TLB Reach} = \text{TLB Size} \times \text{Page Size}$$

Idealmente, il working set di ogni processo dovrebbe essere memorizzato nel TLB, altrimenti c'è un alto grado di page fault.

- **Aumentare la Dimensione della Pagina:** Questo può portare a un aumento della frammentazione interna poiché non tutte le applicazioni richiedono una grande dimensione di pagina
- **Fornire Dimensioni di Pagina Multiple:** Questo consente alle applicazioni che richiedono dimensioni di pagina più grandi l'opportunità di utilizzarle senza un aumento della frammentazione

Esempio: Struttura del programma `int[128,128] data;` - ogni riga è memorizzata in una pagina (512 byte).

```
// Program 1
for (j = 0; j < 128; j++)
  for (i = 0; i < 128; i++)
    data[i,j] = 0;
```

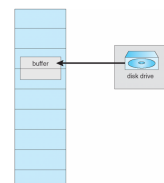
128 × 128 page faults. Ogni accesso ha fatto accesso ad una pagina diversa. Qualsiasi sia il working set avrà sempre page fault.

```
// Program 2
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    data[i,j] = 0;
```

128 × 128 page faults. 1 page fault per ogni pagina.

15.24 I/O interlock

I/O Interlock - Le pagine devono talvolta essere bloccate in memoria. Consideriamo l'I/O - Le pagine che vengono utilizzate per copiare un file da un dispositivo devono essere bloccate ed evitare di essere selezionate per l'espulsione da un algoritmo di sostituzione delle pagine
Pinning delle pagine per bloccarle in memoria.

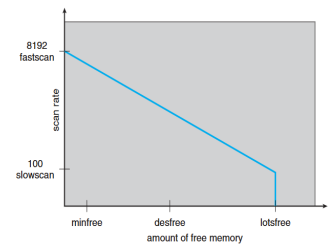


15.24.1 Esempio Windows

Utilizza paginazione su richiesta con clustering. Il clustering carica le pagine circostanti alla pagina che genera il fault. Ai processi vengono assegnati un working set minimo e un working set massimo. Il working set minimo è il numero minimo di pagine che il processo è garantito di avere in memoria. Un processo può essere assegnato fino a un numero di pagine pari al suo working set massimo. Quando la quantità di memoria libera nel sistema scende sotto una soglia, viene eseguita automaticamente la riduzione del working set per ripristinare la quantità di memoria libera. La riduzione del working set rimuove pagine dai processi che hanno pagine in eccesso rispetto al loro working set minimo

15.24.2 Esempio Solaris

Mantiene una lista di pagine libere da assegnare ai processi che generano fault. Lotsfree - parametro di soglia (quantità di memoria libera) per iniziare il paging. Desfree - parametro di soglia per aumentare il paging. Minfree - parametro di soglia per iniziare lo swapping. Il paging viene eseguito dal processo pageout. Pageout scansiona le pagine usando un algoritmo clock modificato. Scanrate è la velocità con cui le pagine vengono scansionate. Questo varia da slowscan a fastscan. Pageout viene chiamato più frequentemente a seconda della quantità di memoria libera disponibile. Il paging con priorità dà priorità alle pagine di codice del processo

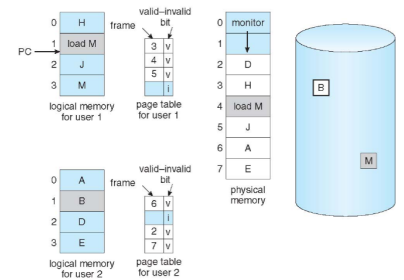


15.25 Performance del Demand Paging

Fasi della Paginazione su Richiesta (caso peggiore)

1. Trap al sistema operativo
2. Salva i registri utente e lo stato del processo
3. Determina che l'interruzione è stata causata da un page fault
4. Verifica che il riferimento alla pagina sia legale e determina la posizione della pagina sul disco
5. Emetti una lettura dal disco a un frame libero:
 - Attendi in una coda per questo dispositivo fino a quando la richiesta di lettura non viene soddisfatta
 - Attendi il tempo di seek e/o latenza del dispositivo
 - Inizia il trasferimento della pagina a un frame libero
6. Mentre aspetti, assegna la CPU a qualche altro utente
7. Ricevi un'interruzione dal sottosistema I/O del disco (I/O completato)
8. Salva i registri e lo stato del processo per l'altro utente
9. Determina che l'interruzione è stata causata dal disco
10. Correggi la tabella delle pagine e altre tabelle per mostrare che la pagina è ora in memoria
11. Attendi che la CPU venga assegnata di nuovo a questo processo
12. Ripristina i registri utente, lo stato del processo e la nuova tabella delle pagine, quindi riprendi l'istruzione interrotta

1. Trap al sistema operativo
2. Salva i registri utente e lo stato del processo
3. Determina che l'interruzione è stata causata da un page fault
4. Verifica che il riferimento alla pagina sia legale e determina la posizione della pagina sul disco
5. Emetti una lettura dal disco a un frame libero:
 - Attendi in una coda per questo dispositivo fino a quando la richiesta di lettura non viene soddisfatta
 - Attendi il tempo di seek e/o latenza del dispositivo
 - Inizia il trasferimento della pagina a un frame libero
6. Mentre aspetti, assegna la CPU a qualche altro utente
7. Ricevi un'interruzione dal sottosistema I/O del disco (I/O completato)
8. Salva i registri e lo stato del processo per l'altro utente
9. Determina che l'interruzione è stata causata dal disco
10. Correggi la tabella delle pagine e altre tabelle per mostrare che la pagina è ora in memoria
11. Attendi che la CPU venga assegnata di nuovo a questo processo
12. Ripristina i registri utente, lo stato del processo e la nuova tabella delle pagine, quindi riprendi l'istruzione interrotta



Cosa è cambiato esattamente: Sostituito
begincenter con
beginflushleft: Questo sposta l'intero blocco principale (quello largo 1
linewidth) verso il margine sinistro della pagina, invece di tenerlo in mezzo.

Rimosso il comando
centering globale: Era ridondante e causava l'allineamento centrale non desiderato del testo.

Aggiunto
raggedright: Inserito subito dopo l'apertura della prima minipage, dice a LaTeX di allineare tutto il contenuto di quella colonna a sinistra, disabilitando la giustificazione automatica che a volte crea spazi vuoti antiestetici nelle colonne strette.

15.25.1 Priority Allocation

Utilizza uno schema di allocazione proporzionale usando le priorità piuttosto che la dimensione. Se il processo P_i genera un page fault:

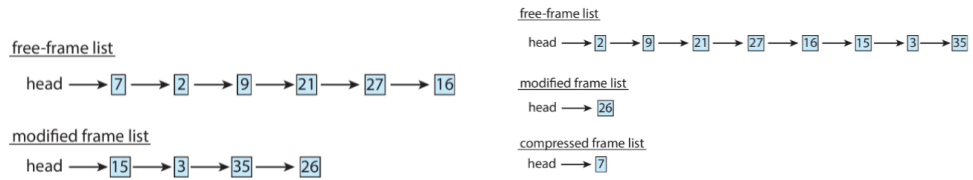
- seleziona per la sostituzione uno dei suoi frame
- seleziona per la sostituzione un frame da un processo con numero di priorità inferiore

15.25.2 Compressione della Memoria

Un'alternativa alla paginazione è la **compressione della memoria**.

Compressione della memoria - invece di effettuare il paging out dei frame modificati nello spazio di swap, comprimi diversi frame in un singolo frame, consentendo al sistema di ridurre l'utilizzo della memoria senza ricorrere allo swapping delle pagine.

Considera la seguente free-frame-list costituita da 6 frame



Supponiamo che questo numero di frame liberi scenda sotto una certa soglia che attiva la sostituzione delle pagine. L'algoritmo di sostituzione (ad esempio, un algoritmo di approssimazione LRU) seleziona quattro frame – 15, 3, 35 e 26 da inserire nella free-frame list. Prima posiziona questi frame in una modified-frame list. Tipicamente, la modified-frame list verrebbe poi scritta nello spazio di swap, rendendo i frame disponibili alla free-frame list. Una strategia alternativa è comprimere un certo numero di frame
 — diciamo, tre
 — e memorizzare le loro versioni compresse in un singolo frame di pagina.

Capitolo 16

Memoria Virtuale

16.1 Swapping delle Pagine

Fino ad ora abbiamo parlato di memoria primaria. Ora introduciamo anche la memoria secondaria, un disco veloce, chiamato **backing store** utilizzato come memoria aggiuntiva.

Un processo, se è fermo per I/O o altro, può essere temporaneamente spostato dalla memoria all'HD e poi riportato in memoria per continuare l'esecuzione. Lo spazio di memoria logico totale dei processi può superare la memoria fisica. In questo modo viene liberato spazio in RAM per eseguire altri processi o per far espandere processi già esistenti.

HD - Backing store - disco veloce abbastanza grande da contenere copie di tutte le immagini di memoria di tutti gli utenti; deve fornire accesso diretto a queste immagini di memoria.

Roll out, roll in - variante dello swapping usata negli algoritmi di scheduling basati sulla priorità; il processo a bassa priorità viene rimosso dalla memoria (swap out) in modo che il processo a priorità più alta possa essere caricato ed eseguito.

La parte principale del tempo di swap è il tempo di trasferimento; il tempo totale di trasferimento è direttamente proporzionale alla quantità di memoria trasferita. Il sistema mantiene una **ready queue** di processi pronti all'esecuzione che possiedono immagini di memoria su disco.

Il processo che è stato rimosso (swap out) deve tornare agli stessi indirizzi fisici?

Di solito no, ma dipende dal metodo di binding degli indirizzi; occorre anche considerare l'I/O in attesa verso/dalla memoria del processo. Ogni OS può decidere come gestire il metodo di swap:

- Swapping normalmente disabilitato
- Avviato se la quantità di memoria allocata supera una soglia
- Disabilitato nuovamente quando la domanda di memoria scende sotto la soglia

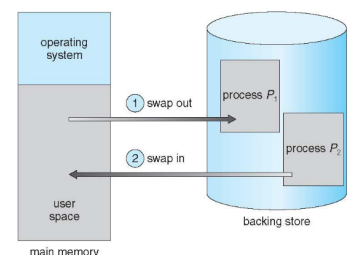


Figura 16.1: Processo di swapping

Quando lo riporto dentro posso portarlo in un posti diverso, è meno vincolato mettendolo in un'altra partizione solo se gli indirizzi sono rilocabili e dipende da come hai fatto binding degli indirizzi.

16.1.1 Tempo di Context Switch con lo Swapping

Se il prossimo processo da eseguire sulla CPU non è in memoria e la RAM è piena, occorre effettuare lo swap out di un processo e lo swap in del processo target. Il tempo di context switch può quindi essere molto elevato, ovvero la latenza per il trasferimento dati verso l'HD.

Esempio: processo da 100MB in swap verso l’hard disk con velocità di trasferimento di 50MB/sec:

Tempo di swap out di 2 secondi

Più swap in di un processo delle stesse dimensioni

Tempo totale del componente di swapping nel context switch: 4 s

È possibile ridurre il tempo sapendo quanta memoria viene effettivamente usata. Chiamate di sistema per informare l’OS sull’uso della memoria tramite `request_memory()` e `release_memory()`.

Altri vincoli: se un task ha I/O in attesa non può essere rimosso (swap out), poiché l’I/O avverrebbe sul processo sbagliato; oppure si trasferisce sempre l’I/O nello spazio kernel e poi al dispositivo I/O, ma questo causa il cosiddetto double buffering, aggiungendo overhead. Allora come funziona nei **sistemi operativi moderni**?

Lo swap viene attivato solo quando la memoria libera è quasi esaurita

Con la memoria a pagine si scambiano intere pagine dentro e fuori dalla RAM

Le pagine spostate su disco vengono in genere salvate in un “file di paging”

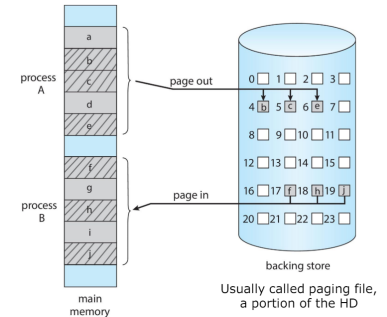


Figura 16.2: Swapping con paginazione

Oltre che per i singoli processi, la tecnica dello swapping è implementata anche a livello di pagine. Non tutte le pagine di un processo devono stare in memoria.

16.1.2 Swapping sui Sistemi Mobili

Lo swapping non viene tipicamente usato perché lo storage è piccolo, il flash drive ha un numero limitato di cicli di scrittura e il bus tra la memoria flash e la CPU è limitato.

Quindi, invece di usare lo swapping quando la memoria libera è bassa:

- **iOS** chiede alle app di cedere volontariamente la memoria allocata; i dati in sola lettura vengono eliminati e ricaricati dal flash se necessario. Il mancato rilascio può portare alla terminazione.
- **Android** termina le app in caso di memoria libera insufficiente, ma prima salva lo stato dell’applicazione nel flash per un riavvio rapido.

Entrambi i sistemi operativi supportano la paginazione come discusso di seguito.

16.2 Struttura della Tabella delle Pagine

Le strutture di memoria per la paginazione possono diventare enormi con metodi diretti. Si consideri uno spazio di indirizzamento logico a 32 bit, con pagine da 4KB: la tabella avrebbe 1 milione di voci e lo spazio totale per OGNI processo è di 4MB solo per ottenere lo spazio di indirizzamento fisico, i quali devono essere contigui.

Una soluzione semplice è dividere la tabella delle pagine in unità più piccole e caricarla solo parzialmente:

1. Shared Pages
2. Paginazione Gerarchica
3. Tabelle delle Pagine con Hashing
4. Tabelle delle Pagine Invertite

16.2.1 Shared Pages

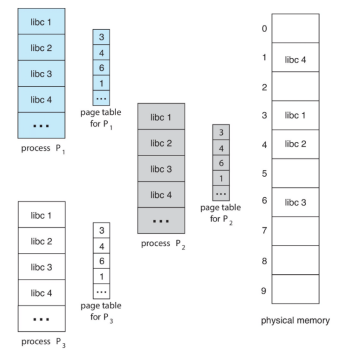
Shared code

Una sola copia in modalità read-only (reentrant), è condivisa tra i processi (eg. text editors, compilers, windows systems). Il concetto è simile a thread multipli che condividono lo stesso process space.

Inoltre è anche molto utile per le comunicazioni interprocesso (IPC), se la condivisione di pagine read-write è consentita.

Codice privato e dati

Ogni processo tiene una copia privata del codice e dei dati. Le pagine di codice e dati privati possono apparire ovunque nella logical address space.

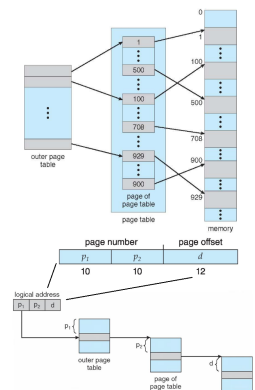


16.2.2 Paginazione gerarchica

Con questo metodo si suddivide lo spazio di indirizzamento logico in più tabelle delle pagine. Una tecnica semplice è la tabella delle pagine a due livelli: in questo modo si carica solo la sezione di interesse.

Con indirizzi a 32 bit e pagine da 4 KB, l'indirizzo logico si divide in:

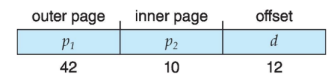
- un numero di pagina di 20 bit, suddiviso in due parti da 10 bit ciascuna:
 - p_1 per selezionare la pagina delle page table, cioè la **pagina esterna**
 - p_2 per selezionare la riga specifica del "blocco" di page table, la **pagina interna**
- un offset di pagina di 12 bit



ATTENZIONE ERRORE: non è 10 10 12 bensì 10 12 10 ETA da cambiare perché è su 3 livelli e non 2

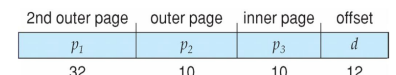
Spazio di Indirizzamento Logico a 64 bit

Anche uno schema di paginazione a due livelli potrebbe non essere sufficiente; se la dimensione della pagina è ancora 4 KB, la tabella delle pagine ha 2^{52} voci e le tabelle delle pagine interne potrebbero essere 2^{10} , vedi figura affianco.



La tabella delle pagine esterna ha 2^{42} voci, ovvero 4 terabyte per processo! Una soluzione potrebbe essere aggiungere un ulteriore livello di tabella delle pagine esterna, vedi figura affianco.

Il problema qui è che sono necessari 4 accessi in memoria per ottenere una singola posizione di memoria fisica, il che non è una soluzione brillante.



16.2.3 Tabelle delle Pagine con Hashing

Gli spazi di indirizzamento comuni sono > 32 bit. Il numero di pagina virtuale viene sottoposto a hashing in una tabella delle pagine; questa tabella contiene un elenco di elementi a cui è stato applicato lo stesso hash.

Ogni elemento contiene:

- il numero di pagina virtuale
- il valore del frame di pagina mappato
- un puntatore all'elemento successivo

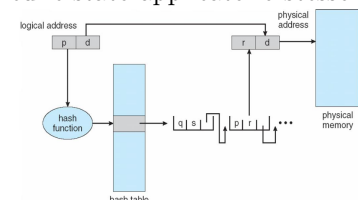


Figura 16.3: Tabella delle pagine con hashing

Se si dispone di uno spazio di indirizzamento a 64 bit, 12 bit sono per l'offset, quindi 52 per l'indirizzo di pagina. Si può usare una tabella hash da 4KB (2^{12}), che tipicamente si adatta a una singola pagina e rimane sempre in memoria per ciascun processo.

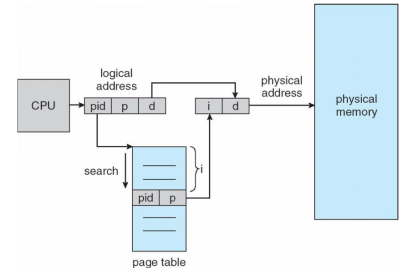
Si accede alla pagina specifica tramite essa. Se tutta la memoria viene usata dal processo, questo non è efficiente. Tuttavia, nella stragrande maggioranza dei casi nessun processo utilizzerà tutti i 2^{52} pagine.

Ovviamente di possono avere collisioni, ma non è un grosso problema se la tabella hash è abbastanza grande. Altrimenti, possono utilizzare **open addressing** (cerca un altro posto) o **linear chaining** (una lista).

16.2.4 Tabella delle Pagine Invertita

Invece di avere una tabella delle pagine per ogni processo che tiene traccia di tutte le pagine logiche possibili, si ha un'unica voce per ogni pagina reale della memoria!

Ogni voce consiste nell'indirizzo virtuale della pagina memorizzata in quella locazione di memoria reale, con informazioni sul processo che possiede quella pagina.



Riduce la memoria necessaria per memorizzare ogni tabella delle pagine, ma aumenta il tempo necessario per ricercare nella tabella quando si verifica un riferimento a una pagina.

Una soluzione potrebbe essere usare l'hashing per limitare la ricerca a una sola voce; anche il TLB può accelerare l'accesso.

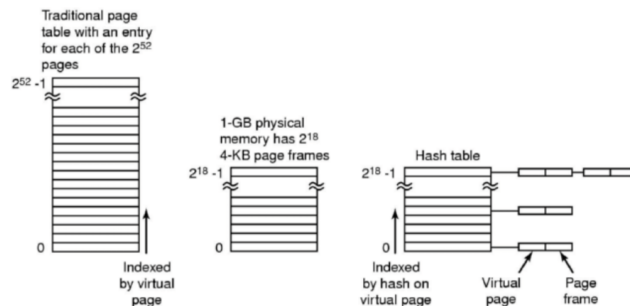


Figura 16.4: Comparativa tra tabella delle pagine tradizionale e tabella delle pagine invertita

16.3 Oracle SPARC Solaris

Oracle SPARC Solaris è un sistema a 64-bit che è molto vicino e integrato con l'hardware. È basato su due hash table:

- Una per il kernel
- Una per i processi utente.

Ogniuna mappa memoria virtuale a memoria fisica. Più efficiente che avere due separate hash table per pagina.

Ogni entry ha un address e una span, che indica il numero di pagine che la entry rappresenta.

TLB possiede la translation table entries (TTEs), per un accesso hardware più rapido. La cache di TTEs risiede nel translation storage buffer (TSB), includendo un entry per le pagine recentemente accedute.

Virtual address reference causa una TLB search:

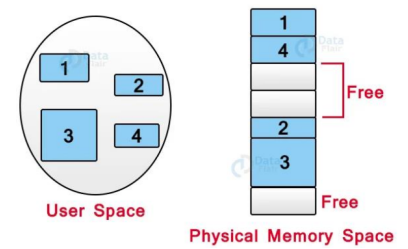
Se è miss, l'hardware va a cercare nella memoria TSB guardando dentro la TTE corrispondente all'address.

- Se è hit, la CPU copia la TSB entry nella TLB e la traduzione viene completata
- Se no match, il kernel interrompe la ricerca nella hash table. Il kernel crea una TTE dall'appropriata hash table e la memorizza nella TSB, Interrupt handler ritorna il controllo alla MMU, la quale completa la traduzione dell'address.

16.4 Segmentazione

L'uso di pagine di dimensioni fisse apre il problema della frammentazione interna: ogni processo viene abbinato a un certo numero di pagine piuttosto che alla memoria di cui ha bisogno.

Esiste quindi un modo per assegnare esattamente la quantità di memoria di cui un processo ha bisogno, evitando così qualsiasi spreco di memoria? La risposta è usare la **segmentazione**.



Un processo è diviso in blocchi; i blocchi in cui viene suddiviso un programma, che non devono essere necessariamente tutti della stessa dimensione, sono chiamati segmenti.

- Un blocco per lo stack
- Un blocco per lo heap
- Un blocco per la libreria 1

Ogni processo è diviso in diversi segmenti, tutti caricati in memoria durante l'esecuzione, anche se non necessariamente in modo contiguo.

Una tabella memorizza le informazioni su tutti questi segmenti: la base e il limite (lunghezza).

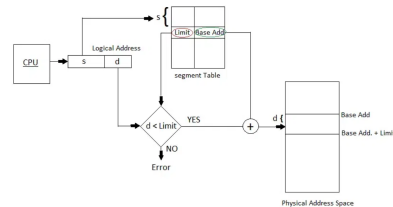
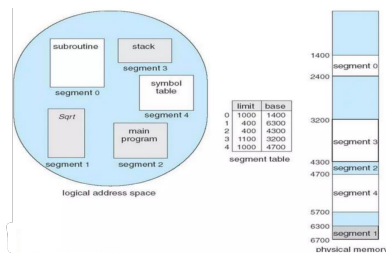
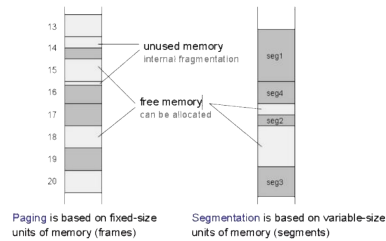


Figura 16.5: Segmentazione: da logico a fisico

16.4.1 Paginazione vs Segmentazione

Feature	Paging	Segmentation	Example
Usage	Systems with a small amount of memory to prevent external fragmentation.	Systems with a large amount of memory to provide better security and flexibility.	A word processing program with a small memory footprint would be a good candidate for paging. A database program with a large memory footprint would be a good candidate for segmentation.
Working	The program is divided into pages and stored in the main memory. When the program needs to access a page, the operating system transfers it from the secondary memory to the main memory.	The program is divided into segments and stored in the secondary memory. When the program needs to access a segment, the operating system transfers it from the secondary memory to the main memory.	A program that is 100 pages long is divided into 4KB pages in paging. In segmentation, the program is divided into 3 segments: code, data, and stack.
Page size	Fixed	Variable	In paging, the page size is typically 4KB. In segmentation, the segment size can be any size.
Loading of pages	Dependent	Independent	In paging, all pages of a process must be loaded into memory before the process can be executed. In segmentation, pages can be loaded into memory independently.
Prevention of internal fragmentation	No	Yes	Paging can lead to internal fragmentation because a free page may not be large enough to accommodate a requesting page. Segmentation can prevent internal fragmentation because segments can be of different sizes.
Prevention of external fragmentation	Yes	No	Paging can prevent external fragmentation because pages are loaded into memory in blocks. Segmentation cannot prevent external fragmentation because segments can be loaded into memory independently.
Memory management overhead	Lower	Higher	The memory management overhead in paging is lower than in segmentation because paging is a simpler technique.
Security	Lower	Higher	Paging cannot provide as much security as segmentation because segments can be loaded into memory independently.
Flexibility	Lower	Higher	Paging is less flexible than segmentation because pages must be loaded into memory in blocks.
Performance	Faster	Slower	Paging is faster than segmentation because it is a simpler technique.



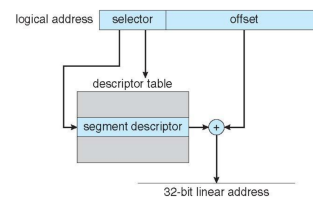
Esempio di Architettura Intel IA-32

Lascia stare.



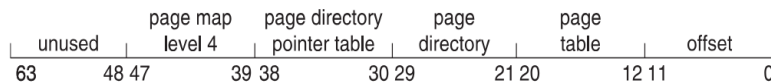
Figura 16.6: Conversione degli indirizzi di memoria

Le unità di paginazione formano l'equivalente dell'MMU; le dimensioni delle pagine possono essere 4 KB o 4 MB, con tabelle delle pagine a due livelli.



Esempio di Intel x86-64

L'attuale generazione dell'architettura CISC ha 64 bit, il che è enorme (> 16 exabyte). In pratica viene implementato solo l'indirizzamento a 48 bit.



16.5 Esempio: Architettura ARM

Nei moderni processori ARM installati sugli smartphone sono CPU 64bit con particolare attenzione ai consumi energetici. Un livello di paginazione per sezione, due livelli si pagine più piccole.

Due livelli di TLBs

- L'outer ha due micro TLBs, uno per le istruzioni e uno per i dati.
- L'inner è una TLB singola
- Viene controllato il primo interno, successivamente vengono controllati gli esterni mancanti, e in caso di mancata corrispondenza viene eseguita la ricerca nella tabella delle pagine da parte della CPU.

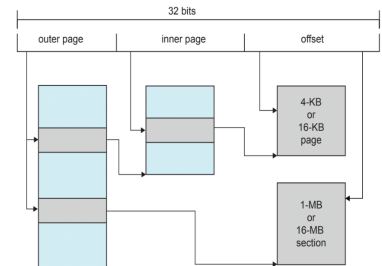


Figura 16.7: Architettura ARM

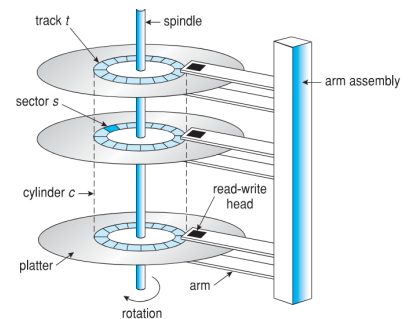
Capitolo 17

Sistemi di Archiviazione di Massa

La maggior parte della memoria secondaria nei computer moderni è costituita da **hard disk drive**, HDD, e dispositivi di **memoria non volatile**, NVM.

HDD fanno ruotare piatti di materiale a rivestimento magnetico sotto testine di lettura-scrittura mobili:

- I dischi ruotano da 60 a 250 volte al secondo
- La velocità di trasferimento è la velocità con cui i dati scorrono tra il disco e il computer, $\approx 1 \text{ Gb/sec}$
- Il tempo di posizionamento (tempo di accesso casuale) è il tempo necessario per spostare il braccio del disco al cilindro desiderato (seek time) e il tempo per far ruotare il settore desiderato sotto la testina (latenza rotazionale), da 3ms a 12ms
- Lo schianto della testina avviene quando la testina del disco entra in contatto con la superficie del disco – molto grave



Latenza di accesso (tempo medio di accesso) = tempo medio di seek + latenza media

Per un disco molto veloce: $3 \text{ ms} + 2 \text{ ms} = 5 \text{ ms}$.

Tempo medio di I/O = tempo medio di accesso + (dati da trasferire / velocità di trasferimento) + overhead del controller

Esempio: trasferire un blocco da 4KB su un disco a 7200 RPM con un seek time medio di 5ms, velocità di trasferimento di 1Gb/sec e overhead del controller di 0.1ms.

A 7200 RPM = 120 giri/sec = 8.333ms/giro; nel caso peggiore bisogna attendere 1 rotazione completa, ma in media si attende mezza rotazione, pari a 4.17ms.

Tempo di trasferimento = $4\text{KB} / 1\text{Gb/s} * 8\text{Gb} / \text{GB} * 1\text{GB} / 1024\text{KB} = 32 / (1024^2) = 0.031 \text{ ms}$

Tempo medio di I/O per un blocco da 4KB = $5\text{ms} + 4.17\text{ms} + 0.1\text{ms} + 0.031\text{ms} = 9.301\text{ms}$

17.1 Dispositivi di Memoria Non Volatile

L'NVM include solid-state drive e chiavette USB. Questi dispositivi sono più affidabili e **più veloci** degli HDD, ma più costosi.

La lettura e scrittura avviene in incrementi di pagina (come i settori), ma non si può sovrascrivere in-place: occorre prima cancellare, e le cancellazioni avvengono in incrementi di blocco più grandi. Questi dispositivi possono essere cancellati un numero limitato di volte, espresso come **drive writes per day**, DWPD.

L'SSD è un dispositivo complesso che integra un processore per verificare lo stato di tutti i settori, cancellare e scrivere blocchi, ecc.

17.2 Algoritmi del Controller NAND Flash

Poiché non è possibile sovrascrivere in-place, le pagine finiscono per contenere un mix di dati validi e non validi. Per tracciare quali blocchi logici sono validi, il controller mantiene una tabella **flash translation layer** (FTL). Implementa inoltre la **garbage collection** per liberare lo spazio occupato dalle pagine non valide. Riserva **overprovisioning** per fornire spazio di lavoro alla GC. Ogni cella ha una vita utile limitata, quindi è necessario il **wear leveling** per distribuire uniformemente le scritture.

valid page	valid page	invalid page	invalid page
invalid page	valid page	invalid page	valid page

17.3 Memoria Volatile

La DRAM viene spesso usata come dispositivo di mass-storage; tecnicamente non è memoria secondaria perché è volatile, ma può ospitare file system ed essere usata come storage secondario molto veloce.

I RAM drive (chiamati anche RAM disk) si presentano come dispositivi a blocchi raw, solitamente formattati con un file system.

I computer hanno già buffering e caching via RAM: perché usare i RAM drive?

- Cache e buffer sono allocati/gestiti da programmatore, sistema operativo e hardware
- I RAM drive sono sotto il controllo diretto dell'utente
- Sono presenti in tutti i principali sistemi operativi. Ad esempio Linux `/dev/ram`, MacOS `diskutil`

Sono usati come storage temporaneo ad alta velocità. I programmi possono condividere rapidamente grandi quantità di dati leggendo/scrivendo sul RAM drive.

17.4 Struttura del Disco

I dischi vengono indirizzati come grandi array monodimensionali di blocchi logici, dove il blocco logico è la minima unità di trasferimento. Il low-level formatting crea i blocchi logici sul supporto fisico.

L'array monodimensionale di blocchi logici viene mappato in modo sequenziale nei settori del disco:

- Il settore 0 è il primo settore della prima traccia del cilindro più esterno
- La mappatura procede in ordine lungo quella traccia, poi sulle altre tracce dello stesso cilindro, quindi sui restanti cilindri dall'esterno verso l'interno
- L'indirizzamento logico-fisico dovrebbe essere semplice, salvo eccezioni dovute a bad sector e al numero non costante di settori per traccia

17.4.1 Collegamento del Disco

Lo storage host-attached viene raggiunto tramite porte di I/O collegate ai bus di I/O. Sono disponibili vari bus, tra cui advanced technology attachment (ATA), serial ATA (SATA), eSATA, serial attached SCSI (SAS), universal serial bus (USB) e fibre channel (FC). Il più comune è SATA.

Poiché la NVM è molto più veloce degli HDD, è stata introdotta una nuova interfaccia ad alte prestazioni, NVMe express (NVMe), che si collega direttamente al bus PCI. I trasferimenti dati su bus sono gestiti da processori elettronici specializzati chiamati controller (o host-bus adapter, HBA).

- Un host controller lato computer del bus e un device controller lato dispositivo
- Il computer inserisce i comandi nell'host controller tramite porte di I/O memory-mapped; l'host controller invia messaggi al device controller e i dati sono trasferiti via DMA tra dispositivo e DRAM del computer

17.5 Scheduling del Disco

Il sistema operativo è responsabile dell'uso efficiente dell'hardware; per il disco ciò significa avere un accesso rapido e una buona **larghezza di banda** del disco. La **bandwidth** è il numero totale di byte trasferiti, diviso per il tempo totale tra la prima richiesta di servizio e il completamento dell'ultimo trasferimento.

Le sorgenti delle richieste di I/O su disco sono molte: OS, processi di sistema, processi utente; includono tutte scrittura/lettura, indirizzo del disco, indirizzo di memoria, numero di settori da trasferire. L'OS mantiene una coda di richieste per dispositivo.

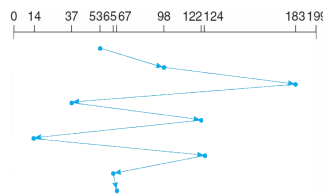
In passato l'HD non era sofisticato come oggi; l'OS doveva occuparsi di tutto: gestione della coda e scheduling della testina del disco. Ora il controllo è affidato all'HD e l'OS deve solo passare gli indirizzi dei blocchi, lasciando che il disco ordini le richieste.

Esempio: vogliamo gestire la seguente lista di richieste usando diversi algoritmi di scheduling:

98, 183, 37, 122, 14, 124, 65, 67

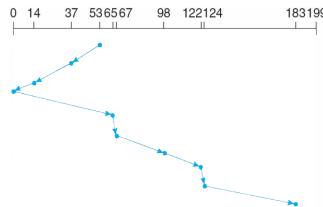
La testina parte dalla posizione 53

17.5.1 FCFS



17.5.2 SCAN - algoritmo dell'ascensore

Il braccio del disco parte da un'estremità e si sposta verso l'altra. Questo non è equo: gli ultimi valori devono aspettare molto a lungo.

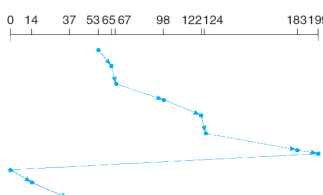


Nota: non ci sono richieste da 0 a 65, quindi possiamo saltare quella zona.

L'algoritmo SCAN è talvolta chiamato algoritmo dell'ascensore. Nell'illustrazione il movimento totale della testina è di 208 cilindri. Si noti però che, se le richieste sono distribuite uniformemente, la densità maggiore si trova all'estremità opposta del disco, e quelle richieste attendono più a lungo.

17.5.3 C-SCAN

Fornisce un tempo di attesa più uniforme rispetto a SCAN: la testina si sposta da un'estremità del disco all'altra, servendo le richieste man mano che procede. Quando raggiunge l'altra estremità, tuttavia, ritorna immediatamente all'inizio del disco senza servire alcuna richiesta durante il percorso di ritorno.



17.5.4 Scelta dell'Algoritmo di Scheduling del Disco

- FCFS è comune e ha un'attrattiva naturale
- SCAN e C-SCAN hanno prestazioni migliori per sistemi con un carico elevato sul disco

Linux implementa lo scheduler deadline

- Mantiene code separate per lettura e scrittura
- Per impostazione predefinita, i batch di lettura hanno la precedenza sui batch di scrittura, poiché le applicazioni tendono a bloccarsi sulle operazioni di I/O in lettura
- Schedula i batch di lettura e scrittura per l'esecuzione in ordine crescente di indirizzamento logico dei blocchi (LBA)
- Dopo ogni batch, seleziona naturalmente i batch di lettura se presenti, ma controlla anche da quanto tempo le operazioni di scrittura sono in attesa (per evitare la starvation)
- Schedula il successivo batch di lettura o scrittura secondo necessità
- Tipicamente, se un batch di lettura è in attesa da più di 500ms, ottiene la priorità sulla scrittura

Questo scheduler è adatto alla maggior parte dei casi d'uso, in particolare quelli in cui le operazioni di scrittura sono prevalentemente **asincrone**.

17.6 Pianificazione per la NVM

Non vi sono testine del disco né latenza rotazionale, ma c'è ancora margine per l'ottimizzazione. L'NVM è migliore nell'I/O casuale, l'HDD nell'I/O sequenziale; le operazioni di input/output al secondo (IOPS) sono molto più elevate con NVM (centinaia di migliaia vs centinaia).

17.7 Rilevamento e Correzione degli Errori

Un aspetto fondamentale di molte parti dell'informatica è il **Rilevamento e Correzione degli Errori**.

Il **rilevamento degli errori** determina se si è verificato un problema; viene spesso implementato tramite bit di parità.

La parità è una forma di checksum; un altro metodo di rilevamento errori è il controllo di ridondanza ciclica (CRC), che usa una funzione hash per rilevare errori su più bit.

Il **codice di correzione degli errori**, ECC, non solo rileva, ma può anche correggere alcuni errori; gli errori soft sono correggibili, quelli hard vengono rilevati ma non corretti.

17.7.1 Gestione dei dispositivi di storage

Il **low-level formatting**, o **formattazione fisica**, divide un disco in settori che il controller può leggere e scrivere. Ogni settore contiene un'intestazione, i dati e il codice di correzione degli errori. In genere ospita 512 byte di dati, ma il valore può essere configurabile.

Per usare un disco per i file, il sistema operativo deve comunque registrare sul disco le proprie strutture dati:

- **Partizionare** il disco in uno o più gruppi di cilindri, ognuno trattato come disco logico
- Eseguire la **formattazione logica**, o “creare un file system”

Per aumentare l'efficienza molti file system raggruppano i blocchi in cluster. L'I/O sul disco avviene in blocchi, quello sui file nei cluster.

La **partizione di boot/root** contiene il sistema operativo; le altre partizioni possono contenere altri sistemi operativi, altri file system, o essere raw. La partizione può essere **montata** al boot, automaticamente o manualmente.

Al momento del montaggio, viene verificata la consistenza del file system e la correttezza di tutti i metadati.

Se non corretta, correggere e riprovare

Se corretta, aggiungere alla tabella di montaggio e consentire l'accesso

Il boot block, o un programma di gestione del boot per sistemi multi-OS, può puntare al volume di boot o al boot loader, ovvero un insieme di blocchi che contengono il codice necessario per caricare il kernel dal file system.

Il boot block inizializza il sistema ed è memorizzato in ROM, firmware. Il programma Bootstrap loader è memorizzato nei boot block della partizione di boot: il master boot record MBR.

17.7.2 Gestione dello Spazio di Swap

Utilizzato per spostare interi processi o pagine dalla DRAM all'archiviazione secondaria quando la DRAM non è sufficiente per tutti i processi. Il sistema operativo fornisce la **gestione dello spazio di swap**:

- L'archiviazione secondaria è più lenta della DRAM, quindi è importante ottimizzare le prestazioni
- Di solito sono possibili più spazi di swap, riducendo il carico di I/O su ogni singolo dispositivo
- È preferibile avere dispositivi dedicati
- Può essere in una partizione raw o in un file all'interno di un file system (per comodità)

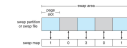


Figura 17.2: Strutture dati per lo swapping nei sistemi Linux

17.7.3 Collegamento dello Storage all'Host

I computer accedono allo storage in tre modi:

- collegato all'host
- collegato in rete
- cloud

L'accesso collegato all'host avviene tramite porte I/O locali, usando una delle varie tecnologie per collegare molti dispositivi (USB, LAN...).

Storage Collegato in Rete: NAS

Il network-attached storage (NAS) è uno storage reso disponibile tramite rete anziché tramite un canale locale (come un bus). **NFS** e **CIFS** sono protocolli comuni, implementati tramite Remote Procedure Call (RPC) tra host e storage, tipicamente su TCP o UDP su rete IP.

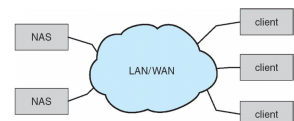


Figura 17.3: NAS

17.7.4 Archiviazione Cloud

Simile al NAS, fornisce accesso allo storage tramite rete. A differenza del NAS, viene acceduto tramite Internet o WAN verso data center remoti.

Il NAS viene presentato come un normale file system, mentre lo storage cloud è basato su API, con programmi che usano le API per fornire l'accesso. Si usano le API a causa della latenza e degli scenari di guasto (i protocolli NAS non funzionerebbero bene).

17.7.5 Storage Array

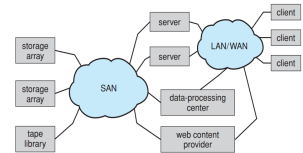
Si possono collegare semplicemente dischi singoli oppure array di dischi. Questo evita lo svantaggio del NAS legato al consumo di banda di rete. Uno storage array ha uno o più controller e offre funzionalità ai sistemi host collegati.

- Porte per connettere gli host all'array
- Memoria e software di controllo (talvolta anche NVRAM, ecc.)
- Da pochi dischi fino a migliaia
- RAID, hot spare, hot swap (discussi in seguito)
- Storage condiviso → maggiore efficienza
- Funzionalità tipiche di alcuni file system: snapshots, cloni, thin provisioning, replication, deduplication, ecc.

17.7.6 Storage Area Network - SAN

Comune negli ambienti di storage di grandi dimensioni: più host collegati a più storage array, con elevata flessibilità.

Una SAN è composta da uno o più storage array collegati a uno o più switch Fibre Channel oppure a una rete InfiniBand (IB). Anche gli host si collegano a questi switch.



Lo storage viene reso disponibile tramite LUN, con masking da specifici array verso specifici server. È facile aggiungere o rimuovere storage, aggiungere nuovi host e assegnare loro capacità. Perché mantenere reti separate per storage e comunicazioni? Si considerino iSCSI e FCoE.

17.7.7 Array Ridondante di Dischi Indipendenti - RAID

RAID - più dischi forniscono affidabilità tramite ridondanza.

- Aumenta il **tempo medio al guasto**.
- **Tempo medio di riparazione - MTTR**: periodo di esposizione durante il quale un altro guasto potrebbe causare perdita di dati
- **Tempo medio alla perdita di dati - MTTF**: basato sui fattori precedenti

Se i dischi speculari si guastano indipendentemente, si consideri un disco con 100.000 (MTTF) ore di tempo medio al guasto e 10 ore di tempo medio di riparazione (MTTR). Il tempo medio alla perdita di dati è:

$$\text{MTDL} = 100\,000^2 / (2 \cdot 10) = 500 \cdot 10^6 \text{ hours or } 57\,000 \text{ years!}$$

RAID mirror: 2 dischi

Con 2 dischi in mirror: $\text{MTTF}_1 = \text{MTTF}/2$ e $P_2 = \text{MTTR}/\text{MTTF}$.

$$\text{MTTDL} = \frac{\text{MTTF}^2}{2 \cdot \text{MTTR}} = \frac{(100000)^2}{2 \cdot 10} = 5 \cdot 10^8 \text{ hours} \approx 57000 \text{ years}$$

Diversi miglioramenti nelle tecniche di utilizzo dei dischi coinvolgono l'uso di più dischi che lavorano in modo cooperativo.

Il **striping** del disco utilizza un gruppo di dischi come un'unica unità di archiviazione. Il RAID è organizzato in sei diversi livelli.

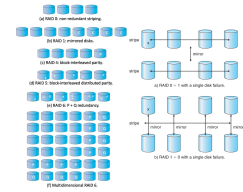
I sistemi RAID migliorano le prestazioni e l'affidabilità dello storage memorizzando dati ridondanti.

- Il **mirroring** o shadowing (**RAID 1**) mantiene una copia duplicata di ogni disco
- Mirror striped (**RAID 1+0**) o stripe mirror (**RAID 0+1**) offre alte prestazioni e alta affidabilità

- La parità a blocchi interlacciati (**RAID 4, 5, 6**) usa molta meno ridondanza

Il RAID all'interno di un array di storage può comunque fallire se l'array si guasta, quindi la replica automatica dei dati tra array è comune. Spesso un piccolo numero di dischi hot-spare viene lasciato non allocato, pronto a sostituire automaticamente un disco guasto e a ricostruire i dati su di esso.

- RAID 0: striping senza ridondanza. La probabilità che si guastino 4 dispositivi assieme è molto molto bassa.
- RAID 1: mirroring, duplica i dati, stesso metodo utilizzato nell'aviazione.



17.8 Altre Funzionalità

Indipendentemente da dove sia implementato il RAID, si possono aggiungere altre funzionalità utili. Uno snapshot è una vista del file system prima che avvenga un insieme di modifiche (cioè in uno specifico istante temporale).

La replication e la duplicazione automatica delle scritture tra siti separati, utile per ridondanza e disaster recovery. Può essere sincrona o asincrona.

Un disco hot spare è un disco non usato normalmente, che entra in funzione automaticamente se un disco si guasta, per sostituirlo e ricostruire il set RAID quando possibile. Questo riduce il tempo medio di riparazione.

17.9 Estensioni

Il solo RAID non previene né rileva la corruzione dei dati o altri errori: copre soprattutto i guasti dei dischi. Solaris ZFS aggiunge checksum su tutti i dati e i metadati. I checksum sono mantenuti insieme ai puntatori agli oggetti, per verificare che l'oggetto sia quello corretto e che non sia stato modificato.

ZFS può rilevare e correggere la corruzione di dati e metadati. Inoltre elimina il concetto tradizionale di volumi e partizioni.

I dischi sono allocati in **pool**. I file system che usano un pool lo condividono, allocando e rilasciando spazio in modo simile alle chiamate `malloc()` e `free()` per la memoria.

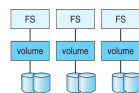


Figura 17.4: Volumi e file system tradizionali

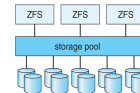


Figura 17.5: ZFS e storage a pool

17.10 Object Storage

Nel calcolo general-purpose, i file system tradizionali non sono sempre sufficienti su scala molto grande. Un approccio alternativo è partire da uno storage pool e inserirvi oggetti.

Un oggetto è semplicemente un contenitore di dati. Non esiste navigazione del pool per trovare oggetti (niente strutture a directory, pochi servizi): è un modello orientato alle macchine, non all'utente.

Sequenza tipica:

- Creare un oggetto nel pool e ricevere un object ID
- Accedere all'oggetto tramite quell'ID
- Eliminare l'oggetto tramite quell'ID

Software di gestione dell'object storage come Hadoop file system (HDFS) e Ceph determinano dove memorizzare gli oggetti e come proteggerli, tipicamente mantenendo N copie su N sistemi nel cluster di object storage. Il modello è scalabile orizzontalmente, content-addressable e non strutturato.

Capitolo 18

Interfaccia del File System

18.1 Cos'è un File?

Spazio di indirizzamento logico contiguo. Tipi:

- File di Dati: Numerico, Carattere, Binario
- File di Programma

18.1.1 Attributi del File

- **Nome** - è la sola informazione mantenuta in forma leggibile dall'uomo
- **Identificatore** - tag unico (numero) che identifica il file all'interno del file system
- **Tipo** - necessario per i sistemi che supportano tipi diversi
- **Posizione** - puntatore alla posizione del file sul dispositivo
- **Dimensione** - dimensione corrente del file
- **Protezione** - controlla chi può leggere, scrivere, eseguire
- **Ora, data e identificazione utente** - dati per protezione, sicurezza e monitoraggio dell'uso

Le informazioni sui file sono conservate nella **struttura delle directory**, mantenuta sul disco. Molte varianti, tra cui attributi estesi del file come il checksum.

18.1.2 Operazioni sui File

Un file è un tipo di dato astratto

- Creare
- Scrivere - alla posizione del puntatore di scrittura
- Leggere - alla posizione del puntatore di lettura
- Riposizionare all'interno del file - seek
- Eliminare
- Truncare
- Aprire (Fi) - cercare la struttura della directory sul disco per la voce Fi, e spostare il contenuto della voce in memoria
- Chiudere (Fi) - spostare il contenuto della voce Fi in memoria alla struttura della directory sul disco

18.1.3 File Aperti

Sono necessari diversi dati per gestire i file aperti:

- Tabella dei file aperti: traccia i file aperti
- Puntatore al file: puntatore all'ultima posizione di lettura/scrittura, specifico per ogni processo che ha il file aperto
- Contatore di aperture del file: contatore del numero di volte in cui un file è aperto, per consentire la rimozione dei dati dalla tabella dei file aperti quando l'ultimo processo lo chiude
- Posizione del file su disco: cache delle informazioni di accesso ai dati
- Diritti di accesso: informazioni sulla modalità di accesso per processo

18.1.4 Open file locking

Provided by some operating systems and file systems. Similar to reader-writer locks

- Shared lock similar to reader lock – several processes can acquire concurrently
- Exclusive lock similar to writer lock

Mediates access to a file. Mandatory or advisory:

- Mandatory - access is denied depending on locks held and requested
- Advisory - processes can find status of locks and decide what to do

18.1.5 Struttura del File

- Nessuna - sequenza di parole, byte
- Struttura semplice - Righe, lunghezza fissa, lunghezza variabile...
- Strutture complesse - Documento formattato, File di caricamento rilocabile

Le ultime due strutture possono essere simulate con la prima inserendo opportuni caratteri di controllo. La decisione sulla struttura spetta all'**OS** o al **Programma**.

18.2 Accesso e Operazioni sui File

Sono necessari diversi dati per gestire i **file aperti**:

- **Tabella dei file aperti**: traccia i file aperti
- Puntatore al file: puntatore all'ultima posizione di lettura/scrittura, specifico per ogni processo che ha il file aperto
- **Contatore di aperture del file**: contatore del numero di volte in cui un file è aperto, per consentire la rimozione dei dati dalla tabella dei file aperti quando l'ultimo processo lo chiude
- Posizione del file su disco: cache delle informazioni di accesso ai dati
- Diritti di accesso: informazioni sulla modalità di accesso per processo

18.2.1 Blocco dei File

Fornito da alcuni **sistemi operativi** e file system. Simile ai lock lettore-scrittore.

Lock condiviso simile al lock lettore - più processi possono acquisirlo contemporaneamente **Lock esclusivo** simile al lock scrittore

Obbligatorio - l'accesso è negato in base ai lock detenuti e richiesti **Consultivo** - i processi possono conoscere lo stato dei lock e decidere come procedere

Esempio: provare a scrivere un file già aperto in Excel genera un errore; aprire un file in gedit di solito non lo fa.

18.2.2 Metodi di Accesso

Un file è un **record logico** di lunghezza fissa:

- Accesso Sequenziale
- Accesso Diretto
- Altri Metodi di Accesso

Accesso Sequenziale

Operazioni: **read next**, **write next**, **reset**; nessuna lettura dopo l'ultima scrittura (rewrite) e nessuna operazione di seek.

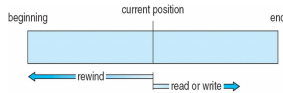


Figura 18.1: Accesso sequenziale

Accesso Diretto

Operazioni: leggi n, scrivi n, Reset, Posiziona (seek) a n. n = numero di blocco relativo

sequential access	implementation for direct access
reset	cp = 0;
read next	read cp; cp = cp + 1;
write next	write cp; cp = cp + 1;

Figura 18.2: Accesso diretto

Accesso Sequenziale vs Diretto

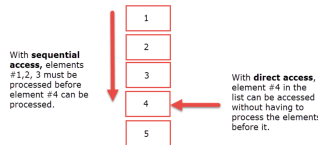


Figura 18.3: Accesso sequenziale vs diretto

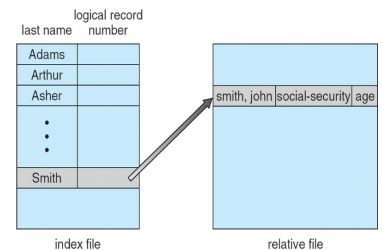
Altri Metodi di Accesso

Possono esistere altri metodi di accesso costruiti sopra i metodi di base. Generalmente comportano la creazione di un indice per il file. L'indice viene mantenuto in memoria per la rapida individuazione della posizione dei dati (si pensi al codice Universal Product Code (UPC) con i relativi record di dati).

Se l'indice è troppo grande, si crea un indice in memoria, che è un indice di un indice su disco.

Il metodo sequenziale ad accesso indicizzato di IBM (ISAM)

- Piccolo indice master, punta ai blocchi su disco dell'indice secondario
- Il file è mantenuto ordinato su una chiave predefinita
- Tutto gestito dall'OS
- Un motore simile, MyISAM, è un modo per indicizzare i DB MySQL



Nota: Questa è una struttura data ad alto livello, più avanti ne vedremo una di più basso livello.

18.3 File Mappati in Memoria

Invece di accedere ai file di dati dall'HD ad ogni accesso, i file di dati possono essere paginati in memoria come i file di processo, risultando in accessi molto più veloci.

Si può vederlo come il "paging" di un file = una tabella delle pagine per file, eccetto ovviamente quando si verificano page-fault. Questo è noto come memory-mapping di un file.

Un file viene mappato su un intervallo di indirizzi all'interno dello spazio di indirizzamento virtuale di un processo e poi paginato secondo necessità usando il normale sistema di demand paging. Può essere usato per caricare un file da 10 GB con soli 2 GB di RAM disponibili!

Le scritture sul file vengono effettuate sui frame di pagina in memoria e non vengono immediatamente scritte su disco. Ecco perché è importante eseguire la `close()` su un file al termine della scrittura.

In questo modo i dati possono essere scritti su disco in sicurezza e i frame di memoria possono essere liberati per altri scopi.

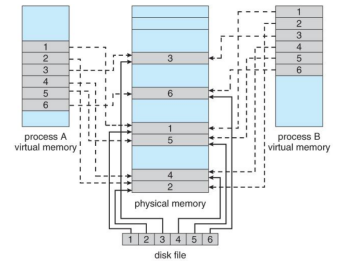


Figura 18.4: File mappati in memoria

18.4 Dischi e File System

Dalla lezione precedente:

- I dischi possono essere suddivisi in partizioni
- I dischi o le partizioni possono essere protetti da RAID contro i guasti
- Le partizioni sono note anche come minidisk, slice

Un file system governa l'organizzazione e l'accesso ai file.

L'entità contenente il file system è nota come volume; un disco o una partizione formattata è un volume.

Il disco o la partizione può essere usato grezzo (raw) - senza file system - oppure formattato con un file system. Ogni volume contenente un file system traccia anche le informazioni di quel file system nella directory del dispositivo o nella tabella dei contenuti del volume. Oltre ai file system di uso generale, esistono molti file system speciali, spesso tutti all'interno dello stesso sistema operativo o computer.

18.4.1 Tipi di File System

Parliamo principalmente di file system di uso generale, ma i sistemi spesso hanno molti file system, sia di uso generale sia speciali.

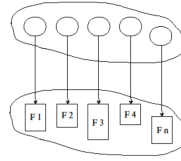
Si consideri Solaris, che ha:

- tmpfs - FS volatile basato su memoria per I/O rapido e temporaneo
- objfs - interfaccia nella memoria del kernel per ottenere i simboli del kernel per il debugging
- ctfs - file system dei contratti per la gestione dei daemon
- lofs - file system loopback che consente l'accesso a un FS al posto di un altro
- procfs - interfaccia del kernel alle strutture dei processi
- ufs, zfs - file system di uso generale

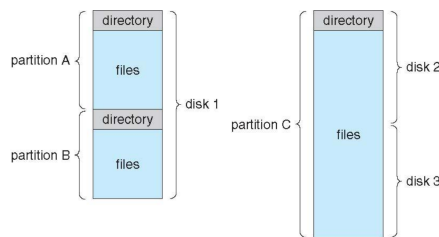
Anche Linux ha più file system.

18.5 Struttura delle Directory

Una collezione di nodi contenenti informazioni su tutti i file, ed alcuni sui attributi come data di creazione, dimensioni, etc.



Sia la struttura delle directory che i file risiedono sul disco. Le informazioni delle directory si trovano di solito in cima a una partizione. I direttori fanno parte dei metadati.



Le operazioni eseguite sulla Directory sono molte: ricerca di un file, creazione di un file, eliminazione di un file, elenco dei contenuti di una directory, rinomina di un file, navigazione del file system verso sottodirectory o directory padre.

18.5.1 Organizzazione delle Directory

La directory è organizzata logicamente per prestazioni, usabilità e controllo degli accessi. In particolare:

- Efficienza - individuare rapidamente un file
- Denominazione - comoda per gli utenti: due utenti possono avere lo stesso nome per file diversi; lo stesso file può avere più nomi diversi per essere compatibili con programmi diversi
- Raggruppamento - raggruppamento logico dei file per proprietà (es. tutti i programmi Java, tutti i giochi...)

18.5.2 Directory a Singolo Livello

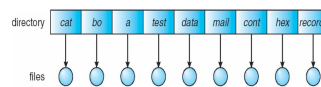


Figura 18.5: Una singola directory per tutti gli utenti

Molti problemi: il problema della denominazione e il problema del raggruppamento ne sono esempi.

18.5.3 Directory a Due Livelli

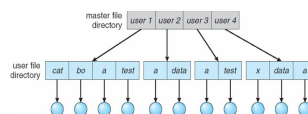
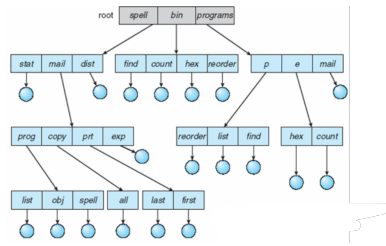


Figura 18.6: Directory separata per ogni utente

Caratteristiche: Percorso lungo (negativo), Possibilità di avere lo stesso nome di file per utenti diversi (neutro), Ricerca efficiente (positivo), Nessuna capacità di raggruppamento (negativo)

18.5.4 Directory ad Albero



Pro	Contro
Concetto di current directory	Inefficiente
Raggruppamento	Nessuna condivisione di file
Facile da cercare	Duplicazione di file in più directory

Traduci in italiano: Nome del percorso assoluto o relativo. Creare un nuovo file viene fatto nella directory corrente. Eliminare un file `rm <file-name>`. Creare una nuova sottodirectory viene fatto nella directory corrente `mkdir <dir-name>` Esempio: se nella directory corrente `/mail mkdir count`

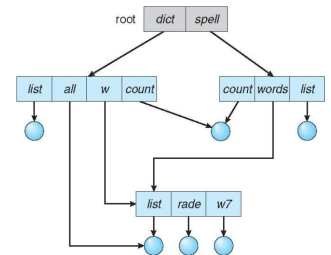
18.5.5 Directory a Grafo Aciclico

Hanno sottodirectory e file condivisi con nomi diversi (alias), ma possono fare riferimento agli stessi dati, quindi la condivisione è possibile.

Quando si elimina una directory, si ottiene un puntatore pendente (dangling pointer) che punta a memoria non più allocata; il suo dereferenzamento costituisce un comportamento indefinito.

Soluzioni:

- **Backpointer - elimina tutto**, in modo da poter eliminare tutti i puntatori alla cartella eliminata; (records di dimensione variabile sono un problema.) Così lo cancelli da una parte e lui di conseguenza sale e cancella i direttori.
- **Backpointer** usando un'organizzazione a catena (daisy chain)
- **entry-hold-count - elimina il puntatore** Soluzione con contatore di riferimenti, elimino solo il puntatore al file e mantengo un contatore ai file. Il file viene cancellato solo se il contatore raggiunge zero.



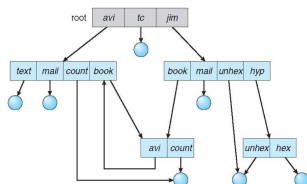
Nuovi tipi di voci nelle directory:

Link - un altro nome (puntatore) a un file esistente

Risolvi il link - segui il puntatore per localizzare il file

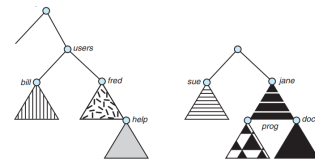
Directory a Grafo Generale

Come garantiamo l'assenza di cicli? Consentiamo solo link a file, non a sottodirectory; **Garbage collection**; ogni volta che viene aggiunto un nuovo link, si usa un algoritmo di rilevamento dei cicli per verificare se è accettabile.



File System Mounting

Un file system deve essere montato prima di poter essere accessibile. Un file system non montato è un'entità separata che non fa parte dell'albero logico principale.



18.6 File Sharing

La condivisione di file su sistemi multi-utente è desiderabile. La condivisione può essere effettuata attraverso uno schema di protezione. Su sistemi distribuiti, i file possono essere condivisi attraverso una rete. Il Network File System (NFS) è un metodo comune di condivisione di file distribuita.

- **User IDs:** Gli ID utente identificano gli utenti, consentendo che i permessi e le protezioni siano per utente.
- **Group IDs:** Gli ID gruppo consentono agli utenti di essere in gruppi, permettendo diritti di accesso per gruppo.
- **Owner of a file / directory:** Il proprietario di un file o directory.
- **Group of a file / directory:** Il gruppo a cui appartiene un file o directory.

Utilizzare la rete per consentire l'accesso al file system tra sistemi.

Il modello client-server consente ai client di montare file system remoti dai server:

- I Sistemi Informativi Distribuiti (servizi di naming distribuiti) come LDAP, DNS, NIS, Active Directory implementano un accesso unificato alle informazioni necessarie per l'elaborazione remota

Tutti i file system presentano modalità di guasto, ad esempio la corruzione delle strutture delle directory o di altri dati non utente, chiamati metadati. I file system remoti aggiungono nuove modalità di guasto, dovute a malfunzionamenti della rete o del server. Il ripristino dai guasti può coinvolgere informazioni di stato relative allo status di ciascuna richiesta remota.

18.7 Protezione

Il proprietario/creatore del file deve poter controllare: cosa può essere fatto/visto, da chi. Tipi di accesso: - Lettura - Scrittura - Esecuzione - Aggiunta - Eliminazione - Elenco.

18.7.1 Liste di Accesso e Gruppi

- Accesso del proprietario (Owner) $7 \rightarrow \text{RWX}$ (111)
- Accesso del gruppo (Group) $6 \rightarrow \text{RW-}$ (110)
- Accesso pubblico (Public) $1 \rightarrow \text{-X}$ (001)

```
owner  group  public
  |      |      |
chmod 761 game
```

Capitolo 19

I/O System

19.1 Panoramica

(fatto alla velox, ma alcune domande all'esame ci possono essere) La gestione dell'I/O è un componente principale della progettazione e del funzionamento dei sistemi operativi.

- Aspetto importante del funzionamento del computer
- I dispositivi di I/O variano notevolmente
- Vari metodi per controllarli
- Gestione delle prestazioni (Performance management)
- Frequente introduzione di nuovi tipi di dispositivi

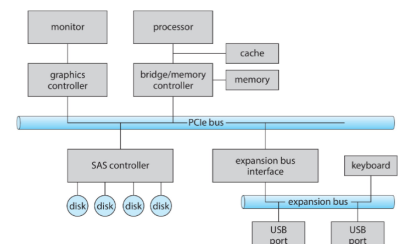
Porte, bus e device controller si connettono a vari dispositivi. I device driver incapsulano i dettagli del dispositivo, presentando un'interfaccia di accesso uniforme al sottosistema di I/O.

19.2 I/O Hardware

Incredibile varietà di dispositivi di I/O come: storage, trasmissione, interfacce umane.

Concetti comuni - i segnali dai dispositivi di I/O si interfacciano con il computer:

- **Porta (Port):** - punto di connessione per il dispositivo
- **Bus:** - daisy chain o accesso diretto condiviso
 - **PCI:** - bus comune nei PC e nei server, PCI Express (PCIe)
 - **expansion bus** - connette dispositivi relativamente lenti
 - **Serial-attached SCSI (SAS)** - interfaccia comune per dischi
- **Controller (host adapter)** - elettronica che gestisce porta, bus, dispositivo:
 - A volte integrato
 - A volte su una scheda a circuito separata (host adapter)
 - Contiene processore, microcodice, memoria privata, bus controller, ecc. Alcuni comunicano con il controller specifico del dispositivo tramite bus controller, microcodice, memoria, ecc.



Il Fibre channel (FC) è un controller complesso, solitamente una scheda a circuito separata (host-bus adapter, HBA) collegata al bus.

Le istruzioni di I/O controllano i dispositivi.

I dispositivi solitamente hanno registri in cui il device driver inserisce comandi, indirizzi e dati da scrivere, o legge dati dai registri dopo l'esecuzione del comando.

- Registro data-in, registro data-out, registro di stato (status register), registro di controllo (control register)
- Tipicamente 1-4 byte, o buffer FIFO

I dispositivi hanno indirizzi, utilizzati da: Istruzioni di I/O dirette o Memory-mapped I/O (MMIO - il dispositivo si prende un pezzo di memoria, non c'è ram ma c'è il dispositivo diverso da quello che verrà presentato nei prossimi capitoli), dati del dispositivo e registri di comando mappati nello spazio degli indirizzi del processore, specialmente per ampi spazi di indirizzamento (grafica).

I/O address range (hexadecimal)	device
000-00F	DMA controller
020-021	interrupt controller
040-043	timer
200-20F	game controller
2F8-2FF	serial port (secondary)
320-32F	hard-disk controller
378-37F	parallel port
3D0-3DF	graphics controller
3F0-3F7	diskette-drive controller
3F8-3FF	serial port (primary)

Figura 19.1: Posizioni delle porte di I/O dei dispositivi sui PC (parziale)

19.3 Polling

(Parte interessante)

In memoria secondaria bisogna controllare se il disco è pronto, se è occupato, etc.

Per ogni byte di I/O:

1. Legge il bit di busy dal registro di stato finché non è 0
2. L'host imposta il bit di lettura o scrittura e, se in scrittura, copia i dati nel registro data-out
3. L'host imposta il bit command-ready
4. Il controller imposta il bit di busy, esegue il trasferimento
5. Il controller azzerava il bit di busy, il bit di errore, il bit command-ready al termine del trasferimento

Il passo 1 è un ciclo di busy-wait per attendere l'I/O dal dispositivo. Questo è ragionevole se il dispositivo è veloce, ma inefficiente se il dispositivo è lento.

La CPU passa ad altri task? Ma se salta un ciclo, i dati vengono sovrascritti / persi.

19.4 Interrupt

Il polling può avvenire in 3 cicli di istruzione. Leggi lo stato, AND logico per estrarre il bit di stato, salta se non è zero. Come essere più efficienti se è diverso da zero solo raramente?

Linea Interrupt-request della CPU attivata dal dispositivo di I/O, controllata dal processore dopo ogni istruzione.

- L'**Interrupt handler** riceve gli interrupt; possono essere **mascherabili** (maskable) per ignorare o ritardare alcuni interrupt
- **Interrupt vector** per inviare l'interrupt all'handler corretto. Context switch all'inizio e alla fine basato sulla priorità, alcuni non mascherabili. Interrupt chaining se c'è più di un dispositivo con lo stesso numero di interrupt

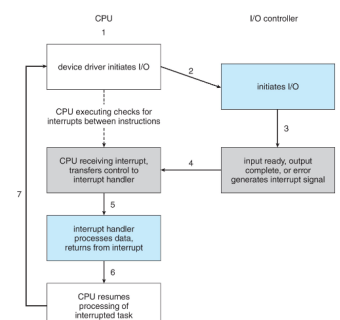
Meccanismo di interrupt utilizzato anche per le eccezioni: terminare un processo, crash del sistema a causa di un errore hardware.

Il page fault viene eseguito in caso di errore di accesso alla memoria. Le system call vengono eseguite tramite trap per attivare il kernel all'esecuzione della richiesta.

I sistemi Multi-CPU possono elaborare gli interrupt in modo concorrente, se il sistema operativo è progettato per gestirlo.

Utilizzato per elaborazioni sensibili al tempo, frequenti, deve essere veloce.

Colori diversi sono attori diversi.



19.5 Latenza (Latency)

Sottolineiamo la gestione degli interrupt perché anche i sistemi mono-utente gestiscono centinaia di interrupt al secondo e i server centinaia di migliaia.

Per esempio, un desktop macOS a riposo ha generato 23.000 interrupt in 10 secondi.

19.6 Direct Memory Access (DMA)

Utilizzato per evitare l'I/O programmato (un byte alla volta) per grandi spostamenti di dati. Richiede un controller DMA.

Bypassa la CPU per trasferire i dati direttamente tra il dispositivo di I/O e la memoria. Tipo thread parallelo.

Il sistema operativo scrive il blocco di comando DMA in memoria:

- Indirizzi di origine e destinazione
- Modalità di lettura o scrittura
- Conteggio dei byte
- Scrive la posizione del blocco di comandi nel controller DMA
- Bus mastering del controller DMA - prende il controllo del bus dalla CPU. Ruba cicli (Cycle stealing) alla CPU ma è comunque molto più efficiente
- Al termine, lancia un interrupt per segnalare il completamento

Una versione consapevole degli indirizzi virtuali può essere ancora più efficiente - DVMA.

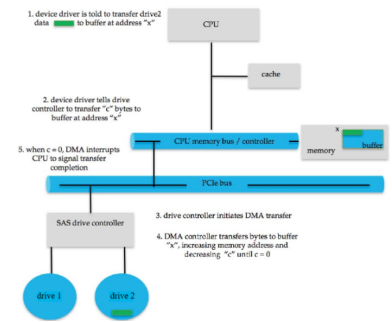
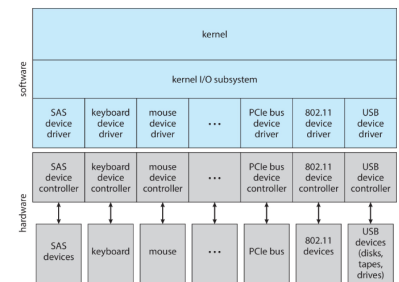


Figura 19.2: Processo in sei fasi per eseguire un trasferimento DMA

19.7 Interfaccia di I/O per le applicazioni

Le system call di I/O incapsulano i comportamenti dei dispositivi in classi generiche. Il livello dei device-driver nasconde le differenze tra i controller di I/O al kernel. Nuovi dispositivi che comunicano con protocolli già implementati non richiedono lavoro aggiuntivo. Ogni sistema operativo ha le proprie strutture del sottosistema di I/O e framework per i device driver. I dispositivi variano in molte dimensioni:

- Stream di caratteri o a blocchi (Character-stream o block)
- Sequenziale o ad accesso casuale (random-access)
- Sincrono o asincrono (o entrambi)
- Condivisibile (Sharable) o dedicato
- Velocità operativa
- lettura-scrittura, sola lettura, o sola scrittura



19.8 Caratteristiche dei dispositivi di I/O

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read-write	CD-ROM graphics controller disk

Le sottigliezze dei dispositivi sono gestite dai device driver. In generale i dispositivi di I/O possono essere raggruppati dal sistema operativo in

Aspetto	Variazione	Esempio
modalità di trasferimento dati	carattere blocco	terminale disco
metodo di accesso	sequenziale casuale	modem CD-ROM
pianificazione del trasferimento	sincrono asincrono	nastro tastiera
condivisione	dedicato condivisibile	nastro tastiera
velocità del dispositivo	latenza tempo di seek velocità di trasferimento ritardo tra le operazioni	
direzione di I/O	sola lettura sola scrittura lettura-scrittura	CD-ROM controller grafico disco

- Block I/O (I/O a blocchi)
- Character I/O (I/O a caratteri / Stream)
- Accesso a file mappati in memoria (Memory-mapped file access)
- Socket di rete

Per la manipolazione diretta delle caratteristiche specifiche del dispositivo di I/O, di solito c'è una scappatoia / porta di servizio (back door). Chiamata Unix `ioctl()` per inviare bit arbitrari a un registro di controllo del dispositivo e dati al registro dati del dispositivo. UNIX e Linux usano tuple di numeri di dispositivo "major" e "minor" per identificare il tipo e l'istanza dei dispositivi (qui major 8 e minor 0-4):

```
% ls -l /dev/sda* brw-rw---- 1 root disk 8, 0 Mar 16 09:18 /dev/sda
brw-rw---- 1 root disk 8, 1 Mar 16 09:18 /dev/sda1
brw-rw---- 1 root disk 8, 2 Mar 16 09:18 /dev/sda2
brw-rw---- 1 root disk 8, 3 Mar 16 09:18 /dev/sda3
```

19.9 Dispositivi a blocchi e a caratteri

I dispositivi a blocchi includono le unità disco (disk drives)

- I comandi includono read, write, seek
- Raw I/O, I/O diretto, o accesso al file system
- Accesso tramite memory-mapped file possibile: File mappato nella memoria virtuale e cluster portati tramite demand paging
- DMA

I dispositivi a caratteri includono tastiere, mouse, porte seriali

- I comandi includono `get()`, `put()`
- Librerie stratificate al di sopra permettono l'editing di linea

19.10 Dispositivi di rete

Variano abbastanza da quelli a blocchi e a caratteri da avere una propria interfaccia. Linux, Unix, Windows e molti altri includono l'interfaccia socket

- Separa il protocollo di rete dall'operazione di rete
- Include la funzionalità `select()`

Gli approcci variano ampiamente (pipe, FIFO, stream, code, mailbox)

19.11 Clock e Timer

Forniscono ora corrente, tempo trascorso, timer. Risoluzione normale circa 1/60 di secondo. Alcuni sistemi forniscono timer a risoluzione più alta. Timer a intervallo programmabile utilizzato per tempistiche, interrupt periodici.

`ioctl()` (su UNIX) copre aspetti particolari dell'I/O come clock e timer

19.12 I/O Non bloccante e Asincrono

Bloccante (Blocking) - processo sospeso fino al completamento dell'I/O

- Facile da usare e comprendere
- Insufficiente per alcune necessità

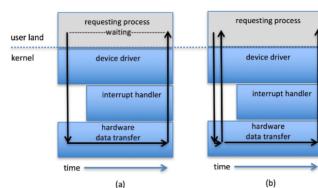
Non bloccante (Nonblocking) - la chiamata di I/O restituisce tutto ciò che è disponibile

- Interfaccia utente, copia di dati (I/O bufferizzato)
- Implementato tramite multi-threading
- Ritorna rapidamente con il conteggio dei byte letti o scritti
- `select()` per verificare se i dati sono pronti, quindi `read()` o `write()` per il trasferimento

Asincrono (Asynchronous) - il processo continua l'esecuzione mentre l'I/O viene eseguito

- Difficile da usare
- Il sottosistema di I/O segnala al processo quando l'I/O è completato

19.13 Due metodi di I/O



19.14 Vectored I/O

(saltato)

Il Vectored I/O consente a una singola system call di eseguire più operazioni di I/O.

Ad esempio, Unix `readve()` accetta un vettore di buffer multipli in cui leggere o da cui scrivere.

Questo metodo scatter-gather è migliore di molteplici chiamate I/O individuali

- Riduce il context switching e l'overhead delle system call
- Alcune versioni forniscono atomicità. Evita, per esempio, di preoccuparsi di thread multipli che modificano i dati mentre si verificano letture/scritture

Capitolo 20

File System Implementation

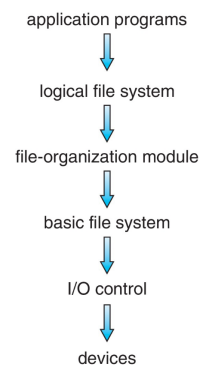
20.1 Struttura del File System

Struttura del file: unità di archiviazione logica e raccolta di informazioni correlate.
Il file system fornisce:

- interfaccia utente allo storage, mappando logico su fisico
- accesso efficiente e comodo al disco, permettendo di memorizzare, localizzare e recuperare facilmente i dati
- Interfaccia all'HW: il disco fornisce riscrittura in-place e accesso casuale - i trasferimenti I/O avvengono in blocchi o settori (solitamente 512 byte)

File control block (FCB) - struttura di archiviazione contenente informazioni su un file.

Driver del dispositivo controlla il dispositivo fisico - molte attività: un File System può essere organizzato a livelli.



20.1.1 Driver dei Dispositivi - I/O Controller

I **driver dei dispositivi** gestiscono i dispositivi I/O al livello di controllo I/O.

Ricevuti comandi come: leggi drive1, cilindro 72, traccia 2, settore 10, in posizione memoria 1060.

Emette comandi hardware di basso livello specifici per il controller hardware

20.1.2 Basic File System

Il **file system di base**, ricevuto un comando come “recupera blocco 123”, lo traduce in comando per il driver del dispositivo.

Gestisce anche i buffer e le cache in memoria (allocazione, liberazione, sostituzione):

- I buffer contengono i dati in transito
- Le cache contengono i dati usati frequentemente

20.1.3 File organization module

Il **modulo di organizzazione** dei file comprende file, indirizzi logici e blocchi fisici.

Traduce il numero di blocco logico nel numero di blocco fisico.

Gestisce lo spazio libero e l'allocazione del disco.

20.1.4 Logical File System

Il file system logico gestisce le informazioni sui metadati:

- Traduce il nome del file in numero di file, handle e posizione, mantenendo i file control block (inode in UNIX)

- Gestione delle directory
- Protezione

La stratificazione è utile per ridurre complessità e ridondanza, ma aggiunge overhead e può diminuire le prestazioni.

I livelli logici possono essere implementati con qualsiasi metodo di codifica, a discrezione del progettista del OS.

Il file system logico viene chiamato direttamente dalle **applicazioni**, es. `C open()`

20.2 Tipi di File System

Molti file system, a volte più di uno nello stesso sistema operativo, ciascuno con il proprio formato:

- CD-ROM è ISO 9660;
- Unix ha UFS, FFS;
- Linux ha più di 130 tipi, con ext3 ed ext4 come estensioni del file system principale;
- Windows ha FAT, FAT32, NTFS oltre a floppy, CD, DVD Blu-ray.

Ne continuano ad arrivare di nuovi - ZFS, GoogleFS, Oracle ASM, FUSE

20.3 Dall'API all'Implementazione

20.3.1 Operazioni del File System

Abbiamo system call al livello API: Open, close, read, write, seek... ma come implementiamo le loro funzioni? Mediante strutture **su disco** e **in memoria**.

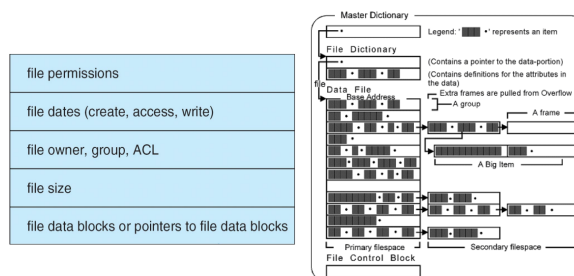
Boot control block (o **MBR**) contiene le informazioni necessarie al sistema per avviare l'OS da quel volume. Necessario se il volume contiene l'OS, di solito il primo blocco del volume.

Volume control block (superblock, master file table) contiene i dettagli del volume: numero totale di blocchi, numero di blocchi liberi, dimensione del blocco, puntatori o array di blocchi liberi.

La struttura delle directory organizza i file: nomi e numeri di inode, master file table.

20.3.2 File Control Block - FCB

L'OS mantiene un FCB per ogni file, che contiene molti dettagli sul file: tipicamente, numero di inode (se Linux), permessi, dimensione, date.



20.3.3 Il Linux iNode

Un inode (noto anche come index node) è una struttura dati usata da Unix/Linux per descrivere un oggetto all'interno di un file system. Tale oggetto può essere un file o una directory.

Ogni inode memorizza puntatori alle posizioni dei blocchi su disco dei dati e dei metadati dell'oggetto.

In generale, i metadati contenuti in un inode sono:

- tipo di file (file normale/directory/link simbolico/file speciale a blocchi/file speciale a caratteri/ecc.),

- permessi,
- ID proprietario/gruppo,
- dimensione,
- ora dell'ultimo accesso/modifica,
- ora di modifica dei metadati
- numero di hard link

Esercizio Blocco: 4KB Indice = 4B

$\# \text{indici/blocco} = 4K/4B = 1K$

file = 4GB = $4GB/4KB = 1M$ blocchi

1M indice nella tabella (un indice per ogni blocco)

tabella richiede 1K blocchi di 2do livello 1 blocco di primo livello

se file > 4GB serve un terzo livello.

Essendo che non sappiamo quanto è grande il file, quando parti (primi 10) sono gratis e li metto nel FCB (inode) se cresce gioco l'indice ad un solo livello ed ospito 1K blocchi.

I file piccoli sono veloci, quelli grandi sono più lenti ma hai tanto da leggere e scrivere.

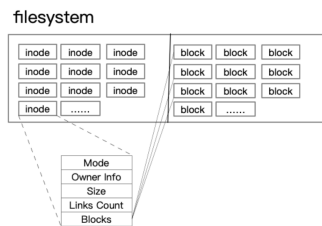


Figura 20.1: Il Linux iNode

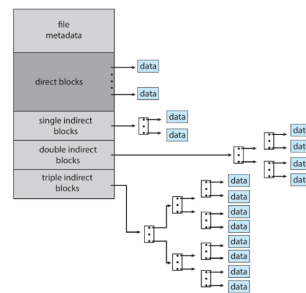


Figura 20.2: UNIX UFS - 4 KB per blocco, indirizzi a 32 bit

20.3.4 Strutture del File System in Memoria

Tabella di montaggio - memorizza i mount dei file system, i punti di montaggio e i tipi di file system.

Tabella globale dei file aperti - contiene una copia del FCB di ogni file e altre informazioni.

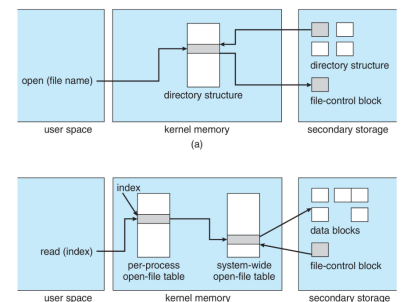
Tabella dei file aperti per processo - contiene puntatori alle voci appropriate nella tabella globale e altre informazioni.

Gli elementi della tabella sono chiamati:

- fd descrittori di file in UNIX/Linux
- fh file handler in Windows
- stessa denominazione delle funzioni C

La `open()` cerca, mentre la `read()` si dimentica dei direttori e legge. Il processo A può avere aperto un file 5. Un'altro processo può avere un altro file 5 ma può essere un'altro file

Se ogni file fosse accessibile esclusivamente per ogni processo (no shering) basterebbe la **tabella dei file aperti per processo** mentre la **tabella globale dei file aperti** tiene traccia dei lock sui vari file.

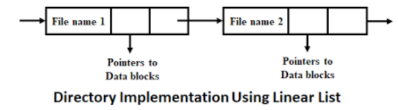


20.4 Implementazione delle Directory

20.4.1 Lista Lineare

Lista lineare di nomi di file con puntatori ai blocchi dati.

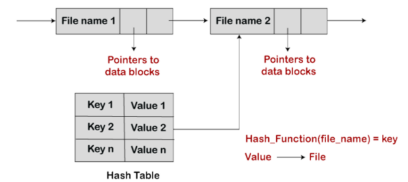
- Semplice da programmare
- Lenta da eseguire: tempo di ricerca lineare; si potrebbe mantenere ordinata alfabeticamente tramite lista collegata o usare un B+ tree



20.4.2 Tabella Hash

Tabella Hash - lista lineare con struttura dati hash:

- Riduce il tempo di ricerca nella directory
- Collisioni - situazioni in cui due nomi di file hanno lo stesso valore hash
- Efficiente solo se le voci hanno dimensione fissa, oppure si usa il metodo a catena di overflow



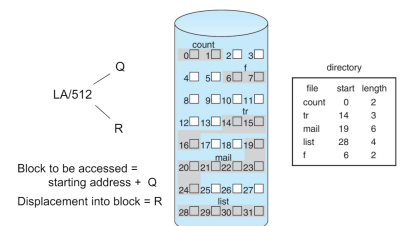
20.5 Metodo di Allocazione

Un metodo di allocazione definisce come i blocchi del disco vengono assegnati ai file: Contigua, Collegata, Tabella di Allocazione dei File (FAT)

20.5.1 Metodo di Allocazione Contigua

Ogni file occupa un insieme di blocchi contigui.

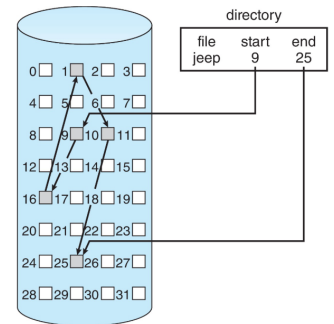
- Migliori prestazioni nella maggior parte dei casi
- Semplice: sono necessari solo la posizione iniziale (numero di blocco) e la lunghezza (numero di blocchi)
- I problemi includono:
 - Trovare spazio sul disco per un file,
 - Conoscere la dimensione del file,
 - Frammentazione esterna nell'HD; necessita di compattazione offline (downtime) o online, detta anche defrag.



Mappatura da logico a fisico (dimensione blocco = 512 byte).

20.5.2 Allocazione Collegata - Linked

- Ogni file è una lista collegata di blocchi
- Il file termina con un puntatore null
- Nessuna frammentazione esterna
- Ogni blocco contiene un puntatore al blocco successivo (minimo overhead)
- Nessuna compattazione, nessuna frammentazione esterna
- Il sistema di gestione dello spazio libero viene chiamato quando è necessario un nuovo blocco
- Si migliora l'efficienza raggruppando i blocchi in cluster, ma aumenta la frammentazione interna
- Se perdi un puntatore perdi il resto del file/ dei file



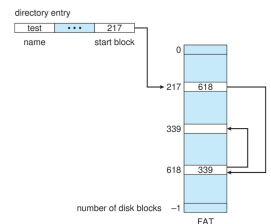
Grande problema: localizzare un blocco richiede molti I/O e seek del disco. Ogni file è una lista collegata di blocchi su disco: i blocchi possono essere sparsi ovunque sul disco

Il blocco da accedere è il Q^{esimo} blocco nella catena collegata di blocchi che rappresenta il file. Spostamento nel blocco = $R + 1$.

Se vogliamo capire dove sta il blocco 1500 dividiamo per (512-1) dato il puntatore.

20.5.3 Metodo di Allocazione File-Allocation Table FAT

Scorporiamo i puntatori dai file e mantengo il pointer di 1 e il blocco 512. Due vettori di dati PARALLELI. Devo stare attento a non corrompere il vettore dei pointer. L'inizio del volume ha una tabella, indicizzata per numero di blocco. Simile a una lista collegata, ma più veloce su disco e memorizzabile in cache. L'allocazione di nuovi blocchi è semplice. Struttura che contiene solo il puntatore

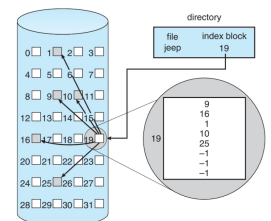


20.5.4 Metodo di Allocazione Indicizzata

C'è un difetto delle liste: l'accesso diretto è $O(n)$. Usando la FAT devo sempre scorrere il vettore dei puntatori anche se molto meglio di una lista collegata. Ogni file ha il proprio/i propri blocco/i indice con puntatori ai propri blocchi dati. Può esserci frammentazione interna, dipende quanto occupa il file. Filosofia simile alla page-table.

Se ti serve più spazio:

- Metti più blocchi indice in lista



20.5.5 Prestazioni

Il metodo migliore dipende dal tipo di accesso al file:

- L'allocazione contigua è ottima per accesso sequenziale e casuale, unico difetto c'è frammentazione
- L'allocazione collegata è buona per l'accesso sequenziale, non per quello casuale
- L'indicizzata è più complessa: per accedere a un singolo blocco potrebbe essere necessario leggere prima il blocco indice e poi il blocco dati.

Dichiarare il tipo di accesso alla creazione: scegliere tra contigua o collegata.

20.6 Gestione dello Spazio Libero

Il file system mantiene la **lista dello spazio libero** per tracciare i blocchi disponibili. Simile alla lista delle pagine libere per la gestione della memoria virtuale.

Vettore di bit o bit map (n blocchi).

Nota: la scelta di che struttura usare per allocare file e della gestione dello spazio libero può essere diversa (inode per i file e lista continua per il free space)

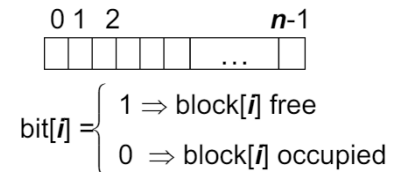


Figura 20.3: bit map

Calcolo del numero di blocco:

Semplice, $(\text{numero di bit per parola}) * (\text{numero di parole con valore 0}) + \text{offset del primo bit a 1}$
 ma la bit map richiede spazio extra.

Esempio:

dimensione blocco = 4KB = 2^{12} byte

dimensione disco = 2^{40} byte (1 TB)

$n = 2^{40} / 2^{12} = 2^{28}$ bit (o 32MB)

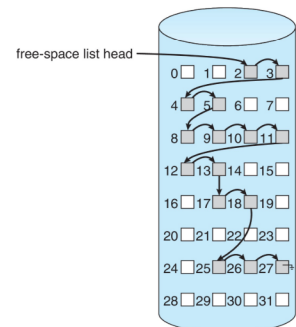
Vantaggio: è facile ottenere spazio per file contigui verificando se ci sono abbastanza blocchi adiacenti liberi

20.6.1 Lista Collegata dello Spazio Libero su Disco

Lista collegata (free list):

- **SVANTAGGIO:** non è facile ottenere spazio contiguo
- **VANTAGGIO:** nessuno spreco. Lista collegata dello spazio libero su disco

Non è necessario attraversare l'intera lista (se il numero di blocchi liberi è registrato)



20.6.2 Lista Collegata dello Spazio Libero

Raggruppamento:

- Modificare la lista collegata per memorizzare l'indirizzo degli n-1 blocchi liberi successivi nel primo blocco libero, più un puntatore al blocco successivo che contiene puntatori a blocchi liberi (come questo)
- I primi n-1 blocchi saranno liberi
- L'n-esimo conterrà puntatori agli n-1 blocchi liberi successivi

Conteggio

- Poiché lo spazio viene spesso usato e liberato in modo contiguo, con allocazione contigua, estensioni o clustering, si mantiene l'indirizzo del primo blocco libero e il conteggio dei blocchi liberi successivi. La lista dello spazio libero contiene quindi voci con indirizzi e conteggi.
- Il primo blocco della lista libera conterrà l'indirizzo del blocco libero successivo + il numero di blocchi liberi dopo di esso
- Gli altri saranno effettivamente blocchi liberi

20.6.3 TRIMing Unused Blocks

Nelle SSD ci sono i blocchi liberi ma ancora da cancellare

20.6.4 Efficiency and Performance of File Systems

- Come organizzati i file
- Come organizzati le directory
- Come organizzati lo spazio libero
- Come gestisci i metadati (file control block)

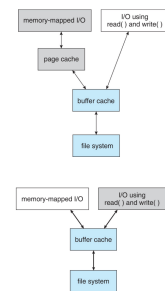
In questo contesto possiamo pensare a strutture dati a lunghezza fissa o a lunghezza variabile. Per migliorare le prestazioni bisogna fare attenzione a: quanto è $O(1)$ o $O(n)$ ma quanto incide?

- tenere dati e metadati vicini
- Buffer cache - non leggo sempre il disco ma lo tengo in memoria per blocchi usati frequentemente. Riduce le richieste al disco.
- Distinguere scritture sincrone e asincrone
- Read-ahead - lettura seq. da file con lettura blocco 3 leggo anche il 4 se sono vicini. Migliore accesso sequenziale

20.6.5 Page Cache

Il *memory-mapped I/O* scrivi su file senza usare le read e write, il SO per essere efficiente usa una **page cache**. Quindi 2 accessi a file presente nell'immagine.

Unified Buffer Cache Per evitare di avere due copie e ottimizzazione a più livelli di cache si è passato ad avere un **unified buffer cache**, meno overhead.



20.6.6 Recovery

Avere ridondanza sui file, come i dischi, posso usare i backup. Ma tra un backup e l'altro si perde qualcosa. Si può usare quindi il *journal*.

20.6.7 Log Structured File Systems

Come agire se si rompe a metà strada? Mantine una copia della transazione e prima di effettuare l'operazione salva cosa si voleva fare questo si ottiene uno stato più consistente.

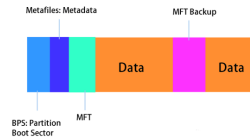
Una volta che sono nel log si considerano fatte. La transazione dunque è come dire "voglio fare il bonifico" e poi in modo asincrono verrà eseguita.

20.7 File System in altri OS

20.7.1 File System di Windows

NTFS, detto anche New Technology File System, è un file system proprietario con journaling sviluppato da Microsoft.

A partire da Windows NT 3.1, è il file system predefinito della famiglia Windows NT. Ha soppiantato la File Allocation Table (FAT) come file system preferito su Windows ed è supportato anche da Linux e BSD.



FAT, nota anche come File Allocation Table, è un file system sviluppato per personal computer.

Originariamente sviluppata nel 1977 per l'uso su floppy disk, è stata adattata per dischi rigidi e altri dispositivi. È spesso supportata per motivi di compatibilità dai sistemi operativi attuali per PC e da molti dispositivi mobili e sistemi embedded, consentendo l'interscambio di dati tra sistemi diversi.

FAT32, successore di FAT16, è stata progettata da Microsoft come nuova versione del file system. FAT32 supporta un numero maggiore di cluster possibili e riutilizza la maggior parte del codice esistente.



Figura 20.4: Struttura del file system FAT32

20.8 Thread in OS161

Il kernel ha bisogno di due tipologie di thread: **user thread** - utilizzate dalle librerie utente - e **kernel thread** - supportate del kernel.

Un **Thread** rappresenta il control state di un processo (programma in esecuzione). Questo context include:

- lo stato del processore come registri, SP, etc;
- Stack, localizzato nell'address space del processo

I thread possono essere many-to-one, one-to-one o many-to-many.

La memoria, come sappiamo già, contiene il codice, i dati e lo stack che contiene il record di attivazione delle funzioni.

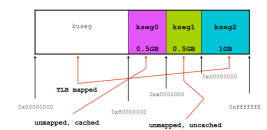
Switch frame, usato in OS161 per il context switch - running, ready, blocked etc. **Trap frame**, usato per gestire le interruzioni - simile al ritorno da una funzione.

20.9 Memoria in OS161

La memoria ha un indirizzo fisico e uno virtuale, in OS161 usa sempre la TLB, anche se non c'è la page table. Se da TLB miss chiama la gestione di un page fault. Se c'è allocazione continua come in DUMBVM è un misto tra allocazione contigua e usa la TLB.

I primi due indirizzi logici sono per user e gli altri due per kernel, così basta guardare l'indirizzo per capire se è user o kernel.

MIPS se l'indirizzo è minore di 2GB uso la TLB, altrimenti: kseg0 usa relocation pensa il processore (aggiungendo 2GB), kseg1 stanno i dispositivi IO mapped in memory e **non** usa la cache (perché io voglio scrivere ma non rileggere quello che ho scritto ma l'effetto), kseg2 alcune strutture del kernel che possono essere paginate (non usato).



20.10 Lab 4

Vogliamo evitare che un thread sia in uno stato zombie, ovvero che nessuno ha collezionato il risultato del thread e i suoi metadati. Si vuole implementare waitpid, attesa di fine processo. Inizialmente crealo lato kernel dato che non serve il PID, perché il kernel ha accesso alle strutture e metadati.

proc.h e in struct proc aggiungere p_id e p_status e un semaforo (o cvar con boolean, cv e un lock).

Nota: la soluzione proposta per cvar non include l'uso del bool. Quindi in proc.h è aggiunto bool p_terminated e poi devi implementarlo.

proc.c `proc_create` creiamo il semaforo o la cvar e poi le distruggo quando uso la `proc_destroy` successivamente quando abbiamo creato ciò creiamo la `proc_wait` un attesa se è un semaforo o un attesa e un while e poi prendo i dati di ritorno. Questa era la prima parte del kernel.

Ora modifichiamo la `sysexit` Quella precedente nel lab 2 distruggeva la struct `proc` (che ora è nella `proc_wait`) mentre ora vogliamo collezionare i dati. `proc_syscall.c`

poi deve fare un `thread_exit`, ma attenzione che se il padre viene svegliato troppo presto potrebbe essere che voglia distruggere un figlio ancora in esecuzione. Per risolvere al posto di scriverla della `proc_destroy` la faccio nella `proc_syscall.c` (`V->p_sem`) e dobbiamo introdurre in `thread.c` in `thread_exit` deve essere chiamata anche quando non ci sono processi in esecuzione.

```
if(cur->t_prc != null) proc_destroy(cur->t_prc); cur->t_prc = null; else
```